```
!pip install -q kagglehub

import kagglehub

print("KaggleHub is ready")
```

```
KaggleHub is ready
```

```
import requests

url =
"https://www.kaggle.com/api/v1/datasets/list?search=awsaf49/asvspoof-
2019-dataset"

response = requests.get(url)
data = response.json()

for d in data:
    print(d["ref"], d["title"])
```

```
awsaf49/asvpoof-2019-dataset ASVspoof 2019 Dataset
awsaf49/asvspoof-2019-tfrecord-dataset ASVspoof 2019 tfrecord Dataset
```

```
!pip install -q kaggle


!kaggle datasets files -d awsaf49/asvspoof-2019-dataset
```

```
403 Client Error: Forbidden for url:
https://api.kaggle.com/v1/datasets.DatasetApiService/ListDatasetFiles
```

```
import os

os.makedirs("/root/.kaggle", exist_ok=True)

%%writefile /root/.kaggle/kaggle.json
{
  "username": "Avanish31",
  "key": "KGAT_a5bfc6e43f87519a12bcb0a4c96d2f30"
}
```

```
Writing /root/.kaggle/kaggle.json
```

```
!chmod 600 /root/.kaggle/kaggle.json

!kaggle datasets list -s "ASVspoof 2019 Dataset"
```

```
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
ref                                                            title
size  lastUpdated                 downloadCount  voteCount
usabilityRating
```

```
-------------------------------------------------------------
-----------------------------------------  -----------
--------------------------  -------------  ---------  ---------------

awsaf49/asvpoof-2019-dataset                                    ASVspoof
2019 Dataset                          25321823144  2022-06-21
17:18:04.330000          17028         73  0.625
anishsarkar22/asvpoof-2019-dataset-la                           ASVSpoof
2019 LA Dataset                        7646913333  2025-10-13
11:12:09.283000            754          2  0.4375
awsaf49/asvspoof-2019-tfrecord-dataset                          ASVspoof
2019 tfrecord Dataset                 1107616818  2022-06-23 06:41:42
226          6  0.375
mohammedabdeldayem/scenefake
SceneFake                                        5767431347  2024-04-20
13:32:41.803000           1811         23  0.875
dhruvtangri1998/asvspoof-dataset-2019                           ASVspoof
dataset 2019                           7646913333  2025-03-17
10:33:31.273000            195          0  0.1875
eminkorkut/deepfakevoice-mfcc-4datasets
DeepFakeVoice-MFCC-4Datasets                      180800358  2026-03-08
14:27:46.507000             43          2  0.7058824
eminkorkut/deepfakevoice-wac2vec-4datasets
DeepFakeVoice-Wac2Vec-4Datasets                  3309490475  2026-03-08
14:21:29.550000             22          1  0.7058824
abhishekkumarabsie/asvspoof-2019-dataset-logical-access-only  ASVspoof
2019 Dataset [Logical Access Only]   7646913333  2026-05-20
13:22:47.210000              1          0  0.3125
eminkorkut/deepfakevoice-hubert-4datasets
DeepFakeVoice-HuBERT-4Datasets                   3456502600  2026-03-08
13:51:10.900000              5          1  0.7058824
eminkorkut/deepfakevoice-google-hear-4datasets
DeepFakeVoice-google-HeAR-4Datasets              4577806640  2026-03-08
15:04:23.780000             24          1  0.75
ictmuhammadaltaf/audio-100k-real-vs-fake                        RAISE
100k                                   4899206703  2025-07-05
16:20:53.473000              8          0  0.4117647
sukiss/asvspoof-dataset
Asvspoof_dataset                                      5355  2025-08-02
18:57:44.890000              7          0  0.25
kavyjaiswal/asvspoof-2019-pa-20-spectrograms
ASVspoof_2019_PA_20_spectrograms                16706883876  2025-11-13
23:18:19.090000             16          0  0.4375

!kaggle datasets files -d anishsarkar22/asvpoof-2019-dataset-la

Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
Next Page Token = CfDJ8ImuQD4OY2pEnVW2WQ-
```

```
kgncO2lyAaQVj3raByv8CvhGO5nFDRCBMAuJcWkX9W-
aDKbFEYdSbUV59aK5iAEDid5YUH7kZpc5Q7qDZFxE_GKgk5_JzCRUvUhVnHrzznXwpHGbQ
lV0bDNKufj98T7BmrKzocBQstMfZ-9nVL3vM4bKOdvHSrNQ
name
size  creationDate
------------------------------------------------------------------------
--  -------  --------------------------
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.dev.female.trl.tx
t    677652  2025-10-13 11:13:31.155000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.dev.female.trn.tx
t       972  2025-10-13 11:13:31.143000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.dev.gi.trl.txt
976804  2025-10-13 11:13:31.148000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.dev.male.trl.txt
299152  2025-10-13 11:13:31.076000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.dev.male.trn.txt
1096  2025-10-13 11:13:31.066000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.eval.female.trl.t
xt  2371140  2025-10-13 11:13:31.119000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.eval.female.trn.t
xt     4374  2025-10-13 11:13:31.134000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.eval.gi.trl.txt
3512112  2025-10-13 11:13:30.985000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.eval.male.trl.txt
1140972  2025-10-13 11:13:31.137000
LA/ASVspoof2019_LA_asv_protocols/ASVspoof2019.LA.asv.eval.male.trn.txt
5754  2025-10-13 11:13:31.118000
LA/ASVspoof2019_LA_asv_scores/ASVspoof2019.LA.asv.dev.gi.trl.scores.tx
t    632703  2025-10-13 11:14:49.401000
LA/ASVspoof2019_LA_asv_scores/ASVspoof2019.LA.asv.eval.gi.trl.scores.t
xt  2422073  2025-10-13 11:14:49.443000
LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.dev.trl.txt
822400  2025-10-13 11:14:49.401000
LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.eval.trl.txt
2358176  2025-10-13 11:14:49.421000
LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.cm.train.trn.txt
840120  2025-10-13 11:14:49.461000
LA/ASVspoof2019_LA_dev/LICENSE.txt
19941  2025-10-13 11:14:49.401000
LA/ASVspoof2019_LA_dev/flac/LA_D_1000265.flac
32573  2025-10-13 11:14:40.280000
LA/ASVspoof2019_LA_dev/flac/LA_D_1000752.flac
38423  2025-10-13 11:14:14.094000
LA/ASVspoof2019_LA_dev/flac/LA_D_1001095.flac
65292  2025-10-13 11:14:27.990000
LA/ASVspoof2019_LA_dev/flac/LA_D_1002130.flac
85044  2025-10-13 11:14:40.786000

!mkdir -p /content/ASVspoof2019
!kaggle datasets download \
```

```
    -d anishsarkar22/asvpoof-2019-dataset-la \
    -p /content/ASVspoof2019 \
    --unzip

Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
Dataset URL: https://www.kaggle.com/datasets/anishsarkar22/asvpoof-
2019-dataset-la
License(s): ODC Attribution License (ODC-By)
Downloading asvpoof-2019-dataset-la.zip to /content/ASVspoof2019
100% 7.12G/7.12G [03:20<00:00, 38.1MB/s]


!find /content/ASVspoof2019/LA -maxdepth 2 -type d

/content/ASVspoof2019/LA
/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols
/content/ASVspoof2019/LA/ASVspoof2019_LA_asv_protocols
/content/ASVspoof2019/LA/ASVspoof2019_LA_eval
/content/ASVspoof2019/LA/ASVspoof2019_LA_eval/flac
/content/ASVspoof2019/LA/ASVspoof2019_LA_asv_scores
/content/ASVspoof2019/LA/ASVspoof2019_LA_dev
/content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac
/content/ASVspoof2019/LA/ASVspoof2019_LA_train
/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac

!find /content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac -type f | wc
-l

25380

!find /content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac -type f | wc -
l

24986

!head -10
/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.
cm.train.trn.txt

LA_0079 LA_T_1138215 - - bonafide
LA_0079 LA_T_1271820 - - bonafide
LA_0079 LA_T_1272637 - - bonafide
LA_0079 LA_T_1276960 - - bonafide
LA_0079 LA_T_1341447 - - bonafide
LA_0079 LA_T_1363611 - - bonafide
LA_0079 LA_T_1596451 - - bonafide
LA_0079 LA_T_1608170 - - bonafide
LA_0079 LA_T_1684951 - - bonafide
LA_0079 LA_T_1699801 - - bonafide
```

```
!find /content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac -type f | wc -
l

24986

!head -10
/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.
cm.dev.trl.txt

LA_0069 LA_D_1047731 - - bonafide
LA_0069 LA_D_1105538 - - bonafide
LA_0069 LA_D_1125976 - - bonafide
LA_0069 LA_D_1293230 - - bonafide
LA_0069 LA_D_1340209 - - bonafide
LA_0069 LA_D_1376638 - - bonafide
LA_0069 LA_D_1377420 - - bonafide
LA_0069 LA_D_1403371 - - bonafide
LA_0069 LA_D_1590530 - - bonafide
LA_0069 LA_D_1595036 - - bonafide

from collections import Counter

protocol =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA
.cm.train.trn.txt"

labels = []

with open(protocol, "r") as f:
    for line in f:
        parts = line.strip().split()
        labels.append(parts[-1])

print(Counter(labels))

Counter({'spoof': 22800, 'bonafide': 2580})

from collections import Counter

protocol =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA
.cm.dev.trl.txt"

labels = []

with open(protocol, "r") as f:
    for line in f:
        parts = line.strip().split()
        labels.append(parts[-1])

print(Counter(labels))
```

```
Counter({'spoof': 22296, 'bonafide': 2548})
```

```
import librosa

audio_path =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac/LA_T_1138215.flac
"

y, sr = librosa.load(audio_path, sr=None)

print("Sample rate:", sr)
print("Number of samples:", len(y))
print("Duration:", len(y) / sr, "seconds")
print("Shape:", y.shape)
```

```
Sample rate: 16000
Number of samples: 55329
Duration: 3.4580625 seconds
Shape: (55329,)
```

```
import librosa
import numpy as np

audio_path =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac/LA_T_1138215.flac
"

# Settings
SR = 16000
DURATION = 4
N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256

# Load
y, sr = librosa.load(audio_path, sr=SR)

# Fixed 4-second length
target_length = SR * DURATION

if len(y) < target_length:
    y = np.pad(y, (0, target_length - len(y)))
else:
    y = y[:target_length]

# Mel spectrogram
mel = librosa.feature.melspectrogram(
    y=y,
    sr=SR,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
```

```
    n_mels=N_MELS
)

# Convert to log scale
log_mel = librosa.power_to_db(mel, ref=np.max)

print("Audio shape:", y.shape)
print("Log-Mel shape:", log_mel.shape)
print("Minimum:", log_mel.min())
print("Maximum:", log_mel.max())
```

```
Audio shape: (64000,)
Log-Mel shape: (128, 251)
Minimum: -80.0
Maximum: 0.0
```

```
import torch
from torch.utils.data import Dataset, DataLoader
import librosa
import numpy as np
import os

class ASVspoofDataset(Dataset):

    def __init__(self, audio_dir, protocol_file):
        self.audio_dir = audio_dir
        self.samples = []

        with open(protocol_file, "r") as f:
            for line in f:
                parts = line.strip().split()

                audio_id = parts[1]
                label = parts[-1]

                if label == "bonafide":
                    label = 0
                else:
                    label = 1

                audio_path = os.path.join(
                    audio_dir,
                    audio_id + ".flac"
                )

                self.samples.append((audio_path, label))

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, index):
```

```
        audio_path, label = self.samples[index]

        # Load audio
        y, sr = librosa.load(
            audio_path,
            sr=16000
        )

        # Fixed 4 seconds
        target_length = 16000 * 4

        if len(y) < target_length:
            y = np.pad(
                y,
                (0, target_length - len(y))
            )
        else:
            y = y[:target_length]

        # Mel spectrogram
        mel = librosa.feature.melspectrogram(
            y=y,
            sr=16000,
            n_fft=1024,
            hop_length=256,
            n_mels=128
        )

        # Log-Mel
        log_mel = librosa.power_to_db(
            mel,
            ref=np.max
        )

        # Convert to tensor
        log_mel = torch.tensor(
            log_mel,
            dtype=torch.float32
        )

        # Add channel dimension
        log_mel = log_mel.unsqueeze(0)

        label = torch.tensor(
            label,
            dtype=torch.float32
        )

        return log_mel, label
```

```
train_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac"

train_protocol =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA
.cm.train.trn.txt"

dev_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac"

dev_protocol =
"/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA
.cm.dev.trl.txt"


train_dataset = ASVspoofDataset(
    train_audio,
    train_protocol
)

dev_dataset = ASVspoofDataset(
    dev_audio,
    dev_protocol
)

print("Training samples:", len(train_dataset))
print("Validation samples:", len(dev_dataset))
```

```
Training samples: 25380
Validation samples: 24844
```

```
x, y = train_dataset[0]

print("Input shape:", x.shape)
print("Label:", y)
print("Min:", x.min().item())
print("Max:", x.max().item())
```

```
Input shape: torch.Size([1, 128, 251])
Label: tensor(0.)
Min: -80.0
Max: 0.0
```

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    num_workers=0
)

x_batch, y_batch = next(iter(train_loader))
```

```
print("Batch input shape:", x_batch.shape)
print("Batch labels shape:", y_batch.shape)
print("Labels:", y_batch)
```

```
Batch input shape: torch.Size([32, 1, 128, 251])
Batch labels shape: torch.Size([32])
Labels: tensor([1., 1., 1., 1., 1., 1., 0., 1., 1., 1., 1., 1., 1.,
1., 1., 1., 1., 1.,
        1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

```
import os
import hashlib
from collections import defaultdict

train_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac"
dev_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac"


def find_exact_duplicates(audio_dir):
    hashes = defaultdict(list)

    for filename in os.listdir(audio_dir):
        if filename.endswith(".flac"):
            filepath = os.path.join(audio_dir, filename)

            with open(filepath, "rb") as f:
                file_hash = hashlib.md5(f.read()).hexdigest()

            hashes[file_hash].append(filename)

    duplicates = {
        h: files for h, files in hashes.items()
        if len(files) > 1
    }

    return duplicates


train_duplicates = find_exact_duplicates(train_audio)
dev_duplicates = find_exact_duplicates(dev_audio)

print("TRAIN")
print("Files:", len(os.listdir(train_audio)))
print("Duplicate groups:", len(train_duplicates))

print("\nDEV")
print("Files:", len(os.listdir(dev_audio)))
print("Duplicate groups:", len(dev_duplicates))
```

```
TRAIN
Files: 25380
Duplicate groups: 0

DEV
Files: 24986
Duplicate groups: 0
```

```
import os
import hashlib

def get_audio_hashes(audio_dir):
    hashes = {}

    for filename in os.listdir(audio_dir):
        if filename.endswith(".flac"):
            filepath = os.path.join(audio_dir, filename)

            with open(filepath, "rb") as f:
                file_hash = hashlib.md5(f.read()).hexdigest()

            hashes[file_hash] = filename

    return hashes


train_hashes = get_audio_hashes(train_audio)
dev_hashes = get_audio_hashes(dev_audio)

common_hashes = set(train_hashes.keys()) & set(dev_hashes.keys())

print("Exact duplicates between TRAIN and DEV:", len(common_hashes))

if common_hashes:
    print("\nDuplicate files:")
    for h in common_hashes:
        print(
            "TRAIN:", train_hashes[h],
            "<-> DEV:", dev_hashes[h]
        )
else:
    print("No exact duplicate recordings between TRAIN and DEV.")
```

```
Exact duplicates between TRAIN and DEV: 0
No exact duplicate recordings between TRAIN and DEV.
```

```
import os
import soundfile as sf
from collections import Counter

def inspect_audio_dataset(audio_dir):
```

```
    sample_rates = Counter()
    channels = Counter()
    file_formats = Counter()
    corrupted = []
    durations = []

    files = [
        f for f in os.listdir(audio_dir)
        if f.endswith(".flac")
    ]

    for i, filename in enumerate(files):
        filepath = os.path.join(audio_dir, filename)

        try:
            info = sf.info(filepath)

            sample_rates[info.samplerate] += 1
            channels[info.channels] += 1
            file_formats[os.path.splitext(filename)[1]] += 1
            durations.append(info.duration)

        except Exception as e:
            corrupted.append((filename, str(e)))

        if (i + 1) % 5000 == 0:
            print(f"Checked {i + 1}/{len(files)} files")

    return sample_rates, channels, file_formats, durations, corrupted


print("========== TRAIN ==========")

train_sr, train_channels, train_formats, train_durations,
train_corrupted = \
    inspect_audio_dataset(train_audio)

print("\nSample rates:")
print(train_sr)

print("\nChannels:")
print(train_channels)

print("\nFormats:")
print(train_formats)

print("\nCorrupted files:", len(train_corrupted))


print("\n========== DEV ==========")
```

```
dev_sr, dev_channels, dev_formats, dev_durations, dev_corrupted = \
    inspect_audio_dataset(dev_audio)

print("\nSample rates:")
print(dev_sr)

print("\nChannels:")
print(dev_channels)

print("\nFormats:")
print(dev_formats)

print("\nCorrupted files:", len(dev_corrupted))
```

```
========== TRAIN ==========
Checked 5000/25380 files
Checked 10000/25380 files
Checked 15000/25380 files
Checked 20000/25380 files
Checked 25000/25380 files

Sample rates:
Counter({16000: 25380})

Channels:
Counter({1: 25380})

Formats:
Counter({'.flac': 25380})

Corrupted files: 0

========== DEV ==========
Checked 5000/24986 files
Checked 10000/24986 files
Checked 15000/24986 files
Checked 20000/24986 files

Sample rates:
Counter({16000: 24986})

Channels:
Counter({1: 24986})

Formats:
Counter({'.flac': 24986})

Corrupted files: 0
```

```
import numpy as np
```

```python
def duration_statistics(durations, name):
    durations = np.array(durations)

    print(f"\n========== {name} ==========")
    print(f"Number of files: {len(durations)}")
    print(f"Minimum duration: {durations.min():.3f} sec")
    print(f"Maximum duration: {durations.max():.3f} sec")
    print(f"Mean duration: {durations.mean():.3f} sec")
    print(f"Median duration: {np.median(durations):.3f} sec")
    print(f"25th percentile: {np.percentile(durations, 25):.3f} sec")
    print(f"75th percentile: {np.percentile(durations, 75):.3f} sec")

    print(f"\nFiles shorter than 4 sec: {(durations < 4).sum()}")
    print(f"Files exactly/longer than 4 sec: {(durations >=
4).sum()}")


duration_statistics(train_durations, "TRAIN")
duration_statistics(dev_durations, "DEV")
```

```
========== TRAIN ==========
Number of files: 25380
Minimum duration: 0.652 sec
Maximum duration: 13.188 sec
Mean duration: 3.426 sec
Median duration: 3.202 sec
25th percentile: 2.435 sec
75th percentile: 4.110 sec

Files shorter than 4 sec: 18389
Files exactly/longer than 4 sec: 6991

========== DEV ==========
Number of files: 24986
Minimum duration: 0.695 sec
Maximum duration: 16.514 sec
Mean duration: 3.494 sec
Median duration: 3.284 sec
25th percentile: 2.452 sec
75th percentile: 4.253 sec

Files shorter than 4 sec: 17415
Files exactly/longer than 4 sec: 7571
```

```python
import os
import numpy as np
import librosa

def analyze_audio_properties(audio_dir, protocol_file):
```

```
    real_rms = []
    fake_rms = []

    real_peak = []
    fake_peak = []

    real_duration = []
    fake_duration = []

    with open(protocol_file, "r") as f:
        for line in f:
            parts = line.strip().split()

            audio_id = parts[1]
            label = parts[-1]

            path = os.path.join(audio_dir, audio_id + ".flac")

            try:
                y, sr = librosa.load(path, sr=None, mono=True)

                rms = np.sqrt(np.mean(y ** 2))
                peak = np.max(np.abs(y))
                duration = len(y) / sr

                if label == "bonafide":
                    real_rms.append(rms)
                    real_peak.append(peak)
                    real_duration.append(duration)

                else:
                    fake_rms.append(rms)
                    fake_peak.append(peak)
                    fake_duration.append(duration)

            except Exception:
                pass

    return (
        np.array(real_rms),
        np.array(fake_rms),
        np.array(real_peak),
        np.array(fake_peak),
        np.array(real_duration),
        np.array(fake_duration)
    )


train_properties = analyze_audio_properties(
    train_audio,
```

```
    train_protocol
)

(
    real_rms,
    fake_rms,
    real_peak,
    fake_peak,
    real_duration,
    fake_duration
) = train_properties


print("========== BONAFIDE / HUMAN ==========")
print("RMS mean:", real_rms.mean())
print("RMS median:", np.median(real_rms))
print("Peak mean:", real_peak.mean())
print("Duration mean:", real_duration.mean())


print("\n========== SPOOF / AI ==========")
print("RMS mean:", fake_rms.mean())
print("RMS median:", np.median(fake_rms))
print("Peak mean:", fake_peak.mean())
print("Duration mean:", fake_duration.mean())
```

```
========== BONAFIDE / HUMAN ==========
RMS mean: 0.11687967
RMS median: 0.11462553
Peak mean: 0.9998638
Duration mean: 3.388754820736434

========== SPOOF / AI ==========
RMS mean: 0.12262276
RMS median: 0.11933981
Peak mean: 0.9999459
Duration mean: 3.430018851425439
```

```
import os

base_dir = "/content/ASVspoof2019/LA"

print("Files containing language-related information:")

found = []

for root, dirs, files in os.walk(base_dir):
    for file in files:
        name = file.lower()
        if "lang" in name or "language" in name:
            found.append(os.path.join(root, file))
```

```
if found:
    for f in found:
        print(f)
else:
    print("No language metadata files found.")
```

```
Files containing language-related information:
No language metadata files found.
```

```
import pandas as pd

def get_speakers(protocol_file):
    data = []

    with open(protocol_file, "r") as f:
        for line in f:
            parts = line.strip().split()

            if len(parts) >= 5:
                speaker_id = parts[0]
                audio_id = parts[1]
                label = parts[-1]

                data.append([speaker_id, audio_id, label])

    return pd.DataFrame(
        data,
        columns=["speaker_id", "audio_id", "label"]
    )

train_info = get_speakers(train_protocol)
dev_info = get_speakers(dev_protocol)

print("TRAIN")
print("Recordings:", len(train_info))
print("Unique speakers:", train_info["speaker_id"].nunique())

print("\nDEV")
print("Recordings:", len(dev_info))
print("Unique speakers:", dev_info["speaker_id"].nunique())

# Check whether any speakers occur in both train and validation
train_speakers = set(train_info["speaker_id"])
dev_speakers = set(dev_info["speaker_id"])

overlap = train_speakers.intersection(dev_speakers)

print("\nSpeaker overlap between TRAIN and DEV:", len(overlap))

if len(overlap) == 0:
```

```
    print("✅ Speaker-disjoint")
else:
    print("❌ Speaker overlap found")
    print("Example overlapping speakers:", list(overlap)[:10])
```

```
TRAIN
Recordings: 25380
Unique speakers: 20

DEV
Recordings: 24844
Unique speakers: 20

Speaker overlap between TRAIN and DEV: 0
✅ Speaker-disjoint
```

```
print("First 10 TRAIN protocol entries:\n")

with open(train_protocol, "r") as f:
    for _ in range(10):
        line = f.readline().strip()
        print(line.split())
```

```
First 10 TRAIN protocol entries:

['LA_0079', 'LA_T_1138215', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1271820', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1272637', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1276960', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1341447', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1363611', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1596451', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1608170', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1684951', '-', '-', 'bonafide']
['LA_0079', 'LA_T_1699801', '-', '-', 'bonafide']
```

```
print("First 20 SPOOF TRAIN entries:\n")

count = 0

with open(train_protocol, "r") as f:
    for line in f:
        parts = line.strip().split()

        if parts[-1] == "spoof":
            print(parts)
            count += 1

            if count == 20:
                break
```

```
First 20 SPOOF TRAIN entries:

['LA_0079', 'LA_T_1004644', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1056709', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1195221', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1265032', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1287124', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1365409', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1477244', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1531370', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1641285', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1647444', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1780590', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1850803', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1886440', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1918731', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1924569', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_1925377', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_2057403', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_2099563', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_2128730', '-', 'A01', 'spoof']
['LA_0079', 'LA_T_2152432', '-', 'A01', 'spoof']
```

```
spoof_systems = set()
spoof_attacks = set()

with open(train_protocol, "r") as f:
    for line in f:
        parts = line.strip().split()

        if parts[-1] == "spoof":
            spoof_systems.add(parts[2])
            spoof_attacks.add(parts[3])

print("System IDs:", len(spoof_systems))
print(sorted(spoof_systems))

print("\nAttack IDs:", len(spoof_attacks))
print(sorted(spoof_attacks))
```

```
System IDs: 1
['-']

Attack IDs: 6
['A01', 'A02', 'A03', 'A04', 'A05', 'A06']
```

```
from collections import Counter

def get_attack_distribution(protocol_file):
    attacks = []
```

```
    with open(protocol_file, "r") as f:
        for line in f:
            parts = line.strip().split()

            if parts[-1] == "spoof":
                attacks.append(parts[3])

    return Counter(attacks)

train_attacks = get_attack_distribution(train_protocol)
dev_attacks = get_attack_distribution(dev_protocol)

print("TRAIN attack distribution:")
for attack, count in sorted(train_attacks.items()):
    print(f"{attack}: {count}")

print("\nDEV attack distribution:")
for attack, count in sorted(dev_attacks.items()):
    print(f"{attack}: {count}")

print("\nAttack IDs in TRAIN:", sorted(train_attacks.keys()))
print("Attack IDs in DEV:", sorted(dev_attacks.keys()))

train_set = set(train_attacks.keys())
dev_set = set(dev_attacks.keys())

print("\nCommon attacks:", sorted(train_set & dev_set))
print("TRAIN-only attacks:", sorted(train_set - dev_set))
print("DEV-only attacks:", sorted(dev_set - train_set))
```

```
TRAIN attack distribution:
A01: 3800
A02: 3800
A03: 3800
A04: 3800
A05: 3800
A06: 3800

DEV attack distribution:
A01: 3716
A02: 3716
A03: 3716
A04: 3716
A05: 3716
A06: 3716

Attack IDs in TRAIN: ['A01', 'A02', 'A03', 'A04', 'A05', 'A06']
Attack IDs in DEV: ['A01', 'A02', 'A03', 'A04', 'A05', 'A06']

Common attacks: ['A01', 'A02', 'A03', 'A04', 'A05', 'A06']
```

```
TRAIN-only attacks: []
DEV-only attacks: []

train_ids = set(train_info["audio_id"])
dev_ids = set(dev_info["audio_id"])

overlap = train_ids.intersection(dev_ids)

print("TRAIN audio IDs:", len(train_ids))
print("DEV audio IDs:", len(dev_ids))
print("Overlapping audio IDs:", len(overlap))

if len(overlap) == 0:
    print("□ No audio-ID leakage between TRAIN and DEV")
else:
    print("□ Leakage detected")
    print(list(overlap)[:20])

TRAIN audio IDs: 25380
DEV audio IDs: 24844
Overlapping audio IDs: 0
□ No audio-ID leakage between TRAIN and DEV

from collections import Counter

# Count classes in the training dataset
class_counts = Counter()

for _, label in train_dataset.samples:
    class_counts[label] += 1

bonafide_count = class_counts[0]
spoof_count = class_counts[1]

total = bonafide_count + spoof_count

# Weight for each class
bonafide_weight = total / (2 * bonafide_count)
spoof_weight = total / (2 * spoof_count)

print("Class counts:")
print("Bonafide:", bonafide_count)
print("Spoof:", spoof_count)

print("\nClass weights:")
print("Bonafide weight:", round(bonafide_weight, 4))
print("Spoof weight:", round(spoof_weight, 4))

Class counts:
Bonafide: 2580
Spoof: 22800
```

```
Class weights:
Bonafide weight: 4.9186
Spoof weight: 0.5566
```

```
# Show all protocol files available
import os

protocol_dir = "/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols"

for file in sorted(os.listdir(protocol_dir)):
    print(file)
```

```
ASVspoof2019.LA.cm.dev.trl.txt
ASVspoof2019.LA.cm.eval.trl.txt
ASVspoof2019.LA.cm.train.trn.txt
```

```
import torch
import torch.nn as nn

class VoiceCNN(nn.Module):
    def __init__(self, dropout=0.3):
        super(VoiceCNN, self).__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 16 * 31, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, 2)
        )

    def forward(self, x):
        x = self.features(x)
```

```
        x = self.classifier(x)
        return x


# Select GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# CREATE THE MODEL
model = VoiceCNN(dropout=0.3).to(device)

print("Device:", device)
print("Model created successfully")
```

```
Device: cpu
Model created successfully
```

```
import torch.optim as optim

LEARNING_RATE = 0.001
WEIGHT_DECAY = 0.0001
BATCH_SIZE = 32
MAX_EPOCHS = 50
DROPOUT = 0.3
PATIENCE = 7

class_weights = torch.tensor(
    [4.9186, 0.5566],
    dtype=torch.float32
).to(device)

criterion = nn.CrossEntropyLoss(weight=class_weights)

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY
)

scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=2
)

print("Training configuration ready")
```

```
Training configuration ready
```

```
# Get one batch
inputs, labels = next(iter(train_loader))
```

```
# Move to GPU/CPU
inputs = inputs.to(device)
labels = labels.long().to(device)

# Forward pass
outputs = model(inputs)

# Calculate loss
loss = criterion(outputs, labels)

print("Input shape :", inputs.shape)
print("Label shape :", labels.shape)
print("Output shape:", outputs.shape)
print("Loss        :", loss.item())
```

```
Input shape : torch.Size([32, 1, 128, 251])
Label shape : torch.Size([32])
Output shape: torch.Size([32, 2])
Loss        : 0.7023959159851074
```

```
import torch

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in loader:

        inputs = inputs.to(device)
        labels = labels.long().to(device)

        # Clear old gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backpropagation
        loss.backward()

        # Update model weights
        optimizer.step()

        # Statistics
```

```
        running_loss += loss.item() * inputs.size(0)

        predictions = torch.argmax(outputs, dim=1)

        correct += (predictions == labels).sum().item()
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_accuracy = correct / total

    return epoch_loss, epoch_accuracy


def validate(model, loader, criterion, device):
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    all_predictions = []
    all_labels = []

    with torch.no_grad():

        for inputs, labels in loader:

            inputs = inputs.to(device)
            labels = labels.long().to(device)

            outputs = model(inputs)

            loss = criterion(outputs, labels)

            running_loss += loss.item() * inputs.size(0)

            predictions = torch.argmax(outputs, dim=1)

            correct += (predictions == labels).sum().item()
            total += labels.size(0)

            all_predictions.extend(predictions.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    epoch_loss = running_loss / total
    epoch_accuracy = correct / total

    return epoch_loss, epoch_accuracy, all_predictions, all_labels

best_val_loss = float("inf")
patience_counter = 0
```

```python
for epoch in range(MAX_EPOCHS):

    train_loss, train_acc = train_one_epoch(
        model,
        train_loader,
        criterion,
        optimizer,
        device
    )

    val_loss, val_acc, val_predictions, val_labels = validate(
        model,
        dev_loader,
        criterion,
        device
    )

    # Reduce learning rate if validation loss stops improving
    scheduler.step(val_loss)

    current_lr = optimizer.param_groups[0]["lr"]

    print(
        f"Epoch [{epoch+1}/{MAX_EPOCHS}] | "
        f"Train Loss: {train_loss:.4f} | "
        f"Train Acc: {train_acc:.4f} | "
        f"Val Loss: {val_loss:.4f} | "
        f"Val Acc: {val_acc:.4f} | "
        f"LR: {current_lr:.6f}"
    )

    # Save best model
    if val_loss < best_val_loss:

        best_val_loss = val_loss
        patience_counter = 0

        torch.save(
            model.state_dict(),
            "/content/best_voice_cnn.pth"
        )

        print("  → Best model saved")

    else:
        patience_counter += 1

    # Early stopping
    if patience_counter >= PATIENCE:
```

```
        print("Early stopping triggered.")
        break
```

```
---------------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_959/441344691.py in <cell line: 0>()
      4 for epoch in range(MAX_EPOCHS):
      5
----> 6     train_loss, train_acc = train_one_epoch(
      7         model,
      8         train_loader,

/tmp/ipykernel_959/2725376992.py in train_one_epoch(model, loader,
criterion, optimizer, device)
     23
     24         # Backpropagation
---> 25         loss.backward()
     26
     27         # Update model weights

/usr/local/lib/python3.13/dist-packages/torch/_tensor.py in
backward(self, gradient, retain_graph, create_graph, inputs)
    629                 inputs=inputs,
    630             )
--> 631         torch.autograd.backward(
    632             self, gradient, retain_graph, create_graph,
inputs=inputs
    633         )

/usr/local/lib/python3.13/dist-packages/torch/autograd/__init__.py in
backward(tensors, grad_tensors, retain_graph, create_graph,
grad_variables, inputs)
    379     # some Python versions print out the first line of a
multi-line function
    380     # calls in the traceback and some print out the last line
--> 381     _engine_run_backward(
    382         tensors,
    383         grad_tensors_,

/usr/local/lib/python3.13/dist-packages/torch/autograd/graph.py in
_engine_run_backward(t_outputs, *args, **kwargs)
    867         unregister_hooks =
_register_logging_hooks_on_whole_graph(t_outputs)
    868     try:
--> 869         return Variable._execution_engine.run_backward(  #
Calls into the C++ engine to run the backward pass
    870             t_outputs, *args, **kwargs
```

```
    871         )  # Calls into the C++ engine to run the backward
pass

KeyboardInterrupt:
```

```
import torch

print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("Using CPU")
```

```
CUDA available: False
Using CPU
```

```
import torch

print(torch.cuda.is_available())

if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

```
False
```

```
import torch

print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA available: True
GPU: Tesla T4
```

```
import os

dataset_path = "/content/ASVspoof2019"

print("Dataset exists:", os.path.exists(dataset_path))

if os.path.exists(dataset_path):
    print(os.listdir(dataset_path))
```

```
Dataset exists: False
```

```
!pip -q install kaggle

import os

print(os.path.exists("/root/.kaggle/kaggle.json"))
```

```
False
```

```
!mkdir -p /content/ASVspoof2019
```

```
!kaggle datasets download \
    -d anishsarkar22/asvspoof-2019-dataset-la \
    -p /content/ASVspoof2019
```

```
403 Client Error: Forbidden for url:
https://api.kaggle.com/v1/datasets.DatasetApiService/GetDatasetMetadat
a
```

```
import os

os.makedirs("/root/.kaggle", exist_ok=True)
```

```
%%writefile /root/.kaggle/kaggle.json
{
  "username": "Avanish31",
  "key": "KGAT_a5bfc6e43f87519a12bcb0a4c96d2f30"
}
```

```
Writing /root/.kaggle/kaggle.json
```

```
!mkdir -p /content/ASVspoof2019
```

```
!kaggle datasets download \
    -d anishsarkar22/asvspoof-2019-dataset-la \
    -p /content/ASVspoof2019
```

```
Warning: Your Kaggle API key is readable by other users on this
system! To fix this, you can run 'chmod 600 /root/.kaggle/kaggle.json'
403 Client Error: Forbidden for url:
https://api.kaggle.com/v1/datasets.DatasetApiService/GetDatasetMetadat
a
```

```
!chmod 600 /root/.kaggle/kaggle.json
```

```
!ls -l /root/.kaggle/kaggle.json
```

```
-rw------- 1 root root 80 Sep  2 19:15 /root/.kaggle/kaggle.json
```

```
!kaggle datasets list -s asvspoof
```

```
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
ref                                                        title
size  lastUpdated                 downloadCount  voteCount
usabilityRating
-----------------------------------------------------------
```

```
------------------------------  -----------
--------------------------  ------------  ---------  ---------------

mohammedabdeldayem/avsspoof-2021                          ASVspoof 2021
Dataset           62250567017  2024-05-18 17:40:31.800000
3164         47  0.875
awsaf49/asvpoof-2019-dataset                              ASVspoof 2019
Dataset           25321823144  2022-06-21 17:18:04.330000
17030         73  0.625
abdallamohamed312/ready-to-input-for-training             Balanced
ASVspoof 2021 PA        5211923567  2024-06-01 05:38:52.957000
523          8  0.875
bishertello/asvspoof-21-df-cqt                            ASVspoof-21-DF-
CQT              21012728750  2024-08-15 22:24:33.100000
171          7  0.75
mohammedabdeldayem/scenefake                              SceneFake
5767431347  2024-04-20 13:32:41.803000           1811         23
0.875
anishsarkar22/asvpoof-2019-dataset-la                     ASVSpoof 2019
LA Dataset         7646913333  2025-10-13 11:12:09.283000
756          2  0.4375
hahunavth/asvspoof2019-la                                 ASVspoof2019_LA
7646913333  2023-09-19 11:54:38.713000            460          2  0.5

awsaf49/asvspoof-2019-tfrecord-dataset                    ASVspoof 2019
tfrecord Dataset   1107616818  2022-06-23 06:41:42
226          6  0.375
beosup/asvspoof-2021                                      ASVspoof-2021
88030113443  2025-08-21 08:37:49.067000             36          1
0.3125
yashwa9t1/asvspoof-wavlm-aug-model                        asvspoof-wavlm-
aug-model         1174948863  2026-07-16 18:21:19.737000
1          1  0.5
azkurniwan/asvspoof-2019-la                               ASVspoof 2019
LA                 7644718211  2026-02-18 12:45:43.993000
218          0  0.5
skenospeniel/asvspoof-21-df                               ASVspoof 21 DF
34625753486  2023-01-09 06:12:55.617000           3065          2
0.375
pratikjodgudri/asvspoof2021-df-audio-dataset
ASVspoof2021_DF_Audio_Dataset    1279232433  2024-11-13
07:45:36.147000            226          4  0.375
shalinisaagrawal/asvspoof-2019-melspectogram              AsvSpoof-2019-
Melspectogram      1755821901  2024-05-12 11:31:08.487000
19          2  0.1875
aniket202411001/asvspoof5-flac                            ASVSpoof5_flac
22336672698  2025-07-31 12:59:26.813000            103          2
0.1875
pankajsomkuwar/asvspoof-2021-df                           ASVspoof 2021
```

```
DF                 34625753486  2025-04-25 14:54:33.573000
75           2  0.25
abdallamohamed312/asv-2021-pa-pt12-into-real-and-fake  ASV 2021 (PA)
Part 1,2           14176529532  2024-05-31 20:10:39.510000
152           7  0.6875
abdallamohamed312/asv-2021-pa-part-3                   ASV 2021 (PA)
part 3              7090016509  2024-05-31 23:59:09.450000
94           7  0.6875
ulianazb/asvspoof-2024                                 Asvspoof 2024
57369198147  2024-07-01 09:54:02.680000            310           0
0.1875
yashwa9t/asvspoof-wavlm-clean-model                    asvspoof-wavlm-
clean-model        1174948936  2026-07-16 16:07:08.527000
2           0  0.375
```

```
!mkdir -p /content/ASVspoof2019

!kaggle datasets download \
    -d anishsarkar22/asvpoof-2019-dataset-la \
    -p /content/ASVspoof2019
```

```
Warning: Looks like you're using an outdated `kaggle` version
(installed: 2.0.2), please consider upgrading to the latest version
(2.2.2)
Dataset URL: https://www.kaggle.com/datasets/anishsarkar22/asvpoof-
2019-dataset-la
License(s): ODC Attribution License (ODC-By)
Downloading asvpoof-2019-dataset-la.zip to /content/ASVspoof2019
100% 7.12G/7.12G [01:37<00:00, 78.8MB/s]
```

```
!unzip -q /content/ASVspoof2019/asvpoof-2019-dataset-la.zip \
    -d /content/ASVspoof2019

import os

print(os.listdir("/content/ASVspoof2019"))
print(os.path.exists("/content/ASVspoof2019/LA"))
```

```
['asvpoof-2019-dataset-la.zip', 'LA']
True
```

```
import os

train_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac"
dev_audio = "/content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac"

print("Train files:", len(os.listdir(train_audio)))
print("Dev files:", len(os.listdir(dev_audio)))
```

```
Train files: 25380
Dev files: 24986
```

```
import os
import numpy as np
import librosa
import soundfile as sf
from collections import Counter

BASE = "/content/ASVspoof2019/LA"

TRAIN_AUDIO = os.path.join(
    BASE, "ASVspoof2019_LA_train", "flac"
)

DEV_AUDIO = os.path.join(
    BASE, "ASVspoof2019_LA_dev", "flac"
)

PROTOCOL_DIR = os.path.join(
    BASE, "ASVspoof2019_LA_cm_protocols"
)

TRAIN_PROTOCOL = os.path.join(
    PROTOCOL_DIR,
    "ASVspoof2019.LA.cm.train.trn.txt"
)

DEV_PROTOCOL = os.path.join(
    PROTOCOL_DIR,
    "ASVspoof2019.LA.cm.dev.trl.txt"
)

CACHE_DIR = "/content/asvspoof_cache"
os.makedirs(CACHE_DIR, exist_ok=True)

print("Train audio:", TRAIN_AUDIO)
print("Dev audio:", DEV_AUDIO)
print("Train protocol:", TRAIN_PROTOCOL)
print("Dev protocol:", DEV_PROTOCOL)
print("Cache:", CACHE_DIR)
```

```
Train audio: /content/ASVspoof2019/LA/ASVspoof2019_LA_train/flac
Dev audio: /content/ASVspoof2019/LA/ASVspoof2019_LA_dev/flac
Train protocol:
/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.
cm.train.trn.txt
Dev protocol:
/content/ASVspoof2019/LA/ASVspoof2019_LA_cm_protocols/ASVspoof2019.LA.
```

```
cm.dev.trl.txt
Cache: /content/asvspoof_cache

from collections import Counter

def read_protocol(protocol_file):
    samples = []

    with open(protocol_file, "r") as f:
        for line in f:
            parts = line.strip().split()

            if len(parts) < 5:
                continue

            samples.append({
                "speaker": parts[0],
                "audio_id": parts[1],
                "system": parts[2],
                "attack": parts[3],
                "label": 0 if parts[4] == "bonafide" else 1
            })

    return samples


train_samples = read_protocol(TRAIN_PROTOCOL)
dev_samples = read_protocol(DEV_PROTOCOL)

print("Train protocol:", len(train_samples))
print("Dev protocol:", len(dev_samples))

print("\nTrain labels:")
print(Counter(x["label"] for x in train_samples))

print("\nDev labels:")
print(Counter(x["label"] for x in dev_samples))

Train protocol: 25380
Dev protocol: 24844

Train labels:
Counter({1: 22800, 0: 2580})

Dev labels:
Counter({1: 22296, 0: 2548})

import os

train_file_ids = {
    os.path.splitext(f)[0]
```

```
    for f in os.listdir(TRAIN_AUDIO)
    if f.endswith(".flac")
}

dev_file_ids = {
    os.path.splitext(f)[0]
    for f in os.listdir(DEV_AUDIO)
    if f.endswith(".flac")
}

train_protocol_ids = {x["audio_id"] for x in train_samples}
dev_protocol_ids = {x["audio_id"] for x in dev_samples}

print("TRAIN")
print("Audio files:", len(train_file_ids))
print("Protocol IDs:", len(train_protocol_ids))
print("Missing:", len(train_protocol_ids - train_file_ids))
print("Extra:", len(train_file_ids - train_protocol_ids))

print("\nDEV")
print("Audio files:", len(dev_file_ids))
print("Protocol IDs:", len(dev_protocol_ids))
print("Missing:", len(dev_protocol_ids - dev_file_ids))
print("Extra:", len(dev_file_ids - dev_protocol_ids))

print("\nTRAIN ↔ DEV OVERLAP")
print("Common audio IDs:", len(train_file_ids & dev_file_ids))
```

```
TRAIN
Audio files: 25380
Protocol IDs: 25380
Missing: 0
Extra: 0

DEV
Audio files: 24986
Protocol IDs: 24844
Missing: 0
Extra: 142

TRAIN ↔ DEV OVERLAP
Common audio IDs: 0
```

```
train_speakers = {x["speaker"] for x in train_samples}
dev_speakers = {x["speaker"] for x in dev_samples}

print("Train speakers:", len(train_speakers))
print("Dev speakers:", len(dev_speakers))

overlap = train_speakers & dev_speakers
```

```
print("Speaker overlap:", len(overlap))

if len(overlap) == 0:
    print("✅ Speaker-disjoint split confirmed")
else:
    print("❌ Speaker leakage detected:")
    print(overlap)
```

```
Train speakers: 20
Dev speakers: 20
Speaker overlap: 0
✅ Speaker-disjoint split confirmed
```

```
from collections import Counter

train_attacks = Counter(
    x["attack"] for x in train_samples
    if x["label"] == 1
)

dev_attacks = Counter(
    x["attack"] for x in dev_samples
    if x["label"] == 1
)

print("Train attack types:")
for attack, count in sorted(train_attacks.items()):
    print(attack, ":", count)

print("\nDev attack types:")
for attack, count in sorted(dev_attacks.items()):
    print(attack, ":", count)
```

```
Train attack types:
A01 : 3800
A02 : 3800
A03 : 3800
A04 : 3800
A05 : 3800
A06 : 3800

Dev attack types:
A01 : 3716
A02 : 3716
A03 : 3716
A04 : 3716
A05 : 3716
A06 : 3716
```

```
import soundfile as sf
```

```
import os
from collections import Counter

def verify_audio_files(samples, audio_dir, name):

    sample_rates = Counter()
    channels = Counter()
    corrupted = []

    for i, item in enumerate(samples):

        path = os.path.join(
            audio_dir,
            item["audio_id"] + ".flac"
        )

        try:
            info = sf.info(path)

            sample_rates[info.samplerate] += 1
            channels[info.channels] += 1

        except Exception as e:
            corrupted.append(
                (item["audio_id"], str(e))
            )

        if (i + 1) % 5000 == 0:
            print(f"{name}: checked {i+1}/{len(samples)}")

    print(f"\n========== {name} ==========")
    print("Files checked:", len(samples))
    print("Sample rates:", sample_rates)
    print("Channels:", channels)
    print("Corrupted:", len(corrupted))

    if corrupted:
        print("First corrupted files:")
        print(corrupted[:5])


verify_audio_files(
    train_samples,
    TRAIN_AUDIO,
    "TRAIN"
)

verify_audio_files(
    dev_samples,
    DEV_AUDIO,
```

```
    "DEV"
)
```

```
TRAIN: checked 5000/25380
TRAIN: checked 10000/25380
TRAIN: checked 15000/25380
TRAIN: checked 20000/25380
TRAIN: checked 25000/25380

========== TRAIN ==========
Files checked: 25380
Sample rates: Counter({16000: 25380})
Channels: Counter({1: 25380})
Corrupted: 0
DEV: checked 5000/24844
DEV: checked 10000/24844
DEV: checked 15000/24844
DEV: checked 20000/24844

========== DEV ==========
Files checked: 24844
Sample rates: Counter({16000: 24844})
Channels: Counter({1: 24844})
Corrupted: 0
```

```
import os

CACHE_DIR = "/content/asvspoof_cache"

os.makedirs(CACHE_DIR, exist_ok=True)

print("Cache directory:", CACHE_DIR)
```

```
Cache directory: /content/asvspoof_cache
```

```
import os
import librosa
import numpy as np

TARGET_SR = 16000
DURATION = 4
TARGET_LENGTH = TARGET_SR * DURATION

N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256


def audio_to_logmel(path):

    # Load audio
```

```
    y, sr = librosa.load(
        path,
        sr=TARGET_SR,
        mono=True
    )

    # Make every recording exactly 4 seconds
    if len(y) < TARGET_LENGTH:
        y = np.pad(
            y,
            (0, TARGET_LENGTH - len(y))
        )
    else:
        y = y[:TARGET_LENGTH]

    # Mel spectrogram
    mel = librosa.feature.melspectrogram(
        y=y,
        sr=TARGET_SR,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        n_mels=N_MELS
    )

    # Convert to Log-Mel
    log_mel = librosa.power_to_db(
        mel,
        ref=np.max
    )

    return log_mel.astype(np.float16)

test_item = train_samples[0]

test_path = os.path.join(
    TRAIN_AUDIO,
    test_item["audio_id"] + ".flac"
)

test_feature = audio_to_logmel(test_path)

print("Audio:", test_item["audio_id"])
print("Feature shape:", test_feature.shape)
print("Data type:", test_feature.dtype)
print("Minimum:", test_feature.min())
print("Maximum:", test_feature.max())
```

```
Audio: LA_T_1138215
Feature shape: (128, 251)
Data type: float16
```

```
Minimum: -80.0
Maximum: 0.0
```

```
import os
import numpy as np

CACHE_DIR = "/content/asvspoof_cache"
os.makedirs(CACHE_DIR, exist_ok=True)

# Train
X_train = np.lib.format.open_memmap(
    os.path.join(CACHE_DIR, "X_train.npy"),
    mode="w+",
    dtype=np.float16,
    shape=(len(train_samples), 128, 251)
)

y_train = np.zeros(len(train_samples), dtype=np.int64)

# Dev
X_dev = np.lib.format.open_memmap(
    os.path.join(CACHE_DIR, "X_dev.npy"),
    mode="w+",
    dtype=np.float16,
    shape=(len(dev_samples), 128, 251)
)

y_dev = np.zeros(len(dev_samples), dtype=np.int64)

print("Train cache shape:", X_train.shape)
print("Dev cache shape:", X_dev.shape)
```

```
Train cache shape: (25380, 128, 251)
Dev cache shape: (24844, 128, 251)
```

```
import os
import time

print("Creating training Log-Mel cache...")

start_time = time.time()

for i, item in enumerate(train_samples):

    audio_path = os.path.join(
        TRAIN_AUDIO,
        item["audio_id"] + ".flac"
    )

    X_train[i] = audio_to_logmel(audio_path)
    y_train[i] = item["label"]
```

```
    if (i + 1) % 500 == 0:
        elapsed = time.time() - start_time
        print(f"{i+1}/{len(train_samples)} processed |
{elapsed/60:.1f} min")

X_train.flush()
np.save(
    os.path.join(CACHE_DIR, "y_train.npy"),
    y_train
)

print("Training cache complete.")
```

```
Creating training Log-Mel cache...
500/25380 processed | 0.1 min
1000/25380 processed | 0.2 min
1500/25380 processed | 0.3 min
2000/25380 processed | 0.4 min
2500/25380 processed | 0.5 min
3000/25380 processed | 0.5 min
3500/25380 processed | 0.7 min
4000/25380 processed | 0.7 min
4500/25380 processed | 0.8 min
5000/25380 processed | 0.9 min
5500/25380 processed | 1.0 min
6000/25380 processed | 1.1 min
6500/25380 processed | 1.2 min
7000/25380 processed | 1.3 min
7500/25380 processed | 1.4 min
8000/25380 processed | 1.5 min
8500/25380 processed | 1.6 min
9000/25380 processed | 1.7 min
9500/25380 processed | 1.8 min
10000/25380 processed | 1.9 min
10500/25380 processed | 2.0 min
11000/25380 processed | 2.1 min
11500/25380 processed | 2.2 min
12000/25380 processed | 2.3 min
12500/25380 processed | 2.4 min
13000/25380 processed | 2.5 min
13500/25380 processed | 2.6 min
14000/25380 processed | 2.7 min
14500/25380 processed | 2.8 min
15000/25380 processed | 2.9 min
15500/25380 processed | 3.0 min
16000/25380 processed | 3.1 min
16500/25380 processed | 3.2 min
17000/25380 processed | 3.3 min
17500/25380 processed | 3.4 min
```

```
18000/25380 processed | 3.5 min
18500/25380 processed | 3.6 min
19000/25380 processed | 3.7 min
19500/25380 processed | 3.8 min
20000/25380 processed | 3.9 min
20500/25380 processed | 4.0 min
21000/25380 processed | 4.1 min
21500/25380 processed | 4.2 min
22000/25380 processed | 4.3 min
22500/25380 processed | 4.4 min
23000/25380 processed | 4.5 min
23500/25380 processed | 4.6 min
24000/25380 processed | 4.7 min
24500/25380 processed | 4.8 min
25000/25380 processed | 4.9 min
Training cache complete.
```

```
import os
import time

print("Creating development Log-Mel cache...")

start_time = time.time()

for i, item in enumerate(dev_samples):

    audio_path = os.path.join(
        DEV_AUDIO,
        item["audio_id"] + ".flac"
    )

    X_dev[i] = audio_to_logmel(audio_path)
    y_dev[i] = item["label"]

    if (i + 1) % 500 == 0:
        elapsed = time.time() - start_time
        print(f"{i+1}/{len(dev_samples)} processed | {elapsed/60:.1f}
min")

X_dev.flush()

np.save(
    os.path.join(CACHE_DIR, "y_dev.npy"),
    y_dev
)

print("Development cache complete.")
```

```
Creating development Log-Mel cache...
500/24844 processed | 0.1 min
1000/24844 processed | 0.2 min
```

```
1500/24844 processed | 0.3 min
2000/24844 processed | 0.4 min
2500/24844 processed | 0.5 min
3000/24844 processed | 0.6 min
3500/24844 processed | 0.6 min
4000/24844 processed | 0.7 min
4500/24844 processed | 0.8 min
5000/24844 processed | 0.9 min
5500/24844 processed | 1.0 min
6000/24844 processed | 1.1 min
6500/24844 processed | 1.2 min
7000/24844 processed | 1.3 min
7500/24844 processed | 1.4 min
8000/24844 processed | 1.5 min
8500/24844 processed | 1.6 min
9000/24844 processed | 1.7 min
9500/24844 processed | 1.7 min
10000/24844 processed | 1.9 min
10500/24844 processed | 1.9 min
11000/24844 processed | 2.0 min
11500/24844 processed | 2.1 min
12000/24844 processed | 2.2 min
12500/24844 processed | 2.3 min
13000/24844 processed | 2.4 min
13500/24844 processed | 2.5 min
14000/24844 processed | 2.6 min
14500/24844 processed | 2.7 min
15000/24844 processed | 2.8 min
15500/24844 processed | 2.9 min
16000/24844 processed | 3.0 min
16500/24844 processed | 3.1 min
17000/24844 processed | 3.1 min
17500/24844 processed | 3.2 min
18000/24844 processed | 3.3 min
18500/24844 processed | 3.4 min
19000/24844 processed | 3.5 min
19500/24844 processed | 3.6 min
20000/24844 processed | 3.7 min
20500/24844 processed | 3.8 min
21000/24844 processed | 3.9 min
21500/24844 processed | 4.0 min
22000/24844 processed | 4.1 min
22500/24844 processed | 4.2 min
23000/24844 processed | 4.3 min
23500/24844 processed | 4.4 min
24000/24844 processed | 4.5 min
24500/24844 processed | 4.6 min
Development cache complete.
```

```python
import os
import numpy as np

train_X_path = os.path.join(CACHE_DIR, "X_train.npy")
train_y_path = os.path.join(CACHE_DIR, "y_train.npy")

dev_X_path = os.path.join(CACHE_DIR, "X_dev.npy")
dev_y_path = os.path.join(CACHE_DIR, "y_dev.npy")

X_train_check = np.load(train_X_path, mmap_mode="r")
y_train_check = np.load(train_y_path)

X_dev_check = np.load(dev_X_path, mmap_mode="r")
y_dev_check = np.load(dev_y_path)

print("TRAIN")
print("X:", X_train_check.shape, X_train_check.dtype)
print("y:", y_train_check.shape, y_train_check.dtype)

print("\nDEV")
print("X:", X_dev_check.shape, X_dev_check.dtype)
print("y:", y_dev_check.shape, y_dev_check.dtype)

print("\nSample feature range:")
print("Min:", X_train_check[0].min())
print("Max:", X_train_check[0].max())

print("\nLabels:")
print("Train:", np.bincount(y_train_check))
print("Dev  :", np.bincount(y_dev_check))
```

```
TRAIN
X: (25380, 128, 251) float16
y: (25380,) int64

DEV
X: (24844, 128, 251) float16
y: (24844,) int64

Sample feature range:
Min: -80.0
Max: 0.0

Labels:
Train: [ 2580 22800]
Dev  : [ 2548 22296]
```

```python
import torch
from torch.utils.data import Dataset
import numpy as np
import os
```

```
class CachedASVspoofDataset(Dataset):

    def __init__(self, feature_path, label_path):
        self.X = np.load(feature_path, mmap_mode="r")
        self.y = np.load(label_path)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, index):
        x = torch.tensor(
            self.X[index],
            dtype=torch.float32
        ).unsqueeze(0)

        y = torch.tensor(
            self.y[index],
            dtype=torch.long
        )

        return x, y

train_cached = CachedASVspoofDataset(
    os.path.join(CACHE_DIR, "X_train.npy"),
    os.path.join(CACHE_DIR, "y_train.npy")
)

dev_cached = CachedASVspoofDataset(
    os.path.join(CACHE_DIR, "X_dev.npy"),
    os.path.join(CACHE_DIR, "y_dev.npy")
)

print("Train:", len(train_cached))
print("Dev:", len(dev_cached))
```

```
Train: 25380
Dev: 24844
```

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
Device: cuda
GPU: Tesla T4
```

```python
import torch
import numpy as np

# Your cached training labels
y_train = np.load(
    os.path.join(CACHE_DIR, "y_train.npy")
)

# Count each class
class_counts = np.bincount(y_train)

bonafide_count = class_counts[0]
spoof_count = class_counts[1]
total = len(y_train)
num_classes = 2

# Balanced class weights
weight_bonafide = total / (num_classes * bonafide_count)
weight_spoof = total / (num_classes * spoof_count)

class_weights = torch.tensor(
    [weight_bonafide, weight_spoof],
    dtype=torch.float32
).to(device)

print("Bonafide:", bonafide_count)
print("Spoof:", spoof_count)

print("\nClass weights:")
print("Bonafide:", weight_bonafide)
print("Spoof:", weight_spoof)

print("\nTensor:", class_weights)
```

```
Bonafide: 2580
Spoof: 22800

Class weights:
Bonafide: 4.9186046511627906
Spoof: 0.5565789473684211

Tensor: tensor([4.9186, 0.5566], device='cuda:0')
```

```python
import torch
import numpy as np
from torch.utils.data import Dataset, DataLoader

class CachedASVspoofDataset(Dataset):

    def __init__(self, feature_path, label_path):
        self.X = np.load(feature_path, mmap_mode="r")
```

```
        self.y = np.load(label_path)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, index):
        # Log-Mel: [128, 251]
        x = torch.tensor(
            self.X[index],
            dtype=torch.float32
        )

        # Add channel dimension
        # [128, 251] → [1, 128, 251]
        x = x.unsqueeze(0)

        y = torch.tensor(
            self.y[index],
            dtype=torch.long
        )

        return x, y

train_cached = CachedASVspoofDataset(
    os.path.join(CACHE_DIR, "X_train.npy"),
    os.path.join(CACHE_DIR, "y_train.npy")
)

dev_cached = CachedASVspoofDataset(
    os.path.join(CACHE_DIR, "X_dev.npy"),
    os.path.join(CACHE_DIR, "y_dev.npy")
)

print("Train:", len(train_cached))
print("Dev:", len(dev_cached))
```

```
Train: 25380
Dev: 24844
```

```
BATCH_SIZE = 32

train_loader = DataLoader(
    train_cached,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_cached,
```

```
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

print("DataLoaders created.")
```

```
DataLoaders created.
```

```
x, y = next(iter(train_loader))

print("Input shape :", x.shape)
print("Label shape :", y.shape)
print("Bonafide in batch:", (y == 0).sum().item())
print("Spoof in batch:", (y == 1).sum().item())
```

```
Input shape : torch.Size([32, 1, 128, 251])
Label shape : torch.Size([32])
Bonafide in batch: 1
Spoof in batch: 31
```

```
import torch
import torch.nn as nn


class VoiceCNN(nn.Module):

    def __init__(self, dropout=0.3):
        super().__init__()

        self.features = nn.Sequential(

            # Block 1
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
```

```
        # Global Average Pooling
        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        # Classifier
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(128, 1)
        )

    def forward(self, x):

        x = self.features(x)
        x = self.gap(x)
        x = self.classifier(x)

        return x


model = VoiceCNN(dropout=0.3).to(device)

print(model)
```

```
VoiceCNN(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
  )
```

```
  (gap): AdaptiveAvgPool2d(output_size=(1, 1))
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=128, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=128, out_features=1, bias=True)
  )
)

model.eval()

with torch.no_grad():
    test_output = model(x.to(device))

print("Input shape :", x.shape)
print("Output shape:", test_output.shape)
print("Output:", test_output[:5].squeeze())

Input shape : torch.Size([32, 1, 128, 251])
Output shape: torch.Size([32, 1])
Output: tensor([-1.4170, -1.3959, -1.3156, -1.3426, -1.3662],
device='cuda:0')

# Class 0 = bonafide
# Class 1 = spoof

bonafide_count = 2580
spoof_count = 22800

pos_weight = torch.tensor(
    [bonafide_count / spoof_count],
    dtype=torch.float32
).to(device)

print("pos_weight:", pos_weight)

pos_weight: tensor([0.1132], device='cuda:0')


import torch.nn as nn

criterion_base = nn.BCEWithLogitsLoss(reduction="none")

class_weights = torch.tensor(
    [4.9186, 0.5566],
    dtype=torch.float32
).to(device)

print("Bonafide weight:", class_weights[0].item())
print("Spoof weight:", class_weights[1].item())
```

```
Bonafide weight: 4.918600082397461
Spoof weight: 0.5565999746322632
```

```
import torch.nn as nn

criterion_base = nn.BCEWithLogitsLoss(reduction="none")

class_weights = torch.tensor(
    [4.9186, 0.5566],
    dtype=torch.float32
).to(device)

print("Weighted BCE loss ready.")
```

```
Weighted BCE loss ready.
```

```
import torch.optim as optim

LEARNING_RATE = 0.001
WEIGHT_DECAY = 0.0001

optimizer = optim.Adam(
    model.parameters(),
    lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY
)

print("Learning rate:", LEARNING_RATE)
print("Weight decay:", WEIGHT_DECAY)
```

```
Learning rate: 0.001
Weight decay: 0.0001
```

```
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=2
)

print("Scheduler ready.")
```

```
Scheduler ready.
```

```
def train_one_epoch(model, loader, optimizer, criterion_base,
class_weights, device):
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0
```

```
    for inputs, labels in loader:

        inputs = inputs.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        # Forward pass
        outputs = model(inputs).squeeze(1)

        # BCE loss for each individual sample
        losses = criterion_base(
            outputs,
            labels.float()
        )

        # Apply class weight
        weights = class_weights[labels]

        # Weighted average loss
        loss = (losses * weights).mean()

        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Statistics
        running_loss += loss.item() * inputs.size(0)

        predictions = (torch.sigmoid(outputs) >= 0.5).long()

        correct += (predictions == labels).sum().item()
        total += labels.size(0)

    epoch_loss = running_loss / total
    epoch_accuracy = correct / total

    return epoch_loss, epoch_accuracy

def validate(model, loader, criterion_base, class_weights, device):
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    all_predictions = []
    all_labels = []
    all_probabilities = []

    with torch.no_grad():
```

```
        for inputs, labels in loader:

            inputs = inputs.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            outputs = model(inputs).squeeze(1)

            # Individual BCE losses
            losses = criterion_base(
                outputs,
                labels.float()
            )

            # Apply class weights
            weights = class_weights[labels]

            loss = (losses * weights).mean()

            running_loss += loss.item() * inputs.size(0)

            # Convert logits → probabilities
            probabilities = torch.sigmoid(outputs)

            predictions = (probabilities >= 0.5).long()

            correct += (predictions == labels).sum().item()
            total += labels.size(0)

            all_predictions.extend(predictions.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
            all_probabilities.extend(probabilities.cpu().numpy())

    epoch_loss = running_loss / total
    epoch_accuracy = correct / total

    return (
        epoch_loss,
        epoch_accuracy,
        all_predictions,
        all_labels,
        all_probabilities
    )

model.train()

inputs, labels = next(iter(train_loader))

inputs = inputs.to(device)
labels = labels.to(device)

outputs = model(inputs).squeeze(1)
```

```
losses = criterion_base(
    outputs,
    labels.float()
)

weights = class_weights[labels]

loss = (losses * weights).mean()

optimizer.zero_grad()
loss.backward()
optimizer.step()

print("Input shape :", inputs.shape)
print("Output shape:", outputs.shape)
print("Loss:", loss.item())
print("Batch processed successfully.")
```

```
Input shape : torch.Size([32, 1, 128, 251])
Output shape: torch.Size([32])
Loss: 0.8429156541824341
Batch processed successfully.
```

```
import time
import torch

MAX_EPOCHS = 50
PATIENCE = 7

best_val_loss = float("inf")
patience_counter = 0

train_history = []
val_history = []

for epoch in range(MAX_EPOCHS):

    start_time = time.time()

    # -------------------------
    # TRAIN
    # -------------------------
    train_loss, train_acc = train_one_epoch(
        model,
        train_loader,
        optimizer,
        criterion_base,
        class_weights,
        device
    )
```

```
    # -------------------------
    # VALIDATION
    # -------------------------
    val_loss, val_acc, val_predictions, val_labels, val_probabilities
= validate(
        model,
        dev_loader,
        criterion_base,
        class_weights,
        device
    )

    # -------------------------
    # LEARNING RATE SCHEDULER
    # -------------------------
    scheduler.step(val_loss)

    current_lr = optimizer.param_groups[0]["lr"]

    epoch_time = time.time() - start_time

    # Save history
    train_history.append((train_loss, train_acc))
    val_history.append((val_loss, val_acc))

    print(
        f"Epoch [{epoch+1:02d}/{MAX_EPOCHS}] | "
        f"Train Loss: {train_loss:.4f} | "
        f"Train Acc: {train_acc:.4f} | "
        f"Val Loss: {val_loss:.4f} | "
        f"Val Acc: {val_acc:.4f} | "
        f"LR: {current_lr:.6f} | "
        f"Time: {epoch_time:.1f}s"
    )

    # -------------------------
    # SAVE BEST MODEL
    # -------------------------
    if val_loss < best_val_loss:

        best_val_loss = val_loss
        patience_counter = 0

        torch.save(
            model.state_dict(),
            "/content/best_voice_cnn.pth"
        )

        print("  ✓ Best model saved")
```

```
    else:

        patience_counter += 1

        print(
            f"  No improvement "
            f"({patience_counter}/{PATIENCE})"
        )

    # -------------------------
    # EARLY STOPPING
    # -------------------------
    if patience_counter >= PATIENCE:

        print("\nEarly stopping triggered.")
        break

print("\nTraining complete.")
print("Best validation loss:", best_val_loss)
```

```
Epoch [01/50] | Train Loss: 0.2810 | Train Acc: 0.8606 | Val Loss:
6.7471 | Val Acc: 0.8976 | LR: 0.001000 | Time: 43.2s
  ✓ Best model saved
Epoch [02/50] | Train Loss: 0.1313 | Train Acc: 0.9491 | Val Loss:
0.0828 | Val Acc: 0.9925 | LR: 0.001000 | Time: 43.7s
  ✓ Best model saved
Epoch [03/50] | Train Loss: 0.0778 | Train Acc: 0.9701 | Val Loss:
0.2131 | Val Acc: 0.9843 | LR: 0.001000 | Time: 43.9s
  No improvement (1/7)
Epoch [04/50] | Train Loss: 0.0611 | Train Acc: 0.9754 | Val Loss:
0.0529 | Val Acc: 0.9664 | LR: 0.001000 | Time: 43.6s
  ✓ Best model saved
Epoch [05/50] | Train Loss: 0.0485 | Train Acc: 0.9804 | Val Loss:
3.7843 | Val Acc: 0.9029 | LR: 0.001000 | Time: 43.4s
  No improvement (1/7)
Epoch [06/50] | Train Loss: 0.0476 | Train Acc: 0.9803 | Val Loss:
4.8406 | Val Acc: 0.8988 | LR: 0.001000 | Time: 43.4s
  No improvement (2/7)
Epoch [07/50] | Train Loss: 0.0442 | Train Acc: 0.9835 | Val Loss:
0.1004 | Val Acc: 0.9912 | LR: 0.000500 | Time: 43.7s
  No improvement (3/7)
Epoch [08/50] | Train Loss: 0.0215 | Train Acc: 0.9914 | Val Loss:
0.0149 | Val Acc: 0.9989 | LR: 0.000500 | Time: 43.5s
  ✓ Best model saved
Epoch [09/50] | Train Loss: 0.0176 | Train Acc: 0.9931 | Val Loss:
0.2603 | Val Acc: 0.9814 | LR: 0.000500 | Time: 43.8s
  No improvement (1/7)
Epoch [10/50] | Train Loss: 0.0188 | Train Acc: 0.9928 | Val Loss:
0.0330 | Val Acc: 0.9967 | LR: 0.000500 | Time: 43.7s
```

```
  No improvement (2/7)
Epoch [11/50] | Train Loss: 0.0213 | Train Acc: 0.9926 | Val Loss:
0.0065 | Val Acc: 0.9980 | LR: 0.000500 | Time: 43.7s
  ✓ Best model saved
Epoch [12/50] | Train Loss: 0.0161 | Train Acc: 0.9941 | Val Loss:
0.2540 | Val Acc: 0.9812 | LR: 0.000500 | Time: 43.8s
  No improvement (1/7)
Epoch [13/50] | Train Loss: 0.0179 | Train Acc: 0.9928 | Val Loss:
0.0091 | Val Acc: 0.9954 | LR: 0.000500 | Time: 43.7s
  No improvement (2/7)
Epoch [14/50] | Train Loss: 0.0163 | Train Acc: 0.9941 | Val Loss:
0.6132 | Val Acc: 0.7275 | LR: 0.000250 | Time: 43.6s
  No improvement (3/7)
Epoch [15/50] | Train Loss: 0.0067 | Train Acc: 0.9974 | Val Loss:
0.0062 | Val Acc: 0.9992 | LR: 0.000250 | Time: 43.4s
  ✓ Best model saved
Epoch [16/50] | Train Loss: 0.0108 | Train Acc: 0.9962 | Val Loss:
0.0315 | Val Acc: 0.9789 | LR: 0.000250 | Time: 43.6s
  No improvement (1/7)
Epoch [17/50] | Train Loss: 0.0052 | Train Acc: 0.9981 | Val Loss:
0.0052 | Val Acc: 0.9993 | LR: 0.000250 | Time: 43.4s
  ✓ Best model saved
Epoch [18/50] | Train Loss: 0.0127 | Train Acc: 0.9959 | Val Loss:
0.3848 | Val Acc: 0.9784 | LR: 0.000250 | Time: 43.8s
  No improvement (1/7)
Epoch [19/50] | Train Loss: 0.0078 | Train Acc: 0.9972 | Val Loss:
0.0041 | Val Acc: 0.9984 | LR: 0.000250 | Time: 43.7s
  ✓ Best model saved
Epoch [20/50] | Train Loss: 0.0068 | Train Acc: 0.9975 | Val Loss:
0.2040 | Val Acc: 0.9859 | LR: 0.000250 | Time: 43.7s
  No improvement (1/7)
Epoch [21/50] | Train Loss: 0.0083 | Train Acc: 0.9968 | Val Loss:
0.0040 | Val Acc: 0.9993 | LR: 0.000250 | Time: 43.7s
  ✓ Best model saved
Epoch [22/50] | Train Loss: 0.0087 | Train Acc: 0.9969 | Val Loss:
0.0049 | Val Acc: 0.9994 | LR: 0.000250 | Time: 43.7s
  No improvement (1/7)
Epoch [23/50] | Train Loss: 0.0044 | Train Acc: 0.9985 | Val Loss:
0.9421 | Val Acc: 0.6770 | LR: 0.000250 | Time: 43.7s
  No improvement (2/7)
Epoch [24/50] | Train Loss: 0.0095 | Train Acc: 0.9962 | Val Loss:
0.0034 | Val Acc: 0.9991 | LR: 0.000250 | Time: 43.5s
  ✓ Best model saved
Epoch [25/50] | Train Loss: 0.0053 | Train Acc: 0.9979 | Val Loss:
0.0116 | Val Acc: 0.9983 | LR: 0.000250 | Time: 43.8s
  No improvement (1/7)
Epoch [26/50] | Train Loss: 0.0059 | Train Acc: 0.9978 | Val Loss:
0.0094 | Val Acc: 0.9994 | LR: 0.000250 | Time: 43.5s
  No improvement (2/7)
```

```
Epoch [27/50] | Train Loss: 0.0069 | Train Acc: 0.9974 | Val Loss:
0.0226 | Val Acc: 0.9982 | LR: 0.000125 | Time: 43.6s
  No improvement (3/7)
Epoch [28/50] | Train Loss: 0.0040 | Train Acc: 0.9985 | Val Loss:
0.0079 | Val Acc: 0.9995 | LR: 0.000125 | Time: 43.5s
  No improvement (4/7)
Epoch [29/50] | Train Loss: 0.0023 | Train Acc: 0.9993 | Val Loss:
0.0069 | Val Acc: 0.9995 | LR: 0.000125 | Time: 43.5s
  No improvement (5/7)
Epoch [30/50] | Train Loss: 0.0020 | Train Acc: 0.9993 | Val Loss:
0.0163 | Val Acc: 0.9987 | LR: 0.000063 | Time: 43.6s
  No improvement (6/7)
Epoch [31/50] | Train Loss: 0.0013 | Train Acc: 0.9997 | Val Loss:
0.0036 | Val Acc: 0.9996 | LR: 0.000063 | Time: 43.8s
  No improvement (7/7)

Early stopping triggered.

Training complete.
Best validation loss: 0.0034144272088965483
```

```
model.load_state_dict(
    torch.load(
        "/content/best_voice_cnn.pth",
        map_location=device
    )
)

model.eval()

print("Best model loaded successfully.")
```

```
Best model loaded successfully.
```

```
# Load best model
model.load_state_dict(
    torch.load("/content/best_voice_cnn.pth", map_location=device)
)

model.to(device)
model.eval()

print("Best model loaded successfully.")
```

```
Best model loaded successfully.
```

```
import numpy as np
import torch
from sklearn.metrics import (
    accuracy_score,
    precision_score,
```

```
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)

model.eval()

all_labels = []
all_probs = []

with torch.no_grad():
    for inputs, labels in dev_loader:
        inputs = inputs.to(device)

        outputs = model(inputs).squeeze(1)
        probs = torch.sigmoid(outputs)

        all_labels.extend(labels.numpy())
        all_probs.extend(probs.cpu().numpy())

all_labels = np.array(all_labels)
all_probs = np.array(all_probs)

# Threshold = 0.5
all_preds = (all_probs >= 0.5).astype(int)

accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
auc = roc_auc_score(all_labels, all_probs)

cm = confusion_matrix(all_labels, all_preds)

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
print("ROC-AUC  :", auc)

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        all_labels,
```

```
        all_preds,
        target_names=["Bonafide", "Spoof"]
    )
)
```

```
Accuracy : 0.9990742231524714
Precision: 0.99977561369654
Recall   : 0.9991926803013994
F1 Score : 0.9994840620022881
ROC-AUC  : 0.9999957225997131

Confusion Matrix:
[[ 2543     5]
 [   18 22278]]

Classification Report:
              precision    recall  f1-score   support

    Bonafide       0.99      1.00      1.00      2548
       Spoof       1.00      1.00      1.00     22296

    accuracy                           1.00     24844
   macro avg       1.00      1.00      1.00     24844
weighted avg       1.00      1.00      1.00     24844
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os

BASE = "/content/drive/MyDrive/voice_cloning_project"

folders = [
    "datasets",
    "features",
    "models",
    "code",
    "results"
]

for folder in folders:
    os.makedirs(os.path.join(BASE, folder), exist_ok=True)

print("Project folder created:")
print(BASE)
```

```
Project folder created:
/content/drive/MyDrive/voice_cloning_project
```

```
import shutil

src = "/content/best_voice_cnn.pth"
dst =
"/content/drive/MyDrive/voice_cloning_project/models/best_voice_cnn.pt
h"

shutil.copy2(src, dst)

print("Model saved to:", dst)
```

```
Model saved to:
/content/drive/MyDrive/voice_cloning_project/models/best_voice_cnn.pth
```

```
import os

print(os.path.exists(dst))
```

```
True
```

```
import shutil
import os

src_dir = "/content/asvspoof_cache"
dst_dir = "/content/drive/MyDrive/voice_cloning_project/features"

for filename in os.listdir(src_dir):
    src = os.path.join(src_dir, filename)
    dst = os.path.join(dst_dir, filename)

    if os.path.isfile(src):
        shutil.copy2(src, dst)

print("Cached features copied to Drive.")
```

```
Cached features copied to Drive.
```

```
!ls -lh "/content/drive/MyDrive/voice_cloning_project/features"
```

```
total 3.1G
-rw------- 1 root root 1.5G Sep  2 19:45 X_dev.npy
-rw------- 1 root root 1.6G Sep  2 19:40 X_train.npy
-rw------- 1 root root 195K Sep  2 19:45 y_dev.npy
-rw------- 1 root root 199K Sep  2 19:40 y_train.npy
```

```
config = """
VOICE CLONING DETECTION — BASELINE CONFIGURATION
```

```
Dataset:
ASVspoof 2019 LA

Labels:
0 = Bonafide
1 = Spoof

Audio:
Sample rate = 16000 Hz
Channels = Mono
Duration = 4 seconds
Samples = 64000

Log-Mel:
N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256
Input = 128 x 251

CNN:
Conv1: 1 -> 32
Conv2: 32 -> 64
Conv3: 64 -> 128
Kernel = 3x3
BatchNorm = Yes
ReLU = Yes
MaxPool = 2x2
Global Average Pooling
Dense = 128
Dropout = 0.3
Output = 1 logit

Loss:
Weighted BCEWithLogitsLoss

Class weights:
Bonafide = 4.9186
Spoof = 0.5566

Optimizer:
Adam

Learning rate:
0.001

Weight decay:
0.0001

Batch size:
```

```
32

Maximum epochs:
50

Early stopping:
7 epochs

Scheduler:
ReduceLROnPlateau
Factor = 0.5
Patience = 2

Best checkpoint:
Epoch 8

Best validation accuracy:
99.89%

Best validation loss:
0.0149
"""

with open(
    "/content/drive/MyDrive/voice_cloning_project/config.txt", "w"
) as f:
    f.write(config)

print("Configuration saved.")
```

```
Configuration saved.
```

```
results = """
ASVSPOOF 2019 EVALUATION

Accuracy  : 0.9990742231524714
Precision : 0.99977561369654
Recall    : 0.9991926803013994
F1 Score  : 0.9994840620022881
ROC-AUC   : 0.9999957225997131

Confusion Matrix:
[[2543, 5],
 [18, 22278]]

Bonafide:
Precision = 0.99
Recall    = 1.00
F1        = 1.00
Support   = 2548
```

```
Spoof:
Precision = 1.00
Recall    = 1.00
F1        = 1.00
Support   = 22296
"""

with open(

"/content/drive/MyDrive/voice_cloning_project/results/asvspoof2019_res
ults.txt",
    "w"
) as f:
    f.write(results)

print("Results saved.")
```

```
Results saved.
```

```
!ls -lh "/content/drive/MyDrive/voice_cloning_project/models/"
```

```
total 440K
-rw------- 1 root root 440K Sep  2 20:13 best_voice_cnn.pth
```

```
import numpy as np
import os

features_dir = "/content/drive/MyDrive/voice_cloning_project/features"

for filename in ["X_train.npy", "y_train.npy", "X_dev.npy",
"y_dev.npy"]:
    path = os.path.join(features_dir, filename)

    print(filename)
    print("Exists:", os.path.exists(path))

    if os.path.exists(path):
        data = np.load(path, mmap_mode="r")
        print("Shape:", data.shape)
        print("Dtype:", data.dtype)
        print()
```

```
X_train.npy
Exists: True
Shape: (25380, 128, 251)
Dtype: float16

y_train.npy
Exists: True
Shape: (25380,)
Dtype: int64
```

```
X_dev.npy
Exists: True
Shape: (24844, 128, 251)
Dtype: float16

y_dev.npy
Exists: True
Shape: (24844,)
Dtype: int64
```

```
from google.colab import drive

drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
BASE = "/content/drive/MyDrive/voice_cloning_project"

MODEL_PATH = BASE + "/models/best_voice_cnn.pth"
FEATURE_DIR = BASE + "/features"

print(MODEL_PATH)
```

```
/content/drive/MyDrive/voice_cloning_project/models/best_voice_cnn.pth
```

```
import os

print("Model exists:", os.path.exists(MODEL_PATH))
```

```
Model exists: True
```

```
import numpy as np

X_train = np.load(FEATURE_DIR + "/X_train.npy")
y_train = np.load(FEATURE_DIR + "/y_train.npy")

X_dev = np.load(FEATURE_DIR + "/X_dev.npy")
y_dev = np.load(FEATURE_DIR + "/y_dev.npy")

print("X_train:", X_train.shape, X_train.dtype)
print("y_train:", y_train.shape, y_train.dtype)

print("X_dev:", X_dev.shape, X_dev.dtype)
print("y_dev:", y_dev.shape, y_dev.dtype)
```

```
X_train: (25380, 128, 251) float16
y_train: (25380,) int64
X_dev: (24844, 128, 251) float16
y_dev: (24844,) int64
```

```python
import torch
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class VoiceCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.gap(x)
        x = self.classifier(x)
        return x


model = VoiceCNN().to(device)

checkpoint = torch.load(
    MODEL_PATH,
    map_location=device
)

model.load_state_dict(checkpoint)
```

```
model.eval()

print("Model loaded successfully!")
print("Device:", device)
```

```
Model loaded successfully!
Device: cuda
```

```
import shutil
import os

# Google Colab local disk
total, used, free = shutil.disk_usage("/content")

print("=== COLAB /content ===")
print("Total :", round(total / (1024**3), 2), "GB")
print("Used  :", round(used / (1024**3), 2), "GB")
print("Free  :", round(free / (1024**3), 2), "GB")

# Drive
drive_path = "/content/drive/MyDrive"

if os.path.exists(drive_path):
    total, used, free = shutil.disk_usage(drive_path)

    print("\n=== GOOGLE DRIVE ===")
    print("Total :", round(total / (1024**3), 2), "GB")
    print("Used  :", round(used / (1024**3), 2), "GB")
    print("Free  :", round(free / (1024**3), 2), "GB")
```

```
=== COLAB /content ===
Total : 112.64 GB
Used  : 50.29 GB
Free  : 62.33 GB

=== GOOGLE DRIVE ===
Total : 15.0 GB
Used  : 13.39 GB
Free  : 1.61 GB
```

```
import requests

zenodo_api = "https://zenodo.org/api/records/4835108"

r = requests.get(zenodo_api)
r.raise_for_status()

data = r.json()

print("Dataset:", data["metadata"]["title"])
```

```
print("\nFiles:")
for f in data["files"]:
    print(f["key"], "->", round(f["size"] / (1024**3), 2), "GB")
```

```
Dataset: ASVspoof 2021 Challenge - Speech Deepfake Database

Files:
ASVspoof2021_DF_eval_part00.tar.gz -> 8.04 GB
ASVspoof2021_DF_eval_part01.tar.gz -> 8.03 GB
ASVspoof2021_DF_eval_part03.tar.gz -> 8.06 GB
ASVspoof2021_DF_eval_part02.tar.gz -> 8.02 GB
```

```
import requests
import os

files = [
    "ASVspoof2021_DF_eval_part00.tar.gz",
    "ASVspoof2021_DF_eval_part01.tar.gz",
    "ASVspoof2021_DF_eval_part02.tar.gz",
    "ASVspoof2021_DF_eval_part03.tar.gz"
]

base_url = "https://zenodo.org/records/4835108/files/"

download_dir = "/content/asvspoof2021_df"
os.makedirs(download_dir, exist_ok=True)

for filename in files:
    path = os.path.join(download_dir, filename)

    if os.path.exists(path):
        print(f"Already exists: {filename}")
        continue

    url = base_url + filename

    print(f"\nDownloading: {filename}")

    with requests.get(url, stream=True) as r:
        r.raise_for_status()

        total = int(r.headers.get("content-length", 0))
        downloaded = 0

        with open(path, "wb") as f:
            for chunk in r.iter_content(chunk_size=1024 * 1024):
                if chunk:
                    f.write(chunk)
                    downloaded += len(chunk)

                    if total:
```

```
                        percent = downloaded / total * 100
                        print(
                            f"\rProgress: {percent:.1f}%",
                            end=""
                        )

    print("\nCompleted:", filename)


Downloading: ASVspoof2021_DF_eval_part00.tar.gz
Progress: 6.5%

---------------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_2577/1059294165.py in <cell line: 0>()
     32
     33         with open(path, "wb") as f:
---> 34             for chunk in r.iter_content(chunk_size=1024 *
1024):
     35                 if chunk:
     36                     f.write(chunk)

/usr/local/lib/python3.13/dist-packages/requests/models.py in
generate()
    818             if hasattr(self.raw, "stream"):
    819                 try:
--> 820                     yield from self.raw.stream(chunk_size,
decode_content=True)
    821                 except ProtocolError as e:
    822                     raise ChunkedEncodingError(e)

/usr/local/lib/python3.13/dist-packages/urllib3/response.py in
stream(self, amt, decode_content)
   1089         else:
   1090             while not is_fp_closed(self._fp) or
len(self._decoded_buffer) > 0:
-> 1091                 data = self.read(amt=amt,
decode_content=decode_content)
   1092
   1093                 if data:

/usr/local/lib/python3.13/dist-packages/urllib3/response.py in
read(self, amt, decode_content, cache_content)
    978                 return self._decoded_buffer.get(amt)
    979
--> 980         data = self._raw_read(amt)
    981
    982         flush_decoder = amt is None or (amt != 0 and not data)
```

```
/usr/local/lib/python3.13/dist-packages/urllib3/response.py in
_raw_read(self, amt, read1)
    902
    903         with self._error_catcher():
--> 904             data = self._fp_read(amt, read1=read1) if not
fp_closed else b""
    905             if amt is not None and amt != 0 and not data:
    906                 # Platform-specific: Buggy versions of Python.

/usr/local/lib/python3.13/dist-packages/urllib3/response.py in
_fp_read(self, amt, read1)
    885         else:
    886             # StringIO doesn't like amt=None
--> 887             return self._fp.read(amt) if amt is not None else
self._fp.read()
    888
    889     def _raw_read(

/usr/lib/python3.13/http/client.py in read(self, amt)
    492                 # clip the read to the "end of response"
    493                 amt = self.length
--> 494             s = self.fp.read(amt)
    495             if not s and amt:
    496                 # Ideally, we would raise IncompleteRead if
the content-length

/usr/lib/python3.13/socket.py in readinto(self, b)
    721             raise OSError("cannot read from timed out object")
    722         try:
--> 723             return self._sock.recv_into(b)
    724         except timeout:
    725             self._timeout_occurred = True

/usr/lib/python3.13/ssl.py in recv_into(self, buffer, nbytes, flags)
   1302                   "non-zero flags not allowed in calls to
recv_into() on %s" %
   1303                   self.__class__)
-> 1304             return self.read(nbytes, buffer)
   1305         else:
   1306             return super().recv_into(buffer, nbytes, flags)

/usr/lib/python3.13/ssl.py in read(self, len, buffer)
   1136         try:
   1137             if buffer is not None:
-> 1138                 return self._sslobj.read(len, buffer)
   1139             else:
   1140                 return self._sslobj.read(len)

KeyboardInterrupt:
```

```
from datasets import load_dataset

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)
```

{"model_id":"1c7fd1e26dac42478c91837b55bd00a8","version_major":2,"version_minor":0}

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```

{"model_id":"9e669e5c14ca4cf4a0518a729691cf69","version_major":2,"version_minor":0}

{"model_id":"9c78fcb6576d442a893522375a5802c2","version_major":2,"version_minor":0}

```
from datasets import load_dataset

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(ds)
```

{"model_id":"376763d7a6d24440b203a35016abbf19","version_major":2,"version_minor":0}

{"model_id":"32aacd952e594e8b8f16ef3e0f0e5965","version_major":2,"version_minor":0}

```
IterableDataset({
    features: ['path', 'audio', 'label', 'notes'],
    num_shards: 80
})

sample = next(iter(ds))

print("Available fields:")
print(sample.keys())

print("\nPath:")
print(sample["path"])
```

```
print("\nLabel:")
print(sample["label"])

print("\nNotes:")
print(sample["notes"])

print("\nAudio information:")
print("Sampling rate:", sample["audio"]["sampling_rate"])
print("Number of samples:", len(sample["audio"]["array"]))
print("Duration:",
      len(sample["audio"]["array"]) / sample["audio"]
["sampling_rate"],
      "seconds")
```

```
Available fields:
dict_keys(['path', 'audio', 'label', 'notes'])

Path:
DF_E_2000011.flac

Label:
1

Notes:
{"utterance_id": "DF_E_2000011", "speaker_id": "LA_0023", "subset":
"progress", "codec": "nocodec", "source": "asvspoof", "attack_id":
"A14", "vocoder": "traditional_vocoder"}

Audio information:
Sampling rate: 16000
Number of samples: 22801
Duration: 1.4250625 seconds
```

```
import numpy as np
import torch
import torchaudio

TARGET_SR = 16000
TARGET_LENGTH = 4 * TARGET_SR

N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=TARGET_SR,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    n_mels=N_MELS
)
```

```python
def preprocess_audio(audio, sr):
    # Convert to float32
    audio = np.asarray(audio, dtype=np.float32)

    # Resample if necessary
    if sr != TARGET_SR:
        waveform = torch.tensor(audio).unsqueeze(0)
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            TARGET_SR
        )
        audio = waveform.squeeze(0).numpy()

    # Convert to tensor
    waveform = torch.tensor(audio, dtype=torch.float32)

    # Pad or crop to exactly 4 seconds
    if len(waveform) < TARGET_LENGTH:
        waveform = torch.nn.functional.pad(
            waveform,
            (0, TARGET_LENGTH - len(waveform))
        )
    else:
        waveform = waveform[:TARGET_LENGTH]

    # Log-Mel spectrogram
    mel = mel_transform(waveform.unsqueeze(0))

    # Convert power Mel → log Mel
    log_mel = torch.log(mel + 1e-9)

    return log_mel

audio = sample["audio"]["array"]
sr = sample["audio"]["sampling_rate"]

feature = preprocess_audio(audio, sr)

print("Original samples:", len(audio))
print("Original duration:", len(audio) / sr)

print("\nFeature shape:", feature.shape)
print("Feature dtype:", feature.dtype)
```

```
Original samples: 22801
Original duration: 1.4250625

Feature shape: torch.Size([1, 128, 251])
Feature dtype: torch.float32
```

```
# Prepare feature for CNN
x = feature.unsqueeze(0).to(device)

# Run inference
model.eval()

with torch.no_grad():
    logit = model(x)
    probability = torch.sigmoid(logit).item()

prediction = 1 if probability >= 0.5 else 0

print("True label :", sample["label"])
print("Prediction :", prediction)
print("Spoof probability:", probability)
```

```
True label : 1
Prediction : 0
Spoof probability: 0.00010990716691594571
```

```
import numpy as np
import torch
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)
from sklearn.metrics import roc_curve
from tqdm.auto import tqdm

# Reload streaming dataset
ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

all_labels = []
all_probs = []

model.eval()

MAX_SAMPLES = 1000   # None = entire dataset

for i, sample in enumerate(tqdm(ds, desc="Evaluating ASVspoof 2021
DF")):

    audio = sample["audio"]["array"]
```

```
    sr = sample["audio"]["sampling_rate"]
    label = int(sample["label"])

    # Same preprocessing
    feature = preprocess_audio(audio, sr)

    # Add batch dimension
    x = feature.unsqueeze(0).to(device)

    # CNN inference
    with torch.no_grad():
        logit = model(x)
        prob = torch.sigmoid(logit).item()

    all_labels.append(label)
    all_probs.append(prob)

    if MAX_SAMPLES is not None and (i + 1) >= MAX_SAMPLES:
        break

all_labels = np.array(all_labels)
all_probs = np.array(all_probs)

all_preds = (all_probs >= 0.5).astype(int)

print("\nEvaluation finished!")
print("Samples evaluated:", len(all_labels))
```


```
{"model_id":"7e046fcc5c72455294218f34ece16a96","version_major":2,"vers
ion_minor":0}

{"model_id":"7eaabca8aa90475a8883118dbbff1154","version_major":2,"vers
ion_minor":0}

{"model_id":"ce11c8f14e6744748e40729a172d22c2","version_major":2,"vers
ion_minor":0}
```


```
Evaluation finished!
Samples evaluated: 1000
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

accuracy = accuracy_score(all_labels, all_preds)
```

```
precision = precision_score(all_labels, all_preds, zero_division=0)
recall = recall_score(all_labels, all_preds, zero_division=0)
f1 = f1_score(all_labels, all_preds, zero_division=0)
roc_auc = roc_auc_score(all_labels, all_probs)
cm = confusion_matrix(all_labels, all_preds)

print("ASVspoof 2021 DF — 1,000 Sample Evaluation")
print("-" * 50)
print(f"Accuracy : {accuracy:.6f}")
print(f"Precision: {precision:.6f}")
print(f"Recall   : {recall:.6f}")
print(f"F1 Score : {f1:.6f}")
print(f"ROC-AUC  : {roc_auc:.6f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClass distribution:")
print("Bonafide (0):", np.sum(all_labels == 0))
print("Spoof    (1):", np.sum(all_labels == 1))
```

```
ASVspoof 2021 DF — 1,000 Sample Evaluation
--------------------------------------------------
Accuracy : 0.038000
Precision: 0.000000
Recall   : 0.000000
F1 Score : 0.000000
ROC-AUC  : 0.565188

Confusion Matrix:
[[ 38   0]
 [962   0]]

Class distribution:
Bonafide (0): 38
Spoof    (1): 962
```

```
import numpy as np

X_train = np.load(

"/content/drive/MyDrive/voice_cloning_project/features/X_train.npy"
)

X_dev = np.load(
    "/content/drive/MyDrive/voice_cloning_project/features/X_dev.npy"
)

print("TRAIN")
print("shape:", X_train.shape)
```

```python
print("dtype:", X_train.dtype)
print("min:", X_train.min())
print("max:", X_train.max())
print("mean:", X_train.mean())
print("std:", X_train.std())

print("\nDEV")
print("shape:", X_dev.shape)
print("dtype:", X_dev.dtype)
print("min:", X_dev.min())
print("max:", X_dev.max())
print("mean:", X_dev.mean())
print("std:", X_dev.std())
```

```
TRAIN
shape: (25380, 128, 251)
dtype: float16
min: -80.0
max: 0.0
mean: -64.4

/usr/local/lib/python3.13/dist-packages/numpy/_core/_methods.py:173:
RuntimeWarning: overflow encountered in reduce
  arrmean = umr_sum(arr, axis, dtype, keepdims=True, where=where)

std: inf

DEV
shape: (24844, 128, 251)
dtype: float16
min: -80.0
max: 0.0
mean: -64.56
std: inf
```

```python
X_train_f32 = X_train.astype(np.float32)
X_dev_f32 = X_dev.astype(np.float32)

print("TRAIN")
print("min :", X_train_f32.min())
print("max :", X_train_f32.max())
print("mean:", X_train_f32.mean())
print("std :", X_train_f32.std())

print("\nDEV")
print("min :", X_dev_f32.min())
print("max :", X_dev_f32.max())
print("mean:", X_dev_f32.mean())
print("std :", X_dev_f32.std())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_10971/2445440430.py in <cell line: 0>()
----> 1 X_train_f32 = X_train.astype(np.float32)
      2 X_dev_f32 = X_dev.astype(np.float32)
      3
      4 print("TRAIN")
      5 print("min :", X_train_f32.min())

NameError: name 'X_train' is not defined
```

```
import numpy as np

TRAIN_PATH =
"/content/drive/MyDrive/voice_cloning_project/features/X_train.npy"
DEV_PATH   =
"/content/drive/MyDrive/voice_cloning_project/features/X_dev.npy"

X_train = np.load(TRAIN_PATH)
X_dev = np.load(DEV_PATH)

# Convert float16 → float32 before calculating statistics
X_train_f32 = X_train.astype(np.float32)
X_dev_f32 = X_dev.astype(np.float32)

print("TRAIN")
print("shape:", X_train.shape)
print("dtype:", X_train.dtype)
print("min :", X_train_f32.min())
print("max :", X_train_f32.max())
print("mean:", X_train_f32.mean())
print("std :", X_train_f32.std())

print("\nDEV")
print("shape:", X_dev.shape)
print("dtype:", X_dev.dtype)
print("min :", X_dev_f32.min())
print("max :", X_dev_f32.max())
print("mean:", X_dev_f32.mean())
print("std :", X_dev_f32.std())
```

```
TRAIN
shape: (25380, 128, 251)
dtype: float16
min : -80.0
max : 0.0
mean: -64.400894
```

```python
import numpy as np

TRAIN_PATH =
"/content/drive/MyDrive/voice_cloning_project/features/X_train.npy"
DEV_PATH   =
"/content/drive/MyDrive/voice_cloning_project/features/X_dev.npy"

# Memory-map: does NOT load the entire array into RAM
X_train = np.load(TRAIN_PATH, mmap_mode="r")
X_dev   = np.load(DEV_PATH, mmap_mode="r")

# Examine only 100 samples
train_sample = np.asarray(X_train[:100], dtype=np.float32)
dev_sample   = np.asarray(X_dev[:100], dtype=np.float32)

print("TRAIN SAMPLE")
print("shape:", train_sample.shape)
print("min :", train_sample.min())
print("max :", train_sample.max())
print("mean:", train_sample.mean())
print("std :", train_sample.std())

print("\nDEV SAMPLE")
print("shape:", dev_sample.shape)
print("min :", dev_sample.min())
print("max :", dev_sample.max())
print("mean:", dev_sample.mean())
print("std :", dev_sample.std())

del train_sample, dev_sample
```

```
TRAIN SAMPLE
shape: (100, 128, 251)
min : -80.0
max : 0.0
mean: -65.11458
std : 17.238668

DEV SAMPLE
shape: (100, 128, 251)
min : -80.0
max : 0.0
mean: -63.787056
std : 17.750332
```

```python
import torch
import torchaudio

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=16000,
```

```
    n_fft=1024,
    hop_length=256,
    n_mels=128,
    power=2.0
)

def preprocess_audio(audio, sr):

    # Convert to float32
    waveform = torch.tensor(audio, dtype=torch.float32)

    # Make mono if necessary
    if waveform.ndim > 1:
        waveform = waveform.mean(dim=0)

    # Resample if necessary
    if sr != 16000:
        waveform = torchaudio.functional.resample(
            waveform, sr, 16000
        )

    # Exactly 4 seconds = 64,000 samples
    target_length = 64000

    if waveform.numel() < target_length:
        waveform = torch.nn.functional.pad(
            waveform,
            (0, target_length - waveform.numel())
        )
    else:
        waveform = waveform[:target_length]

    # Mel spectrogram
    mel = mel_transform(waveform)

    # Convert power Mel → dB
    mel_db = 10.0 * torch.log10(
        torch.clamp(mel, min=1e-10)
    )

    # Reference maximum = 0 dB
    mel_db = mel_db - mel_db.max()

    # Match training range: [-80, 0]
    mel_db = torch.clamp(mel_db, min=-80.0, max=0.0)

    return mel_db.unsqueeze(0)

sample = next(iter(ds))

feature = preprocess_audio(
```

```
    sample["audio"]["array"],
    sample["audio"]["sampling_rate"]
)

print("Shape :", feature.shape)
print("Min   :", feature.min().item())
print("Max   :", feature.max().item())
print("Mean  :", feature.mean().item())
print("Std   :", feature.std().item())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_11709/2549194947.py in <cell line: 0>()
----> 1 sample = next(iter(ds))
      2
      3 feature = preprocess_audio(
      4     sample["audio"]["array"],
      5     sample["audio"]["sampling_rate"]

NameError: name 'ds' is not defined
```

```
# ================================================
# ASVspoof 2021 DF - preprocessing sanity test
# ================================================

import torch
import torchaudio
from datasets import load_dataset

# ------------------------------------------------
# 1. Load streaming dataset
# ------------------------------------------------
ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print("Dataset loaded successfully")


# ------------------------------------------------
# 2. Mel-Spectrogram configuration
# ------------------------------------------------
mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=16000,
    n_fft=1024,
    hop_length=256,
```

```
    n_mels=128,
    power=2.0
)


# ------------------------------------------------
# 3. Preprocessing function
# ------------------------------------------------
def preprocess_audio(audio, sr):

    waveform = torch.tensor(
        audio,
        dtype=torch.float32
    )

    # Convert stereo/multi-channel → mono
    if waveform.ndim > 1:
        waveform = waveform.mean(dim=0)

    # Resample → 16 kHz
    if sr != 16000:
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            16000
        )

    # Exactly 4 seconds
    target_length = 64000

    if waveform.numel() < target_length:
        waveform = torch.nn.functional.pad(
            waveform,
            (0, target_length - waveform.numel())
        )
    else:
        waveform = waveform[:target_length]

    # Mel spectrogram
    mel = mel_transform(waveform)

    # Convert power spectrogram → dB
    mel_db = 10.0 * torch.log10(
        torch.clamp(mel, min=1e-10)
    )

    # Maximum = 0 dB
    mel_db = mel_db - mel_db.max()

    # Match training feature range
```

```
    mel_db = torch.clamp(
        mel_db,
        min=-80.0,
        max=0.0
    )

    # Add channel dimension
    return mel_db.unsqueeze(0)


# ------------------------------------------------
# 4. Get one sample
# ------------------------------------------------
sample = next(iter(ds))

audio = sample["audio"]["array"]
sr = sample["audio"]["sampling_rate"]

feature = preprocess_audio(audio, sr)


# ------------------------------------------------
# 5. Display information
# ------------------------------------------------
print("\nOriginal audio")
print("Sampling rate :", sr)
print("Samples       :", len(audio))
print("Duration      :", len(audio) / sr)

print("\nProcessed feature")
print("Shape :", feature.shape)
print("Dtype :", feature.dtype)
print("Min   :", feature.min().item())
print("Max   :", feature.max().item())
print("Mean  :", feature.mean().item())
print("Std   :", feature.std().item())

print("\nLabel:", sample["label"])
print("Path :", sample["path"])
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"b09f1fad500a44d3b7d4bb5ac1c2872f","version_major":2,"vers
ion_minor":0}
```

{"model_id":"211ddc2a110546e6a643fdb6c206c6e5","version_major":2,"version_minor":0}

```
Dataset loaded successfully

Original audio
Sampling rate : 16000
Samples       : 22801
Duration      : 1.4250625

Processed feature
Shape : torch.Size([1, 128, 251])
Dtype : torch.float32
Min   : -80.0
Max   : 0.0
Mean  : -70.29228973388672
Std   : 17.36210823059082

Label: 1
Path : DF_E_2000011.flac
```

```python
import torch

# Make sure model is in evaluation mode
model.eval()

# Move feature to the same device as the model
x = feature.unsqueeze(0).to(device)

with torch.no_grad():
    logit = model(x)
    probability = torch.sigmoid(logit).item()

prediction = int(probability >= 0.5)

print("True label        :", sample["label"])
print("Raw model output  :", logit.item())
print("Spoof probability :", probability)
print("Prediction        :", prediction)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_11709/1093380957.py in <cell line: 0>()
      2
      3 # Make sure model is in evaluation mode
----> 4 model.eval()
      5
      6 # Move feature to the same device as the model
```

```
NameError: name 'model' is not defined
```

```
import torch
import torch.nn as nn

# ------------------------------------------------
# 1. Select device
# ------------------------------------------------
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)


# ------------------------------------------------
# 2. Recreate the CNN architecture
# ------------------------------------------------
class VoiceCNN(nn.Module):

    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(

            # Block 1
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            # Block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 128),
            nn.ReLU(),
```

```
            nn.Dropout(0.3),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.gap(x)
        x = self.classifier(x)
        return x


# ------------------------------------------------
# 3. Create model
# ------------------------------------------------
model = VoiceCNN().to(device)


# ------------------------------------------------
# 4. Load saved checkpoint
# ------------------------------------------------
MODEL_PATH = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/models/best_voice_cnn.pth"
)

checkpoint = torch.load(
    MODEL_PATH,
    map_location=device
)

# Handle either a complete checkpoint or state_dict
if isinstance(checkpoint, dict) and "model_state_dict" in checkpoint:
    model.load_state_dict(checkpoint["model_state_dict"])
else:
    model.load_state_dict(checkpoint)


# ------------------------------------------------
# 5. Evaluation mode
# ------------------------------------------------
model.eval()

print("Model loaded successfully!")
print("Checkpoint:", MODEL_PATH)
```

```
Device: cuda
Model loaded successfully!
Checkpoint:
/content/drive/MyDrive/voice_cloning_project/models/best_voice_cnn.pth
```

```
# feature and sample were created by the previous preprocessing cell

x = feature.unsqueeze(0).to(device)

with torch.no_grad():
    logit = model(x)
    probability = torch.sigmoid(logit).item()

prediction = int(probability >= 0.5)

print("True label        :", sample["label"])
print("Raw model output  :", logit.item())
print("Spoof probability :", probability)
print("Prediction        :", prediction)
```

```
True label        : 1
Raw model output  : 0.3011232614517212
Spoof probability : 0.574717104434967
Prediction        : 1
```

```
from collections import Counter

subset_counts = Counter()

for i, sample in enumerate(ds):
    subset = sample["notes"].get("subset", "unknown")
    subset_counts[subset] += 1

    if i >= 9999:
        break

print("Subset distribution in first 10,000 samples:")
print(subset_counts)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_11709/3384945448.py in <cell line: 0>()
      4
      5 for i, sample in enumerate(ds):
----> 6     subset = sample["notes"].get("subset", "unknown")
      7     subset_counts[subset] += 1
      8

AttributeError: 'str' object has no attribute 'get'
```

```
import json
from collections import Counter

subset_counts = Counter()
```

```python
for i, sample in enumerate(ds):
    notes = json.loads(sample["notes"])
    subset = notes.get("subset", "unknown")
    subset_counts[subset] += 1

    if i >= 9999:
        break

print("Subset distribution in first 10,000 samples:")
print(subset_counts)
```

```
Subset distribution in first 10,000 samples:
Counter({'eval': 8739, 'progress': 981, 'hidden': 280})
```

```python
import json
from collections import Counter

subset_counts = Counter()

for i, sample in enumerate(ds):
    notes = json.loads(sample["notes"])
    subset = notes.get("subset", "unknown")
    subset_counts[subset] += 1

print("Total subset distribution:")
print(subset_counts)
print("Total samples:", sum(subset_counts.values()))
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_11709/3891875223.py in <cell line: 0>()
      4 subset_counts = Counter()
      5
----> 6 for i, sample in enumerate(ds):
      7     notes = json.loads(sample["notes"])
      8     subset = notes.get("subset", "unknown")

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py in __iter__(self)
   2359             return
   2360
-> 2361         for key, example in ex_iterable:
   2362             # no need to format thanks to FormattedExamplesIterable
   2363             yield example

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py in __iter__(self)
```

```
   1880         if self.ex_iterable.iter_arrow:
   1881             # feature casting (inc column addition) handled
within self._iter_arrow()
-> 1882             for key, pa_table in self._iter_arrow():
   1883                 batch = formatter.format_batch(pa_table)
   1884                 for example in _batch_to_examples(batch):

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
   1903         if not self.features:
   1904             yield from self.ex_iterable._iter_arrow()
-> 1905         for key, pa_table in self.ex_iterable._iter_arrow():
   1906             columns = set(pa_table.column_names)
   1907             schema = self.features.arrow_schema

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    497             previous_state = self.ex_iterable.state_dict()
    498             self._state_dict["previous_state"] =
previous_state
--> 499         for key, pa_table in iterator:
    500             for num_chunks_since_previous_state, chunk in
enumerate(pa_table.to_reader(max_chunksize=self.batch_size)):
    501                 if num_chunks_to_skip > 1:

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    344             shard_example_idx_start =
self._state_dict["shard_example_idx"] if self._state_dict else 0
    345             shard_example_idx = 0
--> 346             for key, pa_table in
self.generate_tables_fn(**gen_kwags):
    347                 shard_example_idx += len(pa_table)
    348                 if shard_example_idx <=
shard_example_idx_start:

/usr/local/lib/python3.13/dist-packages/datasets/packaged_modules/
parquet/parquet.py in _generate_tables(self, files)
     91                     batch_size = self.config.batch_size or
parquet_fragment.row_groups[0].num_rows
     92                     try:
---> 93                         for batch_idx, record_batch in
enumerate(
     94                             parquet_fragment.to_batches(
     95                                 batch_size=batch_size,

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
_iterator()
   3804         """
   3805         def _iterator(batch_iter):
```

```
-> 3806             for batch in batch_iter:
   3807                 yield batch.record_batch
   3808         # Don't make ourselves a generator so errors are
raised immediately

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
pyarrow._dataset.TaggedRecordBatchIterator.__next__()
   3425         if batch.record_batch == NULL:
   3426             raise StopIteration
-> 3427         return TaggedRecordBatch(
   3428
record_batch=pyarrow_wrap_batch(batch.record_batch),
   3429             fragment=Fragment.wrap(batch.fragment))

<string> in <lambda>(_cls, record_batch, fragment)

KeyboardInterrupt:
```

```python
import json
import numpy as np
import torch
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)

# Evaluate only 1,000 ASVspoof 2021 DF "eval" samples
MAX_EVAL_SAMPLES = 1000

y_true = []
y_prob = []

eval_seen = 0

for sample in ds:
    notes = json.loads(sample["notes"])

    # Keep ONLY official eval subset
    if notes.get("subset") != "eval":
        continue

    # Preprocess
    feature = preprocess_audio(
        sample["audio"]["array"],
        sample["audio"]["sampling_rate"]
    )
```

```
    feature = feature.unsqueeze(0).to(device)

    # Model inference
    with torch.no_grad():
        logit = model(feature)
        prob = torch.sigmoid(logit).item()

    y_true.append(int(sample["label"]))
    y_prob.append(prob)

    eval_seen += 1

    if eval_seen >= MAX_EVAL_SAMPLES:
        break

print("Evaluation samples:", eval_seen)

# Convert probabilities to predictions
y_pred = [int(p >= 0.5) for p in y_prob]

# Metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, zero_division=0)
recall = recall_score(y_true, y_pred, zero_division=0)
f1 = f1_score(y_true, y_pred, zero_division=0)

try:
    roc_auc = roc_auc_score(y_true, y_prob)
except ValueError:
    roc_auc = float("nan")

cm = confusion_matrix(y_true, y_pred)

print("\nASVspoof 2021 DF – Eval Subset")
print("--------------------------------")
print(f"Accuracy : {accuracy:.6f}")
print(f"Precision: {precision:.6f}")
print(f"Recall   : {recall:.6f}")
print(f"F1 Score : {f1:.6f}")
print(f"ROC-AUC  : {roc_auc:.6f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClass distribution:")
print("Bonafide (0):", y_true.count(0))
print("Spoof    (1):", y_true.count(1))
```

```
Evaluation samples: 1000
```

```
ASVspoof 2021 DF — Eval Subset
--------------------------------
Accuracy : 0.590000
Precision: 0.977740
Recall   : 0.589876
F1 Score : 0.735825
ROC-AUC  : 0.631505

Confusion Matrix:
[[ 19  13]
 [397 571]]

Class distribution:
Bonafide (0): 32
Spoof    (1): 968
```

```python
import json
import numpy as np
import torch

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)

MAX_EVAL_SAMPLES = 10000

y_true = []
y_prob = []

eval_seen = 0

print("Starting 10,000-sample ASVspoof 2021 DF evaluation...")

for sample in ds:
    notes = json.loads(sample["notes"])

    # Only ASVspoof 2021 DF eval subset
    if notes.get("subset") != "eval":
        continue

    # Preprocess audio
    feature = preprocess_audio(
        sample["audio"]["array"],
        sample["audio"]["sampling_rate"]
```

```
    )

    # Add batch dimension
    feature = feature.unsqueeze(0).to(device)

    # Inference
    with torch.no_grad():
        logit = model(feature)
        prob = torch.sigmoid(logit).item()

    y_true.append(int(sample["label"]))
    y_prob.append(prob)

    eval_seen += 1

    if eval_seen % 1000 == 0:
        print(f"Processed {eval_seen}/{MAX_EVAL_SAMPLES}")

    if eval_seen >= MAX_EVAL_SAMPLES:
        break


# Convert to NumPy
y_true = np.array(y_true)
y_prob = np.array(y_prob)

# Fixed threshold = 0.5
y_pred = (y_prob >= 0.5).astype(int)


# ----------------------------------------------------
# Classification metrics
# ----------------------------------------------------

accuracy = accuracy_score(y_true, y_pred)

precision = precision_score(
    y_true,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_true,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_true,
    y_pred,
```

```
    zero_division=0
)

roc_auc = roc_auc_score(
    y_true,
    y_prob
)

cm = confusion_matrix(
    y_true,
    y_pred
)


# ----------------------------------------------------
# EER
# ----------------------------------------------------

fpr, tpr, thresholds = roc_curve(
    y_true,
    y_prob
)

fnr = 1 - tpr

eer_index = np.nanargmin(
    np.abs(fpr - fnr)
)

eer = (fpr[eer_index] + fnr[eer_index]) / 2


# ----------------------------------------------------
# Results
# ----------------------------------------------------

print("\n" + "=" * 55)
print("ASVspoof 2021 DF — 10,000 Eval Samples")
print("=" * 55)

print(f"Samples evaluated : {eval_seen}")

print(f"\nAccuracy : {accuracy:.6f}")
print(f"Precision: {precision:.6f}")
print(f"Recall   : {recall:.6f}")
print(f"F1 Score : {f1:.6f}")
print(f"ROC-AUC  : {roc_auc:.6f}")
print(f"EER      : {eer:.6f} ({eer * 100:.2f}%)")

print("\nConfusion Matrix:")
print(cm)
```

```python
print("\nClass Distribution:")
print(f"Bonafide (0): {np.sum(y_true == 0)}")
print(f"Spoof    (1): {np.sum(y_true == 1)}")

print("\nPrediction Distribution:")
print(f"Predicted Bonafide (0): {np.sum(y_pred == 0)}")
print(f"Predicted Spoof    (1): {np.sum(y_pred == 1)}")
```

```
Starting 10,000-sample ASVspoof 2021 DF evaluation...
Processed 1000/10000
Processed 2000/10000
Processed 3000/10000
Processed 4000/10000
Processed 5000/10000
Processed 6000/10000
Processed 7000/10000
Processed 8000/10000
Processed 9000/10000
Processed 10000/10000

============================================================
ASVspoof 2021 DF — 10,000 Eval Samples
============================================================
Samples evaluated : 10000

Accuracy : 0.599600
Precision: 0.980844
Recall   : 0.600082
F1 Score : 0.744610
ROC-AUC  : 0.643612
EER      : 0.406905 (40.69%)

Confusion Matrix:
[[ 159  114]
 [3890 5837]]

Class Distribution:
Bonafide (0): 273
Spoof    (1): 9727

Prediction Distribution:
Predicted Bonafide (0): 4049
Predicted Spoof    (1): 5951
```

```python
import json
from collections import Counter, defaultdict
import numpy as np
import torch
```

```
MAX_EVAL_SAMPLES = 10000

# Store results by attack/generator
attack_stats = defaultdict(lambda: {
    "total": 0,
    "correct": 0,
    "spoof": 0,
    "detected_spoof": 0
})

vocoder_stats = defaultdict(lambda: {
    "total": 0,
    "correct": 0,
    "spoof": 0,
    "detected_spoof": 0
})

codec_stats = defaultdict(lambda: {
    "total": 0,
    "correct": 0,
    "spoof": 0,
    "detected_spoof": 0
})

eval_seen = 0

print("Analyzing 10,000 ASVspoof 2021 DF eval samples...")

for sample in ds:

    notes = json.loads(sample["notes"])

    if notes.get("subset") != "eval":
        continue

    # Preprocess
    feature = preprocess_audio(
        sample["audio"]["array"],
        sample["audio"]["sampling_rate"]
    )

    feature = feature.unsqueeze(0).to(device)

    # Inference
    with torch.no_grad():
        logit = model(feature)
        prob = torch.sigmoid(logit).item()

    true_label = int(sample["label"])
    prediction = int(prob >= 0.5)
```

```
    attack_id = notes.get("attack_id", "unknown")
    vocoder = notes.get("vocoder", "unknown")
    codec = notes.get("codec", "unknown")

    # -------------------------
    # Attack statistics
    # -------------------------

    a = attack_stats[attack_id]

    a["total"] += 1
    a["correct"] += int(prediction == true_label)

    if true_label == 1:
        a["spoof"] += 1
        a["detected_spoof"] += int(prediction == 1)

    # -------------------------
    # Vocoder statistics
    # -------------------------

    v = vocoder_stats[vocoder]

    v["total"] += 1
    v["correct"] += int(prediction == true_label)

    if true_label == 1:
        v["spoof"] += 1
        v["detected_spoof"] += int(prediction == 1)

    # -------------------------
    # Codec statistics
    # -------------------------

    c = codec_stats[codec]

    c["total"] += 1
    c["correct"] += int(prediction == true_label)

    if true_label == 1:
        c["spoof"] += 1
        c["detected_spoof"] += int(prediction == 1)

    eval_seen += 1

    if eval_seen % 1000 == 0:
        print(f"Processed {eval_seen}/{MAX_EVAL_SAMPLES}")

    if eval_seen >= MAX_EVAL_SAMPLES:
        break
```

```python
# ================================================
# Print attack results
# ================================================

print("\n" + "=" * 70)
print("PER-ATTACK PERFORMANCE")
print("=" * 70)

for attack, stats in sorted(attack_stats.items()):

    accuracy = stats["correct"] / stats["total"]

    if stats["spoof"] > 0:
        spoof_recall = stats["detected_spoof"] / stats["spoof"]
    else:
        spoof_recall = float("nan")

    print(
        f"{attack:8s} | "
        f"Samples: {stats['total']:5d} | "
        f"Accuracy: {accuracy:.4f} | "
        f"Spoof Recall: {spoof_recall:.4f}"
    )


# ================================================
# Print vocoder results
# ================================================

print("\n" + "=" * 70)
print("VOCODER / GENERATOR PERFORMANCE")
print("=" * 70)

for vocoder, stats in sorted(vocoder_stats.items()):

    accuracy = stats["correct"] / stats["total"]

    if stats["spoof"] > 0:
        spoof_recall = stats["detected_spoof"] / stats["spoof"]
    else:
        spoof_recall = float("nan")

    print(
        f"{vocoder:25s} | "
        f"Samples: {stats['total']:5d} | "
        f"Accuracy: {accuracy:.4f} | "
        f"Spoof Recall: {spoof_recall:.4f}"
    )
```

```python
# ================================================
# Print codec results
# ================================================

print("\n" + "=" * 70)
print("CODEC PERFORMANCE")
print("=" * 70)

for codec, stats in sorted(codec_stats.items()):

    accuracy = stats["correct"] / stats["total"]

    if stats["spoof"] > 0:
        spoof_recall = stats["detected_spoof"] / stats["spoof"]
    else:
        spoof_recall = float("nan")

    print(
        f"{codec:25s} | "
        f"Samples: {stats['total']:5d} | "
        f"Accuracy: {accuracy:.4f} | "
        f"Spoof Recall: {spoof_recall:.4f}"
    )
```

```
Analyzing 10,000 ASVspoof 2021 DF eval samples...
Processed 1000/10000
Processed 2000/10000
Processed 3000/10000
Processed 4000/10000
Processed 5000/10000
Processed 6000/10000
Processed 7000/10000
Processed 8000/10000
Processed 9000/10000
Processed 10000/10000

======================================================================
PER-ATTACK PERFORMANCE
======================================================================
-          | Samples:   273 | Accuracy: 0.5824 | Spoof Recall: nan
A07        | Samples:   127 | Accuracy: 1.0000 | Spoof Recall: 1.0000
A08        | Samples:    84 | Accuracy: 0.9405 | Spoof Recall: 0.9405
A09        | Samples:   103 | Accuracy: 0.4466 | Spoof Recall: 0.4466
A10        | Samples:   110 | Accuracy: 0.4818 | Spoof Recall: 0.4818
A11        | Samples:   118 | Accuracy: 0.5085 | Spoof Recall: 0.5085
A12        | Samples:   115 | Accuracy: 0.3565 | Spoof Recall: 0.3565
A13        | Samples:    99 | Accuracy: 0.3636 | Spoof Recall: 0.3636
A14        | Samples:   102 | Accuracy: 0.5882 | Spoof Recall: 0.5882
A15        | Samples:   108 | Accuracy: 0.3333 | Spoof Recall: 0.3333
```

```
A16       | Samples:    97 | Accuracy: 0.9794 | Spoof Recall: 0.9794
A17       | Samples:   106 | Accuracy: 0.5755 | Spoof Recall: 0.5755
A18       | Samples:    99 | Accuracy: 0.6970 | Spoof Recall: 0.6970
A19       | Samples:   104 | Accuracy: 0.9904 | Spoof Recall: 0.9904
HUB-B00   | Samples:    10 | Accuracy: 0.3000 | Spoof Recall: 0.3000
HUB-B01   | Samples:    93 | Accuracy: 0.5484 | Spoof Recall: 0.5484
HUB-D01   | Samples:   106 | Accuracy: 0.5566 | Spoof Recall: 0.5566
HUB-D02   | Samples:    94 | Accuracy: 0.4681 | Spoof Recall: 0.4681
HUB-D03   | Samples:   100 | Accuracy: 0.5800 | Spoof Recall: 0.5800
HUB-D04   | Samples:   111 | Accuracy: 0.9820 | Spoof Recall: 0.9820
HUB-D05   | Samples:    90 | Accuracy: 0.9111 | Spoof Recall: 0.9111
HUB-N03   | Samples:   100 | Accuracy: 0.8900 | Spoof Recall: 0.8900
HUB-N04   | Samples:   101 | Accuracy: 0.8911 | Spoof Recall: 0.8911
HUB-N05   | Samples:    97 | Accuracy: 0.6082 | Spoof Recall: 0.6082
HUB-N06   | Samples:    88 | Accuracy: 0.5000 | Spoof Recall: 0.5000
HUB-N07   | Samples:    82 | Accuracy: 0.9146 | Spoof Recall: 0.9146
HUB-N08   | Samples:    82 | Accuracy: 0.5610 | Spoof Recall: 0.5610
HUB-N09   | Samples:   104 | Accuracy: 1.0000 | Spoof Recall: 1.0000
HUB-N10   | Samples:    95 | Accuracy: 0.4632 | Spoof Recall: 0.4632
HUB-N11   | Samples:    98 | Accuracy: 0.9082 | Spoof Recall: 0.9082
HUB-N12   | Samples:    93 | Accuracy: 0.4409 | Spoof Recall: 0.4409
HUB-N13   | Samples:   104 | Accuracy: 0.5288 | Spoof Recall: 0.5288
HUB-N14   | Samples:    82 | Accuracy: 0.6707 | Spoof Recall: 0.6707
HUB-N15   | Samples:    95 | Accuracy: 0.8737 | Spoof Recall: 0.8737
HUB-N16   | Samples:    87 | Accuracy: 1.0000 | Spoof Recall: 1.0000
HUB-N17   | Samples:   104 | Accuracy: 0.5000 | Spoof Recall: 0.5000
HUB-N18   | Samples:    96 | Accuracy: 0.7917 | Spoof Recall: 0.7917
HUB-N19   | Samples:    96 | Accuracy: 0.9583 | Spoof Recall: 0.9583
HUB-N20   | Samples:    91 | Accuracy: 0.6813 | Spoof Recall: 0.6813
SPO-B00   | Samples:     8 | Accuracy: 0.2500 | Spoof Recall: 0.2500
SPO-B01   | Samples:    88 | Accuracy: 0.6023 | Spoof Recall: 0.6023
SPO-N03   | Samples:    94 | Accuracy: 0.9043 | Spoof Recall: 0.9043
SPO-N04   | Samples:   105 | Accuracy: 0.9524 | Spoof Recall: 0.9524
SPO-N05   | Samples:    94 | Accuracy: 0.4894 | Spoof Recall: 0.4894
SPO-N06   | Samples:   113 | Accuracy: 0.5398 | Spoof Recall: 0.5398
SPO-N10   | Samples:    99 | Accuracy: 0.5152 | Spoof Recall: 0.5152
SPO-N11   | Samples:    91 | Accuracy: 0.8791 | Spoof Recall: 0.8791
SPO-N12   | Samples:   112 | Accuracy: 0.3839 | Spoof Recall: 0.3839
SPO-N13   | Samples:    94 | Accuracy: 0.5213 | Spoof Recall: 0.5213
SPO-N16   | Samples:    93 | Accuracy: 1.0000 | Spoof Recall: 1.0000
SPO-N17   | Samples:   100 | Accuracy: 0.4500 | Spoof Recall: 0.4500
SPO-N18   | Samples:    95 | Accuracy: 0.7895 | Spoof Recall: 0.7895
Task1-team01 | Samples:    57 | Accuracy: 0.3509 | Spoof Recall:
0.3509
Task1-team02 | Samples:    63 | Accuracy: 0.7302 | Spoof Recall:
0.7302
Task1-team03 | Samples:    57 | Accuracy: 0.9123 | Spoof Recall:
0.9123
Task1-team04 | Samples:    84 | Accuracy: 0.2738 | Spoof Recall:
```

```
0.2738
Task1-team06 | Samples:    66 | Accuracy: 0.7879 | Spoof Recall:
0.7879
Task1-team07 | Samples:    63 | Accuracy: 0.2540 | Spoof Recall:
0.2540
Task1-team08 | Samples:    58 | Accuracy: 0.3276 | Spoof Recall:
0.3276
Task1-team09 | Samples:    71 | Accuracy: 0.9718 | Spoof Recall:
0.9718
Task1-team10 | Samples:    70 | Accuracy: 0.3000 | Spoof Recall:
0.3000
Task1-team11 | Samples:    75 | Accuracy: 0.3867 | Spoof Recall:
0.3867
Task1-team12 | Samples:    65 | Accuracy: 0.8615 | Spoof Recall:
0.8615
Task1-team13 | Samples:    65 | Accuracy: 0.3692 | Spoof Recall:
0.3692
Task1-team14 | Samples:    58 | Accuracy: 0.4138 | Spoof Recall:
0.4138
Task1-team16 | Samples:    69 | Accuracy: 0.3043 | Spoof Recall:
0.3043
Task1-team17 | Samples:    78 | Accuracy: 0.2564 | Spoof Recall:
0.2564
Task1-team18 | Samples:    74 | Accuracy: 0.8649 | Spoof Recall:
0.8649
Task1-team19 | Samples:    72 | Accuracy: 0.3056 | Spoof Recall:
0.3056
Task1-team20 | Samples:    86 | Accuracy: 0.3605 | Spoof Recall:
0.3605
Task1-team21 | Samples:    60 | Accuracy: 0.2667 | Spoof Recall:
0.2667
Task1-team22 | Samples:    73 | Accuracy: 0.2192 | Spoof Recall:
0.2192
Task1-team23 | Samples:    87 | Accuracy: 0.3563 | Spoof Recall:
0.3563
Task1-team24 | Samples:    66 | Accuracy: 0.7576 | Spoof Recall:
0.7576
Task1-team25 | Samples:    81 | Accuracy: 0.3704 | Spoof Recall:
0.3704
Task1-team26 | Samples:    71 | Accuracy: 0.3099 | Spoof Recall:
0.3099
Task1-team27 | Samples:    66 | Accuracy: 0.5909 | Spoof Recall:
0.5909
Task1-team28 | Samples:    61 | Accuracy: 0.8689 | Spoof Recall:
0.8689
Task1-team29 | Samples:    59 | Accuracy: 0.3729 | Spoof Recall:
0.3729
Task1-team30 | Samples:    67 | Accuracy: 0.9403 | Spoof Recall:
0.9403
```

```
Task1-team31 | Samples:    54 | Accuracy: 0.8519 | Spoof Recall:
0.8519
Task1-team32 | Samples:    67 | Accuracy: 0.2239 | Spoof Recall:
0.2239
Task1-team33 | Samples:    70 | Accuracy: 0.3429 | Spoof Recall:
0.3429
Task2-team02 | Samples:   110 | Accuracy: 0.8091 | Spoof Recall:
0.8091
Task2-team03 | Samples:   107 | Accuracy: 0.9907 | Spoof Recall:
0.9907
Task2-team05 | Samples:   102 | Accuracy: 0.6176 | Spoof Recall:
0.6176
Task2-team06 | Samples:    81 | Accuracy: 0.7901 | Spoof Recall:
0.7901
Task2-team07 | Samples:   110 | Accuracy: 0.3727 | Spoof Recall:
0.3727
Task2-team08 | Samples:    85 | Accuracy: 0.3176 | Spoof Recall:
0.3176
Task2-team09 | Samples:    99 | Accuracy: 1.0000 | Spoof Recall:
1.0000
Task2-team10 | Samples:    99 | Accuracy: 0.3535 | Spoof Recall:
0.3535
Task2-team11 | Samples:    96 | Accuracy: 0.3333 | Spoof Recall:
0.3333
Task2-team12 | Samples:   102 | Accuracy: 0.7157 | Spoof Recall:
0.7157
Task2-team13 | Samples:   103 | Accuracy: 0.3883 | Spoof Recall:
0.3883
Task2-team15 | Samples:    94 | Accuracy: 0.2660 | Spoof Recall:
0.2660
Task2-team16 | Samples:   103 | Accuracy: 0.3204 | Spoof Recall:
0.3204
Task2-team18 | Samples:    91 | Accuracy: 0.6374 | Spoof Recall:
0.6374
Task2-team19 | Samples:    99 | Accuracy: 0.3030 | Spoof Recall:
0.3030
Task2-team20 | Samples:    90 | Accuracy: 0.2778 | Spoof Recall:
0.2778
Task2-team22 | Samples:    93 | Accuracy: 0.4194 | Spoof Recall:
0.4194
Task2-team23 | Samples:   102 | Accuracy: 0.3627 | Spoof Recall:
0.3627
Task2-team24 | Samples:   100 | Accuracy: 0.7900 | Spoof Recall:
0.7900
Task2-team25 | Samples:   102 | Accuracy: 0.4216 | Spoof Recall:
0.4216
Task2-team26 | Samples:   107 | Accuracy: 0.3458 | Spoof Recall:
0.3458
Task2-team27 | Samples:    81 | Accuracy: 0.4815 | Spoof Recall:
```

```
0.4815
Task2-team28 | Samples:   122 | Accuracy: 0.9426 | Spoof Recall:
0.9426
Task2-team29 | Samples:    99 | Accuracy: 0.3838 | Spoof Recall:
0.3838
Task2-team30 | Samples:    93 | Accuracy: 0.9892 | Spoof Recall:
0.9892
Task2-team31 | Samples:    90 | Accuracy: 0.9444 | Spoof Recall:
0.9444
Task2-team32 | Samples:   105 | Accuracy: 0.2857 | Spoof Recall:
0.2857
Task2-team33 | Samples:    92 | Accuracy: 0.3152 | Spoof Recall:
0.3152

======================================================================
VOCODER / GENERATOR PERFORMANCE
======================================================================
bonafide                   | Samples:   273 | Accuracy: 0.5824 | Spoof
Recall: nan
neural_vocoder_autoregressive | Samples:  2624 | Accuracy: 0.4775 |
Spoof Recall: 0.4775
neural_vocoder_nonautoregressive | Samples:  2101 | Accuracy: 0.5045 |
Spoof Recall: 0.5045
traditional_vocoder        | Samples:  4368 | Accuracy: 0.6806 | Spoof
Recall: 0.6806
unknown                    | Samples:   537 | Accuracy: 0.8492 | Spoof
Recall: 0.8492
waveform_concatenation     | Samples:    97 | Accuracy: 0.9794 | Spoof
Recall: 0.9794

======================================================================
CODEC PERFORMANCE
======================================================================
high_m4a                   | Samples:  1030 | Accuracy: 0.3845 | Spoof
Recall: 0.3673
high_mp3                   | Samples:  1006 | Accuracy: 0.8400 | Spoof
Recall: 0.8512
high_ogg                   | Samples:  1216 | Accuracy: 0.3150 | Spoof
Recall: 0.2906
low_m4a                    | Samples:  1026 | Accuracy: 0.3782 | Spoof
Recall: 0.3703
low_mp3                    | Samples:   983 | Accuracy: 0.9420 | Spoof
Recall: 0.9595
low_ogg                    | Samples:  1274 | Accuracy: 0.7567 | Spoof
Recall: 0.7709
mp3m4a                     | Samples:  1220 | Accuracy: 0.8549 | Spoof
Recall: 0.8797
nocodec                    | Samples:  1030 | Accuracy: 0.3835 | Spoof
Recall: 0.3754
```

```
oggm4a                    | Samples:  1215 | Accuracy: 0.5399 | Spoof Recall: 0.5321

import numpy as np
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

thresholds_to_test = [
    0.10,
    0.20,
    0.30,
    0.40,
    0.50,
    0.60,
    0.70,
    0.80,
    0.90
]

print("=" * 85)
print("THRESHOLD ANALYSIS — ASVspoof 2021 DF")
print("=" * 85)

print(
    f"{'Threshold':<12}"
    f"{'Accuracy':<12}"
    f"{'Precision':<12}"
    f"{'Recall':<12}"
    f"{'F1':<12}"
)

for threshold in thresholds_to_test:

    y_pred_threshold = (
        y_prob >= threshold
    ).astype(int)

    accuracy = accuracy_score(
        y_true,
        y_pred_threshold
    )

    precision = precision_score(
        y_true,
        y_pred_threshold,
```

```
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred_threshold,
        zero_division=0
    )

    f1 = f1_score(
        y_true,
        y_pred_threshold,
        zero_division=0
    )

    print(
        f"{threshold:<12.2f}"
        f"{accuracy:<12.4f}"
        f"{precision:<12.4f}"
        f"{recall:<12.4f}"
        f"{f1:<12.4f}"
    )
```

```
================================================================================
THRESHOLD ANALYSIS — ASVspoof 2021 DF
================================================================================
Threshold   Accuracy    Precision   Recall      F1
0.10        0.6526      0.9792      0.6568      0.7862
0.20        0.6337      0.9791      0.6370      0.7718
0.30        0.6205      0.9793      0.6230      0.7615
0.40        0.6100      0.9802      0.6114      0.7531
0.50        0.5996      0.9808      0.6001      0.7446
0.60        0.5887      0.9816      0.5882      0.7356
0.70        0.5739      0.9815      0.5727      0.7234
0.80        0.5584      0.9822      0.5561      0.7101
0.90        0.5338      0.9834      0.5297      0.6885
```

```
import os
import json
import numpy as np

RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"
os.makedirs(RESULTS_DIR, exist_ok=True)

baseline = {
    "dataset": "ASVspoof 2021 DF",
    "samples_evaluated": 10000,
    "accuracy": 0.599600,
```

```
    "precision": 0.980844,
    "recall": 0.600082,
    "f1": 0.744610,
    "roc_auc": 0.643612,
    "eer_approx": 0.406905,
    "confusion_matrix": [[159, 114], [3890, 5837]],
    "model": "VoiceCNN baseline",
    "preprocessing": {
        "sample_rate": 16000,
        "duration_seconds": 4,
        "n_mels": 128,
        "n_fft": 1024,
        "hop_length": 256
    },
    "note": "Frozen baseline; must not be used for hyperparameter
tuning."
}

with open(os.path.join(RESULTS_DIR, "2021_DF_baseline_10000.json"),
"w") as f:
    json.dump(baseline, f, indent=4)

print("Baseline saved.")
```

```
Baseline saved.
```

```
import os
import shutil

DRIVE_FEATURES =
"/content/drive/MyDrive/voice_cloning_project/features"
LOCAL_FEATURES = "/content/features"

os.makedirs(LOCAL_FEATURES, exist_ok=True)

files = [
    "X_train.npy",
    "y_train.npy",
    "X_dev.npy",
    "y_dev.npy"
]

for f in files:
    src = os.path.join(DRIVE_FEATURES, f)
    dst = os.path.join(LOCAL_FEATURES, f)

    if not os.path.exists(dst):
        print("Copying:", f)
        shutil.copy2(src, dst)
    else:
```

```
        print("Already exists:", f)

print("\nDone.")
```

```
Copying: X_train.npy
Copying: y_train.npy
Copying: X_dev.npy
Copying: y_dev.npy

Done.
```

```
import numpy as np
import os

for f in files:
    path = os.path.join(LOCAL_FEATURES, f)
    arr = np.load(path, mmap_mode="r")

    print(
        f,
        "shape =", arr.shape,
        "dtype =", arr.dtype
    )
```

```
X_train.npy shape = (25380, 128, 251) dtype = float16
y_train.npy shape = (25380,) dtype = int64
X_dev.npy shape = (24844, 128, 251) dtype = float16
y_dev.npy shape = (24844,) dtype = int64
```

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

class MelDataset(Dataset):

    def __init__(self, x_path, y_path, augment=False):
        self.X = np.load(x_path, mmap_mode="r")
        self.y = np.load(y_path, mmap_mode="r")
        self.augment = augment

    def __len__(self):
        return len(self.y)

    def spec_augment(self, x):

        # x: [1, 128, 251]

        # Frequency masking
        if torch.rand(1).item() < 0.5:
            max_width = 12
```

```
            width = torch.randint(
                0,
                max_width + 1,
                (1,)
            ).item()

            if width > 0:
                start = torch.randint(
                    0,
                    x.shape[1] - width + 1,
                    (1,)
                ).item()

                x[:, start:start + width, :] = -80.0

        # Time masking
        if torch.rand(1).item() < 0.5:
            max_width = 25

            width = torch.randint(
                0,
                max_width + 1,
                (1,)
            ).item()

            if width > 0:
                start = torch.randint(
                    0,
                    x.shape[2] - width + 1,
                    (1,)
                ).item()

                x[:, :, start:start + width] = -80.0

        return x

    def __getitem__(self, idx):

        x = torch.from_numpy(
            np.asarray(self.X[idx])
        ).float()

        # [128,251] → [1,128,251]
        x = x.unsqueeze(0)

        if self.augment:
            x = self.spec_augment(x)

        y = torch.tensor(
            self.y[idx],
```

```
            dtype=torch.float32
        )

        return x, y

TRAIN_X = "/content/features/X_train.npy"
TRAIN_Y = "/content/features/y_train.npy"

DEV_X = "/content/features/X_dev.npy"
DEV_Y = "/content/features/y_dev.npy"

train_dataset = MelDataset(
    TRAIN_X,
    TRAIN_Y,
    augment=True
)

dev_dataset = MelDataset(
    DEV_X,
    DEV_Y,
    augment=False
)

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=128,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Train:", len(train_dataset))
print("Dev  :", len(dev_dataset))
```

```
Train: 25380
Dev  : 24844
```

```
import torch.nn as nn

class VoiceCNNv2(nn.Module):

    def __init__(self):
        super().__init__()
```

```
        self.features = nn.Sequential(

            nn.Conv2d(1, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Flatten(),

            nn.Linear(128, 128),
            nn.ReLU(),

            nn.Dropout(0.3),

            nn.Linear(128, 1)
        )

    def forward(self, x):

        x = self.features(x)
        x = self.gap(x)
        x = self.classifier(x)

        return x

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = VoiceCNNv2().to(device)

class_weights = torch.tensor(
    [4.9186, 0.5566],
    dtype=torch.float32,
    device=device
)
```

```
criterion = nn.BCEWithLogitsLoss(
    weight=None,
    reduction="none"
)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)

print("Device:", device)
```

```
Device: cuda
```

```
from sklearn.metrics import (
    roc_auc_score,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

def calculate_metrics(y_true, y_prob):

    y_pred = (y_prob >= 0.5).astype(int)

    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(
            y_true,
            y_pred,
            zero_division=0
        ),
        "recall": recall_score(
            y_true,
            y_pred,
            zero_division=0
        ),
        "f1": f1_score(
            y_true,
            y_pred,
```

```
            zero_division=0
        ),
        "roc_auc": roc_auc_score(
            y_true,
            y_prob
        )
    }

def train_one_epoch(model, loader):

    model.train()

    total_loss = 0

    for x, y in loader:

        x = x.to(
            device,
            non_blocking=True
        )

        y = y.to(
            device,
            non_blocking=True
        )

        optimizer.zero_grad()

        logits = model(x).squeeze(1)

        raw_loss = criterion(
            logits,
            y
        )

        sample_weights = torch.where(
            y == 0,
            class_weights[0],
            class_weights[1]
        )

        loss = (
            raw_loss * sample_weights
        ).mean()

        loss.backward()

        optimizer.step()

        total_loss += (
            loss.item() * x.size(0)
```

```
        )

    return total_loss / len(loader.dataset)

@torch.no_grad()
def evaluate(model, loader):

    model.eval()

    all_probs = []
    all_labels = []

    total_loss = 0

    for x, y in loader:

        x = x.to(
            device,
            non_blocking=True
        )

        y = y.to(
            device,
            non_blocking=True
        )

        logits = model(x).squeeze(1)

        raw_loss = criterion(
            logits,
            y
        )

        sample_weights = torch.where(
            y == 0,
            class_weights[0],
            class_weights[1]
        )

        loss = (
            raw_loss * sample_weights
        ).mean()

        total_loss += (
            loss.item() * x.size(0)
        )

        probs = torch.sigmoid(logits)

        all_probs.extend(
            probs.cpu().numpy()
```

```
        )

        all_labels.extend(
            y.cpu().numpy()
        )

    all_probs = np.array(all_probs)
    all_labels = np.array(all_labels)

    metrics = calculate_metrics(
        all_labels,
        all_probs
    )

    metrics["loss"] = (
        total_loss /
        len(loader.dataset)
    )

    return metrics

import os
import copy

MODEL_DIR = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/models"
)

os.makedirs(MODEL_DIR, exist_ok=True)

best_auc = -1
best_state = None

patience = 7
epochs_without_improvement = 0

MAX_EPOCHS = 50

for epoch in range(1, MAX_EPOCHS + 1):

    train_loss = train_one_epoch(
        model,
        train_loader
    )

    metrics = evaluate(
        model,
        dev_loader
    )
```

```
    val_auc = metrics["roc_auc"]

    scheduler.step(val_auc)

    print(
        f"Epoch {epoch:02d} | "
        f"Train Loss: {train_loss:.4f} | "
        f"Val Loss: {metrics['loss']:.4f} | "
        f"AUC: {metrics['roc_auc']:.5f} | "
        f"F1: {metrics['f1']:.5f} | "
        f"Recall: {metrics['recall']:.5f}"
    )

    if val_auc > best_auc:

        best_auc = val_auc

        best_state = copy.deepcopy(
            model.state_dict()
        )

        torch.save(
            {
                "model_state_dict": best_state,
                "epoch": epoch,
                "val_roc_auc": val_auc
            },
            os.path.join(
                MODEL_DIR,
                "voice_cnn_v2_best.pth"
            )
        )

        epochs_without_improvement = 0

        print("  ✓ New best model saved")

    else:

        epochs_without_improvement += 1

    if epochs_without_improvement >= patience:

        print("Early stopping.")
        break
```

```
/tmp/ipykernel_11709/2543098741.py:61: UserWarning: The given NumPy array is not writable, and PyTorch does not support non-writable tensors. This means writing to this tensor will result in undefined behavior. You may want to copy the array to protect its data or make it writable before converting it to a tensor. This type of warning
```

```
will be suppressed for the rest of this program. (Triggered internally
at /pytorch/torch/csrc/utils/tensor_numpy.cpp:213.)
  x = torch.from_numpy(

Epoch 01 | Train Loss: 0.3573 | Val Loss: 0.0793 | AUC: 0.99917 | F1:
0.99015 | Recall: 0.98103
  ✓ New best model saved
Epoch 02 | Train Loss: 0.1128 | Val Loss: 2.6615 | AUC: 0.98251 | F1:
0.94832 | Recall: 1.00000
Epoch 03 | Train Loss: 0.0831 | Val Loss: 0.6707 | AUC: 0.99675 | F1:
0.68986 | Recall: 0.52655
Epoch 04 | Train Loss: 0.0684 | Val Loss: 1.6830 | AUC: 0.98022 | F1:
0.96002 | Recall: 0.99996
Epoch 05 | Train Loss: 0.0397 | Val Loss: 0.1432 | AUC: 0.99914 | F1:
0.94355 | Recall: 0.89321
Epoch 06 | Train Loss: 0.0462 | Val Loss: 0.0135 | AUC: 0.99997 | F1:
0.99913 | Recall: 0.99883
  ✓ New best model saved
Epoch 07 | Train Loss: 0.0362 | Val Loss: 0.5320 | AUC: 0.99881 | F1:
0.97942 | Recall: 1.00000
Epoch 08 | Train Loss: 0.0405 | Val Loss: 0.0433 | AUC: 0.99986 | F1:
0.98350 | Recall: 0.96753
Epoch 09 | Train Loss: 0.0349 | Val Loss: 0.9767 | AUC: 0.99940 | F1:
0.97038 | Recall: 1.00000
Epoch 10 | Train Loss: 0.0258 | Val Loss: 0.0090 | AUC: 0.99997 | F1:
0.99924 | Recall: 0.99879
  ✓ New best model saved
Epoch 11 | Train Loss: 0.0226 | Val Loss: 0.0454 | AUC: 0.99981 | F1:
0.99778 | Recall: 0.99937
Epoch 12 | Train Loss: 0.0261 | Val Loss: 0.0085 | AUC: 0.99996 | F1:
0.99809 | Recall: 0.99628
Epoch 13 | Train Loss: 0.0130 | Val Loss: 0.0080 | AUC: 0.99998 | F1:
0.99960 | Recall: 0.99964
  ✓ New best model saved
Epoch 14 | Train Loss: 0.0151 | Val Loss: 0.0234 | AUC: 0.99998 | F1:
0.99102 | Recall: 0.98219
Epoch 15 | Train Loss: 0.0183 | Val Loss: 0.0075 | AUC: 0.99999 | F1:
0.99966 | Recall: 0.99964
  ✓ New best model saved
Epoch 16 | Train Loss: 0.0110 | Val Loss: 0.0088 | AUC: 0.99998 | F1:
0.99948 | Recall: 0.99937
Epoch 17 | Train Loss: 0.0112 | Val Loss: 0.0052 | AUC: 0.99999 | F1:
0.99937 | Recall: 0.99892
Epoch 18 | Train Loss: 0.0095 | Val Loss: 0.0063 | AUC: 0.99999 | F1:
0.99975 | Recall: 0.99982
  ✓ New best model saved
Epoch 19 | Train Loss: 0.0075 | Val Loss: 0.0041 | AUC: 0.99999 | F1:
0.99951 | Recall: 0.99915
  ✓ New best model saved
Epoch 20 | Train Loss: 0.0161 | Val Loss: 0.0102 | AUC: 0.99999 | F1:
```

```
0.99960 | Recall: 0.99982
Epoch 21 | Train Loss: 0.0094 | Val Loss: 0.0108 | AUC: 0.99999 | F1:
0.99964 | Recall: 0.99991
Epoch 22 | Train Loss: 0.0112 | Val Loss: 0.0058 | AUC: 0.99999 | F1:
0.99973 | Recall: 0.99978
Epoch 23 | Train Loss: 0.0112 | Val Loss: 0.0078 | AUC: 0.99999 | F1:
0.99973 | Recall: 0.99978
Epoch 24 | Train Loss: 0.0075 | Val Loss: 0.0052 | AUC: 0.99999 | F1:
0.99960 | Recall: 0.99951
Epoch 25 | Train Loss: 0.0111 | Val Loss: 0.0073 | AUC: 0.99999 | F1:
0.99978 | Recall: 0.99987
Epoch 26 | Train Loss: 0.0093 | Val Loss: 0.0060 | AUC: 0.99999 | F1:
0.99973 | Recall: 0.99973
Early stopping.
```

```python
import torch
import numpy as np
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)

MODEL_PATH = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/models/"
    "voice_cnn_v2_best.pth"
)

checkpoint = torch.load(
    MODEL_PATH,
    map_location=device,
    weights_only=False
)

model = VoiceCNNv2().to(device)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

model.eval()

print("Loaded epoch:", checkpoint["epoch"])
print("Saved validation AUC:", checkpoint["val_roc_auc"])
```

```
Loaded epoch: 19
Saved validation AUC: 0.9999943144020877
```

```
model = VoiceCNNv2().to(device)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

model.eval()

print("Loaded epoch:", checkpoint["epoch"])
print("Saved validation AUC:", checkpoint["val_roc_auc"])
```

```
Loaded epoch: 19
Saved validation AUC: 0.9999943144020877
```

```
y_true_v2, y_prob_v2 = get_predictions(
    model,
    dev_loader
)

y_pred_v2 = (
    y_prob_v2 >= 0.5
).astype(int)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_11709/1893386933.py in <cell line: 0>()
----> 1 y_true_v2, y_prob_v2 = get_predictions(
      2     model,
      3     dev_loader
      4 )
      5

NameError: name 'get_predictions' is not defined
```

```
import numpy as np
import torch
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)
```

```python
@torch.no_grad()
def get_predictions(model, loader):

    model.eval()

    all_probs = []
    all_labels = []

    for x, y in loader:

        x = x.to(
            device,
            non_blocking=True
        )

        logits = model(x).squeeze(1)

        probs = torch.sigmoid(logits)

        all_probs.extend(
            probs.cpu().numpy()
        )

        all_labels.extend(
            y.numpy()
        )

    return (
        np.array(all_labels),
        np.array(all_probs)
    )


# Get predictions
y_true_v2, y_prob_v2 = get_predictions(
    model,
    dev_loader
)

# Classification at threshold 0.5
y_pred_v2 = (
    y_prob_v2 >= 0.5
).astype(int)


# Metrics
accuracy = accuracy_score(
    y_true_v2,
    y_pred_v2
)
```

```
precision = precision_score(
    y_true_v2,
    y_pred_v2,
    zero_division=0
)

recall = recall_score(
    y_true_v2,
    y_pred_v2,
    zero_division=0
)

f1 = f1_score(
    y_true_v2,
    y_pred_v2,
    zero_division=0
)

auc = roc_auc_score(
    y_true_v2,
    y_prob_v2
)

cm = confusion_matrix(
    y_true_v2,
    y_pred_v2
)


# EER
fpr, tpr, thresholds = roc_curve(
    y_true_v2,
    y_prob_v2
)

fnr = 1 - tpr

eer_index = np.nanargmin(
    np.abs(fpr - fnr)
)

eer = (
    fpr[eer_index] +
    fnr[eer_index]
) / 2

eer_threshold = thresholds[eer_index]


# Print results
```

```python
print("V2 — ASVspoof 2019 LA DEV")
print("=" * 40)

print(f"Accuracy : {accuracy:.6f}")
print(f"Precision: {precision:.6f}")
print(f"Recall   : {recall:.6f}")
print(f"F1 Score : {f1:.6f}")
print(f"ROC-AUC  : {auc:.6f}")
print(f"EER      : {eer:.6f}")
print(f"EER (%)  : {eer * 100:.2f}%")

print("\nConfusion Matrix:")
print(cm)

print(f"\nEER Threshold: {eer_threshold:.6f}")
```

```
V2 — ASVspoof 2019 LA DEV
========================================
Accuracy : 0.999114
Precision: 0.999865
Recall   : 0.999148
F1 Score : 0.999506
ROC-AUC  : 0.999994
EER      : 0.001149
EER (%)  : 0.11%

Confusion Matrix:
[[ 2545     3]
 [   19 22277]]

EER Threshold: 0.572226
```

```python
import torch
import torch.nn as nn
import numpy as np

MODEL_PATH = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/models/"
    "voice_cnn_v2_best.pth"
)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = VoiceCNNv2().to(device)

checkpoint = torch.load(
    MODEL_PATH,
    map_location=device,
```

```
    weights_only=False
)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

model.eval()

print("Model loaded successfully")
print("Checkpoint epoch:", checkpoint["epoch"])
print("2019 Dev AUC:", checkpoint["val_roc_auc"])
```

```
Model loaded successfully
Checkpoint epoch: 19
2019 Dev AUC: 0.9999943144020877
```

```
from datasets import load_dataset

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(ds)
```


```
{"model_id":"bcfec3fe0efb4f38b72f2a640d79046f","version_major":2,"vers
ion_minor":0}

{"model_id":"2ad94d33d0b54dada14faa43cceabae3","version_major":2,"vers
ion_minor":0}
```


```
IterableDataset({
    features: ['path', 'audio', 'label', 'notes'],
    num_shards: 80
})
```

```
from datasets import load_dataset

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(ds)
```


```
{"model_id":"7c0ec7b018474f5ab0ea8f73c781e0a2","version_major":2,"vers
ion_minor":0}
```

{"model_id":"893f6fc30e43451f8c9c811e96bce874","version_major":2,"version_minor":0}

```
IterableDataset({
    features: ['path', 'audio', 'label', 'notes'],
    num_shards: 80
})
```

```
import torchaudio
import torch.nn.functional as F

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=16000,
    n_fft=1024,
    hop_length=256,
    n_mels=128,
    power=2.0
)

def preprocess_audio(audio, sr):

    waveform = torch.tensor(
        audio,
        dtype=torch.float32
    )

    if waveform.ndim > 1:
        waveform = waveform.mean(dim=0)

    if sr != 16000:
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            16000
        )

    target_length = 64000

    if waveform.numel() < target_length:

        waveform = F.pad(
            waveform,
            (0, target_length - waveform.numel())
        )

    else:

        waveform = waveform[:target_length]

    mel = mel_transform(waveform)
```

```
    mel_db = 10.0 * torch.log10(
        torch.clamp(
            mel,
            min=1e-10
        )
    )

    mel_db = mel_db - mel_db.max()

    mel_db = torch.clamp(
        mel_db,
        min=-80.0,
        max=0.0
    )

    return mel_db.unsqueeze(0)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)

y_true = []
y_prob = []

model.eval()

for i, sample in enumerate(ds):

    audio = sample["audio"]

    waveform = audio["array"]
    sr = audio["sampling_rate"]

    label = int(sample["label"])

    feature = preprocess_audio(
        waveform,
        sr
    )

    feature = feature.unsqueeze(0).to(device)

    with torch.no_grad():

        logit = model(feature)
```

```
        probability = torch.sigmoid(
            logit
        ).item()

    y_true.append(label)
    y_prob.append(probability)

    if (i + 1) % 1000 == 0:

        print(
            f"Processed: {i + 1:,}"
        )
```

```
Processed: 1,000
Processed: 2,000
Processed: 3,000
Processed: 4,000
Processed: 5,000
Processed: 6,000
Processed: 7,000
Processed: 8,000
Processed: 9,000
Processed: 10,000
Processed: 11,000
Processed: 12,000
Processed: 13,000
Processed: 14,000
Processed: 15,000
Processed: 16,000
Processed: 17,000
Processed: 18,000
Processed: 19,000
Processed: 20,000
```

```
---------------------------------------------------------------------------
-----
RuntimeError                              Traceback (most recent call
last)
/tmp/ipykernel_11709/1990754140.py in <cell line: 0>()
     18     audio = sample["audio"]
     19
---> 20     waveform = audio["array"]
     21     sr = audio["sampling_rate"]
     22

/usr/local/lib/python3.13/dist-packages/datasets/features/_torchcodec.
py in __getitem__(self, key)
      6     def __getitem__(self, key: str):
      7         if key == "array":
```

```
----> 8             y = self.get_all_samples().data.cpu().numpy()
      9             return np.mean(y, axis=tuple(range(y.ndim - 1)))
if y.ndim > 1 else y
     10         elif key == "sampling_rate":

/usr/local/lib/python3.13/dist-packages/torchcodec/decoders/_audio_dec
oder.py in get_all_samples(self)
     85             AudioSamples: The samples within the file.
     86         """
---> 87         return self.get_samples_played_in_range()
     88
     89     def get_samples_played_in_range(

/usr/local/lib/python3.13/dist-packages/torchcodec/decoders/_audio_dec
oder.py in get_samples_played_in_range(self, start_seconds,
stop_seconds)
    114                 f"Invalid start seconds: {start_seconds}. It
must be less than or equal to stop seconds ({stop_seconds})."
    115             )
--> 116         frames, first_pts =
core.get_frames_by_pts_in_range_audio(
    117             self._decoder,
    118             start_seconds=start_seconds,

/usr/local/lib/python3.13/dist-packages/torch/_ops.py in
__call__(self, *args, **kwargs)
    863     # that are named "self". This way, all the aten ops can be
called by kwargs.
    864     def __call__(self, /, *args: _P.args, **kwargs: _P.kwargs)
-> _T:
--> 865         return self._op(*args, **kwargs)
    866
    867     # Use positional-only argument to avoid naming collision
with aten ops arguments

RuntimeError: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
```

```
import numpy as np
import torch
import time

y_true = []
y_prob = []

processed = 0
failed = 0
```

```
model.eval()

start_time = time.time()

for i, sample in enumerate(ds):

    try:
        audio = sample["audio"]

        waveform = audio["array"]
        sr = audio["sampling_rate"]

        label = int(sample["label"])

        feature = preprocess_audio(
            waveform,
            sr
        )

        feature = (
            feature
            .unsqueeze(0)
            .to(device)
        )

        with torch.no_grad():

            logit = model(feature)

            probability = torch.sigmoid(
                logit
            ).item()

        y_true.append(label)
        y_prob.append(probability)

        processed += 1

        if processed % 1000 == 0:

            elapsed = time.time() - start_time

            print(
                f"Processed: {processed:,} | "
                f"Failed: {failed:,} | "
                f"Elapsed: {elapsed/60:.1f} min"
            )

    except Exception as e:

        failed += 1
```

```
        print(
            f"⚠ Skipping undecodable sample "
            f"at dataset index {i}"
        )

        if failed <= 10:
            print("   Error:", str(e)[:200])

        continue
```

```
Processed: 1,000 | Failed: 0 | Elapsed: 0.2 min
Processed: 2,000 | Failed: 0 | Elapsed: 0.3 min
Processed: 3,000 | Failed: 0 | Elapsed: 0.4 min
Processed: 4,000 | Failed: 0 | Elapsed: 0.4 min
Processed: 5,000 | Failed: 0 | Elapsed: 0.6 min
Processed: 6,000 | Failed: 0 | Elapsed: 0.6 min
Processed: 7,000 | Failed: 0 | Elapsed: 0.7 min
Processed: 8,000 | Failed: 0 | Elapsed: 0.9 min
Processed: 9,000 | Failed: 0 | Elapsed: 1.0 min
Processed: 10,000 | Failed: 0 | Elapsed: 1.1 min
Processed: 11,000 | Failed: 0 | Elapsed: 1.2 min
Processed: 12,000 | Failed: 0 | Elapsed: 1.3 min
Processed: 13,000 | Failed: 0 | Elapsed: 1.4 min
Processed: 14,000 | Failed: 0 | Elapsed: 1.5 min
Processed: 15,000 | Failed: 0 | Elapsed: 1.6 min
Processed: 16,000 | Failed: 0 | Elapsed: 1.7 min
Processed: 17,000 | Failed: 0 | Elapsed: 1.8 min
Processed: 18,000 | Failed: 0 | Elapsed: 1.9 min
Processed: 19,000 | Failed: 0 | Elapsed: 2.0 min
Processed: 20,000 | Failed: 0 | Elapsed: 2.1 min
⚠ Skipping undecodable sample at dataset index 20504
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 21,000 | Failed: 1 | Elapsed: 2.2 min
Processed: 22,000 | Failed: 1 | Elapsed: 2.3 min
Processed: 23,000 | Failed: 1 | Elapsed: 2.4 min
Processed: 24,000 | Failed: 1 | Elapsed: 2.5 min
Processed: 25,000 | Failed: 1 | Elapsed: 2.7 min
Processed: 26,000 | Failed: 1 | Elapsed: 2.7 min
⚠ Skipping undecodable sample at dataset index 26942
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 27,000 | Failed: 2 | Elapsed: 2.8 min
Processed: 28,000 | Failed: 2 | Elapsed: 2.9 min
Processed: 29,000 | Failed: 2 | Elapsed: 3.0 min
```

```
⚠ Skipping undecodable sample at dataset index 29320
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 30,000 | Failed: 3 | Elapsed: 3.1 min
Processed: 31,000 | Failed: 3 | Elapsed: 3.3 min
Processed: 32,000 | Failed: 3 | Elapsed: 3.4 min
Processed: 33,000 | Failed: 3 | Elapsed: 3.5 min
Processed: 34,000 | Failed: 3 | Elapsed: 3.6 min
Processed: 35,000 | Failed: 3 | Elapsed: 3.7 min
Processed: 36,000 | Failed: 3 | Elapsed: 3.8 min
Processed: 37,000 | Failed: 3 | Elapsed: 3.9 min
Processed: 38,000 | Failed: 3 | Elapsed: 4.0 min
Processed: 39,000 | Failed: 3 | Elapsed: 4.1 min
Processed: 40,000 | Failed: 3 | Elapsed: 4.2 min
Processed: 41,000 | Failed: 3 | Elapsed: 4.3 min
Processed: 42,000 | Failed: 3 | Elapsed: 4.4 min
Processed: 43,000 | Failed: 3 | Elapsed: 4.5 min
Processed: 44,000 | Failed: 3 | Elapsed: 4.6 min
Processed: 45,000 | Failed: 3 | Elapsed: 4.7 min
Processed: 46,000 | Failed: 3 | Elapsed: 4.8 min
Processed: 47,000 | Failed: 3 | Elapsed: 4.9 min
Processed: 48,000 | Failed: 3 | Elapsed: 5.0 min
Processed: 49,000 | Failed: 3 | Elapsed: 5.1 min
Processed: 50,000 | Failed: 3 | Elapsed: 5.2 min
Processed: 51,000 | Failed: 3 | Elapsed: 5.3 min
Processed: 52,000 | Failed: 3 | Elapsed: 5.4 min
Processed: 53,000 | Failed: 3 | Elapsed: 5.5 min
Processed: 54,000 | Failed: 3 | Elapsed: 5.7 min
Processed: 55,000 | Failed: 3 | Elapsed: 5.8 min
Processed: 56,000 | Failed: 3 | Elapsed: 5.9 min
Processed: 57,000 | Failed: 3 | Elapsed: 6.0 min
Processed: 58,000 | Failed: 3 | Elapsed: 6.1 min
Processed: 59,000 | Failed: 3 | Elapsed: 6.2 min
Processed: 60,000 | Failed: 3 | Elapsed: 6.3 min
Processed: 61,000 | Failed: 3 | Elapsed: 6.4 min
⚠ Skipping undecodable sample at dataset index 61912
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 62,000 | Failed: 4 | Elapsed: 6.6 min
Processed: 63,000 | Failed: 4 | Elapsed: 6.7 min
Processed: 64,000 | Failed: 4 | Elapsed: 6.8 min
Processed: 65,000 | Failed: 4 | Elapsed: 6.9 min
Processed: 66,000 | Failed: 4 | Elapsed: 7.0 min
Processed: 67,000 | Failed: 4 | Elapsed: 7.1 min
Processed: 68,000 | Failed: 4 | Elapsed: 7.2 min
```

```
Processed: 69,000 | Failed: 4 | Elapsed: 7.3 min
Processed: 70,000 | Failed: 4 | Elapsed: 7.4 min
Processed: 71,000 | Failed: 4 | Elapsed: 7.5 min
Processed: 72,000 | Failed: 4 | Elapsed: 7.6 min
Processed: 73,000 | Failed: 4 | Elapsed: 7.7 min
Processed: 74,000 | Failed: 4 | Elapsed: 7.8 min
Processed: 75,000 | Failed: 4 | Elapsed: 7.9 min
Processed: 76,000 | Failed: 4 | Elapsed: 8.0 min
Processed: 77,000 | Failed: 4 | Elapsed: 8.2 min
Processed: 78,000 | Failed: 4 | Elapsed: 8.3 min
Processed: 79,000 | Failed: 4 | Elapsed: 8.4 min
Processed: 80,000 | Failed: 4 | Elapsed: 8.5 min
Processed: 81,000 | Failed: 4 | Elapsed: 8.6 min
Processed: 82,000 | Failed: 4 | Elapsed: 8.7 min
Processed: 83,000 | Failed: 4 | Elapsed: 8.8 min
Processed: 84,000 | Failed: 4 | Elapsed: 8.9 min
⚠ Skipping undecodable sample at dataset index 84702
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 85,000 | Failed: 5 | Elapsed: 9.0 min
Processed: 86,000 | Failed: 5 | Elapsed: 9.1 min
Processed: 87,000 | Failed: 5 | Elapsed: 9.2 min
Processed: 88,000 | Failed: 5 | Elapsed: 9.3 min
Processed: 89,000 | Failed: 5 | Elapsed: 9.4 min
Processed: 90,000 | Failed: 5 | Elapsed: 9.5 min
Processed: 91,000 | Failed: 5 | Elapsed: 9.6 min
Processed: 92,000 | Failed: 5 | Elapsed: 9.7 min
Processed: 93,000 | Failed: 5 | Elapsed: 9.8 min
Processed: 94,000 | Failed: 5 | Elapsed: 9.9 min
Processed: 95,000 | Failed: 5 | Elapsed: 10.1 min
Processed: 96,000 | Failed: 5 | Elapsed: 10.1 min
Processed: 97,000 | Failed: 5 | Elapsed: 10.2 min
Processed: 98,000 | Failed: 5 | Elapsed: 10.3 min
Processed: 99,000 | Failed: 5 | Elapsed: 10.4 min
Processed: 100,000 | Failed: 5 | Elapsed: 10.6 min
Processed: 101,000 | Failed: 5 | Elapsed: 10.7 min
Processed: 102,000 | Failed: 5 | Elapsed: 10.8 min
Processed: 103,000 | Failed: 5 | Elapsed: 10.9 min
Processed: 104,000 | Failed: 5 | Elapsed: 11.0 min
Processed: 105,000 | Failed: 5 | Elapsed: 11.1 min
Processed: 106,000 | Failed: 5 | Elapsed: 11.2 min
Processed: 107,000 | Failed: 5 | Elapsed: 11.3 min
Processed: 108,000 | Failed: 5 | Elapsed: 11.4 min
Processed: 109,000 | Failed: 5 | Elapsed: 11.5 min
Processed: 110,000 | Failed: 5 | Elapsed: 11.7 min
Processed: 111,000 | Failed: 5 | Elapsed: 11.7 min
Processed: 112,000 | Failed: 5 | Elapsed: 11.8 min
```

```
Processed: 113,000 | Failed: 5 | Elapsed: 11.9 min
Processed: 114,000 | Failed: 5 | Elapsed: 12.0 min
Processed: 115,000 | Failed: 5 | Elapsed: 12.2 min
Processed: 116,000 | Failed: 5 | Elapsed: 12.3 min
Processed: 117,000 | Failed: 5 | Elapsed: 12.4 min
Processed: 118,000 | Failed: 5 | Elapsed: 12.5 min
Processed: 119,000 | Failed: 5 | Elapsed: 12.6 min
Processed: 120,000 | Failed: 5 | Elapsed: 12.7 min
Processed: 121,000 | Failed: 5 | Elapsed: 12.8 min
Processed: 122,000 | Failed: 5 | Elapsed: 12.9 min
Processed: 123,000 | Failed: 5 | Elapsed: 13.0 min
Processed: 124,000 | Failed: 5 | Elapsed: 13.2 min
Processed: 125,000 | Failed: 5 | Elapsed: 13.3 min
Processed: 126,000 | Failed: 5 | Elapsed: 13.4 min
Processed: 127,000 | Failed: 5 | Elapsed: 13.5 min
Processed: 128,000 | Failed: 5 | Elapsed: 13.6 min
⚠ Skipping undecodable sample at dataset index 128812
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 129,000 | Failed: 6 | Elapsed: 13.7 min
Processed: 130,000 | Failed: 6 | Elapsed: 13.8 min
Processed: 131,000 | Failed: 6 | Elapsed: 13.9 min
Processed: 132,000 | Failed: 6 | Elapsed: 14.0 min
Processed: 133,000 | Failed: 6 | Elapsed: 14.1 min
Processed: 134,000 | Failed: 6 | Elapsed: 14.2 min
Processed: 135,000 | Failed: 6 | Elapsed: 14.3 min
Processed: 136,000 | Failed: 6 | Elapsed: 14.4 min
Processed: 137,000 | Failed: 6 | Elapsed: 14.5 min
Processed: 138,000 | Failed: 6 | Elapsed: 14.6 min
Processed: 139,000 | Failed: 6 | Elapsed: 14.8 min
Processed: 140,000 | Failed: 6 | Elapsed: 14.9 min
Processed: 141,000 | Failed: 6 | Elapsed: 15.0 min
Processed: 142,000 | Failed: 6 | Elapsed: 15.1 min
Processed: 143,000 | Failed: 6 | Elapsed: 15.2 min
Processed: 144,000 | Failed: 6 | Elapsed: 15.3 min
Processed: 145,000 | Failed: 6 | Elapsed: 15.4 min
Processed: 146,000 | Failed: 6 | Elapsed: 15.5 min
Processed: 147,000 | Failed: 6 | Elapsed: 15.6 min
Processed: 148,000 | Failed: 6 | Elapsed: 15.7 min
Processed: 149,000 | Failed: 6 | Elapsed: 15.8 min
Processed: 150,000 | Failed: 6 | Elapsed: 15.9 min
Processed: 151,000 | Failed: 6 | Elapsed: 16.0 min
Processed: 152,000 | Failed: 6 | Elapsed: 16.1 min
Processed: 153,000 | Failed: 6 | Elapsed: 16.3 min
⚠ Skipping undecodable sample at dataset index 153414
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
```

```
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 154,000 | Failed: 7 | Elapsed: 16.4 min
Processed: 155,000 | Failed: 7 | Elapsed: 16.5 min
Processed: 156,000 | Failed: 7 | Elapsed: 16.6 min
Processed: 157,000 | Failed: 7 | Elapsed: 16.7 min
Processed: 158,000 | Failed: 7 | Elapsed: 16.8 min
Processed: 159,000 | Failed: 7 | Elapsed: 16.9 min
Processed: 160,000 | Failed: 7 | Elapsed: 17.0 min
Processed: 161,000 | Failed: 7 | Elapsed: 17.1 min
Processed: 162,000 | Failed: 7 | Elapsed: 17.2 min
Processed: 163,000 | Failed: 7 | Elapsed: 17.3 min
Processed: 164,000 | Failed: 7 | Elapsed: 17.4 min
Processed: 165,000 | Failed: 7 | Elapsed: 17.5 min
Processed: 166,000 | Failed: 7 | Elapsed: 17.6 min
Processed: 167,000 | Failed: 7 | Elapsed: 17.7 min
⚠ Skipping undecodable sample at dataset index 167807
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 168,000 | Failed: 8 | Elapsed: 17.8 min
Processed: 169,000 | Failed: 8 | Elapsed: 17.9 min
Processed: 170,000 | Failed: 8 | Elapsed: 18.1 min
Processed: 171,000 | Failed: 8 | Elapsed: 18.1 min
Processed: 172,000 | Failed: 8 | Elapsed: 18.2 min
Processed: 173,000 | Failed: 8 | Elapsed: 18.4 min
Processed: 174,000 | Failed: 8 | Elapsed: 18.4 min
Processed: 175,000 | Failed: 8 | Elapsed: 18.6 min
Processed: 176,000 | Failed: 8 | Elapsed: 18.7 min
⚠ Skipping undecodable sample at dataset index 176936
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 177,000 | Failed: 9 | Elapsed: 18.8 min
Processed: 178,000 | Failed: 9 | Elapsed: 18.9 min
Processed: 179,000 | Failed: 9 | Elapsed: 19.0 min
Processed: 180,000 | Failed: 9 | Elapsed: 19.1 min
Processed: 181,000 | Failed: 9 | Elapsed: 19.2 min
Processed: 182,000 | Failed: 9 | Elapsed: 19.3 min
Processed: 183,000 | Failed: 9 | Elapsed: 19.4 min
Processed: 184,000 | Failed: 9 | Elapsed: 19.6 min
Processed: 185,000 | Failed: 9 | Elapsed: 19.7 min
Processed: 186,000 | Failed: 9 | Elapsed: 19.8 min
Processed: 187,000 | Failed: 9 | Elapsed: 19.9 min
Processed: 188,000 | Failed: 9 | Elapsed: 20.0 min
Processed: 189,000 | Failed: 9 | Elapsed: 20.1 min
Processed: 190,000 | Failed: 9 | Elapsed: 20.2 min
```

```
Processed: 191,000 | Failed: 9 | Elapsed: 20.3 min
Processed: 192,000 | Failed: 9 | Elapsed: 20.4 min
Processed: 193,000 | Failed: 9 | Elapsed: 20.5 min
Processed: 194,000 | Failed: 9 | Elapsed: 20.6 min
Processed: 195,000 | Failed: 9 | Elapsed: 20.7 min
Processed: 196,000 | Failed: 9 | Elapsed: 20.8 min
Processed: 197,000 | Failed: 9 | Elapsed: 20.9 min
Processed: 198,000 | Failed: 9 | Elapsed: 21.1 min
Processed: 199,000 | Failed: 9 | Elapsed: 21.2 min
Processed: 200,000 | Failed: 9 | Elapsed: 21.3 min
Processed: 201,000 | Failed: 9 | Elapsed: 21.4 min
Processed: 202,000 | Failed: 9 | Elapsed: 21.5 min
Processed: 203,000 | Failed: 9 | Elapsed: 21.6 min
Processed: 204,000 | Failed: 9 | Elapsed: 21.7 min
Processed: 205,000 | Failed: 9 | Elapsed: 21.8 min
Processed: 206,000 | Failed: 9 | Elapsed: 21.9 min
Processed: 207,000 | Failed: 9 | Elapsed: 22.1 min
Processed: 208,000 | Failed: 9 | Elapsed: 22.2 min
Processed: 209,000 | Failed: 9 | Elapsed: 22.3 min
Processed: 210,000 | Failed: 9 | Elapsed: 22.4 min
Processed: 211,000 | Failed: 9 | Elapsed: 22.5 min
Processed: 212,000 | Failed: 9 | Elapsed: 22.6 min
Processed: 213,000 | Failed: 9 | Elapsed: 22.7 min
Processed: 214,000 | Failed: 9 | Elapsed: 22.8 min
Processed: 215,000 | Failed: 9 | Elapsed: 23.0 min
Processed: 216,000 | Failed: 9 | Elapsed: 23.1 min
Processed: 217,000 | Failed: 9 | Elapsed: 23.2 min
Processed: 218,000 | Failed: 9 | Elapsed: 23.3 min
Processed: 219,000 | Failed: 9 | Elapsed: 23.4 min
Processed: 220,000 | Failed: 9 | Elapsed: 23.5 min
⚠ Skipping undecodable sample at dataset index 220394
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 221,000 | Failed: 10 | Elapsed: 23.6 min
Processed: 222,000 | Failed: 10 | Elapsed: 23.8 min
Processed: 223,000 | Failed: 10 | Elapsed: 23.9 min
Processed: 224,000 | Failed: 10 | Elapsed: 24.0 min
Processed: 225,000 | Failed: 10 | Elapsed: 24.1 min
Processed: 226,000 | Failed: 10 | Elapsed: 24.2 min
Processed: 227,000 | Failed: 10 | Elapsed: 24.3 min
Processed: 228,000 | Failed: 10 | Elapsed: 24.4 min
Processed: 229,000 | Failed: 10 | Elapsed: 24.5 min
Processed: 230,000 | Failed: 10 | Elapsed: 24.7 min
Processed: 231,000 | Failed: 10 | Elapsed: 24.8 min
Processed: 232,000 | Failed: 10 | Elapsed: 24.9 min
Processed: 233,000 | Failed: 10 | Elapsed: 25.0 min
Processed: 234,000 | Failed: 10 | Elapsed: 25.1 min
```

```
Processed: 235,000 | Failed: 10 | Elapsed: 25.2 min
Processed: 236,000 | Failed: 10 | Elapsed: 25.3 min
Processed: 237,000 | Failed: 10 | Elapsed: 25.4 min
Processed: 238,000 | Failed: 10 | Elapsed: 25.6 min
Processed: 239,000 | Failed: 10 | Elapsed: 25.7 min
Processed: 240,000 | Failed: 10 | Elapsed: 25.8 min
Processed: 241,000 | Failed: 10 | Elapsed: 25.9 min
Processed: 242,000 | Failed: 10 | Elapsed: 26.0 min
Processed: 243,000 | Failed: 10 | Elapsed: 26.1 min
Processed: 244,000 | Failed: 10 | Elapsed: 26.2 min
Processed: 245,000 | Failed: 10 | Elapsed: 26.3 min
Processed: 246,000 | Failed: 10 | Elapsed: 26.4 min
Processed: 247,000 | Failed: 10 | Elapsed: 26.5 min
Processed: 248,000 | Failed: 10 | Elapsed: 26.6 min
Processed: 249,000 | Failed: 10 | Elapsed: 26.7 min
⚠ Skipping undecodable sample at dataset index 249230
Processed: 250,000 | Failed: 11 | Elapsed: 26.9 min
Processed: 251,000 | Failed: 11 | Elapsed: 26.9 min
Processed: 252,000 | Failed: 11 | Elapsed: 27.1 min
Processed: 253,000 | Failed: 11 | Elapsed: 27.2 min
Processed: 254,000 | Failed: 11 | Elapsed: 27.3 min
Processed: 255,000 | Failed: 11 | Elapsed: 27.4 min
Processed: 256,000 | Failed: 11 | Elapsed: 27.5 min
Processed: 257,000 | Failed: 11 | Elapsed: 27.6 min
Processed: 258,000 | Failed: 11 | Elapsed: 27.7 min
Processed: 259,000 | Failed: 11 | Elapsed: 27.8 min
Processed: 260,000 | Failed: 11 | Elapsed: 27.9 min
Processed: 261,000 | Failed: 11 | Elapsed: 28.1 min
Processed: 262,000 | Failed: 11 | Elapsed: 28.2 min
Processed: 263,000 | Failed: 11 | Elapsed: 28.3 min
Processed: 264,000 | Failed: 11 | Elapsed: 28.4 min
Processed: 265,000 | Failed: 11 | Elapsed: 28.5 min
Processed: 266,000 | Failed: 11 | Elapsed: 28.6 min
Processed: 267,000 | Failed: 11 | Elapsed: 28.7 min
⚠ Skipping undecodable sample at dataset index 267139
Processed: 268,000 | Failed: 12 | Elapsed: 28.8 min
Processed: 269,000 | Failed: 12 | Elapsed: 29.0 min
Processed: 270,000 | Failed: 12 | Elapsed: 29.1 min
Processed: 271,000 | Failed: 12 | Elapsed: 29.2 min
Processed: 272,000 | Failed: 12 | Elapsed: 29.3 min
Processed: 273,000 | Failed: 12 | Elapsed: 29.4 min
Processed: 274,000 | Failed: 12 | Elapsed: 29.5 min
Processed: 275,000 | Failed: 12 | Elapsed: 29.6 min
Processed: 276,000 | Failed: 12 | Elapsed: 29.7 min
Processed: 277,000 | Failed: 12 | Elapsed: 29.8 min
Processed: 278,000 | Failed: 12 | Elapsed: 29.9 min
Processed: 279,000 | Failed: 12 | Elapsed: 30.0 min
⚠ Skipping undecodable sample at dataset index 279312
Processed: 280,000 | Failed: 13 | Elapsed: 30.1 min
```

```
Processed: 281,000 | Failed: 13 | Elapsed: 30.2 min
Processed: 282,000 | Failed: 13 | Elapsed: 30.3 min
Processed: 283,000 | Failed: 13 | Elapsed: 30.5 min
Processed: 284,000 | Failed: 13 | Elapsed: 30.6 min
Processed: 285,000 | Failed: 13 | Elapsed: 30.7 min
Processed: 286,000 | Failed: 13 | Elapsed: 30.8 min
Processed: 287,000 | Failed: 13 | Elapsed: 30.9 min
Processed: 288,000 | Failed: 13 | Elapsed: 31.0 min
Processed: 289,000 | Failed: 13 | Elapsed: 31.1 min
⚠ Skipping undecodable sample at dataset index 289756
Processed: 290,000 | Failed: 14 | Elapsed: 31.2 min
Processed: 291,000 | Failed: 14 | Elapsed: 31.3 min
Processed: 292,000 | Failed: 14 | Elapsed: 31.5 min
Processed: 293,000 | Failed: 14 | Elapsed: 31.6 min
Processed: 294,000 | Failed: 14 | Elapsed: 31.7 min
Processed: 295,000 | Failed: 14 | Elapsed: 31.8 min
Processed: 296,000 | Failed: 14 | Elapsed: 31.9 min
Processed: 297,000 | Failed: 14 | Elapsed: 32.0 min
Processed: 298,000 | Failed: 14 | Elapsed: 32.1 min
Processed: 299,000 | Failed: 14 | Elapsed: 32.2 min
Processed: 300,000 | Failed: 14 | Elapsed: 32.4 min
Processed: 301,000 | Failed: 14 | Elapsed: 32.5 min
Processed: 302,000 | Failed: 14 | Elapsed: 32.6 min
Processed: 303,000 | Failed: 14 | Elapsed: 32.7 min
Processed: 304,000 | Failed: 14 | Elapsed: 32.8 min
Processed: 305,000 | Failed: 14 | Elapsed: 32.9 min
Processed: 306,000 | Failed: 14 | Elapsed: 33.0 min
Processed: 307,000 | Failed: 14 | Elapsed: 33.1 min
Processed: 308,000 | Failed: 14 | Elapsed: 33.2 min
Processed: 309,000 | Failed: 14 | Elapsed: 33.3 min
Processed: 310,000 | Failed: 14 | Elapsed: 33.4 min
Processed: 311,000 | Failed: 14 | Elapsed: 33.5 min
Processed: 312,000 | Failed: 14 | Elapsed: 33.6 min
Processed: 313,000 | Failed: 14 | Elapsed: 33.7 min
Processed: 314,000 | Failed: 14 | Elapsed: 33.9 min
Processed: 315,000 | Failed: 14 | Elapsed: 34.0 min
Processed: 316,000 | Failed: 14 | Elapsed: 34.1 min
Processed: 317,000 | Failed: 14 | Elapsed: 34.2 min
Processed: 318,000 | Failed: 14 | Elapsed: 34.3 min
Processed: 319,000 | Failed: 14 | Elapsed: 34.4 min
Processed: 320,000 | Failed: 14 | Elapsed: 34.5 min
Processed: 321,000 | Failed: 14 | Elapsed: 34.6 min
Processed: 322,000 | Failed: 14 | Elapsed: 34.8 min
Processed: 323,000 | Failed: 14 | Elapsed: 34.9 min
Processed: 324,000 | Failed: 14 | Elapsed: 35.0 min
Processed: 325,000 | Failed: 14 | Elapsed: 35.1 min
Processed: 326,000 | Failed: 14 | Elapsed: 35.2 min
Processed: 327,000 | Failed: 14 | Elapsed: 35.3 min
Processed: 328,000 | Failed: 14 | Elapsed: 35.4 min
```

```
Processed: 329,000 | Failed: 14 | Elapsed: 35.5 min
Processed: 330,000 | Failed: 14 | Elapsed: 35.6 min
Processed: 331,000 | Failed: 14 | Elapsed: 35.8 min
Processed: 332,000 | Failed: 14 | Elapsed: 35.9 min
Processed: 333,000 | Failed: 14 | Elapsed: 36.0 min
Processed: 334,000 | Failed: 14 | Elapsed: 36.1 min
Processed: 335,000 | Failed: 14 | Elapsed: 36.2 min
Processed: 336,000 | Failed: 14 | Elapsed: 36.3 min
Processed: 337,000 | Failed: 14 | Elapsed: 36.4 min
Processed: 338,000 | Failed: 14 | Elapsed: 36.5 min
Processed: 339,000 | Failed: 14 | Elapsed: 36.7 min
Processed: 340,000 | Failed: 14 | Elapsed: 36.8 min
Processed: 341,000 | Failed: 14 | Elapsed: 36.9 min
Processed: 342,000 | Failed: 14 | Elapsed: 37.0 min
Processed: 343,000 | Failed: 14 | Elapsed: 37.1 min
Processed: 344,000 | Failed: 14 | Elapsed: 37.2 min
Processed: 345,000 | Failed: 14 | Elapsed: 37.4 min
Processed: 346,000 | Failed: 14 | Elapsed: 37.5 min
Processed: 347,000 | Failed: 14 | Elapsed: 37.6 min
Processed: 348,000 | Failed: 14 | Elapsed: 37.7 min
Processed: 349,000 | Failed: 14 | Elapsed: 37.8 min
Processed: 350,000 | Failed: 14 | Elapsed: 37.9 min
Processed: 351,000 | Failed: 14 | Elapsed: 38.0 min
Processed: 352,000 | Failed: 14 | Elapsed: 38.2 min
Processed: 353,000 | Failed: 14 | Elapsed: 38.3 min
Processed: 354,000 | Failed: 14 | Elapsed: 38.4 min
Processed: 355,000 | Failed: 14 | Elapsed: 38.5 min
Processed: 356,000 | Failed: 14 | Elapsed: 38.6 min
⚠ Skipping undecodable sample at dataset index 356544
Processed: 357,000 | Failed: 15 | Elapsed: 38.7 min
Processed: 358,000 | Failed: 15 | Elapsed: 38.8 min
Processed: 359,000 | Failed: 15 | Elapsed: 38.9 min
Processed: 360,000 | Failed: 15 | Elapsed: 39.1 min
Processed: 361,000 | Failed: 15 | Elapsed: 39.2 min
Processed: 362,000 | Failed: 15 | Elapsed: 39.3 min
Processed: 363,000 | Failed: 15 | Elapsed: 39.4 min
Processed: 364,000 | Failed: 15 | Elapsed: 39.5 min
Processed: 365,000 | Failed: 15 | Elapsed: 39.6 min
Processed: 366,000 | Failed: 15 | Elapsed: 39.7 min
Processed: 367,000 | Failed: 15 | Elapsed: 39.8 min
Processed: 368,000 | Failed: 15 | Elapsed: 40.0 min
Processed: 369,000 | Failed: 15 | Elapsed: 40.1 min
Processed: 370,000 | Failed: 15 | Elapsed: 40.2 min
Processed: 371,000 | Failed: 15 | Elapsed: 40.3 min
Processed: 372,000 | Failed: 15 | Elapsed: 40.4 min
Processed: 373,000 | Failed: 15 | Elapsed: 40.5 min
Processed: 374,000 | Failed: 15 | Elapsed: 40.6 min
Processed: 375,000 | Failed: 15 | Elapsed: 40.7 min
Processed: 376,000 | Failed: 15 | Elapsed: 40.8 min
```

```
Processed: 377,000 | Failed: 15 | Elapsed: 40.9 min
Processed: 378,000 | Failed: 15 | Elapsed: 41.0 min
Processed: 379,000 | Failed: 15 | Elapsed: 41.1 min
Processed: 380,000 | Failed: 15 | Elapsed: 41.2 min
Processed: 381,000 | Failed: 15 | Elapsed: 41.3 min
Processed: 382,000 | Failed: 15 | Elapsed: 41.4 min
Processed: 383,000 | Failed: 15 | Elapsed: 41.6 min
Processed: 384,000 | Failed: 15 | Elapsed: 41.7 min
Processed: 385,000 | Failed: 15 | Elapsed: 41.8 min
Processed: 386,000 | Failed: 15 | Elapsed: 41.9 min
Processed: 387,000 | Failed: 15 | Elapsed: 42.0 min
Processed: 388,000 | Failed: 15 | Elapsed: 42.1 min
Processed: 389,000 | Failed: 15 | Elapsed: 42.2 min
Processed: 390,000 | Failed: 15 | Elapsed: 42.3 min
Processed: 391,000 | Failed: 15 | Elapsed: 42.5 min
Processed: 392,000 | Failed: 15 | Elapsed: 42.6 min
Processed: 393,000 | Failed: 15 | Elapsed: 42.7 min
Processed: 394,000 | Failed: 15 | Elapsed: 42.8 min
Processed: 395,000 | Failed: 15 | Elapsed: 42.9 min
Processed: 396,000 | Failed: 15 | Elapsed: 43.0 min
Processed: 397,000 | Failed: 15 | Elapsed: 43.1 min
Processed: 398,000 | Failed: 15 | Elapsed: 43.2 min
Processed: 399,000 | Failed: 15 | Elapsed: 43.3 min
Processed: 400,000 | Failed: 15 | Elapsed: 43.5 min
Processed: 401,000 | Failed: 15 | Elapsed: 43.6 min
Processed: 402,000 | Failed: 15 | Elapsed: 43.7 min
Processed: 403,000 | Failed: 15 | Elapsed: 43.8 min
Processed: 404,000 | Failed: 15 | Elapsed: 43.9 min
Processed: 405,000 | Failed: 15 | Elapsed: 44.0 min
Processed: 406,000 | Failed: 15 | Elapsed: 44.1 min
Processed: 407,000 | Failed: 15 | Elapsed: 44.2 min
Processed: 408,000 | Failed: 15 | Elapsed: 44.4 min
Processed: 409,000 | Failed: 15 | Elapsed: 44.5 min
⚠ Skipping undecodable sample at dataset index 409768
Processed: 410,000 | Failed: 16 | Elapsed: 44.6 min
Processed: 411,000 | Failed: 16 | Elapsed: 44.7 min
Processed: 412,000 | Failed: 16 | Elapsed: 44.8 min
Processed: 413,000 | Failed: 16 | Elapsed: 44.9 min
Processed: 414,000 | Failed: 16 | Elapsed: 45.0 min
Processed: 415,000 | Failed: 16 | Elapsed: 45.2 min
Processed: 416,000 | Failed: 16 | Elapsed: 45.3 min
Processed: 417,000 | Failed: 16 | Elapsed: 45.4 min
Processed: 418,000 | Failed: 16 | Elapsed: 45.5 min
Processed: 419,000 | Failed: 16 | Elapsed: 45.6 min
Processed: 420,000 | Failed: 16 | Elapsed: 45.7 min
Processed: 421,000 | Failed: 16 | Elapsed: 45.8 min
Processed: 422,000 | Failed: 16 | Elapsed: 45.9 min
Processed: 423,000 | Failed: 16 | Elapsed: 46.1 min
Processed: 424,000 | Failed: 16 | Elapsed: 46.2 min
```

```
Processed: 425,000 | Failed: 16 | Elapsed: 46.3 min
Processed: 426,000 | Failed: 16 | Elapsed: 46.4 min
Processed: 427,000 | Failed: 16 | Elapsed: 46.5 min
Processed: 428,000 | Failed: 16 | Elapsed: 46.6 min
Processed: 429,000 | Failed: 16 | Elapsed: 46.7 min
Processed: 430,000 | Failed: 16 | Elapsed: 46.9 min
Processed: 431,000 | Failed: 16 | Elapsed: 47.0 min
Processed: 432,000 | Failed: 16 | Elapsed: 47.1 min
Processed: 433,000 | Failed: 16 | Elapsed: 47.2 min
Processed: 434,000 | Failed: 16 | Elapsed: 47.3 min
Processed: 435,000 | Failed: 16 | Elapsed: 47.4 min
Processed: 436,000 | Failed: 16 | Elapsed: 47.5 min
Processed: 437,000 | Failed: 16 | Elapsed: 47.6 min
Processed: 438,000 | Failed: 16 | Elapsed: 47.7 min
Processed: 439,000 | Failed: 16 | Elapsed: 47.9 min
Processed: 440,000 | Failed: 16 | Elapsed: 47.9 min
Processed: 441,000 | Failed: 16 | Elapsed: 48.1 min
Processed: 442,000 | Failed: 16 | Elapsed: 48.2 min
Processed: 443,000 | Failed: 16 | Elapsed: 48.3 min
Processed: 444,000 | Failed: 16 | Elapsed: 48.4 min
Processed: 445,000 | Failed: 16 | Elapsed: 48.5 min
Processed: 446,000 | Failed: 16 | Elapsed: 48.6 min
Processed: 447,000 | Failed: 16 | Elapsed: 48.7 min
Processed: 448,000 | Failed: 16 | Elapsed: 48.8 min
Processed: 449,000 | Failed: 16 | Elapsed: 49.0 min
Processed: 450,000 | Failed: 16 | Elapsed: 49.0 min
Processed: 451,000 | Failed: 16 | Elapsed: 49.1 min
Processed: 452,000 | Failed: 16 | Elapsed: 49.3 min
Processed: 453,000 | Failed: 16 | Elapsed: 49.4 min
Processed: 454,000 | Failed: 16 | Elapsed: 49.5 min
Processed: 455,000 | Failed: 16 | Elapsed: 49.6 min
Processed: 456,000 | Failed: 16 | Elapsed: 49.7 min
Processed: 457,000 | Failed: 16 | Elapsed: 49.8 min
Processed: 458,000 | Failed: 16 | Elapsed: 49.9 min
Processed: 459,000 | Failed: 16 | Elapsed: 50.1 min
Processed: 460,000 | Failed: 16 | Elapsed: 50.2 min
Processed: 461,000 | Failed: 16 | Elapsed: 50.3 min
Processed: 462,000 | Failed: 16 | Elapsed: 50.4 min
Processed: 463,000 | Failed: 16 | Elapsed: 50.5 min
Processed: 464,000 | Failed: 16 | Elapsed: 50.6 min
Processed: 465,000 | Failed: 16 | Elapsed: 50.7 min
Processed: 466,000 | Failed: 16 | Elapsed: 50.8 min
Processed: 467,000 | Failed: 16 | Elapsed: 50.9 min
Processed: 468,000 | Failed: 16 | Elapsed: 51.1 min
Processed: 469,000 | Failed: 16 | Elapsed: 51.2 min
Processed: 470,000 | Failed: 16 | Elapsed: 51.3 min
Processed: 471,000 | Failed: 16 | Elapsed: 51.4 min
Processed: 472,000 | Failed: 16 | Elapsed: 51.5 min
⚠ Skipping undecodable sample at dataset index 472629
Processed: 473,000 | Failed: 17 | Elapsed: 51.6 min
```

```
Processed: 474,000 | Failed: 17 | Elapsed: 51.7 min
Processed: 475,000 | Failed: 17 | Elapsed: 51.8 min
Processed: 476,000 | Failed: 17 | Elapsed: 52.0 min
Processed: 477,000 | Failed: 17 | Elapsed: 52.1 min
Processed: 478,000 | Failed: 17 | Elapsed: 52.2 min
Processed: 479,000 | Failed: 17 | Elapsed: 52.3 min
Processed: 480,000 | Failed: 17 | Elapsed: 52.4 min
Processed: 481,000 | Failed: 17 | Elapsed: 52.5 min
Processed: 482,000 | Failed: 17 | Elapsed: 52.6 min
Processed: 483,000 | Failed: 17 | Elapsed: 52.7 min
Processed: 484,000 | Failed: 17 | Elapsed: 52.8 min
Processed: 485,000 | Failed: 17 | Elapsed: 52.9 min
Processed: 486,000 | Failed: 17 | Elapsed: 53.0 min
Processed: 487,000 | Failed: 17 | Elapsed: 53.1 min
Processed: 488,000 | Failed: 17 | Elapsed: 53.2 min
Processed: 489,000 | Failed: 17 | Elapsed: 53.3 min
Processed: 490,000 | Failed: 17 | Elapsed: 53.5 min
Processed: 491,000 | Failed: 17 | Elapsed: 53.6 min
Processed: 492,000 | Failed: 17 | Elapsed: 53.7 min
Processed: 493,000 | Failed: 17 | Elapsed: 53.8 min
Processed: 494,000 | Failed: 17 | Elapsed: 53.9 min
Processed: 495,000 | Failed: 17 | Elapsed: 54.0 min
Processed: 496,000 | Failed: 17 | Elapsed: 54.1 min
Processed: 497,000 | Failed: 17 | Elapsed: 54.2 min
Processed: 498,000 | Failed: 17 | Elapsed: 54.3 min
Processed: 499,000 | Failed: 17 | Elapsed: 54.5 min
⚠ Skipping undecodable sample at dataset index 499244
Processed: 500,000 | Failed: 18 | Elapsed: 54.6 min
Processed: 501,000 | Failed: 18 | Elapsed: 54.6 min
Processed: 502,000 | Failed: 18 | Elapsed: 54.8 min
Processed: 503,000 | Failed: 18 | Elapsed: 54.8 min
Processed: 504,000 | Failed: 18 | Elapsed: 55.0 min
Processed: 505,000 | Failed: 18 | Elapsed: 55.1 min
⚠ Skipping undecodable sample at dataset index 505582
Processed: 506,000 | Failed: 19 | Elapsed: 55.2 min
Processed: 507,000 | Failed: 19 | Elapsed: 55.3 min
Processed: 508,000 | Failed: 19 | Elapsed: 55.4 min
Processed: 509,000 | Failed: 19 | Elapsed: 55.5 min
Processed: 510,000 | Failed: 19 | Elapsed: 55.6 min
Processed: 511,000 | Failed: 19 | Elapsed: 55.7 min
Processed: 512,000 | Failed: 19 | Elapsed: 55.8 min
Processed: 513,000 | Failed: 19 | Elapsed: 56.0 min
Processed: 514,000 | Failed: 19 | Elapsed: 56.1 min
Processed: 515,000 | Failed: 19 | Elapsed: 56.2 min
Processed: 516,000 | Failed: 19 | Elapsed: 56.3 min
Processed: 517,000 | Failed: 19 | Elapsed: 56.4 min
Processed: 518,000 | Failed: 19 | Elapsed: 56.5 min
Processed: 519,000 | Failed: 19 | Elapsed: 56.6 min
Processed: 520,000 | Failed: 19 | Elapsed: 56.7 min
```

```
Processed: 521,000 | Failed: 19 | Elapsed: 56.8 min
Processed: 522,000 | Failed: 19 | Elapsed: 57.0 min
Processed: 523,000 | Failed: 19 | Elapsed: 57.1 min
Processed: 524,000 | Failed: 19 | Elapsed: 57.2 min
Processed: 525,000 | Failed: 19 | Elapsed: 57.3 min
Processed: 526,000 | Failed: 19 | Elapsed: 57.4 min
Processed: 527,000 | Failed: 19 | Elapsed: 57.5 min
Processed: 528,000 | Failed: 19 | Elapsed: 57.6 min
Processed: 529,000 | Failed: 19 | Elapsed: 57.7 min
Processed: 530,000 | Failed: 19 | Elapsed: 57.8 min
Processed: 531,000 | Failed: 19 | Elapsed: 57.9 min
Processed: 532,000 | Failed: 19 | Elapsed: 58.0 min
Processed: 533,000 | Failed: 19 | Elapsed: 58.1 min
Processed: 534,000 | Failed: 19 | Elapsed: 58.2 min
Processed: 535,000 | Failed: 19 | Elapsed: 58.3 min
Processed: 536,000 | Failed: 19 | Elapsed: 58.5 min
Processed: 537,000 | Failed: 19 | Elapsed: 58.6 min
Processed: 538,000 | Failed: 19 | Elapsed: 58.7 min
⚠ Skipping undecodable sample at dataset index 538275
Processed: 539,000 | Failed: 20 | Elapsed: 58.8 min
Processed: 540,000 | Failed: 20 | Elapsed: 58.9 min
Processed: 541,000 | Failed: 20 | Elapsed: 59.0 min
Processed: 542,000 | Failed: 20 | Elapsed: 59.1 min
Processed: 543,000 | Failed: 20 | Elapsed: 59.2 min
Processed: 544,000 | Failed: 20 | Elapsed: 59.4 min
Processed: 545,000 | Failed: 20 | Elapsed: 59.5 min
⚠ Skipping undecodable sample at dataset index 545495
Processed: 546,000 | Failed: 21 | Elapsed: 59.6 min
Processed: 547,000 | Failed: 21 | Elapsed: 59.7 min
Processed: 548,000 | Failed: 21 | Elapsed: 59.8 min
Processed: 549,000 | Failed: 21 | Elapsed: 59.9 min
Processed: 550,000 | Failed: 21 | Elapsed: 60.0 min
Processed: 551,000 | Failed: 21 | Elapsed: 60.1 min
Processed: 552,000 | Failed: 21 | Elapsed: 60.3 min
Processed: 553,000 | Failed: 21 | Elapsed: 60.4 min
Processed: 554,000 | Failed: 21 | Elapsed: 60.5 min
Processed: 555,000 | Failed: 21 | Elapsed: 60.6 min
Processed: 556,000 | Failed: 21 | Elapsed: 60.7 min
Processed: 557,000 | Failed: 21 | Elapsed: 60.8 min
Processed: 558,000 | Failed: 21 | Elapsed: 60.9 min
Processed: 559,000 | Failed: 21 | Elapsed: 61.0 min
Processed: 560,000 | Failed: 21 | Elapsed: 61.1 min
Processed: 561,000 | Failed: 21 | Elapsed: 61.2 min
Processed: 562,000 | Failed: 21 | Elapsed: 61.3 min
Processed: 563,000 | Failed: 21 | Elapsed: 61.4 min
Processed: 564,000 | Failed: 21 | Elapsed: 61.5 min
Processed: 565,000 | Failed: 21 | Elapsed: 61.6 min
Processed: 566,000 | Failed: 21 | Elapsed: 61.8 min
Processed: 567,000 | Failed: 21 | Elapsed: 61.9 min
```

```
Processed: 568,000 | Failed: 21 | Elapsed: 62.0 min
Processed: 569,000 | Failed: 21 | Elapsed: 62.1 min
Processed: 570,000 | Failed: 21 | Elapsed: 62.2 min
Processed: 571,000 | Failed: 21 | Elapsed: 62.3 min
Processed: 572,000 | Failed: 21 | Elapsed: 62.4 min
Processed: 573,000 | Failed: 21 | Elapsed: 62.5 min
Processed: 574,000 | Failed: 21 | Elapsed: 62.6 min
Processed: 575,000 | Failed: 21 | Elapsed: 62.8 min
Processed: 576,000 | Failed: 21 | Elapsed: 62.9 min
Processed: 577,000 | Failed: 21 | Elapsed: 63.0 min
⚠ Skipping undecodable sample at dataset index 577169
Processed: 578,000 | Failed: 22 | Elapsed: 63.1 min
Processed: 579,000 | Failed: 22 | Elapsed: 63.2 min
Processed: 580,000 | Failed: 22 | Elapsed: 63.3 min
Processed: 581,000 | Failed: 22 | Elapsed: 63.4 min
Processed: 582,000 | Failed: 22 | Elapsed: 63.5 min
Processed: 583,000 | Failed: 22 | Elapsed: 63.6 min
Processed: 584,000 | Failed: 22 | Elapsed: 63.7 min
Processed: 585,000 | Failed: 22 | Elapsed: 63.8 min
Processed: 586,000 | Failed: 22 | Elapsed: 63.9 min
Processed: 587,000 | Failed: 22 | Elapsed: 64.0 min
Processed: 588,000 | Failed: 22 | Elapsed: 64.1 min
⚠ Skipping undecodable sample at dataset index 588960
Processed: 589,000 | Failed: 23 | Elapsed: 64.3 min
Processed: 590,000 | Failed: 23 | Elapsed: 64.4 min
Processed: 591,000 | Failed: 23 | Elapsed: 64.5 min
Processed: 592,000 | Failed: 23 | Elapsed: 64.6 min
Processed: 593,000 | Failed: 23 | Elapsed: 64.7 min
Processed: 594,000 | Failed: 23 | Elapsed: 64.8 min
Processed: 595,000 | Failed: 23 | Elapsed: 64.9 min
Processed: 596,000 | Failed: 23 | Elapsed: 65.0 min
Processed: 597,000 | Failed: 23 | Elapsed: 65.1 min
Processed: 598,000 | Failed: 23 | Elapsed: 65.2 min
Processed: 599,000 | Failed: 23 | Elapsed: 65.3 min
Processed: 600,000 | Failed: 23 | Elapsed: 65.4 min
Processed: 601,000 | Failed: 23 | Elapsed: 65.5 min
Processed: 602,000 | Failed: 23 | Elapsed: 65.6 min
Processed: 603,000 | Failed: 23 | Elapsed: 65.7 min
Processed: 604,000 | Failed: 23 | Elapsed: 65.8 min
Processed: 605,000 | Failed: 23 | Elapsed: 66.0 min
Processed: 606,000 | Failed: 23 | Elapsed: 66.1 min
Processed: 607,000 | Failed: 23 | Elapsed: 66.2 min
Processed: 608,000 | Failed: 23 | Elapsed: 66.3 min
Processed: 609,000 | Failed: 23 | Elapsed: 66.4 min
Processed: 610,000 | Failed: 23 | Elapsed: 66.5 min
Processed: 611,000 | Failed: 23 | Elapsed: 66.6 min
```

```
print("Successful predictions:", len(y_prob))
print("Failed samples:", failed)
```

```
Successful predictions: 611806
Failed samples: 23
```

```python
import numpy as np
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    roc_curve
)

y_true_np = np.array(y_true)
y_prob_np = np.array(y_prob)

# Default classification threshold
threshold = 0.5
y_pred = (y_prob_np >= threshold).astype(int)

# Metrics
accuracy = accuracy_score(y_true_np, y_pred)
precision = precision_score(y_true_np, y_pred, zero_division=0)
recall = recall_score(y_true_np, y_pred, zero_division=0)
f1 = f1_score(y_true_np, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_true_np, y_prob_np)

cm = confusion_matrix(y_true_np, y_pred)

# EER
fpr, tpr, thresholds = roc_curve(y_true_np, y_prob_np)

fnr = 1 - tpr

eer_index = np.nanargmin(np.abs(fpr - fnr))

eer = (fpr[eer_index] + fnr[eer_index]) / 2
eer_threshold = thresholds[eer_index]

print("\n========== ASVSPOOF 2021 DF RESULTS ==========")

print(f"Samples evaluated : {len(y_true_np):,}")
print(f"Failed samples    : {failed:,}")

print(f"\nAccuracy  : {accuracy:.6f}")
print(f"Precision : {precision:.6f}")
print(f"Recall    : {recall:.6f}")
print(f"F1 Score  : {f1:.6f}")
print(f"ROC-AUC   : {roc_auc:.6f}")
```

```
print(f"\nEER       : {eer:.6f}")
print(f"EER (%)   : {eer*100:.2f}%")
print(f"EER Threshold : {eer_threshold:.6f}")

print("\nConfusion Matrix:")
print(cm)
```

```
========== ASVSPOOF 2021 DF RESULTS ==========
Samples evaluated : 611,806
Failed samples    : 23

Accuracy  : 0.628098
Precision : 0.978741
Recall    : 0.627451
F1 Score  : 0.764680
ROC-AUC   : 0.706488

EER       : 0.365386
EER (%)   : 36.54%
EER Threshold : 0.444269

Confusion Matrix:
[[ 14587   8030]
 [219502 369687]]
```

```
from datasets import load_dataset

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(ds)
```


```
{"model_id":"42357fbeb83b4401929c7c9130f6d6e9","version_major":2,"vers
ion_minor":0}

{"model_id":"367d4bd727434008a94cbd77316cf319","version_major":2,"vers
ion_minor":0}
```


```
IterableDataset({
    features: ['path', 'audio', 'label', 'notes'],
    num_shards: 80
})
```

```
import numpy as np

np.savez(
```

```
"/content/drive/MyDrive/voice_cloning_project/results/asvspoof2021_v2_
predictions.npz",
    y_true=np.array(y_true),
    y_prob=np.array(y_prob)
)

print("Saved:", len(y_true), "predictions")
```

```
Saved: 611806 predictions
```

```
import os
import json

results_dir = "/content/drive/MyDrive/voice_cloning_project/results"
os.makedirs(results_dir, exist_ok=True)

results = {
    "model": "VoiceCNN V2",
    "train_dataset": "ASVspoof 2019 LA",
    "test_dataset": "ASVspoof 2021 DF",
    "samples_evaluated": len(y_true),
    "failed_samples": failed,
    "accuracy": 0.628098,
    "precision": 0.978741,
    "recall": 0.627451,
    "f1": 0.764680,
    "roc_auc": 0.706488,
    "eer": 0.365386,
    "eer_percent": 36.54,
    "confusion_matrix": [
        [14587, 8030],
        [219502, 369687]
    ]
}

with open(
    f"{results_dir}/v2_asvspoof2021_results.json",
    "w"
) as f:
    json.dump(results, f, indent=4)

print("Results saved.")
```

```
Results saved.
```

```
import numpy as np

np.savez(
    f"{results_dir}/v2_asvspoof2021_predictions.npz",
    y_true=np.array(y_true),
    y_prob=np.array(y_prob)
```

```
)

print("Predictions saved:", len(y_true))
```

```
Predictions saved: 611806
```

```
import numpy as np
import matplotlib.pyplot as plt

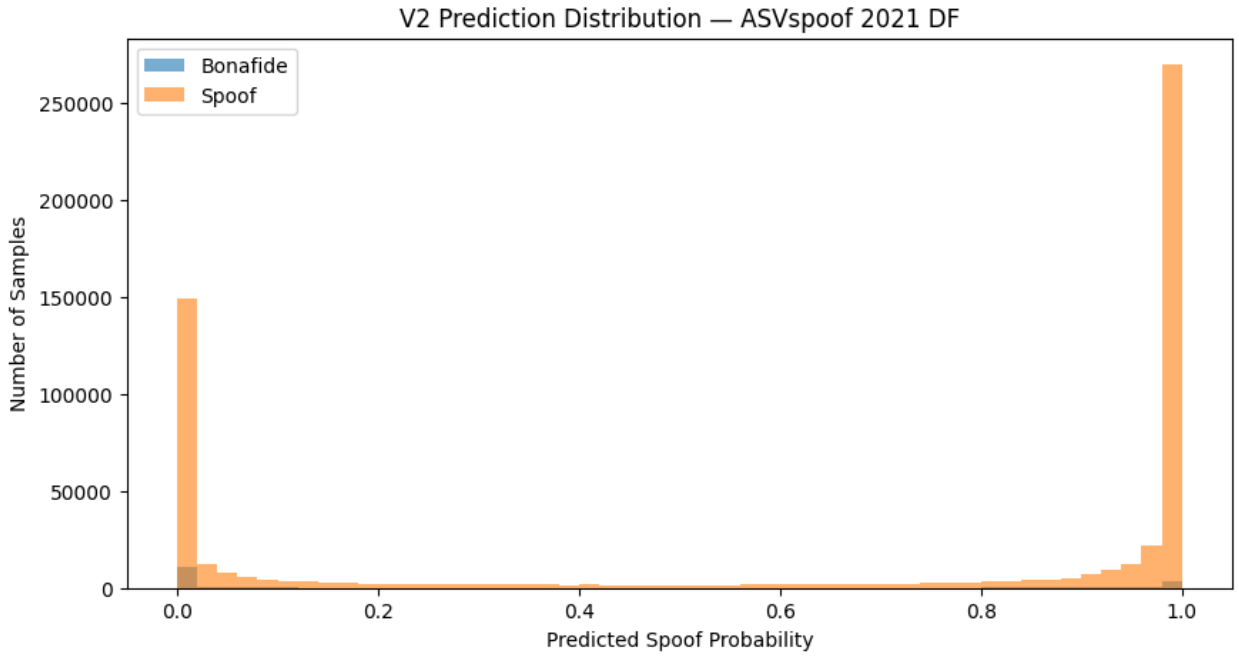
y_true_np = np.array(y_true)
y_prob_np = np.array(y_prob)

plt.figure(figsize=(10, 5))

plt.hist(
    y_prob_np[y_true_np == 0],
    bins=50,
    alpha=0.6,
    label="Bonafide"
)

plt.hist(
    y_prob_np[y_true_np == 1],
    bins=50,
    alpha=0.6,
    label="Spoof"
)

plt.xlabel("Predicted Spoof Probability")
plt.ylabel("Number of Samples")
plt.title("V2 Prediction Distribution — ASVspoof 2021 DF")
plt.legend()
plt.show()
```




```
y_true_np = np.array(y_true)
y_prob_np = np.array(y_prob)

y_pred = (y_prob_np >= 0.5).astype(int)

false_negatives = (y_true_np == 1) & (y_pred == 0)
false_positives = (y_true_np == 0) & (y_pred == 1)

print("========== ERROR ANALYSIS ==========")

print("\nFalse Negatives (Spoof → Human)")
print("Count :", false_negatives.sum())
print("Mean  :", y_prob_np[false_negatives].mean())
print("Median:", np.median(y_prob_np[false_negatives]))
print("Min   :", y_prob_np[false_negatives].min())
print("Max   :", y_prob_np[false_negatives].max())

print("\nFalse Positives (Human → Spoof)")
print("Count :", false_positives.sum())
print("Mean  :", y_prob_np[false_positives].mean())
print("Median:", np.median(y_prob_np[false_positives]))
print("Min   :", y_prob_np[false_positives].min())
print("Max   :", y_prob_np[false_positives].max())
```

```
========== ERROR ANALYSIS ==========

False Negatives (Spoof → Human)
Count : 219502
Mean  : 0.05879987017224508
Median: 0.0033250913256779313
```

```
Min   : 6.729940338345841e-08
Max   : 0.49998998641967773

False Positives (Human → Spoof)
Count : 8030
Mean  : 0.9021485214215584
Median: 0.9636968374252319
Min   : 0.5000438690185547
Max   : 1.0
```

```python
import os
import json
import time
import numpy as np
import torch
from datasets import load_dataset

# ----------------------------------------------------
# Paths
# ----------------------------------------------------

results_dir = "/content/drive/MyDrive/voice_cloning_project/results"
os.makedirs(results_dir, exist_ok=True)

checkpoint_path = f"{results_dir}/v2_2021_detailed_eval.npz"

# ----------------------------------------------------
# Dataset
# ----------------------------------------------------

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

# ----------------------------------------------------
# Storage
# ----------------------------------------------------

y_true = []
y_prob = []

attack_ids = []
vocoders = []
codecs = []
speakers = []
subsets = []
paths = []
```

```
failed_indices = []

processed = 0
failed = 0

model.eval()

start_time = time.time()

# ------------------------------------------------
# Evaluation
# ------------------------------------------------

for i, sample in enumerate(ds):

    try:

        # ---------- metadata ----------
        notes = json.loads(sample["notes"])

        attack_id = notes.get("attack_id", "unknown")
        vocoder = notes.get("vocoder", "unknown")
        codec = notes.get("codec", "unknown")
        speaker = notes.get("speaker_id", "unknown")
        subset = notes.get("subset", "unknown")
        path = sample["path"]

        # ---------- audio ----------
        audio = sample["audio"]

        waveform = audio["array"]
        sr = audio["sampling_rate"]

        label = int(sample["label"])

        # ---------- preprocessing ----------
        feature = preprocess_audio(
            waveform,
            sr
        )

        feature = (
            feature
            .unsqueeze(0)
            .to(device)
        )

        # ---------- inference ----------
        with torch.no_grad():

            logit = model(feature)
```

```
            probability = torch.sigmoid(
                logit
            ).item()

        # ---------- store ----------
        y_true.append(label)
        y_prob.append(probability)

        attack_ids.append(attack_id)
        vocoders.append(vocoder)
        codecs.append(codec)
        speakers.append(speaker)
        subsets.append(subset)
        paths.append(path)

        processed += 1

        # ---------- checkpoint ----------
        if processed % 10000 == 0:

            np.savez(
                checkpoint_path,
                y_true=np.array(y_true),
                y_prob=np.array(y_prob),
                attack_ids=np.array(attack_ids),
                vocoders=np.array(vocoders),
                codecs=np.array(codecs),
                speakers=np.array(speakers),
                subsets=np.array(subsets),
                paths=np.array(paths),
                failed_indices=np.array(
                    failed_indices
                )
            )

            elapsed = time.time() - start_time

            print(
                f"Processed: {processed:,} | "
                f"Failed: {failed:,} | "
                f"Elapsed: {elapsed/60:.1f} min"
            )

    except Exception as e:

        failed += 1
        failed_indices.append(i)

        if failed <= 10:
```

```
            print(
                f"⚠ Failed dataset index: {i}"
            )

            print(
                "   Error:",
                str(e)[:200]
            )

# ------------------------------------------------
# Final save
# ------------------------------------------------

np.savez(
    checkpoint_path,
    y_true=np.array(y_true),
    y_prob=np.array(y_prob),
    attack_ids=np.array(attack_ids),
    vocoders=np.array(vocoders),
    codecs=np.array(codecs),
    speakers=np.array(speakers),
    subsets=np.array(subsets),
    paths=np.array(paths),
    failed_indices=np.array(failed_indices)
)

print("\n========== EVALUATION COMPLETE ==========")
print("Processed:", processed)
print("Failed:", failed)
print("Saved to:", checkpoint_path)
```

{"model_id":"6482ce5af3664e0d954623152270bd6a","version_major":2,"version_minor":0}

{"model_id":"99dc363dd5d04be3a4221e291be95e80","version_major":2,"version_minor":0}

```
Processed: 10,000 | Failed: 0 | Elapsed: 1.3 min
Processed: 20,000 | Failed: 0 | Elapsed: 2.4 min
⚠ Failed dataset index: 20504
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
⚠ Failed dataset index: 26942
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
```

```
⚠ Failed dataset index: 29320
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 30,000 | Failed: 3 | Elapsed: 3.5 min
Processed: 40,000 | Failed: 3 | Elapsed: 4.6 min
Processed: 50,000 | Failed: 3 | Elapsed: 5.6 min
Processed: 60,000 | Failed: 3 | Elapsed: 6.7 min
⚠ Failed dataset index: 61912
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 70,000 | Failed: 4 | Elapsed: 7.8 min
Processed: 80,000 | Failed: 4 | Elapsed: 8.9 min
⚠ Failed dataset index: 84702
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 90,000 | Failed: 5 | Elapsed: 9.9 min
Processed: 100,000 | Failed: 5 | Elapsed: 11.0 min
Processed: 110,000 | Failed: 5 | Elapsed: 12.0 min
Processed: 120,000 | Failed: 5 | Elapsed: 13.5 min
⚠ Failed dataset index: 128812
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 130,000 | Failed: 6 | Elapsed: 14.5 min
Processed: 140,000 | Failed: 6 | Elapsed: 15.6 min
Processed: 150,000 | Failed: 6 | Elapsed: 16.7 min
⚠ Failed dataset index: 153414
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 160,000 | Failed: 7 | Elapsed: 17.8 min
⚠ Failed dataset index: 167807
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 170,000 | Failed: 8 | Elapsed: 18.9 min
⚠ Failed dataset index: 176936
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
```

```
Invalid data found when processing input
Processed: 180,000 | Failed: 9 | Elapsed: 20.0 min
Processed: 190,000 | Failed: 9 | Elapsed: 21.0 min
Processed: 200,000 | Failed: 9 | Elapsed: 22.1 min
Processed: 210,000 | Failed: 9 | Elapsed: 23.2 min
Processed: 220,000 | Failed: 9 | Elapsed: 24.2 min
⚠ Failed dataset index: 220394
   Error: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input
Processed: 230,000 | Failed: 10 | Elapsed: 25.4 min
Processed: 240,000 | Failed: 10 | Elapsed: 26.4 min
Processed: 250,000 | Failed: 11 | Elapsed: 27.5 min
Processed: 260,000 | Failed: 11 | Elapsed: 28.6 min
Processed: 270,000 | Failed: 12 | Elapsed: 29.7 min

print("y_true:", len(y_true))
print("y_prob:", len(y_prob))
print("attack_ids:", len(attack_ids))
print("vocoders:", len(vocoders))
print("codecs:", len(codecs))

print("\nTypes:")
print(type(attack_ids))
print(type(vocoders))
print(type(codecs))

import numpy as np
import os

# ----------------------------------------------------
# 1. Mount Google Drive
# ----------------------------------------------------
from google.colab import drive
drive.mount('/content/drive')

# ----------------------------------------------------
# 2. Results directory
# ----------------------------------------------------
RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"

print("Checking:", RESULTS_DIR)
print(os.listdir(RESULTS_DIR))

# ----------------------------------------------------
# 3. Load saved arrays
# ----------------------------------------------------
y_true = np.load(os.path.join(RESULTS_DIR, "y_true.npy"))
y_prob = np.load(os.path.join(RESULTS_DIR, "y_prob.npy"))
```

```
attack_ids = np.load(os.path.join(RESULTS_DIR, "attack_ids.npy"))
vocoders = np.load(os.path.join(RESULTS_DIR, "vocoders.npy"))
codecs = np.load(os.path.join(RESULTS_DIR, "codecs.npy"))
speakers = np.load(os.path.join(RESULTS_DIR, "speakers.npy"))
subsets = np.load(os.path.join(RESULTS_DIR, "subsets.npy"))
paths = np.load(os.path.join(RESULTS_DIR, "paths.npy"))

# ------------------------------------------------
# 4. Check lengths
# ------------------------------------------------
arrays = {
    "y_true": y_true,
    "y_prob": y_prob,
    "attack_ids": attack_ids,
    "vocoders": vocoders,
    "codecs": codecs,
    "speakers": speakers,
    "subsets": subsets,
    "paths": paths
}

print("\nArray lengths:")
for name, arr in arrays.items():
    print(f"{name:12s}: {len(arr):,}")

# ------------------------------------------------
# 5. Verify alignment
# ------------------------------------------------
lengths = [len(arr) for arr in arrays.values()]

if len(set(lengths)) == 1:
    print("\n ALL ARRAYS ARE ALIGNED")
    print(f"Total samples: {lengths[0]:,}")
else:
    print("\n ARRAY LENGTH MISMATCH")
    print("DO NOT CONTINUE WITH ATTACK ANALYSIS.")

# ------------------------------------------------
# 6. Basic sanity checks
# ------------------------------------------------
print("\nData types:")
for name, arr in arrays.items():
    print(f"{name:12s}: {arr.dtype}")

print("\nLabel distribution:")
unique, counts = np.unique(y_true, return_counts=True)

for label, count in zip(unique, counts):
    print(f"Label {label}: {count:,}")
```

```
print("\nProbability range:")
print("Minimum:", y_prob.min())
print("Maximum:", y_prob.max())
```

```
Mounted at /content/drive
Checking: /content/drive/MyDrive/voice_cloning_project/results
['asvspoof2019_results.txt', '2021_DF_baseline_10000.json',
'asvspoof2021_v2_predictions.npz', 'v2_asvspoof2021_results.json',
'v2_asvspoof2021_predictions.npz', 'v2_2021_detailed_eval.npz']

---------------------------------------------------------------------------
-----
FileNotFoundError                         Traceback (most recent call
last)
/tmp/ipykernel_4544/163378807.py in <cell line: 0>()
     19 # 3. Load saved arrays
     20 # ------------------------------------------------
---> 21 y_true = np.load(os.path.join(RESULTS_DIR, "y_true.npy"))
     22 y_prob = np.load(os.path.join(RESULTS_DIR, "y_prob.npy"))
     23 attack_ids = np.load(os.path.join(RESULTS_DIR,
"attack_ids.npy"))

/usr/local/lib/python3.13/dist-packages/numpy/lib/_npyio_impl.py in
load(file, mmap_mode, allow_pickle, fix_imports, encoding,
max_header_size)
    457             own_fid = False
    458         else:
--> 459             fid = stack.enter_context(open(os.fspath(file),
"rb"))
    460             own_fid = True
    461

FileNotFoundError: [Errno 2] No such file or directory:
'/content/drive/MyDrive/voice_cloning_project/results/y_true.npy'
```

```
import numpy as np
import os

RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"

file_path = os.path.join(
    RESULTS_DIR,
    "v2_2021_detailed_eval.npz"
)

print("Loading:", file_path)

data = np.load(file_path, allow_pickle=True)

print("\nFiles stored inside NPZ:")
print(data.files)
```

```
print("\nArray information:")
for key in data.files:
    arr = data[key]
    print(f"{key:20s} shape={arr.shape}, dtype={arr.dtype}")
```

```
Loading:
/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed_
eval.npz

Files stored inside NPZ:
['y_true', 'y_prob', 'attack_ids', 'vocoders', 'codecs', 'speakers',
'subsets', 'paths', 'failed_indices']

Array information:
y_true               shape=(480000,), dtype=int64
y_prob               shape=(480000,), dtype=float64
attack_ids           shape=(480000,), dtype=<U12
vocoders             shape=(480000,), dtype=<U32
codecs               shape=(480000,), dtype=<U8
speakers             shape=(480000,), dtype=<U7
subsets              shape=(480000,), dtype=<U8
paths                shape=(480000,), dtype=<U17
failed_indices       shape=(17,), dtype=int64
```

```
# ============================================================
# STEP 2 — ALIGNMENT + SANITY CHECK
# ============================================================

# Extract arrays from the loaded NPZ
y_true = data["y_true"]
y_prob = data["y_prob"]
attack_ids = data["attack_ids"]
vocoders = data["vocoders"]
codecs = data["codecs"]
speakers = data["speakers"]
subsets = data["subsets"]
paths = data["paths"]
failed_indices = data["failed_indices"]

# ------------------------------------------------------------
# 1. Check lengths
# ------------------------------------------------------------

arrays = {
    "y_true": y_true,
    "y_prob": y_prob,
    "attack_ids": attack_ids,
    "vocoders": vocoders,
    "codecs": codecs,
```

```
    "speakers": speakers,
    "subsets": subsets,
    "paths": paths
}

print("ARRAY ALIGNMENT CHECK")
print("=" * 50)

lengths = []

for name, arr in arrays.items():
    print(f"{name:15s}: {len(arr):,}")
    lengths.append(len(arr))

if len(set(lengths)) == 1:
    print("\n ALL ARRAYS ARE ALIGNED")
else:
    print("\n ARRAY LENGTH MISMATCH")
    raise ValueError("Arrays are not aligned!")

# ------------------------------------------------------------
# 2. Number of evaluated samples
# ------------------------------------------------------------

N = len(y_true)

print(f"\nTotal evaluated samples: {N:,}")
print(f"Failed indices recorded: {len(failed_indices)}")

# ------------------------------------------------------------
# 3. Label distribution
# ------------------------------------------------------------

print("\nLABEL DISTRIBUTION")
print("=" * 50)

unique_labels, label_counts = np.unique(y_true, return_counts=True)

for label, count in zip(unique_labels, label_counts):
    percentage = count / N * 100
    name = "BONAFIDE" if label == 0 else "SPOOF"
    print(f"Class {label} ({name:8s}): {count:9,} ({percentage:.2f}
%)")

# ------------------------------------------------------------
# 4. Probability sanity check
# ------------------------------------------------------------

print("\nPROBABILITY CHECK")
print("=" * 50)
```

```python
print(f"Minimum probability: {y_prob.min():.8f}")
print(f"Maximum probability: {y_prob.max():.8f}")
print(f"Mean probability:    {y_prob.mean():.8f}")

if np.all((y_prob >= 0) & (y_prob <= 1)):
    print("✓ All probabilities are between 0 and 1")
else:
    print("✗ Invalid probability detected")

# ------------------------------------------------------------
# 5. Metadata examples
# ------------------------------------------------------------

print("\nMETADATA EXAMPLES")
print("=" * 50)

for i in range(min(5, N)):
    print(
        f"\nSample {i}:"
        f"\n  Label     = {y_true[i]}"
        f"\n  Probability = {y_prob[i]:.6f}"
        f"\n  Attack    = {attack_ids[i]}"
        f"\n  Vocoder   = {vocoders[i]}"
        f"\n  Codec     = {codecs[i]}"
        f"\n  Speaker   = {speakers[i]}"
        f"\n  Subset    = {subsets[i]}"
        f"\n  Path      = {paths[i]}"
    )
```

```
ARRAY ALIGNMENT CHECK
==================================================
y_true         : 480,000
y_prob         : 480,000
attack_ids     : 480,000
vocoders       : 480,000
codecs         : 480,000
speakers       : 480,000
subsets        : 480,000
paths          : 480,000

✓ ALL ARRAYS ARE ALIGNED

Total evaluated samples: 480,000
Failed indices recorded: 17

LABEL DISTRIBUTION
==================================================
Class 0 (BONAFIDE):    17,761 (3.70%)
Class 1 (SPOOF   ):   462,239 (96.30%)
```

```
PROBABILITY CHECK
==================================================
Minimum probability: 0.00000007
Maximum probability: 1.00000000
Mean probability:    0.61221773
 All probabilities are between 0 and 1

METADATA EXAMPLES
==================================================

Sample 0:
  Label     = 1
  Probability = 0.759155
  Attack    = A14
  Vocoder   = traditional_vocoder
  Codec     = nocodec
  Speaker   = LA_0023
  Subset    = progress
  Path      = DF_E_2000011.flac

Sample 1:
  Label     = 1
  Probability = 0.000021
  Attack    = Task1-team20
  Vocoder   = neural_vocoder_nonautoregressive
  Codec     = low_m4a
  Speaker   = TEF2
  Subset    = eval
  Path      = DF_E_2000013.flac

Sample 2:
  Label     = 1
  Probability = 0.997268
  Attack    = Task2-team12
  Vocoder   = traditional_vocoder
  Codec     = mp3m4a
  Speaker   = TGF1
  Subset    = eval
  Path      = DF_E_2000024.flac

Sample 3:
  Label     = 1
  Probability = 0.989950
  Attack    = A09
  Vocoder   = traditional_vocoder
  Codec     = mp3m4a
  Speaker   = LA_0043
  Subset    = eval
  Path      = DF_E_2000026.flac
```

```
Sample 4:
  Label     = 1
  Probability = 0.877060
  Attack    = A12
  Vocoder   = neural_vocoder_autoregressive
  Codec     = mp3m4a
  Speaker   = LA_0021
  Subset    = eval
  Path      = DF_E_2000027.flac

# ============================================================
# STEP 3 — ATTACK-WISE ANALYSIS
# ============================================================

import numpy as np

THRESHOLD = 0.5

# ------------------------------------------------------------
# Only spoof samples are relevant for spoof recall
# ------------------------------------------------------------

spoof_mask = (y_true == 1)

spoof_attack_ids = attack_ids[spoof_mask]
spoof_probs = y_prob[spoof_mask]

# Prediction:
# probability >= 0.5 -> spoof
spoof_predictions = spoof_probs >= THRESHOLD

# ------------------------------------------------------------
# Find unique attacks
# ------------------------------------------------------------

unique_attacks = np.unique(spoof_attack_ids)

results = []

for attack in unique_attacks:

    mask = (spoof_attack_ids == attack)

    probs = spoof_probs[mask]
    predictions = spoof_predictions[mask]

    total = len(probs)

    detected = np.sum(predictions)

    false_negatives = total - detected
```

```
    recall = detected / total if total > 0 else 0

    avg_probability = np.mean(probs)

    median_probability = np.median(probs)

    results.append({
        "Attack ID": attack,
        "Samples": total,
        "Detected Spoof": detected,
        "False Negatives": false_negatives,
        "Spoof Recall": recall,
        "Avg Spoof Probability": avg_probability,
        "Median Spoof Probability": median_probability
    })

# ------------------------------------------------------------
# Sort by recall: WORST -> BEST
# ------------------------------------------------------------

results = sorted(
    results,
    key=lambda x: x["Spoof Recall"]
)

# ------------------------------------------------------------
# Print table
# ------------------------------------------------------------

print("=" * 110)
print("ATTACK-WISE SPOOF DETECTION PERFORMANCE")
print("=" * 110)

print(
    f"{'Attack':<18}"
    f"{'Samples':>10}"
    f"{'Detected':>12}"
    f"{'FN':>12}"
    f"{'Recall':>12}"
    f"{'Avg Prob':>15}"
    f"{'Median Prob':>15}"
)

print("-" * 110)

for r in results:

    print(
        f"{r['Attack ID']:<18}"
```

```
        f"{r['Samples']:>10,}"
        f"{r['Detected Spoof']:>12,}"
        f"{r['False Negatives']:>12,}"
        f"{r['Spoof Recall']*100:>11.2f}%"
        f"{r['Avg Spoof Probability']:>15.4f}"
        f"{r['Median Spoof Probability']:>15.4f}"
    )

print("=" * 110)

# ------------------------------------------------------------
# Summary
# ------------------------------------------------------------

print(f"\nTotal unique spoof attacks: {len(results)}")

print("\nWORST 5 ATTACKS")
print("-" * 60)

for r in results[:5]:
    print(
        f"{r['Attack ID']:18s} "
        f"Recall={r['Spoof Recall']*100:6.2f}% | "
        f"Samples={r['Samples']:8,} | "
        f"FN={r['False Negatives']:8,}"
    )

print("\nBEST 5 ATTACKS")
print("-" * 60)

for r in results[-5:][::-1]:
    print(
        f"{r['Attack ID']:18s} "
        f"Recall={r['Spoof Recall']*100:6.2f}% | "
        f"Samples={r['Samples']:8,} | "
        f"FN={r['False Negatives']:8,}"
    )
```

```
==============================================================================
==========================================
ATTACK-WISE SPOOF DETECTION PERFORMANCE
==============================================================================
==========================================
Attack               Samples    Detected          FN      Recall
Avg Prob    Median Prob
------------------------------------------------------------------------------
------------------------------------------
Task1-team04           2,814         761       2,053      27.04%
0.2677         0.0022
Task1-team10           2,812         851       1,961      30.26%
```

0.3031	0.0039				
Task1-team25		2,853	865	1,988	30.32%
0.2929	0.0035				
Task2-team15		4,214	1,324	2,890	31.42%
0.3013	0.0011				
Task1-team20		2,845	894	1,951	31.42%
0.3033	0.0038				
Task1-team32		2,829	919	1,910	32.48%
0.3107	0.0022				
Task2-team20		4,246	1,387	2,859	32.67%
0.3240	0.0044				
Task1-team07		2,889	944	1,945	32.68%
0.3145	0.0024				
Task1-team21		2,801	920	1,881	32.85%
0.3211	0.0041				
Task1-team22		2,785	926	1,859	33.25%
0.3209	0.0046				
Task1-team01		2,800	951	1,849	33.96%
0.3298	0.0089				
Task1-team23		2,817	959	1,858	34.04%
0.3325	0.0051				
Task2-team23		4,197	1,447	2,750	34.48%
0.3387	0.0031				
Task1-team16		2,833	980	1,853	34.59%
0.3407	0.0073				
Task2-team07		4,221	1,472	2,749	34.87%
0.3452	0.0059				
Task2-team11		4,271	1,498	2,773	35.07%
0.3575	0.0089				
Task2-team33		4,201	1,476	2,725	35.13%
0.3423	0.0039				
Task1-team11		2,783	984	1,799	35.36%
0.3468	0.0030				
Task2-team19		4,264	1,514	2,750	35.51%
0.3524	0.0079				
Task2-team32		4,261	1,515	2,746	35.56%
0.3560	0.0062				
Task2-team26		4,220	1,511	2,709	35.81%
0.3606	0.0494				
Task1-team33		2,860	1,025	1,835	35.84%
0.3473	0.0047				
Task1-team29		2,819	1,014	1,805	35.97%
0.3511	0.0063				
Task1-team19		2,785	1,005	1,780	36.09%
0.3482	0.0038				
A12		8,590	3,101	5,489	36.10%
0.3574	0.0318				
Task1-team26		2,800	1,015	1,785	36.25%
0.3455	0.0126				

Task1-team17		2,828	1,036	1,792	36.63%
0.3546	0.0022				
Task2-team29		4,282	1,608	2,674	37.55%
0.3791	0.0302				
Task2-team16		4,195	1,577	2,618	37.59%
0.3809	0.0276				
A13		8,879	3,370	5,509	37.95%
0.3701	0.0271				
A15		8,641	3,307	5,334	38.27%
0.3902	0.0527				
Task2-team10		4,183	1,601	2,582	38.27%
0.3786	0.0218				
Task2-team25		4,256	1,638	2,618	38.49%
0.3739	0.0233				
Task1-team14		2,839	1,094	1,745	38.53%
0.3916	0.0144				
Task1-team08		2,801	1,137	1,664	40.59%
0.4015	0.0460				
Task2-team22		4,213	1,716	2,497	40.73%
0.4039	0.0575				
A11		8,818	3,674	5,144	41.66%
0.4146	0.2401				
Task2-team08		4,192	1,772	2,420	42.27%
0.4205	0.0423				
SPO-N17		3,989	1,742	2,247	43.67%
0.4359	0.1618				
A10		8,821	3,859	4,962	43.75%
0.4294	0.2133				
Task1-team13		2,786	1,267	1,519	45.48%
0.4459	0.2176				
SPO-N13		3,959	1,834	2,125	46.32%
0.4730	0.2902				
HUB-N17		3,940	1,915	2,025	48.60%
0.4938	0.4186				
A17		8,610	4,226	4,384	49.08%
0.4927	0.4506				
HUB-N13		4,029	2,015	2,014	50.01%
0.5097	0.5069				
A14		8,772	4,446	4,326	50.68%
0.5109	0.5406				
Task2-team13		4,247	2,156	2,091	50.77%
0.5077	0.5556				
A09		8,537	4,401	4,136	51.55%
0.5258	0.5555				
SPO-N05		3,939	2,033	1,906	51.61%
0.5137	0.5982				
HUB-N06		3,986	2,120	1,866	53.19%
0.5378	0.6260				
HUB-D02		3,934	2,114	1,820	53.74%

0.5425	0.6923				
HUB-N10		3,980	2,156	1,824	54.17%
0.5502	0.7597				
HUB-B00		495	270	225	54.55%
0.5691	0.7167				
A18		8,472	4,631	3,841	54.66%
0.5413	0.8156				
SPO-N10		3,974	2,173	1,801	54.68%
0.5516	0.7803				
HUB-N12		3,959	2,198	1,761	55.52%
0.5523	0.7410				
SPO-B00		280	160	120	57.14%
0.5842	0.8571				
SPO-N06		3,994	2,303	1,691	57.66%
0.5796	0.8149				
SPO-N12		3,990	2,330	1,660	58.40%
0.5852	0.8375				
HUB-N05		3,991	2,347	1,644	58.81%
0.5895	0.8352				
SPO-B01		3,868	2,452	1,416	63.39%
0.6279	0.9419				
HUB-B01		4,013	2,544	1,469	63.39%
0.6305	0.9374				
Task2-team05		4,202	2,777	1,425	66.09%
0.6657	0.9766				
HUB-N20		3,904	2,675	1,229	68.52%
0.6789	0.9555				
HUB-N14		3,953	2,720	1,233	68.81%
0.6787	0.9673				
HUB-D03		3,957	2,730	1,227	68.99%
0.6808	0.9506				
Task1-team31		2,802	1,944	858	69.38%
0.6959	0.9184				
HUB-D01		3,989	2,796	1,193	70.09%
0.6955	0.9579				
Task2-team18		4,194	3,095	1,099	73.80%
0.7367	0.9915				
Task1-team27		2,805	2,079	726	74.12%
0.7332	0.9866				
HUB-N08		3,978	3,006	972	75.57%
0.7471	0.9895				
Task1-team06		2,817	2,130	687	75.61%
0.7510	0.9926				
Task1-team24		2,775	2,105	670	75.86%
0.7472	0.9826				
Task2-team24		4,281	3,325	956	77.67%
0.7699	0.9895				
Task2-team27		4,187	3,278	909	78.29%
0.7776	0.9963				

Task1-team02		2,853	2,252	601	78.93%
0.7808	0.9946				
SPO-N18		3,948	3,128	820	79.23%
0.7880	0.9948				
Task2-team02		4,243	3,403	840	80.20%
0.7909	0.9937				
HUB-N18		4,009	3,223	786	80.39%
0.7973	0.9945				
Task2-team06		4,222	3,410	812	80.77%
0.8046	0.9984				
Task2-team12		4,234	3,468	766	81.91%
0.8141	0.9984				
Task1-team12		2,806	2,306	500	82.18%
0.8212	0.9989				
Task1-team18		2,769	2,436	333	87.97%
0.8658	0.9977				
Task2-team31		4,246	3,866	380	91.05%
0.8895	0.9979				
Task1-team28		2,827	2,582	245	91.33%
0.9036	1.0000				
Task1-team30		2,869	2,694	175	93.90%
0.9258	0.9999				
A19		8,584	8,074	510	94.06%
0.9236	0.9999				
HUB-N15		3,936	3,731	205	94.79%
0.9391	0.9997				
HUB-N03		3,913	3,714	199	94.91%
0.9411	0.9997				
HUB-D05		3,912	3,739	173	95.58%
0.9487	0.9998				
HUB-N07		3,975	3,808	167	95.80%
0.9491	0.9998				
HUB-N04		3,969	3,803	166	95.82%
0.9404	0.9991				
SPO-N03		3,960	3,839	121	96.94%
0.9610	0.9998				
Task2-team28		4,257	4,132	125	97.06%
0.9537	1.0000				
A16		8,851	8,612	239	97.30%
0.9560	0.9999				
Task1-team09		2,833	2,773	60	97.88%
0.9615	1.0000				
A07		8,701	8,521	180	97.93%
0.9648	1.0000				
SPO-N11		3,964	3,887	77	98.06%
0.9602	0.9988				
SPO-N04		3,951	3,881	70	98.23%
0.9730	0.9999				
Task2-team30		4,213	4,148	65	98.46%

```
0.9714          1.0000
Task1-team03            2,837       2,795           42       98.52%
0.9707          0.9996
A08                     8,715       8,592          123       98.59%
0.9738          0.9999
Task2-team09            4,281       4,222           59       98.62%
0.9743          1.0000
HUB-N11                 3,967       3,920           47       98.82%
0.9671          0.9984
HUB-N19                 3,946       3,904           42       98.94%
0.9814          0.9999
SPO-N16                 3,969       3,967            2       99.95%
0.9973          1.0000
HUB-D04                 4,002       4,000            2       99.95%
0.9968          1.0000
HUB-N09                 3,950       3,949            1       99.97%
0.9974          1.0000
Task2-team03            4,224       4,223            1       99.98%
0.9935          0.9999
HUB-N16                 3,957       3,957            0      100.00%
0.9982          1.0000
================================================================================
========================================

Total unique spoof attacks: 110

WORST 5 ATTACKS
------------------------------------------------------------
Task1-team04       Recall= 27.04% | Samples=   2,814 | FN=   2,053
Task1-team10       Recall= 30.26% | Samples=   2,812 | FN=   1,961
Task1-team25       Recall= 30.32% | Samples=   2,853 | FN=   1,988
Task2-team15       Recall= 31.42% | Samples=   4,214 | FN=   2,890
Task1-team20       Recall= 31.42% | Samples=   2,845 | FN=   1,951

BEST 5 ATTACKS
------------------------------------------------------------
HUB-N16            Recall=100.00% | Samples=   3,957 | FN=       0
Task2-team03       Recall= 99.98% | Samples=   4,224 | FN=       1
HUB-N09            Recall= 99.97% | Samples=   3,950 | FN=       1
HUB-D04            Recall= 99.95% | Samples=   4,002 | FN=       2
SPO-N16            Recall= 99.95% | Samples=   3,969 | FN=       2
```

```python
import numpy as np

# Load saved detailed evaluation
RESULTS_PATH = "/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed_eval.npz"

data = np.load(RESULTS_PATH, allow_pickle=True)
```

```python
y_true = data["y_true"]
y_prob = data["y_prob"]
attack_ids = data["attack_ids"]
vocoders = data["vocoders"]

THRESHOLD = 0.5

# Only spoof samples
spoof_mask = (y_true == 1)

attack = attack_ids[spoof_mask]
vocoder = vocoders[spoof_mask]
prob = y_prob[spoof_mask]

unique_vocoders = np.unique(vocoder)

print("=" * 100)
print("VOCODER-WISE SPOOF DETECTION PERFORMANCE")
print("=" * 100)

results = []

for v in unique_vocoders:

    mask = (vocoder == v)

    probs_v = prob[mask]

    detected = np.sum(probs_v >= THRESHOLD)
    total = len(probs_v)
    fn = total - detected

    recall = detected / total if total > 0 else 0
    avg_prob = np.mean(probs_v)
    median_prob = np.median(probs_v)

    results.append(
        (
            v,
            total,
            detected,
            fn,
            recall,
            avg_prob,
            median_prob
        )
    )

# Sort worst → best recall
results.sort(key=lambda x: x[4])
```

```python
print(
    f"{'Vocoder':35s}"
    f"{'Samples':>10s}"
    f"{'Detected':>10s}"
    f"{'FN':>10s}"
    f"{'Recall':>12s}"
    f"{'Avg Prob':>12s}"
    f"{'Median':>12s}"
)

print("-" * 100)

for r in results:
    v, total, detected, fn, recall, avg_prob, median_prob = r

    print(
        f"{str(v):35s}"
        f"{total:10d}"
        f"{detected:10d}"
        f"{fn:10d}"
        f"{recall*100:11.2f}%"
        f"{avg_prob:12.4f}"
        f"{median_prob:12.4f}"
    )

print("=" * 100)
print(f"Total vocoders: {len(results)}")
```

```
====================================================================================================
VOCODER-WISE SPOOF DETECTION PERFORMANCE
====================================================================================================
Vocoder                                Samples  Detected        FN      Recall    Avg Prob      Median
----------------------------------------------------------------------------------------------------
neural_vocoder_autoregressive           119639     57955     61684      48.44%      0.4809      0.3776
neural_vocoder_nonautoregressive         94693     53970     40723      56.99%      0.5623      0.8738
traditional_vocoder                     216363    149206     67157      68.96%      0.6853      0.9844
unknown                                  22693     20356      2337      89.70%      0.8888      0.9996
waveform_concatenation                    8851      8612       239      97.30%      0.9560      0.9999
====================================================================================================
```

```
==============================
Total vocoders: 5

import numpy as np

RESULTS_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed
_eval.npz"

data = np.load(RESULTS_PATH, allow_pickle=True)

y_true = data["y_true"]
y_prob = data["y_prob"]
codecs = data["codecs"]

THRESHOLD = 0.5

# Only spoof samples
spoof_mask = (y_true == 1)

codec = codecs[spoof_mask]
prob = y_prob[spoof_mask]

unique_codecs = np.unique(codec)

print("=" * 100)
print("CODEC-WISE SPOOF DETECTION PERFORMANCE")
print("=" * 100)

results = []

for c in unique_codecs:

    mask = (codec == c)

    probs_c = prob[mask]

    detected = np.sum(probs_c >= THRESHOLD)
    total = len(probs_c)
    fn = total - detected

    recall = detected / total if total > 0 else 0
    avg_prob = np.mean(probs_c)
    median_prob = np.median(probs_c)

    results.append(
        (
            c,
            total,
            detected,
            fn,
```

```
            recall,
            avg_prob,
            median_prob
        )
    )

# Worst → best
results.sort(key=lambda x: x[4])

print(
    f"{'Codec':20s}"
    f"{'Samples':>10s}"
    f"{'Detected':>10s}"
    f"{'FN':>10s}"
    f"{'Recall':>12s}"
    f"{'Avg Prob':>12s}"
    f"{'Median':>12s}"
)

print("-" * 100)

for r in results:

    c, total, detected, fn, recall, avg_prob, median_prob = r

    print(
        f"{str(c):20s}"
        f"{total:10d}"
        f"{detected:10d}"
        f"{fn:10d}"
        f"{recall*100:11.2f}%"
        f"{avg_prob:12.4f}"
        f"{median_prob:12.4f}"
    )

print("=" * 100)
print(f"Total codecs: {len(results)}")
```

```
====================================================================================================
CODEC-WISE SPOOF DETECTION PERFORMANCE
====================================================================================================
Codec                Samples  Detected        FN      Recall    Avg Prob      Median
----------------------------------------------------------------------------------------------------
high_m4a               51318     19206     32112      37.43%      0.3815      0.0344
high_ogg               51296     19639     31657      38.29%
```

```
0.3898      0.0434
low_m4a                 51397     19731     31666      38.39%
0.3913      0.0476
nocodec                 51395     20644     30751      40.17%
0.4075      0.0548
oggm4a                  51288     28241     23047      55.06%
0.5544      0.7681
low_ogg                 51409     37047     14362      72.06%
0.7198      0.9966
high_mp3                51280     47806      3474      93.23%
0.9077      0.9950
mp3m4a                  51512     48129      3383      93.43%
0.9023      0.9894
low_mp3                 51344     49656      1688      96.71%
0.9463      0.9974
================================================================================
==============================
Total codecs: 9

import numpy as np

RESULTS_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed
_eval.npz"

data = np.load(RESULTS_PATH, allow_pickle=True)

y_true = data["y_true"]
y_prob = data["y_prob"]
vocoders = data["vocoders"]
codecs = data["codecs"]

THRESHOLD = 0.5

# Only spoof samples
spoof_mask = (y_true == 1)

vocoder = vocoders[spoof_mask]
codec = codecs[spoof_mask]
prob = y_prob[spoof_mask]

unique_vocoders = np.unique(vocoder)
unique_codecs = np.unique(codec)

print("=" * 120)
print("VOCODER × CODEC SPOOF DETECTION PERFORMANCE")
print("=" * 120)

for v in unique_vocoders:
```

```
    print("\n" + "-" * 120)
    print(f"VOCODER: {v}")
    print("-" * 120)

    print(
        f"{'Codec':20s}"
        f"{'Samples':>10s}"
        f"{'Detected':>10s}"
        f"{'FN':>10s}"
        f"{'Recall':>12s}"
        f"{'Avg Prob':>12s}"
        f"{'Median':>12s}"
    )

    print("-" * 100)

    for c in unique_codecs:

        mask = (vocoder == v) & (codec == c)

        total = np.sum(mask)

        if total == 0:
            continue

        probs = prob[mask]

        detected = np.sum(probs >= THRESHOLD)
        fn = total - detected

        recall = detected / total
        avg_prob = np.mean(probs)
        median_prob = np.median(probs)

        print(
            f"{str(c):20s}"
            f"{total:10d}"
            f"{detected:10d}"
            f"{fn:10d}"
            f"{recall*100:11.2f}%"
            f"{avg_prob:12.4f}"
            f"{median_prob:12.4f}"
        )

print("\n" + "=" * 120)
print("ANALYSIS COMPLETE")
print("=" * 120)
```

```
============================================================================================================
VOCODER × CODEC SPOOF DETECTION PERFORMANCE
```

```
================================================================================
================================================

--------------------------------------------------------------------------------
--------------------------------------------------
VOCODER: neural_vocoder_autoregressive
--------------------------------------------------------------------------------
--------------------------------------------------
Codec                   Samples  Detected        FN      Recall    Avg
Prob      Median
--------------------------------------------------------------------------------
------------------------------
high_m4a                  13200      2038     11162      15.44%
0.1655      0.0018
high_mp3                  13258     12204      1054      92.05%
0.8887      0.9880
high_ogg                  13246      2029     11217      15.32%
0.1652      0.0020
low_m4a                   13403      2466     10937      18.40%
0.1954      0.0027
low_mp3                   13298     12852       446      96.65%
0.9432      0.9953
low_ogg                   13274      7395      5879      55.71%
0.5625      0.7385
mp3m4a                    13354     12402       952      92.87%
0.8893      0.9806
nocodec                   13300      2044     11256      15.37%
0.1649      0.0017
oggm4a                    13306      4525      8781      34.01%
0.3519      0.0544

--------------------------------------------------------------------------------
--------------------------------------------------
VOCODER: neural_vocoder_nonautoregressive
--------------------------------------------------------------------------------
--------------------------------------------------
Codec                   Samples  Detected        FN      Recall    Avg
Prob      Median
--------------------------------------------------------------------------------
------------------------------
high_m4a                  10568      3321      7247      31.43%
0.3181      0.0037
high_mp3                  10472      9481       991      90.54%
0.8784      0.9908
high_ogg                  10484      3708      6776      35.37%
0.3533      0.0045
low_m4a                   10447      3147      7300      30.12%
0.3065      0.0038
low_mp3                   10561     10208       353      96.66%
```

```
0.9405      0.9960
low_ogg                 10556      5883      4673      55.73%
0.5655      0.8972
mp3m4a                  10610      9693       917      91.36%
0.8775      0.9822
nocodec                 10512      3979      6533      37.85%
0.3779      0.0049
oggm4a                  10483      4550      5933      43.40%
0.4383      0.1146

------------------------------------------------------------------------
-------------------------------------------------
VOCODER: traditional_vocoder
------------------------------------------------------------------------
-------------------------------------------------
Codec                  Samples  Detected        FN      Recall    Avg
Prob      Median
------------------------------------------------------------------------
-------------------------------
high_m4a                24037     10814     13223      44.99%
0.4580      0.2468
high_mp3                24034     22840      1194      95.03%
0.9312      0.9981
high_ogg                24036     10958     13078      45.59%
0.4644      0.2784
low_m4a                 24044     11004     13040      45.77%
0.4653      0.2836
low_mp3                 23947     23132       815      96.60%
0.9488      0.9986
low_ogg                 24094     20317      3777      84.32%
0.8351      0.9995
mp3m4a                  24049     22699      1350      94.39%
0.9178      0.9945
nocodec                 24133     11642     12491      48.24%
0.4895      0.3966
oggm4a                  23989     15800      8189      65.86%
0.6587      0.9647

------------------------------------------------------------------------
-------------------------------------------------
VOCODER: unknown
------------------------------------------------------------------------
-------------------------------------------------
Codec                  Samples  Detected        FN      Recall    Avg
Prob      Median
------------------------------------------------------------------------
-------------------------------
high_m4a                 2534      2055       479      81.10%
0.8083      0.9989
```

```
high_mp3                  2526      2432        94      96.28%
0.9464      0.9990
high_ogg                  2531      1946       585      76.89%
0.7669      0.9964
low_m4a                   2527      2138       389      84.61%
0.8412      0.9997
low_mp3                   2541      2492        49      98.07%
0.9667      0.9992
low_ogg                   2517      2484        33      98.69%
0.9859      1.0000
mp3m4a                    2519      2424        95      96.23%
0.9391      0.9963
nocodec                   2468      1998       470      80.96%
0.8040      0.9989
oggm4a                    2530      2387       143      94.35%
0.9393      1.0000

--------------------------------------------------------------------------------
------------------------------------------------
VOCODER: waveform_concatenation
--------------------------------------------------------------------------------
------------------------------------------------
Codec                   Samples  Detected        FN      Recall    Avg
Prob      Median
--------------------------------------------------------------------------------
------------------------------
high_m4a                   979       978         1      99.90%
0.9979      1.0000
high_mp3                   990       849       141      85.76%
0.8042      0.9294
high_ogg                   999       998         1      99.90%
0.9984      1.0000
low_m4a                    976       976         0     100.00%
0.9993      1.0000
low_mp3                    997       972        25      97.49%
0.9374      0.9892
low_ogg                    968       968         0     100.00%
0.9998      1.0000
mp3m4a                     980       911        69      92.96%
0.8713      0.9559
nocodec                    982       981         1      99.90%
0.9983      1.0000
oggm4a                     980       979         1      99.90%
0.9993      1.0000

================================================================================
================================================
ANALYSIS COMPLETE
```

```
================================================================================
================================================

import numpy as np

RESULTS_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed
_eval.npz"

data = np.load(RESULTS_PATH, allow_pickle=True)

y_true = data["y_true"]
y_prob = data["y_prob"]
vocoders = data["vocoders"]
codecs = data["codecs"]

THRESHOLD = 0.5

# All spoof samples
spoof_mask = (y_true == 1)

vocoder = vocoders[spoof_mask]
codec = codecs[spoof_mask]
prob = y_prob[spoof_mask]

# Total false negatives
fn_mask = (prob < THRESHOLD)
total_fn = np.sum(fn_mask)

unique_vocoders = np.unique(vocoder)
unique_codecs = np.unique(codec)

results = []

for v in unique_vocoders:
    for c in unique_codecs:

        mask = (
            (vocoder == v) &
            (codec == c)
        )

        total = np.sum(mask)

        if total == 0:
            continue

        fn = np.sum(mask & fn_mask)

        detected = total - fn
        recall = detected / total
```

```
        fn_percentage = (fn / total_fn) * 100

        results.append(
            (
                v,
                c,
                total,
                detected,
                fn,
                recall,
                fn_percentage
            )
        )

# Sort by contribution to total FN
results.sort(key=lambda x: x[6], reverse=True)

print("=" * 120)
print("FALSE-NEGATIVE CONTRIBUTION BY VOCODER × CODEC")
print("=" * 120)

print(f"\nTotal spoof samples : {len(prob):,}")
print(f"Total false negatives: {total_fn:,}\n")

print(
    f"{'Vocoder':35s}"
    f"{'Codec':15s}"
    f"{'Samples':>10s}"
    f"{'Detected':>10s}"
    f"{'FN':>10s}"
    f"{'Recall':>10s}"
    f"{'% All FN':>12s}"
)

print("-" * 120)

for r in results:

    v, c, total, detected, fn, recall, fn_percentage = r

    print(
        f"{str(v):35s}"
        f"{str(c):15s}"
        f"{total:10d}"
        f"{detected:10d}"
        f"{fn:10d}"
        f"{recall*100:9.2f}%"
        f"{fn_percentage:11.2f}%"
    )
```

```
print("=" * 120)
```

```
========================================================================================================
FALSE-NEGATIVE CONTRIBUTION BY VOCODER × CODEC
========================================================================================================

Total spoof samples : 462,239
Total false negatives: 172,140

Vocoder                              Codec              Samples  Detected     FN    Recall    % All FN
--------------------------------------------------------------------------------------------------------
traditional_vocoder                  high_m4a             24037     10814  13223    44.99%       7.68%
traditional_vocoder                  high_ogg             24036     10958  13078    45.59%       7.60%
traditional_vocoder                  low_m4a              24044     11004  13040    45.77%       7.58%
traditional_vocoder                  nocodec              24133     11642  12491    48.24%       7.26%
neural_vocoder_autoregressive        nocodec              13300      2044  11256    15.37%       6.54%
neural_vocoder_autoregressive        high_ogg             13246      2029  11217    15.32%       6.52%
neural_vocoder_autoregressive        high_m4a             13200      2038  11162    15.44%       6.48%
neural_vocoder_autoregressive        low_m4a              13403      2466  10937    18.40%       6.35%
neural_vocoder_autoregressive        oggm4a               13306      4525   8781    34.01%       5.10%
traditional_vocoder                  oggm4a               23989     15800   8189    65.86%       4.76%
neural_vocoder_nonautoregressive     low_m4a              10447      3147   7300    30.12%       4.24%
neural_vocoder_nonautoregressive     high_m4a             10568      3321   7247    31.43%       4.21%
neural_vocoder_nonautoregressive     high_ogg             10484      3708   6776    35.37%       3.94%
neural_vocoder_nonautoregressive     nocodec              10512      3979   6533    37.85%       3.80%
neural_vocoder_nonautoregressive     oggm4a               10483      4550   5933    43.40%       3.45%
neural_vocoder_autoregressive        low_ogg              13274      7395   5879    55.71%       3.42%
neural_vocoder_nonautoregressive     low_ogg              10556      5883   4673    55.73%       2.71%
```

```
traditional_vocoder                 low_ogg              24094     20317
3777    84.32%       2.19%
traditional_vocoder                 mp3m4a               24049     22699
1350    94.39%       0.78%
traditional_vocoder                 high_mp3             24034     22840
1194    95.03%       0.69%
neural_vocoder_autoregressive       high_mp3             13258     12204
1054    92.05%       0.61%
neural_vocoder_nonautoregressive    high_mp3             10472      9481
991    90.54%       0.58%
neural_vocoder_autoregressive       mp3m4a               13354     12402
952    92.87%       0.55%
neural_vocoder_nonautoregressive    mp3m4a               10610      9693
917    91.36%       0.53%
traditional_vocoder                 low_mp3              23947     23132
815    96.60%       0.47%
unknown                             high_ogg              2531      1946
585    76.89%       0.34%
unknown                             high_m4a              2534      2055
479    81.10%       0.28%
unknown                             nocodec               2468      1998
470    80.96%       0.27%
neural_vocoder_autoregressive       low_mp3              13298     12852
446    96.65%       0.26%
unknown                             low_m4a               2527      2138
389    84.61%       0.23%
neural_vocoder_nonautoregressive    low_mp3              10561     10208
353    96.66%       0.21%
unknown                             oggm4a                2530      2387
143    94.35%       0.08%
waveform_concatenation              high_mp3               990       849
141    85.76%       0.08%
unknown                             mp3m4a                2519      2424
95    96.23%       0.06%
unknown                             high_mp3              2526      2432
94    96.28%       0.05%
waveform_concatenation              mp3m4a                 980       911
69    92.96%       0.04%
unknown                             low_mp3               2541      2492
49    98.07%       0.03%
unknown                             low_ogg               2517      2484
33    98.69%       0.02%
waveform_concatenation              low_mp3                997       972
25    97.49%       0.01%
waveform_concatenation              high_m4a               979       978
1    99.90%       0.00%
waveform_concatenation              high_ogg               999       998
1    99.90%       0.00%
waveform_concatenation              nocodec                982       981
```

```
1    99.90%       0.00%
waveform_concatenation             oggm4a                980       979
1    99.90%       0.00%
waveform_concatenation             low_m4a               976       976
0   100.00%       0.00%
waveform_concatenation             low_ogg               968       968
0   100.00%       0.00%
================================================================================
================================================
```

```python
import numpy as np

# Load existing detailed evaluation
path = "/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed_eval.npz"

data = np.load(path, allow_pickle=True)

y_true = data["y_true"]
y_prob = data["y_prob"]
vocoders = data["vocoders"]
codecs = data["codecs"]

# Only spoof samples
mask = (y_true == 1)

p = y_prob[mask]
v = vocoders[mask]
c = codecs[mask]

print("Total spoof samples:", len(p))

# Probability regions
regions = [
    ("Very low (<0.10)", p < 0.10),
    ("Low (0.10–0.30)", (p >= 0.10) & (p < 0.30)),
    ("Borderline (0.30–0.50)", (p >= 0.30) & (p < 0.50)),
    ("Just above threshold (0.50–0.70)", (p >= 0.50) & (p < 0.70)),
    ("High (0.70–0.90)", (p >= 0.70) & (p < 0.90)),
    ("Very high (>=0.90)", p >= 0.90),
]

print("\n=== SPOOF PROBABILITY DISTRIBUTION ===")

for name, r in regions:
    count = np.sum(r)
    pct = count / len(p) * 100

    print(f"{name:35s}: {count:8d} ({pct:6.2f}%)")
```

```
Total spoof samples: 462239

=== SPOOF PROBABILITY DISTRIBUTION ===
Very low (<0.10)                    :   140395 ( 30.37%)
Low (0.10–0.30)                     :    19057 (  4.12%)
Borderline (0.30–0.50)              :    12688 (  2.74%)
Just above threshold (0.50–0.70)    :    13797 (  2.98%)
High (0.70–0.90)                    :    25515 (  5.52%)
Very high (>=0.90)                  :   250787 ( 54.25%)
```

```
print("\n=== CODEC ANALYSIS ===")

unique_codecs = np.unique(c)

for codec in unique_codecs:
    m = (c == codec)
    probs = p[m]

    recall = np.mean(probs >= 0.5)

    print(
        f"{codec:12s} | "
        f"N={len(probs):6d} | "
        f"Recall={recall*100:6.2f}% | "
        f"Mean={np.mean(probs):.4f} | "
        f"Median={np.median(probs):.4f} | "
        f"<0.1={np.mean(probs < 0.1)*100:6.2f}%"
    )
```

```
=== CODEC ANALYSIS ===
high_m4a     | N= 51318 | Recall= 37.43% | Mean=0.3815 | Median=0.0344
| <0.1= 54.73%
high_mp3     | N= 51280 | Recall= 93.23% | Mean=0.9077 | Median=0.9950
| <0.1=  1.98%
high_ogg     | N= 51296 | Recall= 38.29% | Mean=0.3898 | Median=0.0434
| <0.1= 53.77%
low_m4a      | N= 51397 | Recall= 38.39% | Mean=0.3913 | Median=0.0476
| <0.1= 53.52%
low_mp3      | N= 51344 | Recall= 96.71% | Mean=0.9463 | Median=0.9974
| <0.1=  0.86%
low_ogg      | N= 51409 | Recall= 72.06% | Mean=0.7198 | Median=0.9966
| <0.1= 19.27%
mp3m4a       | N= 51512 | Recall= 93.43% | Mean=0.9023 | Median=0.9894
| <0.1=  1.75%
nocodec      | N= 51395 | Recall= 40.17% | Mean=0.4075 | Median=0.0548
| <0.1= 52.49%
oggm4a       | N= 51288 | Recall= 55.06% | Mean=0.5544 | Median=0.7681
| <0.1= 35.06%
```

```
import os

for root in [
    "/content/drive/MyDrive",
]:
    print("\nROOT:", root)

    for name in os.listdir(root):
        if "wave" in name.lower() or "dfadd" in name.lower():
            print("FOUND:", name)
```

```
ROOT: /content/drive/MyDrive
```

```
import os

search_root = "/content/drive/MyDrive"

targets = ["wavefake", "dfadd"]

for root, dirs, files in os.walk(search_root):
    for d in dirs:
        if any(t in d.lower() for t in targets):
            print("FOUND FOLDER:", os.path.join(root, d))
```

```
import os

root = "/content/drive/MyDrive"

for dirpath, dirnames, filenames in os.walk(root):
    # Print directories that look related to ASVspoof 2021
    if "2021" in dirpath.lower() or "df" in dirpath.lower():
        print(dirpath)

for dirpath, dirnames, filenames in os.walk(root):
    for f in filenames:
        if "2021" in f.lower() or "df" in f.lower():
            print(os.path.join(dirpath, f))
```

```
/content/drive/MyDrive/avanish cv.pdf
/content/drive/MyDrive/Adityaoops.pdf
/content/drive/MyDrive/voice_cloning_project/results/2021_DF_baseline_
10000.json
/content/drive/MyDrive/voice_cloning_project/results/asvspoof2021_v2_p
redictions.npz
/content/drive/MyDrive/voice_cloning_project/results/v2_asvspoof2021_r
esults.json
/content/drive/MyDrive/voice_cloning_project/results/v2_asvspoof2021_p
redictions.npz
/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed_
eval.npz
```

```
!pip -q install datasets soundfile librosa

from datasets import load_dataset

wavefake = load_dataset("DeepFense/WaveFake")

print(wavefake)
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"a9404ecb3c414be99af4a06f4ab28a50","version_major":2,"vers
ion_minor":0}

{"model_id":"a60d0cc14a9b46389a3b9af121979db8","version_major":2,"vers
ion_minor":0}

{"model_id":"5cbce3ada5bd4842b435531fe520f07a","version_major":2,"vers
ion_minor":0}

{"model_id":"827cdc2ee5dd44f5b42baa19e585c4b0","version_major":2,"vers
ion_minor":0}
```


```
DatasetDict({
    train: Dataset({
        features: ['ID', 'path', 'label', 'dataset_name'],
        num_rows: 134266
    })
})

wf = wavefake["train"]

print("Columns:", wf.column_names)
print("Total:", len(wf))

# Show first 10 samples
for i in range(10):
    print(wf[i])
```

```
Columns: ['ID', 'path', 'label', 'dataset_name']
Total: 134266
{'ID': 'BASIC5000_0658_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_0658_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_0596_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_0596_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_4917_gen', 'path':
```

```
'jsut_multi_band_melgan/BASIC5000_4917_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_4412_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_4412_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_2331_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_2331_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_1545_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_1545_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_4572_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_4572_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_1826_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_1826_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_3631_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_3631_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
{'ID': 'BASIC5000_1899_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_1899_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
```

```python
from collections import Counter
import re

wf = wavefake["train"]

# 1. Label distribution
print("=== LABEL DISTRIBUTION ===")
print(Counter(wf["label"]))

# 2. Generation systems from path
print("\n=== GENERATOR / PATH PREFIXES ===")

generators = Counter()

for path in wf["path"]:
    generator = path.split("/")[0]
    generators[generator] += 1

for generator, count in generators.most_common():
    print(f"{generator:40s} {count:7d}")

# 3. Dataset names
print("\n=== DATASET NAMES ===")
print(Counter(wf["dataset_name"]))

# 4. Real samples, if present
```

```python
print("\n=== REAL SAMPLE EXAMPLES ===")

real_indices = [
    i for i, x in enumerate(wf["label"])
    if x.lower() in ["real", "bonafide", "bona-fide"]
][:10]

for i in real_indices:
    print(wf[i])

# 5. Check whether generated samples have corresponding source IDs
print("\n=== ID PATTERN CHECK ===")

for i in range(min(20, len(wf))):
    print(wf[i]["ID"])
```

```
=== LABEL DISTRIBUTION ===
Counter({'spoof': 134266})

=== GENERATOR / PATH PREFIXES ===
common_voices_prompts_from_conformer_fastspeech2_pwg_ljspeech   32566
ljspeech_parallel_wavegan                                       13100
ljspeech_full_band_melgan                                       13100
ljspeech_hifiGAN                                                13100
ljspeech_melgan_large                                           13100
ljspeech_melgan                                                 13100
ljspeech_multi_band_melgan                                      13100
ljspeech_waveglow                                               13100
jsut_multi_band_melgan                                           5000
jsut_parallel_wavegan                                            5000

=== DATASET NAMES ===
Counter({'WaveFake': 134266})

=== REAL SAMPLE EXAMPLES ===

=== ID PATTERN CHECK ===
BASIC5000_0658_gen
BASIC5000_0596_gen
BASIC5000_4917_gen
BASIC5000_4412_gen
BASIC5000_2331_gen
BASIC5000_1545_gen
BASIC5000_4572_gen
BASIC5000_1826_gen
BASIC5000_3631_gen
BASIC5000_1899_gen
BASIC5000_1107_gen
BASIC5000_2040_gen
BASIC5000_4602_gen
```

```
BASIC5000_2253_gen
BASIC5000_0034_gen
BASIC5000_3670_gen
BASIC5000_4182_gen
BASIC5000_0863_gen
BASIC5000_3113_gen
BASIC5000_1510_gen
```

```python
import numpy as np
from collections import defaultdict

wf = wavefake["train"]

SEED = 42
rng = np.random.default_rng(SEED)

MAX_PER_GENERATOR = 5000

# Group indices by generator
generator_indices = defaultdict(list)

for i, row in enumerate(wf):
    generator = row["path"].split("/")[0]
    generator_indices[generator].append(i)

selected_indices = []

for generator, indices in generator_indices.items():

    indices = np.array(indices)

    n = min(MAX_PER_GENERATOR, len(indices))

    chosen = rng.choice(
        indices,
        size=n,
        replace=False
    )

    selected_indices.extend(chosen.tolist())

    print(
        f"{generator:55s} "
        f"available={len(indices):6d} "
        f"selected={n:5d}"
    )

print("\nTotal WaveFake selected:", len(selected_indices))
```

```
jsut_multi_band_melgan                                  available=
5000 selected= 5000
```

```
ljspeech_parallel_wavegan                                       available=
13100 selected= 5000
ljspeech_full_band_melgan                                       available=
13100 selected= 5000
ljspeech_hifiGAN                                                available=
13100 selected= 5000
jsut_parallel_wavegan                                           available=
5000 selected= 5000
ljspeech_melgan_large                                           available=
13100 selected= 5000
common_voices_prompts_from_conformer_fastspeech2_pwg_ljspeech
available= 32566 selected= 5000
ljspeech_melgan                                                 available=
13100 selected= 5000
ljspeech_multi_band_melgan                                      available=
13100 selected= 5000
ljspeech_waveglow                                               available=
13100 selected= 5000

Total WaveFake selected: 50000
```

```python
# Inspect one selected WaveFake sample

idx = selected_indices[0]
sample = wf[idx]

print("ID       :", sample["ID"])
print("Path     :", sample["path"])
print("Label    :", sample["label"])
print("Dataset  :", sample["dataset_name"])

print("\nPath type:", type(sample["path"]))
```

```
ID       : BASIC5000_1476_gen
Path     : jsut_multi_band_melgan/BASIC5000_1476_gen.wav
Label    : spoof
Dataset  : WaveFake

Path type: <class 'str'>
```

```python
# Check the dataset cache location
import os
from huggingface_hub import scan_cache_dir

cache_info = scan_cache_dir()

for repo in cache_info.repos:
    if "WaveFake" in str(repo.repo_id):
        print("Repository:", repo.repo_id)
        print("Cache path:", repo.repo_path)
```

```
Repository: DeepFense/WaveFake
Cache path: /root/.cache/huggingface/hub/datasets--DeepFense--WaveFake
```

```
import os

cache_path = "/root/.cache/huggingface/hub/datasets--DeepFense--
WaveFake"

print("Cache exists:", os.path.exists(cache_path))

# Show directory structure (limited)
for root, dirs, files in os.walk(cache_path):
    level = root.replace(cache_path, "").count(os.sep)

    if level <= 3:
        print("\n" + "  " * level + os.path.basename(root) + "/")

        for f in files[:10]:
            print("  " * (level + 1) + f)
```

```
Cache exists: True

datasets--DeepFense--WaveFake/

  snapshots/

    6d5b1040c2a2d08150fe015b119ba5180556bd85/
      README.md
      wavefake.parquet

  blobs/
    97de8da472d460146fa940c4caa8071416450952
    3bc8a79a7b8651725276c06559e1a598d577c41bb1434803a9f06ae90aefc84e

  .no_exist/

    6d5b1040c2a2d08150fe015b119ba5180556bd85/
      .huggingface.yaml
      dataset_infos.json
      WaveFake.py

  refs/
    main
```

```
from datasets import load_dataset

# Try streaming the dataset
wavefake_stream = load_dataset(
    "DeepFense/WaveFake",
    split="train",
    streaming=True
```

```
)

sample = next(iter(wavefake_stream))

print(sample)
```

```
{'ID': 'BASIC5000_0658_gen', 'path':
'jsut_multi_band_melgan/BASIC5000_0658_gen.wav', 'label': 'spoof',
'dataset_name': 'WaveFake'}
```

```
from huggingface_hub import list_repo_files

files = list_repo_files(
    repo_id="DeepFense/WaveFake",
    repo_type="dataset"
)

print("Total files:", len(files))

for f in files[:50]:
    print(f)
```

```
Total files: 3
.gitattributes
README.md
wavefake.parquet
```

```
import os

wavefake_dir = "/content/drive/MyDrive/voice_cloning_project/wavefake"
os.makedirs(wavefake_dir, exist_ok=True)

print(wavefake_dir)
```

```
/content/drive/MyDrive/voice_cloning_project/wavefake
```

```
import os

wavefake_dir = "/content/wavefake"
os.makedirs(wavefake_dir, exist_ok=True)

print(wavefake_dir)
```

```
/content/wavefake
```

```
!wget -c \
"https://zenodo.org/records/5642694/files/generated_audio.zip" \
-O "/content/wavefake/generated_audio.zip"
```

```
--2026-09-05 21:08:53--
https://zenodo.org/records/5642694/files/generated_audio.zip
Resolving zenodo.org (zenodo.org)... 188.184.103.118, 188.184.98.114,
```

```
137.138.153.219, ...
Connecting to zenodo.org (zenodo.org)|188.184.103.118|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 28918626084 (27G) [application/octet-stream]
Saving to: ‘/content/wavefake/generated_audio.zip’

/generated_audio.zi   9%[>                    ]   2.44G  3.70MB/s
eta 99m 22s^C
```

```
from datasets import load_dataset

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

sample = next(iter(wf_audio))

print(sample.keys())
print(sample["audio"])
print(sample["real_or_fake"])
```


```
{"model_id":"c8e49f25a0bf439b9433260d2c2e0070","version_major":2,"vers
ion_minor":0}

{"model_id":"42ab9aae30a7432f94dee4a442b4f4f7","version_major":2,"vers
ion_minor":0}

{"model_id":"4942f14bc5674dd6862f2a9112a2dd52","version_major":2,"vers
ion_minor":0}
```


```
dict_keys(['audio', 'audio_id', 'real_or_fake'])
<datasets.features._torchcodec.AudioDecoder object at 0x7a730517e900>
WF1
```

```
sample = next(iter(wf_audio))

audio = sample["audio"]

print("Audio object:", audio)
print("ID:", sample["audio_id"])
print("Label:", sample["real_or_fake"])

# Decode the audio
audio_data = audio.get_all_samples()

print("Sample rate:", audio_data.sample_rate)
print("Shape:", audio_data.data.shape)
```

```
Audio object: <datasets.features._torchcodec.AudioDecoder object at
0x7a717d5c0910>
ID: LJ001-0001
Label: WF1
Sample rate: 22050
Shape: torch.Size([1, 212992])
```

```
from collections import Counter

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

generator_counts = Counter()

for sample in wf_audio:
    generator_counts[sample["real_or_fake"]] += 1

print("Generator distribution:")
for generator, count in sorted(generator_counts.items()):
    print(f"{generator}: {count:,}")

print("\nTotal:", sum(generator_counts.values()))
```


```
{"model_id":"d5583d18a860450598dadfbffe3c211f","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"896c7c4e11d44507aa2270a23a26e381","version_major":2,"vers
ion_minor":0}
```


```
---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_4544/1358186967.py in <cell line: 0>()
      9 generator_counts = Counter()
     10
---> 11 for sample in wf_audio:
     12     generator_counts[sample["real_or_fake"]] += 1
     13

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in __iter__(self)
   2359             return
   2360
-> 2361         for key, example in ex_iterable:
   2362             # no need to format thanks to
FormattedExamplesIterable
   2363             yield example
```

```
/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in __iter__(self)
   1880         if self.ex_iterable.iter_arrow:
   1881             # feature casting (inc column addition) handled
within self._iter_arrow()
-> 1882             for key, pa_table in self._iter_arrow():
   1883                 batch = formatter.format_batch(pa_table)
   1884                 for example in _batch_to_examples(batch):

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
   1903         if not self.features:
   1904             yield from self.ex_iterable._iter_arrow()
-> 1905         for key, pa_table in self.ex_iterable._iter_arrow():
   1906             columns = set(pa_table.column_names)
   1907             schema = self.features.arrow_schema

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    497             previous_state = self.ex_iterable.state_dict()
    498             self._state_dict["previous_state"] =
previous_state
--> 499         for key, pa_table in iterator:
    500             for num_chunks_since_previous_state, chunk in
enumerate(pa_table.to_reader(max_chunksize=self.batch_size)):
    501                 if num_chunks_to_skip > 1:

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    344             shard_example_idx_start =
self._state_dict["shard_example_idx"] if self._state_dict else 0
    345             shard_example_idx = 0
--> 346             for key, pa_table in
self.generate_tables_fn(**gen_kwags):
    347                 shard_example_idx += len(pa_table)
    348                 if shard_example_idx <=
shard_example_idx_start:

/usr/local/lib/python3.13/dist-packages/datasets/packaged_modules/
parquet/parquet.py in _generate_tables(self, files)
     91                     batch_size = self.config.batch_size or
parquet_fragment.row_groups[0].num_rows
     92                     try:
---> 93                         for batch_idx, record_batch in
enumerate(
     94                             parquet_fragment.to_batches(
     95                                 batch_size=batch_size,

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
```

```
_iterator()
   3804         """
   3805         def _iterator(batch_iter):
-> 3806             for batch in batch_iter:
   3807                 yield batch.record_batch
   3808         # Don't make ourselves a generator so errors are
raised immediately

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
pyarrow._dataset.TaggedRecordBatchIterator.__next__()
   3425         if batch.record_batch == NULL:
   3426             raise StopIteration
-> 3427         return TaggedRecordBatch(
   3428
record_batch=pyarrow_wrap_batch(batch.record_batch),
   3429             fragment=Fragment.wrap(batch.fragment))

<string> in <lambda>(_cls, record_batch, fragment)

KeyboardInterrupt:
```

```python
from datasets import load_dataset

wf_meta = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=False
)

print(wf_meta)
print(wf_meta.features)
```


```
{"model_id":"88996a5c653f42d392b1a748a29f85f9","version_major":2,"vers
ion_minor":0}

{"model_id":"ad1683c3e9bd49678a0e3a17697262a7","version_major":2,"vers
ion_minor":0}

{"model_id":"a4f60097df80404c975c170933ec59d7","version_major":2,"vers
ion_minor":0}

{"model_id":"f909a7cf9df4426b986f3e475d74bff0","version_major":2,"vers
ion_minor":0}

{"model_id":"85d7fec94a6a4fe0a6de03508f88fd8c","version_major":2,"vers
ion_minor":0}

{"model_id":"af9acbb81057450a99026e4455d31183","version_major":2,"vers
ion_minor":0}
```

{"model_id":"a3aef05d6a0548538a0920cd451acc6f","version_major":2,"version_minor":0}

{"model_id":"03b3bff6a56b4effb5100fddd3e6cc89","version_major":2,"version_minor":0}

{"model_id":"bef46cdba3d44dd59ae3632379f614d6","version_major":2,"version_minor":0}

{"model_id":"1af98c92289e4ce596af10f8e33807aa","version_major":2,"version_minor":0}

{"model_id":"a2445248b423466d8e6ad23b7bfb28f9","version_major":2,"version_minor":0}

{"model_id":"05dbbc0015824f62b8535eb2507f63f7","version_major":2,"version_minor":0}

{"model_id":"423c299c52184c6697c19e47023002a9","version_major":2,"version_minor":0}

{"model_id":"b69ba013d76b4a4bb80111c3c456e6a4","version_major":2,"version_minor":0}

{"model_id":"a3f477956b7e4a13b2e633b068e5de6f","version_major":2,"version_minor":0}

{"model_id":"7559a40a182041c28cba74bb47a5f7ec","version_major":2,"version_minor":0}

{"model_id":"946708964f5d431d9e58f92812506581","version_major":2,"version_minor":0}

{"model_id":"f13514ca30644764bb4159c0d309db8a","version_major":2,"version_minor":0}

{"model_id":"72cc04290dad4ff5a1c3f99518af3879","version_major":2,"version_minor":0}

{"model_id":"4d932d58a33b45a9b503971e9b25dae6","version_major":2,"version_minor":0}

{"model_id":"65fac17df80a4a1ba309ea4a6adf5bce","version_major":2,"version_minor":0}

{"model_id":"2c19cd5a1c65465a93e7ee2cc8770d08","version_major":2,"version_minor":0}

{"model_id":"bb5b04279f194a68a1ac1452ae1aafbb","version_major":2,"version_minor":0}

{"model_id":"ea7f3060f52e44d6bd957c424857a3c7","version_major":2,"version_minor":0}

{"model_id":"e4ae1ad40e7f41b3a2dff6d5b8d05353","version_major":2,"version_minor":0}

{"model_id":"f4f39888cf0247b2b2b3125fbbd80082","version_major":2,"version_minor":0}

{"model_id":"f224f79f92aa45e9bf1b31d6d96b52fc","version_major":2,"version_minor":0}

{"model_id":"fe0eebd225a04fb7a58a204352e7b86b","version_major":2,"version_minor":0}

{"model_id":"0d34841b45884b46a635c638d6968b52","version_major":2,"version_minor":0}

{"model_id":"764afc0250864de4b0ab7ce0ae96fc48","version_major":2,"version_minor":0}

{"model_id":"84aedd713475455793db28c2fb4cb937","version_major":2,"version_minor":0}

{"model_id":"3b2dd15564fe4be6a829df6d7a51229e","version_major":2,"version_minor":0}

{"model_id":"78d23c72f18d46edac6ad67d7ea27d2e","version_major":2,"version_minor":0}

{"model_id":"0e1446aa10c3418fafcf5b038a7b950d","version_major":2,"version_minor":0}

{"model_id":"1a4aa901de5e4b19bfb3355209a21b8e","version_major":2,"version_minor":0}

{"model_id":"008c28e8f8f4425297e2c8463d2a4707","version_major":2,"version_minor":0}

{"model_id":"3581acbd86fe40499df72e0deb6298b7","version_major":2,"version_minor":0}

{"model_id":"721464541ced463381ae9a05102fe6f6","version_major":2,"version_minor":0}

{"model_id":"5e619d626bb74b16acf119ae4b4e9702","version_major":2,"version_minor":0}

{"model_id":"477d65886c1f4c7c9e52ac8a88895f5d","version_major":2,"version_minor":0}

{"model_id":"1970b961334c4759878978f00cb7c270","version_major":2,"version_minor":0}

{"model_id":"62380f306b0044d5925e5a6fbcb65e47","version_major":2,"version_minor":0}

{"model_id":"51b1ea5d9c694629aeec4963b8a3f0a6","version_major":2,"version_minor":0}

{"model_id":"d8579219367041389a44d10fabdb64aa","version_major":2,"version_minor":0}

{"model_id":"5a7d3092f62e495385879c103ccfa401","version_major":2,"version_minor":0}

{"model_id":"ce10bd59c99d47c9a965458390235a97","version_major":2,"version_minor":0}

{"model_id":"0586a776b3c34fa3a1e0e48064af1abf","version_major":2,"version_minor":0}

{"model_id":"415ae0d06734433ca89ea6bd359c29eb","version_major":2,"version_minor":0}

{"model_id":"53fd34b7a64b4621a2c5f628caae9d44","version_major":2,"version_minor":0}

{"model_id":"9a8680b12a12434791437816ec9e4541","version_major":2,"version_minor":0}

{"model_id":"02b90d2205034aa8a7cf384f9f6f2982","version_major":2,"version_minor":0}

{"model_id":"0be3f8c289594547855b77d33cb4c04a","version_major":2,"version_minor":0}

{"model_id":"f9a7b4e087b441ceba2d297040d138a9","version_major":2,"version_minor":0}

{"model_id":"2ddcca2d5c05485a9544719b8548d0ee","version_major":2,"version_minor":0}

{"model_id":"b725276b688b4919bf3037cd20490fa9","version_major":2,"version_minor":0}

{"model_id":"648f014a55194511a093b9840d0e74f6","version_major":2,"version_minor":0}

{"model_id":"929ec2eeba024dbcb630a2c64828d198","version_major":2,"version_minor":0}

{"model_id":"6024eac849654abcac173f2c22a2c906","version_major":2,"version_minor":0}

{"model_id":"74dfbf169b254e6bbef8006dfaad2fb1","version_major":2,"version_minor":0}

{"model_id":"8252dc4c1b5c4a5aa120b694bee3c999","version_major":2,"version_minor":0}

{"model_id":"41d3a6acafe74b4e8a05f535cb2888e3","version_major":2,"version_minor":0}

{"model_id":"a9e264e7042941d6b46e006947aee898","version_major":2,"version_minor":0}

{"model_id":"f5a480fc120b40e7ac82043a1ada483d","version_major":2,"version_minor":0}

{"model_id":"99661dcff4cc4a60a5ce943b37636bea","version_major":2,"version_minor":0}

{"model_id":"d2b99a5e3f8a4df6a599caf3934de6d5","version_major":2,"version_minor":0}

{"model_id":"ae446ad1113b4927b9cfb4ad198ef57d","version_major":2,"version_minor":0}

{"model_id":"3083c37f902545ed904c9b7474c11e04","version_major":2,"version_minor":0}

{"model_id":"a91cbb9cc43d4a40949b376ad3f18ec0","version_major":2,"version_minor":0}

{"model_id":"b0c8ea8e32ef4014814d450626608625","version_major":2,"version_minor":0}

{"model_id":"060689ca2fc445588c7ac0e0a0262100","version_major":2,"version_minor":0}

{"model_id":"b3729173258541348a0ed174c27afaa6","version_major":2,"version_minor":0}

{"model_id":"785d1bfc8063442da252a7c332bcd367","version_major":2,"version_minor":0}

{"model_id":"d275456f3944418dafe568c99379ad4b","version_major":2,"version_minor":0}

{"model_id":"4ea411f18beb47eaa1cbedd58c8cfbed","version_major":2,"version_minor":0}

{"model_id":"890ed9ee71a940179ee9c3f11bce7408","version_major":2,"version_minor":0}

{"model_id":"2a557524d9ff42beab9a37b0c1ece94c","version_major":2,"version_minor":0}

{"model_id":"30da47594e7547f4b482a729763a235d","version_major":2,"version_minor":0}

{"model_id":"0f2664b4515d41c3ac88e3787923cc5e","version_major":2,"version_minor":0}

{"model_id":"4bcb901f07034ad09525584b821c1dcd","version_major":2,"version_minor":0}

{"model_id":"80b1de0890d84f8d9eefd902c7d62a7d","version_major":2,"version_minor":0}

{"model_id":"3421282989bf4373ba7c07f77598563d","version_major":2,"version_minor":0}

{"model_id":"a4333a5e7ed744628df2c3529f2e8652","version_major":2,"version_minor":0}

{"model_id":"31677dab64d54881b94bf392c1ddf951","version_major":2,"version_minor":0}

{"model_id":"b89b9b44c0a94514bc16aa494ebafc2a","version_major":2,"version_minor":0}

{"model_id":"925a688954844adda493798596adebc2","version_major":2,"version_minor":0}

{"model_id":"f1ab707c6c6e441d9f4a439f1e0495ea","version_major":2,"version_minor":0}

{"model_id":"5c7e1bd7d8e54a3f83394febbd53b4e2","version_major":2,"version_minor":0}

{"model_id":"99e3fc9b13e74e96b3946119a3b29315","version_major":2,"version_minor":0}

{"model_id":"8cde06c824e840a1b85875327fc6ceff","version_major":2,"version_minor":0}

{"model_id":"0153ce8364624349a60596d7f4b6e01a","version_major":2,"version_minor":0}

{"model_id":"f686841d79c843f98deb566b9d59e9fc","version_major":2,"version_minor":0}

{"model_id":"e1c3c3eca1d344eca6d9024bf0a67016","version_major":2,"version_minor":0}

{"model_id":"691ff3a3cd95444aa16aac6dd6554119","version_major":2,"version_minor":0}

{"model_id":"dc3ea551d4e94365a20ec9f0b9b85a2e","version_major":2,"version_minor":0}

{"model_id":"f17f638f69b24ce29d51be805abdef8e","version_major":2,"version_minor":0}

{"model_id":"bc1487f380eb47aea53280c3e78b1980","version_major":2,"version_minor":0}

{"model_id":"f5e3f56907184593a3b7e6a2cb247e39","version_major":2,"version_minor":0}

{"model_id":"d8f9557b073145d9a5ced86a56ecc116","version_major":2,"version_minor":0}

{"model_id":"2b8fb2ae93c44c4c8ac751e4a68f8c5a","version_major":2,"version_minor":0}

{"model_id":"a14a560420974839a4048f1f92d47d1d","version_major":2,"version_minor":0}

{"model_id":"8aaa742c82754eeba7182ed930f6bbe9","version_major":2,"version_minor":0}

{"model_id":"92c552f690a64516a19fe7f1eebbd60d","version_major":2,"version_minor":0}

{"model_id":"0aaab611d4c04735937db74a289482ae","version_major":2,"version_minor":0}

{"model_id":"5d0d2c9340994693844a484a09d70ca4","version_major":2,"version_minor":0}

{"model_id":"310d6d4e55d74e4db9ca32c99efe804a","version_major":2,"version_minor":0}

{"model_id":"28389802978d420da7a60fd3c44bf821","version_major":2,"version_minor":0}

{"model_id":"9ad4abe3dd204e63b3d8fa10a1381058","version_major":2,"version_minor":0}

{"model_id":"9f316b8e15064e3e826b87899f888909","version_major":2,"version_minor":0}

{"model_id":"201dae25af504a46ae203b4d69a7a190","version_major":2,"version_minor":0}

{"model_id":"46b9c83417594df0a008da16e3903a52","version_major":2,"version_minor":0}

{"model_id":"e582fed8b34e481ba47b0f8325d60552","version_major":2,"version_minor":0}

{"model_id":"3cf2ace5b3c14951a96779838124927c","version_major":2,"version_minor":0}

{"model_id":"d05b3a731b6045ae944341cb3427f083","version_major":2,"version_minor":0}

{"model_id":"fb7c11161e2b4d20bf1cc0d001962f9f","version_major":2,"version_minor":0}

{"model_id":"a87fc7f7d03f4401aca3640b0562d621","version_major":2,"version_minor":0}

{"model_id":"c4ea16e63b38475fa1097bbaec124c2c","version_major":2,"version_minor":0}

{"model_id":"83f94409402e40f7802e77564050a082","version_major":2,"version_minor":0}

{"model_id":"fa79071f20f24a64a4f49f2ca42a3d01","version_major":2,"version_minor":0}

{"model_id":"1b0faaca9208412fb4c212939632259b","version_major":2,"version_minor":0}

{"model_id":"971eb5a477504cc18d3f0d9d680e4cea","version_major":2,"version_minor":0}

{"model_id":"374da76a687a4061821cb8fa93f897b6","version_major":2,"version_minor":0}

{"model_id":"d14f6e083cf44d03823ac45bdfa62d83","version_major":2,"version_minor":0}

{"model_id":"f9764f154a794889b6d7a546d0db2b56","version_major":2,"version_minor":0}

{"model_id":"df1b003e1a3f40c7a543f08bb1fc5e5d","version_major":2,"version_minor":0}

{"model_id":"9e81965adff54224933a3552065d9077","version_major":2,"version_minor":0}

{"model_id":"44fab116ff1044db8d1142e3b22be0f7","version_major":2,"version_minor":0}

{"model_id":"c4f5947297ad488cbc27c0b58fa0a832","version_major":2,"version_minor":0}

{"model_id":"6e7e55d212264d629f3972d501734809","version_major":2,"version_minor":0}

{"model_id":"0faa83f7168b4969876d2b82343452ca","version_major":2,"version_minor":0}

{"model_id":"04391fb374264b3db93178b1e69e25e3","version_major":2,"version_minor":0}

{"model_id":"bc100d6c4fb84f98a834bec5aeed0caf","version_major":2,"version_minor":0}

{"model_id":"9056c8e4b0454aff926d988dc567f7e4","version_major":2,"version_minor":0}

{"model_id":"0c2e97ccc01342af9a14edb5461c2da2","version_major":2,"version_minor":0}

{"model_id":"e8e3d0ecae57469581e37ae8b8df13d5","version_major":2,"version_minor":0}

{"model_id":"8fd3feacc49d486da0c2a567cc564cce","version_major":2,"version_minor":0}

{"model_id":"db2a35ffb290496d841be536846143d6","version_major":2,"version_minor":0}

{"model_id":"82282e17ae474e0e98f2092e20ca5832","version_major":2,"version_minor":0}

{"model_id":"8c02df103c7e4e8aa6505beb807a3ba2","version_major":2,"version_minor":0}

{"model_id":"26bb14050ea9467693688c5923fbdc8d","version_major":2,"version_minor":0}

{"model_id":"bc5fd6ebd9b54774b895da111de51a5c","version_major":2,"version_minor":0}

{"model_id":"a64d561f4c284b6f886cbeeadf297388","version_major":2,"version_minor":0}

{"model_id":"5dd8f38eb53b438088056651021187f7","version_major":2,"version_minor":0}

{"model_id":"4babce0907da4521b2cd88cc2ad0dfd9","version_major":2,"version_minor":0}

{"model_id":"bbbdee12e4f04a5a9908ffe35049f313","version_major":2,"version_minor":0}

{"model_id":"cee1ddb200774845b04dd7b16a424240","version_major":2,"version_minor":0}

{"model_id":"4a8bd0d587a049c899e08ac2905c10ca","version_major":2,"version_minor":0}

{"model_id":"a25a22218d79482f9a2fb930dfd3b742","version_major":2,"version_minor":0}

{"model_id":"04917c6fd1fe45f1a2bc7c1db9c576e7","version_major":2,"version_minor":0}

{"model_id":"31aed29312c54513b1323feb8abf8263","version_major":2,"version_minor":0}

{"model_id":"9c9d9a5e313c4495a799705798956fae","version_major":2,"version_minor":0}

{"model_id":"2ac355ad902b4b91967fb3913778c410","version_major":2,"version_minor":0}

{"model_id":"0ac354a54222411a9fdb7d645c03c411","version_major":2,"version_minor":0}

{"model_id":"6589f750ec244b98823f8cec04f74732","version_major":2,"version_minor":0}

{"model_id":"11d87a63742546acb9451b24ea76e8bc","version_major":2,"version_minor":0}

{"model_id":"9a1bc8d60e474b64b630f459f9127835","version_major":2,"version_minor":0}

{"model_id":"7b04983621ba4798968670c5c71a5b8a","version_major":2,"version_minor":0}

{"model_id":"90054f098cf44e4592b70caba7bd7098","version_major":2,"version_minor":0}

{"model_id":"14da680b376144c5b7dab8045ae51e57","version_major":2,"version_minor":0}

{"model_id":"fa554c504a16403996b91ee7dd083dfa","version_major":2,"version_minor":0}

{"model_id":"50365733372b4e02898564d820c139ce","version_major":2,"version_minor":0}

{"model_id":"eccbf27a4273405d891b0872377e0a89","version_major":2,"version_minor":0}

{"model_id":"8eb72bf0b9ce4ad79f02375c48c120d3","version_major":2,"version_minor":0}

{"model_id":"98a18079200141909526059ab7622ab4","version_major":2,"version_minor":0}

{"model_id":"c5f337379e6240ea85f99f3440dc962d","version_major":2,"version_minor":0}

{"model_id":"8332bab925e0422da798eaf3b9ca4856","version_major":2,"version_minor":0}

{"model_id":"9e7fd15d10f742bb892f30de2bdc535e","version_major":2,"version_minor":0}

{"model_id":"f8e6ca98b6cd4bc6a1821919273801ba","version_major":2,"version_minor":0}

{"model_id":"8a65c6b3618540adb4d6819e4dcbe063","version_major":2,"version_minor":0}

{"model_id":"9352f9a5d54d4267830a677993e101e9","version_major":2,"version_minor":0}

{"model_id":"56b60ae854084f2ba04924440d67f829","version_major":2,"version_minor":0}

{"model_id":"6421fdcbbdc94c36a1707116e0eaa0f6","version_major":2,"version_minor":0}

{"model_id":"cbf575c0747b42c59ed9ddc32cf9a0ed","version_major":2,"version_minor":0}

{"model_id":"c7a56d22efaa4c9595f5892246ee4303","version_major":2,"version_minor":0}

{"model_id":"abaf25e82db54d638d348ba5c1800975","version_major":2,"version_minor":0}

{"model_id":"7440b571951740be8c7f6cea1a0e41e9","version_major":2,"version_minor":0}

{"model_id":"109fb8c097ba4aee935c537e7af79c21","version_major":2,"version_minor":0}

{"model_id":"66dc64eab7124070abc0d0cc56b78038","version_major":2,"version_minor":0}

{"model_id":"900ce164e359484890093ea3d97ddee1","version_major":2,"version_minor":0}

{"model_id":"9673d1299e8e4117b82cb1305fd58764","version_major":2,"version_minor":0}

{"model_id":"a9735870a39a4336801138ecff3861dc","version_major":2,"version_minor":0}

{"model_id":"276c1fed08d14024b3bb419a33790032","version_major":2,"version_minor":0}

{"model_id":"e7788dee261641f3aca6ef6d367ebd30","version_major":2,"version_minor":0}

{"model_id":"60a8a36cc2c24c7da31525da1e2acf43","version_major":2,"version_minor":0}

{"model_id":"24f8e4ee615d411fb7ec7d07d2d2479e","version_major":2,"version_minor":0}

{"model_id":"416efc63daf343df91fa3cbc737bd9da","version_major":2,"version_minor":0}

{"model_id":"e9f7dbdcca9843b1b6126e10c2c815e1","version_major":2,"version_minor":0}

{"model_id":"43be7f6bc555418cbb6ee72c9aa5b3bb","version_major":2,"version_minor":0}

{"model_id":"3779069a58324f3bacad6c917a62e482","version_major":2,"version_minor":0}

{"model_id":"3849463e02f44466a50cc8e207ea9b6d","version_major":2,"version_minor":0}

{"model_id":"259886fdf94644f9a8cf8eebdb198dad","version_major":2,"version_minor":0}

{"model_id":"24099d649da24d2c9cdd614a053ddadf","version_major":2,"version_minor":0}

{"model_id":"252a933b90b946dd996f7b550c17d206","version_major":2,"version_minor":0}

{"model_id":"a3e6993ac38c4e1693386e8341567337","version_major":2,"version_minor":0}

{"model_id":"4104a2225c7b44c7a5bebe664d69d02f","version_major":2,"version_minor":0}

{"model_id":"5dc8c44164ff4f81b19480b09ec0d76a","version_major":2,"version_minor":0}

{"model_id":"a2a7bbc746aa4fc1ad6f62ec491e9822","version_major":2,"version_minor":0}

{"model_id":"4e010b5b2bd6469a90a93e1d0bed91b5","version_major":2,"version_minor":0}

{"model_id":"81676296ce764a9fbaaa8c3e3fd3e1cf","version_major":2,"version_minor":0}

{"model_id":"b3a7adcc3b984d6cb494fb72274eb8b7","version_major":2,"version_minor":0}

{"model_id":"67084d38e79e4c6788b056e810678941","version_major":2,"version_minor":0}

{"model_id":"4daf1c1bf1204d2999499fd6ce52330f","version_major":2,"version_minor":0}

{"model_id":"baf7b36fa6644407a54acb11a75b75cc","version_major":2,"version_minor":0}

{"model_id":"25ff50ee42a441ee9a8fe915a8115eb1","version_major":2,"version_minor":0}

{"model_id":"5e2673de5274483bbcee032e07b1d22f","version_major":2,"version_minor":0}

{"model_id":"4c2dccbc0e9146599edb770e927b5881","version_major":2,"version_minor":0}

{"model_id":"4d30b2ba4f1d4069a34ed40a9884c0fa","version_major":2,"version_minor":0}

{"model_id":"16db6af090704b458a6fad21c615ed23","version_major":2,"version_minor":0}

{"model_id":"b2e2caf5aa0c487ebcf4cdaea82b809f","version_major":2,"version_minor":0}

{"model_id":"4471a79e890d470a8ffa4d480ef43ec2","version_major":2,"version_minor":0}

{"model_id":"fa163fbd2b3340a198a987c44f5c8655","version_major":2,"version_minor":0}

{"model_id":"d45742b0214b42daa37d69e02303cd2c","version_major":2,"version_minor":0}

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_4544/3701923846.py in <cell line: 0>()
      1 from datasets import load_dataset
      2
----> 3 wf_meta = load_dataset(
      4     "ajaykarthick/wavefake-audio",
      5     split="train",

/usr/local/lib/python3.13/dist-packages/datasets/load.py in load_dataset(path, name, data_dir, data_files, split, cache_dir, features, download_config, download_mode, verification_mode, keep_in_memory, save_infos, revision, token, streaming, num_proc, storage_options, **config_kwargs)
   1410
   1411     # Download and prepare data
-> 1412     builder_instance.download_and_prepare(
   1413         download_config=download_config,
   1414         download_mode=download_mode,

/usr/local/lib/python3.13/dist-packages/datasets/builder.py in download_and_prepare(self, output_dir, download_config, download_mode, verification_mode, dl_manager, base_path, file_format, max_shard_size, num_proc, storage_options, **download_and_prepare_kwargs)
    892                     if num_proc is not None:
    893                         prepare_split_kwargs["num_proc"] = num_proc
--> 894                     self._download_and_prepare(
```

```
    895                         dl_manager=dl_manager,
    896                         verification_mode=verification_mode,

/usr/local/lib/python3.13/dist-packages/datasets/builder.py in
_download_and_prepare(self, dl_manager, verification_mode,
**prepare_split_kwargs)
    946         split_dict = SplitDict(dataset_name=self.dataset_name)
    947         split_generators_kwargs =
self._make_split_generators_kwargs(prepare_split_kwargs)
--> 948         split_generators = self._split_generators(dl_manager,
**split_generators_kwargs)
    949
    950         # Checksums verification

/usr/local/lib/python3.13/dist-packages/datasets/packaged_modules/
parquet/parquet.py in _split_generators(self, dl_manager)
     47             raise ValueError(f"At least one data file must be
specified, but got data_files={self.config.data_files}")
     48         dl_manager.download_config.extract_on_the_fly = True
---> 49         data_files =
dl_manager.download_and_extract(self.config.data_files)
     50         splits = []
     51         for split_name, files in data_files.items():

/usr/local/lib/python3.13/dist-packages/datasets/download/download_man
ager.py in download_and_extract(self, url_or_urls)
    324             extracted_path(s): `str`, extracted paths of given
URL(s).
    325         """
--> 326         return self.extract(self.download(url_or_urls))
    327
    328     def get_recorded_sizes_checksums(self):

/usr/local/lib/python3.13/dist-packages/datasets/download/download_man
ager.py in download(self, url_or_urls)
    157         start_time = datetime.now()
    158         with stack_multiprocessing_download_progress_bars():
--> 159             downloaded_path_or_paths = map_nested(
    160                 download_func,
    161                 url_or_urls,

/usr/local/lib/python3.13/dist-packages/datasets/utils/py_utils.py in
map_nested(function, data_struct, dict_only, map_list, map_tuple,
map_numpy, num_proc, parallel_min_length, batched, batch_size, types,
disable_tqdm, desc)
    502     if any(isinstance(v, types) and len(v) > len(iterable) for
v in iterable):
    503         mapped = [
--> 504             map_nested(
    505                 function=function,
```

```
    506                 data_struct=obj,

/usr/local/lib/python3.13/dist-packages/datasets/utils/py_utils.py in
map_nested(function, data_struct, dict_only, map_list, map_tuple,
map_numpy, num_proc, parallel_min_length, batched, batch_size, types,
disable_tqdm, desc)
    519             iterable = list(iter_batched(iterable,
batch_size))
    520         mapped = [
--> 521             _single_map_nested((function, obj, batched,
batch_size, types, None, True, None))
    522             for obj in hf_tqdm(iterable, disable=disable_tqdm,
desc=desc)
    523         ]

/usr/local/lib/python3.13/dist-packages/datasets/utils/py_utils.py in
_single_map_nested(args)
    387         and all(not isinstance(v, (dict, types)) for v in
data_struct)
    388     ):
--> 389         return [mapped_item for batch in
iter_batched(data_struct, batch_size) for mapped_item in
function(batch)]
    390
    391     # Reduce logging to keep things readable in
multiprocessing with tqdm

/usr/local/lib/python3.13/dist-packages/datasets/download/download_man
ager.py in _download_batched(self, url_or_filenames, download_config)
    204             )  # enable multithreading if files are small
    205
--> 206             return thread_map(
    207                 download_func,
    208                 url_or_filenames,

/usr/local/lib/python3.13/dist-packages/tqdm/contrib/concurrent.py in
thread_map(fn, *iterables, **tqdm_kwargs)
     67     """
     68     from concurrent.futures import ThreadPoolExecutor
---> 69     return _executor_map(ThreadPoolExecutor, fn, *iterables,
**tqdm_kwargs)
     70
     71

/usr/local/lib/python3.13/dist-packages/tqdm/contrib/concurrent.py in
_executor_map(PoolExecutor, fn, *iterables, **tqdm_kwargs)
     49         with PoolExecutor(max_workers=max_workers,
initializer=tqdm_class.set_lock,
     50                           initargs=(lk,)) as ex:
---> 51             return list(tqdm_class(ex.map(fn, *iterables,
```

```
chunksize=chunksize), **kwargs))
     52
     53

/usr/local/lib/python3.13/dist-packages/tqdm/notebook.py in
__iter__(self)
    248         try:
    249             it = super().__iter__()
--> 250             yield from it
    251         # NB: except ... [ as ...] breaks IPython async
KeyboardInterrupt
    252         except:  # NOQA

/usr/local/lib/python3.13/dist-packages/tqdm/std.py in __iter__(self)
   1179
   1180         try:
-> 1181             for obj in iterable:
   1182                 yield obj
   1183                 # Update and possibly print the progressbar.

/usr/lib/python3.13/concurrent/futures/_base.py in result_iterator()
    623                     # Careful not to keep a reference to the
popped future
    624                     if timeout is None:
--> 625                         yield _result_or_cancel(fs.pop())
    626                     else:
    627                         yield _result_or_cancel(fs.pop(),
end_time - time.monotonic())

/usr/lib/python3.13/concurrent/futures/_base.py in
_result_or_cancel(***failed resolving arguments***)
    316     try:
    317         try:
--> 318             return fut.result(timeout)
    319         finally:
    320             fut.cancel()

/usr/lib/python3.13/concurrent/futures/_base.py in result(self,
timeout)
    453                     return self.__get_result()
    454
--> 455                 self._condition.wait(timeout)
    456
    457                 if self._state in [CANCELLED,
CANCELLED_AND_NOTIFIED]:

/usr/lib/python3.13/threading.py in wait(self, timeout)
    357         try:    # restore state no matter what (e.g.,
KeyboardInterrupt)
    358             if timeout is None:
```

```
--> 359                 waiter.acquire()
    360                 gotit = True
    361             else:

KeyboardInterrupt:
```

```
wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)
```

{"model_id":"ecb604a7108348b383ce1c6b8ae69867","version_major":2,"version_minor":0}

{"model_id":"128eee0d450c4d909a75f44d031d2a83","version_major":2,"version_minor":0}

```
from datasets import load_dataset
from itertools import islice

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

for i, sample in enumerate(islice(wf_audio, 16)):
    print(
        i,
        sample["audio_id"],
        sample["real_or_fake"]
    )
```

{"model_id":"7a267610e68946b2b26c1152bd28dcd4","version_major":2,"version_minor":0}

{"model_id":"f425747217644909a15fdba52a4fbc49","version_major":2,"version_minor":0}

```
0 LJ001-0001 WF1
1 LJ001-0001 WF2
2 LJ001-0001 WF3
3 LJ001-0001 WF4
4 LJ001-0001 WF5
5 LJ001-0001 WF6
6 LJ001-0001 WF7
7 LJ001-0001 R
8 LJ001-0002 WF1
9 LJ001-0002 WF2
10 LJ001-0002 WF3
```

```
11 LJ001-0002 WF4
12 LJ001-0002 WF5
13 LJ001-0002 WF6
14 LJ001-0002 WF7
15 LJ001-0002 R

import numpy as np
from itertools import islice

# Re-create streaming dataset from the beginning
wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

TARGET_PER_GENERATOR = 7000
GENERATORS = {f"WF{i}" for i in range(1, 8)}

selected = []
counts = {g: 0 for g in GENERATORS}

# Because the confirmed structure is:
# WF1, WF2, ..., WF7, R
#
# We take the first 7,000 utterance IDs.
# That gives 7,000 samples for every WF generator.

for i, sample in enumerate(wf_audio):

    label = sample["real_or_fake"]

    if label in GENERATORS and counts[label] < TARGET_PER_GENERATOR:

        selected.append({
            "audio_id": sample["audio_id"],
            "generator": label
        })

        counts[label] += 1

    # Stop once every generator has 7,000 samples
    if all(counts[g] == TARGET_PER_GENERATOR for g in GENERATORS):
        break

print("Selection complete.")
print("Total selected:", len(selected))
print("Counts:", counts)

{"model_id":"3e74f5bbe65648d4be74e39070ebb494","version_major":2,"vers
ion_minor":0}
```

{"model_id":"dc4d248699bb4edd9ee69972f43edc45","version_major":2,"version_minor":0}

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_4544/129321496.py in <cell line: 0>()
     21 # That gives 7,000 samples for every WF generator.
     22
---> 23 for i, sample in enumerate(wf_audio):
     24
     25     label = sample["real_or_fake"]

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py in __iter__(self)
   2359             return
   2360
-> 2361         for key, example in ex_iterable:
   2362             # no need to format thanks to FormattedExamplesIterable
   2363             yield example

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py in __iter__(self)
   1881             # feature casting (inc column addition) handled within self._iter_arrow()
   1882             for key, pa_table in self._iter_arrow():
-> 1883                 batch = formatter.format_batch(pa_table)
   1884                 for example in _batch_to_examples(batch):
   1885                     yield key, example

/usr/local/lib/python3.13/dist-packages/datasets/formatting/formatting.py in format_batch(self, pa_table)
    469             return LazyBatch(pa_table, self)
    470         batch = self.python_arrow_extractor().extract_batch(pa_table)
--> 471         batch = self.python_features_decoder.decode_batch(batch)
    472         return batch
    473

/usr/local/lib/python3.13/dist-packages/datasets/formatting/formatting.py in decode_batch(self, batch)
    231
    232     def decode_batch(self, batch: dict) -> dict:
--> 233         return self.features.decode_batch(batch, token_per_repo_id=self.token_per_repo_id) if self.features else batch
    234
    235
```

```
/usr/local/lib/python3.13/dist-packages/datasets/features/features.py
in decode_batch(self, batch, token_per_repo_id)
   2141             decoded_batch[column_name] = (
   2142                 [
-> 2143                     decode_nested_example(self[column_name],
value, token_per_repo_id=token_per_repo_id)
   2144                     if value is not None
   2145                     else None

/usr/local/lib/python3.13/dist-packages/datasets/features/features.py
in decode_nested_example(schema, obj, token_per_repo_id)
   1403     elif hasattr(schema, "decode_example") and getattr(schema,
"decode", True):
   1404         # we pass the token to read and decode files from
private repositories in streaming mode
-> 1405         return schema.decode_example(obj,
token_per_repo_id=token_per_repo_id) if obj is not None else None
   1406     return obj
   1407

/usr/local/lib/python3.13/dist-packages/datasets/features/audio.py in
decode_example(self, value, token_per_repo_id)
    196
    197         else:
--> 198             audio = AudioDecoder(bytes,
stream_index=self.stream_index, sample_rate=self.sampling_rate)
    199         audio._hf_encoded = {"path": path, "bytes": bytes}
    200         audio.metadata.path = path

/usr/local/lib/python3.13/dist-packages/torchcodec/decoders/_audio_dec
oder.py in __init__(self, source, stream_index, sample_rate,
num_channels)
     64             self.stream_index,
     65             self.metadata,
---> 66         ) = create_audio_decoder(
     67             source=source,
     68             seek_mode="approximate",

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/_decoder_util
s.py in create_audio_decoder(source, seek_mode, stream_index,
sample_rate, num_channels)
     73 ) -> tuple[Tensor, int, AudioStreamMetadata]:
     74
---> 75     decoder = create_decoder(source=source,
seek_mode=seek_mode)
     76
     77     container_metadata = get_container_metadata(decoder)

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/_decoder_util
```

```
s.py in create_decoder(source, seek_mode)
     46         return create_from_file_like(source, seek_mode)
     47     elif isinstance(source, bytes):
---> 48         return create_from_bytes(source, seek_mode)
     49     elif isinstance(source, Tensor):
     50         return create_from_tensor(source, seek_mode)

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/ops.py in
create_from_bytes(video_bytes, seek_mode)
    151         warnings.filterwarnings("ignore",
category=UserWarning)
    152         buffer = torch.frombuffer(video_bytes,
dtype=torch.uint8)
--> 153     return create_from_tensor(buffer, seek_mode)
    154
    155

/usr/local/lib/python3.13/dist-packages/torch/_ops.py in
__call__(self, *args, **kwargs)
    863     # that are named "self". This way, all the aten ops can be
called by kwargs.
    864     def __call__(self, /, *args: _P.args, **kwargs: _P.kwargs)
-> _T:
--> 865         return self._op(*args, **kwargs)
    866
    867     # Use positional-only argument to avoid naming collision
with aten ops arguments

KeyboardInterrupt:
```

```python
from datasets import load_dataset
import numpy as np

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

TARGET_UTTERANCES = 7000

selected = []

for i, sample in enumerate(wf_audio):

    # Each utterance has:
    # WF1, WF2, ..., WF7, R
    position = i % 8

    # Skip R
```

```
    if position == 7:
        continue

    selected.append({
        "audio_id": sample["audio_id"],
        "generator": sample["real_or_fake"]
    })

    # After 7000 complete utterances
    if i >= TARGET_UTTERANCES * 8 - 1:
        break

print("Total selected:", len(selected))

for g in [f"WF{i}" for i in range(1, 8)]:
    print(g, sum(x["generator"] == g for x in selected))
```


```
{"model_id":"995a253614d04ed4b16a5227c9936aae","version_major":2,"vers
ion_minor":0}

{"model_id":"394ee336106f48f99c66d0d286181185","version_major":2,"vers
ion_minor":0}
```


```
---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_4544/2154507304.py in <cell line: 0>()
     12 selected = []
     13
---> 14 for i, sample in enumerate(wf_audio):
     15
     16     # Each utterance has:

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in __iter__(self)
   2359             return
   2360
-> 2361         for key, example in ex_iterable:
   2362             # no need to format thanks to
FormattedExamplesIterable
   2363             yield example

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in __iter__(self)
   1880         if self.ex_iterable.iter_arrow:
   1881             # feature casting (inc column addition) handled
within self._iter_arrow()
-> 1882             for key, pa_table in self._iter_arrow():
   1883                 batch = formatter.format_batch(pa_table)
   1884                 for example in _batch_to_examples(batch):
```

```
/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
   1903         if not self.features:
   1904             yield from self.ex_iterable._iter_arrow()
-> 1905         for key, pa_table in self.ex_iterable._iter_arrow():
   1906             columns = set(pa_table.column_names)
   1907             schema = self.features.arrow_schema

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    497             previous_state = self.ex_iterable.state_dict()
    498             self._state_dict["previous_state"] =
previous_state
--> 499         for key, pa_table in iterator:
    500             for num_chunks_since_previous_state, chunk in
enumerate(pa_table.to_reader(max_chunksize=self.batch_size)):
    501                 if num_chunks_to_skip > 1:

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in _iter_arrow(self)
    344             shard_example_idx_start =
self._state_dict["shard_example_idx"] if self._state_dict else 0
    345             shard_example_idx = 0
--> 346             for key, pa_table in
self.generate_tables_fn(**gen_kwags):
    347                 shard_example_idx += len(pa_table)
    348                 if shard_example_idx <=
shard_example_idx_start:

/usr/local/lib/python3.13/dist-packages/datasets/packaged_modules/
parquet/parquet.py in _generate_tables(self, files)
     91                     batch_size = self.config.batch_size or
parquet_fragment.row_groups[0].num_rows
     92                     try:
---> 93                         for batch_idx, record_batch in
enumerate(
     94                             parquet_fragment.to_batches(
     95                                 batch_size=batch_size,

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
_iterator()
   3804         """
   3805         def _iterator(batch_iter):
-> 3806             for batch in batch_iter:
   3807                 yield batch.record_batch
   3808         # Don't make ourselves a generator so errors are
raised immediately

/usr/local/lib/python3.13/dist-packages/pyarrow/_dataset.pyx in
```

```
pyarrow._dataset.TaggedRecordBatchIterator.__next__()
   3425         if batch.record_batch == NULL:
   3426             raise StopIteration
-> 3427         return TaggedRecordBatch(
   3428
record_batch=pyarrow_wrap_batch(batch.record_batch),
   3429             fragment=Fragment.wrap(batch.fragment))

<string> in <lambda>(_cls, record_batch, fragment)

KeyboardInterrupt:
```

```
from datasets import load_dataset

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

# Test retrieving one known sample
sample = next(iter(wf_audio))

print("ID:", sample["audio_id"])
print("Generator:", sample["real_or_fake"])
print("Audio:", sample["audio"])
```


```
{"model_id":"e830b573402941c5a89f0670ca5adc93","version_major":2,"vers
ion_minor":0}

{"model_id":"e2ad188190b340b5a400931f1f94865d","version_major":2,"vers
ion_minor":0}
```


```
ID: LJ001-0001
Generator: WF1
Audio: <datasets.features._torchcodec.AudioDecoder object at
0x7a72adc4dc50>
```

```
import torch
import torchaudio
import numpy as np
from datasets import load_dataset

# Streaming dataset
wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

TARGET_SR = 16000
```

```
DURATION = 4
TARGET_LENGTH = TARGET_SR * DURATION

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=TARGET_SR,
    n_fft=1024,
    hop_length=256,
    n_mels=128
)

log_transform = torchaudio.transforms.AmplitudeToDB()

features = []
labels = []

# First 100 complete utterances
# 100 × 7 generators = 700 spoof samples
for i, sample in enumerate(wf_audio):

    if i >= 100 * 8:
        break

    generator = sample["real_or_fake"]

    # Skip real samples
    if generator == "R":
        continue

    # Decode audio
    audio = sample["audio"].get_all_samples()

    waveform = audio.data.float()
    sr = audio.sample_rate

    # Convert stereo → mono
    if waveform.ndim == 2:
        waveform = waveform.mean(dim=0, keepdim=True)

    # Resample
    if sr != TARGET_SR:
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            TARGET_SR
        )

    # Fixed 4 seconds
    if waveform.shape[1] < TARGET_LENGTH:
        waveform = torch.nn.functional.pad(
            waveform,
```

```
            (0, TARGET_LENGTH - waveform.shape[1])
        )
    else:
        waveform = waveform[:, :TARGET_LENGTH]

    # Log-Mel
    mel = mel_transform(waveform)
    mel = log_transform(mel)

    features.append(mel.squeeze(0).numpy().astype(np.float16))
    labels.append(1)

print("Samples:", len(features))
print("Feature shape:", features[0].shape)
print("Labels:", len(labels))
```

{"model_id":"3e4f609513b74ea98a40013a53507862","version_major":2,"version_minor":0}

{"model_id":"b5d3cae2d13b4887a3b8800a1b0819c8","version_major":2,"version_minor":0}

```
Samples: 700
Feature shape: (128, 251)
Labels: 700
```

```
import os

WAVEFAKE_FEATURE_DIR = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/features/wavefake_v3"
)

os.makedirs(WAVEFAKE_FEATURE_DIR, exist_ok=True)

print("Directory:", WAVEFAKE_FEATURE_DIR)
```

```
Directory:
/content/drive/MyDrive/voice_cloning_project/features/wavefake_v3
```

```
import os
import torch
import torchaudio
import numpy as np
from datasets import load_dataset

# ========================
# Configuration
# ========================

TARGET_SR = 16000
```

```
TARGET_LENGTH = 16000 * 4

TARGET_UTTERANCES = 7000
CHUNK_SIZE = 1000

GENERATORS = {f"WF{i}" for i in range(1, 8)}

OUTPUT_DIR = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/features/wavefake_v3"
)

os.makedirs(OUTPUT_DIR, exist_ok=True)

# ========================
# Feature extraction
# ========================

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=TARGET_SR,
    n_fft=1024,
    hop_length=256,
    n_mels=128
)

log_transform = torchaudio.transforms.AmplitudeToDB()

# ========================
# Streaming dataset
# ========================

wf_audio = load_dataset(
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

features = []
labels = []

sample_count = 0
chunk_number = 0

# 7000 utterances × 8 rows
MAX_ROWS = TARGET_UTTERANCES * 8

for i, sample in enumerate(wf_audio):

    if i >= MAX_ROWS:
        break
```

```
    generator = sample["real_or_fake"]

    # Skip real/reference samples
    if generator not in GENERATORS:
        continue

    # ========================
    # Decode
    # ========================

    audio = sample["audio"].get_all_samples()

    waveform = audio.data.float()
    sr = audio.sample_rate

    # ========================
    # Mono
    # ========================

    if waveform.ndim == 2:
        waveform = waveform.mean(
            dim=0,
            keepdim=True
        )

    # ========================
    # Resample
    # ========================

    if sr != TARGET_SR:
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            TARGET_SR
        )

    # ========================
    # Fixed 4 seconds
    # ========================

    if waveform.shape[1] < TARGET_LENGTH:

        waveform = torch.nn.functional.pad(
            waveform,
            (0, TARGET_LENGTH - waveform.shape[1])
        )

    else:

        waveform = waveform[:, :TARGET_LENGTH]
```

```
    # ========================
    # Log-Mel
    # ========================

    mel = mel_transform(waveform)
    mel = log_transform(mel)

    feature = mel.squeeze(0).numpy().astype(
        np.float16
    )

    features.append(feature)
    labels.append(1)

    sample_count += 1

    # ========================
    # Save chunk
    # ========================

    if len(features) >= CHUNK_SIZE:

        X = np.stack(features)

        chunk_path = os.path.join(
            OUTPUT_DIR,
            f"X_chunk_{chunk_number:03d}.npy"
        )

        y_path = os.path.join(
            OUTPUT_DIR,
            f"y_chunk_{chunk_number:03d}.npy"
        )

        np.save(chunk_path, X)
        np.save(y_path, np.array(labels, dtype=np.int64))

        print(
            f"Chunk {chunk_number:03d} saved | "
            f"samples: {sample_count} | "
            f"shape: {X.shape}"
        )

        features.clear()
        labels.clear()

        chunk_number += 1

# ========================
# Save final partial chunk
# ========================
```

```
if features:

    X = np.stack(features)

    chunk_path = os.path.join(
        OUTPUT_DIR,
        f"X_chunk_{chunk_number:03d}.npy"
    )

    y_path = os.path.join(
        OUTPUT_DIR,
        f"y_chunk_{chunk_number:03d}.npy"
    )

    np.save(chunk_path, X)
    np.save(y_path, np.array(labels, dtype=np.int64))

    print(
        f"Chunk {chunk_number:03d} saved | "
        f"samples: {sample_count} | "
        f"shape: {X.shape}"
    )

print("\nExtraction complete.")
print("Total WaveFake samples:", sample_count)
```

{"model_id":"3f6c725852104f5c8211c81e8b99c6d3","version_major":2,"version_minor":0}

{"model_id":"6b320161555443a5bfbbf76310dd5f88","version_major":2,"version_minor":0}

```
Chunk 000 saved | samples: 1000 | shape: (1000, 128, 251)
Chunk 001 saved | samples: 2000 | shape: (1000, 128, 251)
Chunk 002 saved | samples: 3000 | shape: (1000, 128, 251)
Chunk 003 saved | samples: 4000 | shape: (1000, 128, 251)

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_4544/413048842.py in <cell line: 0>()
     56 MAX_ROWS = TARGET_UTTERANCES * 8
     57
---> 58 for i, sample in enumerate(wf_audio):
     59
     60     if i >= MAX_ROWS:

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py in __iter__(self)
```

```
   2359             return
   2360
-> 2361         for key, example in ex_iterable:
   2362             # no need to format thanks to
FormattedExamplesIterable
   2363             yield example

/usr/local/lib/python3.13/dist-packages/datasets/iterable_dataset.py
in __iter__(self)
   1881             # feature casting (inc column addition) handled
within self._iter_arrow()
   1882             for key, pa_table in self._iter_arrow():
-> 1883                 batch = formatter.format_batch(pa_table)
   1884                 for example in _batch_to_examples(batch):
   1885                     yield key, example

/usr/local/lib/python3.13/dist-packages/datasets/formatting/formatting
.py in format_batch(self, pa_table)
    469             return LazyBatch(pa_table, self)
    470         batch =
self.python_arrow_extractor().extract_batch(pa_table)
--> 471         batch =
self.python_features_decoder.decode_batch(batch)
    472         return batch
    473

/usr/local/lib/python3.13/dist-packages/datasets/formatting/formatting
.py in decode_batch(self, batch)
    231
    232     def decode_batch(self, batch: dict) -> dict:
--> 233         return self.features.decode_batch(batch,
token_per_repo_id=self.token_per_repo_id) if self.features else batch
    234
    235

/usr/local/lib/python3.13/dist-packages/datasets/features/features.py
in decode_batch(self, batch, token_per_repo_id)
   2141             decoded_batch[column_name] = (
   2142                 [
-> 2143                     decode_nested_example(self[column_name],
value, token_per_repo_id=token_per_repo_id)
   2144                     if value is not None
   2145                     else None

/usr/local/lib/python3.13/dist-packages/datasets/features/features.py
in decode_nested_example(schema, obj, token_per_repo_id)
   1403     elif hasattr(schema, "decode_example") and getattr(schema,
"decode", True):
   1404         # we pass the token to read and decode files from
private repositories in streaming mode
```

```
-> 1405         return schema.decode_example(obj,
token_per_repo_id=token_per_repo_id) if obj is not None else None
   1406     return obj
   1407

/usr/local/lib/python3.13/dist-packages/datasets/features/audio.py in
decode_example(self, value, token_per_repo_id)
    196
    197         else:
--> 198             audio = AudioDecoder(bytes,
stream_index=self.stream_index, sample_rate=self.sampling_rate)
    199         audio._hf_encoded = {"path": path, "bytes": bytes}
    200         audio.metadata.path = path

/usr/local/lib/python3.13/dist-packages/torchcodec/decoders/_audio_dec
oder.py in __init__(self, source, stream_index, sample_rate,
num_channels)
     64             self.stream_index,
     65             self.metadata,
---> 66         ) = create_audio_decoder(
     67             source=source,
     68             seek_mode="approximate",

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/_decoder_util
s.py in create_audio_decoder(source, seek_mode, stream_index,
sample_rate, num_channels)
     73 ) -> tuple[Tensor, int, AudioStreamMetadata]:
     74
---> 75     decoder = create_decoder(source=source,
seek_mode=seek_mode)
     76
     77     container_metadata = get_container_metadata(decoder)

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/_decoder_util
s.py in create_decoder(source, seek_mode)
     46         return create_from_file_like(source, seek_mode)
     47     elif isinstance(source, bytes):
---> 48         return create_from_bytes(source, seek_mode)
     49     elif isinstance(source, Tensor):
     50         return create_from_tensor(source, seek_mode)

/usr/local/lib/python3.13/dist-packages/torchcodec/_core/ops.py in
create_from_bytes(video_bytes, seek_mode)
    151         warnings.filterwarnings("ignore",
category=UserWarning)
    152         buffer = torch.frombuffer(video_bytes,
dtype=torch.uint8)
--> 153     return create_from_tensor(buffer, seek_mode)
    154
    155
```

```
/usr/local/lib/python3.13/dist-packages/torch/_ops.py in
__call__(self, *args, **kwargs)
    863     # that are named "self". This way, all the aten ops can be
called by kwargs.
    864     def __call__(self, /, *args: _P.args, **kwargs: _P.kwargs)
-> _T:
--> 865         return self._op(*args, **kwargs)
    866
    867     # Use positional-only argument to avoid naming collision
with aten ops arguments

KeyboardInterrupt:
```

```
import os

WAVEFAKE_FEATURE_DIR = "/content/wavefake_v3"

os.makedirs(WAVEFAKE_FEATURE_DIR, exist_ok=True)

print("Saving to:", WAVEFAKE_FEATURE_DIR)
```

```
Saving to: /content/wavefake_v3
```

```
import os
import torch
import torchaudio
import numpy as np
from datasets import load_dataset

TARGET_SR = 16000
TARGET_LENGTH = 16000 * 4

TARGET_UTTERANCES = 7000
CHUNK_SIZE = 1000

GENERATORS = {f"WF{i}" for i in range(1, 8)}

OUTPUT_DIR = "/content/wavefake_v3"
os.makedirs(OUTPUT_DIR, exist_ok=True)

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=TARGET_SR,
    n_fft=1024,
    hop_length=256,
    n_mels=128
)

log_transform = torchaudio.transforms.AmplitudeToDB()

wf_audio = load_dataset(
```

```
    "ajaykarthick/wavefake-audio",
    split="train",
    streaming=True
)

features = []
labels = []

sample_count = 0
chunk_number = 0

MAX_ROWS = TARGET_UTTERANCES * 8

for i, sample in enumerate(wf_audio):

    if i >= MAX_ROWS:
        break

    generator = sample["real_or_fake"]

    # Skip real samples
    if generator not in GENERATORS:
        continue

    # Decode
    audio = sample["audio"].get_all_samples()

    waveform = audio.data.float()
    sr = audio.sample_rate

    # Mono
    if waveform.ndim == 2:
        waveform = waveform.mean(
            dim=0,
            keepdim=True
        )

    # Resample
    if sr != TARGET_SR:
        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            TARGET_SR
        )

    # 4 seconds
    if waveform.shape[1] < TARGET_LENGTH:

        waveform = torch.nn.functional.pad(
            waveform,
```

```
            (0, TARGET_LENGTH - waveform.shape[1])
        )

    else:
        waveform = waveform[:, :TARGET_LENGTH]

    # Log-Mel
    mel = mel_transform(waveform)
    mel = log_transform(mel)

    feature = mel.squeeze(0).numpy().astype(np.float16)

    features.append(feature)
    labels.append(1)

    sample_count += 1

    # Save every 1000 samples
    if len(features) == CHUNK_SIZE:

        X = np.stack(features)

        np.save(
            os.path.join(
                OUTPUT_DIR,
                f"X_chunk_{chunk_number:03d}.npy"
            ),
            X
        )

        np.save(
            os.path.join(
                OUTPUT_DIR,
                f"y_chunk_{chunk_number:03d}.npy"
            ),
            np.array(labels, dtype=np.int64)
        )

        print(
            f"Chunk {chunk_number:03d} | "
            f"Total: {sample_count}"
        )

        features.clear()
        labels.clear()

        chunk_number += 1

# Final chunk
if features:
```

```
    X = np.stack(features)

    np.save(
        os.path.join(
            OUTPUT_DIR,
            f"X_chunk_{chunk_number:03d}.npy"
        ),
        X
    )

    np.save(
        os.path.join(
            OUTPUT_DIR,
            f"y_chunk_{chunk_number:03d}.npy"
        ),
        np.array(labels, dtype=np.int64)
    )

    print(
        f"Final chunk {chunk_number:03d} | "
        f"Total: {sample_count}"
    )

print("\nDONE")
print("Total samples:", sample_count)
print("Location:", OUTPUT_DIR)
```

{"model_id":"a0c0813d842040198e608cb5b2ad8c38","version_major":2,"version_minor":0}

{"model_id":"cd422dd6f821432db6d22f15c94aacfe","version_major":2,"version_minor":0}

```
Chunk 000 | Total: 1000
Chunk 001 | Total: 2000
Chunk 002 | Total: 3000
Chunk 003 | Total: 4000
Chunk 004 | Total: 5000
Chunk 005 | Total: 6000
Chunk 006 | Total: 7000
Chunk 007 | Total: 8000
Chunk 008 | Total: 9000
Chunk 009 | Total: 10000
Chunk 010 | Total: 11000
Chunk 011 | Total: 12000
Chunk 012 | Total: 13000
Chunk 013 | Total: 14000
Chunk 014 | Total: 15000
Chunk 015 | Total: 16000
Chunk 016 | Total: 17000
```

```
Chunk 017 | Total: 18000
Chunk 018 | Total: 19000
Chunk 019 | Total: 20000
Chunk 020 | Total: 21000
Chunk 021 | Total: 22000
Chunk 022 | Total: 23000
Chunk 023 | Total: 24000
Chunk 024 | Total: 25000
Chunk 025 | Total: 26000
Chunk 026 | Total: 27000
Chunk 027 | Total: 28000
Chunk 028 | Total: 29000
Chunk 029 | Total: 30000
Chunk 030 | Total: 31000
Chunk 031 | Total: 32000
Chunk 032 | Total: 33000
Chunk 033 | Total: 34000
Chunk 034 | Total: 35000
Chunk 035 | Total: 36000
Chunk 036 | Total: 37000
Chunk 037 | Total: 38000
Chunk 038 | Total: 39000
Chunk 039 | Total: 40000
Chunk 040 | Total: 41000
Chunk 041 | Total: 42000
Chunk 042 | Total: 43000
Chunk 043 | Total: 44000
Chunk 044 | Total: 45000
Chunk 045 | Total: 46000
Chunk 046 | Total: 47000
Chunk 047 | Total: 48000
Chunk 048 | Total: 49000

DONE
Total samples: 49000
Location: /content/wavefake_v3
```

```python
import os
import glob
import numpy as np

WAVEFAKE_DIR = "/content/wavefake_v3"

X_files = sorted(glob.glob(os.path.join(WAVEFAKE_DIR,
"X_chunk_*.npy")))
y_files = sorted(glob.glob(os.path.join(WAVEFAKE_DIR,
"y_chunk_*.npy")))

print("X chunks:", len(X_files))
print("y chunks:", len(y_files))
```

```
total_samples = 0

for X_path, y_path in zip(X_files, y_files):

    X = np.load(X_path, mmap_mode="r")
    y = np.load(y_path, mmap_mode="r")

    print(
        os.path.basename(X_path),
        X.shape,
        y.shape,
        X.dtype,
        y.dtype
    )

    assert X.shape[0] == y.shape[0]
    assert X.shape[1:] == (128, 251)
    assert np.all(y == 1)

    total_samples += X.shape[0]

print("\nTotal samples:", total_samples)
print("Expected:", 49000)

assert total_samples == 49000

print("\n WaveFake V3 feature cache verified.")
```

```
X chunks: 49
y chunks: 49
X_chunk_000.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_001.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_002.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_003.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_004.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_005.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_006.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_007.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_008.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_009.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_010.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_011.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_012.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_013.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_014.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_015.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_016.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_017.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_018.npy (1000, 128, 251) (1000,) float16 int64
```

```
X_chunk_019.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_020.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_021.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_022.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_023.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_024.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_025.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_026.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_027.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_028.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_029.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_030.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_031.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_032.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_033.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_034.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_035.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_036.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_037.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_038.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_039.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_040.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_041.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_042.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_043.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_044.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_045.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_046.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_047.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_048.npy (1000, 128, 251) (1000,) float16 int64

Total samples: 49000
Expected: 49000

 WaveFake V3 feature cache verified.
```

```python
import os
import glob
import numpy as np

WAVEFAKE_DIR = "/content/wavefake_v3"

X_files = sorted(glob.glob(os.path.join(WAVEFAKE_DIR,
"X_chunk_*.npy")))
y_files = sorted(glob.glob(os.path.join(WAVEFAKE_DIR,
"y_chunk_*.npy")))

print("X chunks:", len(X_files))
print("y chunks:", len(y_files))
```

```
total_samples = 0

for X_path, y_path in zip(X_files, y_files):

    X = np.load(X_path, mmap_mode="r")
    y = np.load(y_path, mmap_mode="r")

    print(
        os.path.basename(X_path),
        X.shape,
        y.shape,
        X.dtype,
        y.dtype
    )

    assert X.shape[0] == y.shape[0]
    assert X.shape[1:] == (128, 251)
    assert np.all(y == 1)

    total_samples += X.shape[0]

print("\nTotal samples:", total_samples)
print("Expected:", 49000)

assert total_samples == 49000

print("\n WaveFake V3 feature cache verified.")
```

```
X chunks: 49
y chunks: 49
X_chunk_000.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_001.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_002.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_003.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_004.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_005.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_006.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_007.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_008.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_009.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_010.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_011.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_012.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_013.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_014.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_015.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_016.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_017.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_018.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_019.npy (1000, 128, 251) (1000,) float16 int64
```

```
X_chunk_020.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_021.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_022.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_023.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_024.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_025.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_026.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_027.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_028.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_029.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_030.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_031.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_032.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_033.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_034.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_035.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_036.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_037.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_038.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_039.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_040.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_041.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_042.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_043.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_044.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_045.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_046.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_047.npy (1000, 128, 251) (1000,) float16 int64
X_chunk_048.npy (1000, 128, 251) (1000,) float16 int64

Total samples: 49000
Expected: 49000

 WaveFake V3 feature cache verified.
```

```python
import os
import numpy as np

FEATURE_DIR = "/content/drive/MyDrive/voice_cloning_project/features"

print("Files in features directory:\n")

for f in sorted(os.listdir(FEATURE_DIR)):
    print(f)
```

```
Files in features directory:

X_dev.npy
X_train.npy
wavefake_v3
```

```
y_dev.npy
y_train.npy

import numpy as np
import os

FEATURE_DIR = "/content/drive/MyDrive/voice_cloning_project/features"
WF_DIR = "/content/wavefake_v3"

# ASVspoof
X_train = np.load(os.path.join(FEATURE_DIR, "X_train.npy"),
mmap_mode="r")
y_train = np.load(os.path.join(FEATURE_DIR, "y_train.npy"),
mmap_mode="r")

X_dev = np.load(os.path.join(FEATURE_DIR, "X_dev.npy"), mmap_mode="r")
y_dev = np.load(os.path.join(FEATURE_DIR, "y_dev.npy"), mmap_mode="r")

print("ASVspoof Train:")
print("X:", X_train.shape, X_train.dtype)
print("y:", y_train.shape, y_train.dtype)

print("\nASVspoof Dev:")
print("X:", X_dev.shape, X_dev.dtype)
print("y:", y_dev.shape, y_dev.dtype)

# WaveFake chunks
x_chunks = sorted(
    f for f in os.listdir(WF_DIR)
    if f.startswith("X_chunk_") and f.endswith(".npy")
)

y_chunks = sorted(
    f for f in os.listdir(WF_DIR)
    if f.startswith("y_chunk_") and f.endswith(".npy")
)

print("\nWaveFake:")
print("X chunks:", len(x_chunks))
print("y chunks:", len(y_chunks))

# Check first and last chunks
for f in [x_chunks[0], x_chunks[-1]]:
    arr = np.load(os.path.join(WF_DIR, f), mmap_mode="r")
    print(f, arr.shape, arr.dtype)

for f in [y_chunks[0], y_chunks[-1]]:
    arr = np.load(os.path.join(WF_DIR, f), mmap_mode="r")
    print(f, arr.shape, arr.dtype)

# Total WaveFake samples
```

```
total_wf = sum(
    np.load(os.path.join(WF_DIR, f), mmap_mode="r").shape[0]
    for f in x_chunks
)

print("\nTotal WaveFake samples:", total_wf)
print("Combined training samples:", len(y_train) + total_wf)
```

```
ASVspoof Train:
X: (25380, 128, 251) float16
y: (25380,) int64

ASVspoof Dev:
X: (24844, 128, 251) float16
y: (24844,) int64

WaveFake:
X chunks: 49
y chunks: 49
X_chunk_000.npy (1000, 128, 251) float16
X_chunk_048.npy (1000, 128, 251) float16
y_chunk_000.npy (1000,) int64
y_chunk_048.npy (1000,) int64

Total WaveFake samples: 49000
Combined training samples: 74380
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

FEATURE_DIR = "/content/drive/MyDrive/voice_cloning_project/features"
WF_DIR = "/content/wavefake_v3"


class V3TrainDataset(Dataset):

    def __init__(self, asv_x_path, asv_y_path, wavefake_dir):

        # ASVspoof 2019
        self.asv_x = np.load(asv_x_path, mmap_mode="r")
        self.asv_y = np.load(asv_y_path, mmap_mode="r")

        # WaveFake chunks
        self.x_files = sorted([
            f for f in os.listdir(wavefake_dir)
            if f.startswith("X_chunk_") and f.endswith(".npy")
        ])

        self.y_files = sorted([
```

```
            f for f in os.listdir(wavefake_dir)
            if f.startswith("y_chunk_") and f.endswith(".npy")
        ])

        self.wavefake_dir = wavefake_dir

        # Number of samples per chunk
        self.chunk_sizes = []

        for f in self.x_files:
            arr = np.load(
                os.path.join(wavefake_dir, f),
                mmap_mode="r"
            )
            self.chunk_sizes.append(arr.shape[0])

        self.chunk_offsets = np.cumsum(
            [0] + self.chunk_sizes
        )

        self.asv_len = len(self.asv_y)
        self.wavefake_len = sum(self.chunk_sizes)

        self.total_len = self.asv_len + self.wavefake_len

    def __len__(self):
        return self.total_len

    def __getitem__(self, idx):

        # ------------------------------
        # ASVspoof sample
        # ------------------------------
        if idx < self.asv_len:

            x = self.asv_x[idx]
            y = self.asv_y[idx]

        # ------------------------------
        # WaveFake sample
        # ------------------------------
        else:

            wf_idx = idx - self.asv_len

            # Find which chunk contains this index
            chunk_idx = np.searchsorted(
                self.chunk_offsets,
                wf_idx,
                side="right"
```

```
            ) - 1

            local_idx = (
                wf_idx - self.chunk_offsets[chunk_idx]
            )

            x_file = self.x_files[chunk_idx]
            y_file = self.y_files[chunk_idx]

            X_chunk = np.load(
                os.path.join(self.wavefake_dir, x_file),
                mmap_mode="r"
            )

            y_chunk = np.load(
                os.path.join(self.wavefake_dir, y_file),
                mmap_mode="r"
            )

            x = X_chunk[local_idx]
            y = y_chunk[local_idx]

        # CNN expects [channel, mel, time]
        x = torch.from_numpy(
            np.asarray(x, dtype=np.float32)
        ).unsqueeze(0)

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        return x, y


train_dataset = V3TrainDataset(
    os.path.join(FEATURE_DIR, "X_train.npy"),
    os.path.join(FEATURE_DIR, "y_train.npy"),
    WF_DIR
)

print("V3 training dataset created.")
print("Total samples:", len(train_dataset))
```

```
V3 training dataset created.
Total samples: 74380
```

```
print("Expected:", 74380)
print("Actual:  ", len(train_dataset))

# Check beginning: ASVspoof
```

```
x0, y0 = train_dataset[0]

# Check end: WaveFake
x_last, y_last = train_dataset[len(train_dataset) - 1]

print("\nASVspoof sample:")
print("Shape:", x0.shape)
print("Label:", y0.item())

print("\nWaveFake sample:")
print("Shape:", x_last.shape)
print("Label:", y_last.item())
```

```
Expected: 74380
Actual:   74380

ASVspoof sample:
Shape: torch.Size([1, 128, 251])
Label: 0.0

WaveFake sample:
Shape: torch.Size([1, 128, 251])
Label: 1.0
```

```
import torch

N_BONAFIDE = 2580
N_SPOOF = 71800
N_TOTAL = N_BONAFIDE + N_SPOOF

# Balanced class weights
weight_bonafide = N_TOTAL / (2 * N_BONAFIDE)
weight_spoof = N_TOTAL / (2 * N_SPOOF)

print("V3 class weights:")
print("Bonafide (0):", weight_bonafide)
print("Spoof (1):   ", weight_spoof)
```

```
V3 class weights:
Bonafide (0): 14.414728682170542
Spoof (1):    0.517966573816156
```

```
from torch.utils.data import DataLoader

BATCH_SIZE = 64

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0,
```

```
    pin_memory=True
)

print("Total samples:", len(train_dataset))
print("Batch size:", BATCH_SIZE)
print("Batches per epoch:", len(train_loader))
```

```
Total samples: 74380
Batch size: 64
Batches per epoch: 1163
```

```
X_batch, y_batch = next(iter(train_loader))

print("X shape:", X_batch.shape)
print("y shape:", y_batch.shape)
print("Unique labels:", torch.unique(y_batch))

print("Bonafide:", (y_batch == 0).sum().item())
print("Spoof:", (y_batch == 1).sum().item())
```

```
X shape: torch.Size([64, 1, 128, 251])
y shape: torch.Size([64])
Unique labels: tensor([0., 1.])
Bonafide: 3
Spoof: 61
```

```
import os

RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"

print("Files in results:")
for f in sorted(os.listdir(RESULTS_DIR)):
    print(f)
```

```
Files in results:
2021_DF_baseline_10000.json
asvspoof2019_results.txt
asvspoof2021_v2_predictions.npz
v2_2021_detailed_eval.npz
v2_asvspoof2021_predictions.npz
v2_asvspoof2021_results.json
```

```
import torch
import torch.nn as nn

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = VoiceCNN().to(device)
```

```
print("Device:", device)
print("Fresh V3 model created.")
```

```
Device: cuda
Fresh V3 model created.
```

```
dev_dataset = ASVspoofDevDataset(
    os.path.join(FEATURE_DIR, "X_dev.npy"),
    os.path.join(FEATURE_DIR, "y_dev.npy")
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Validation samples:", len(dev_dataset))
print("Validation batches:", len(dev_loader))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_4544/2133998916.py in <cell line: 0>()
----> 1 dev_dataset = ASVspoofDevDataset(
      2     os.path.join(FEATURE_DIR, "X_dev.npy"),
      3     os.path.join(FEATURE_DIR, "y_dev.npy")
      4 )
      5

NameError: name 'ASVspoofDevDataset' is not defined
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader


class ASVspoofDevDataset(Dataset):

    def __init__(self, x_path, y_path):
        self.X = np.load(x_path, mmap_mode="r")
        self.y = np.load(y_path, mmap_mode="r")

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
```

```
        x = self.X[idx]
        y = self.y[idx]

        # Convert [128, 251] → [1, 128, 251]
        x = torch.from_numpy(
            np.asarray(x, dtype=np.float32)
        ).unsqueeze(0)

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        return x, y

dev_dataset = ASVspoofDevDataset(
    os.path.join(FEATURE_DIR, "X_dev.npy"),
    os.path.join(FEATURE_DIR, "y_dev.npy")
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Validation samples:", len(dev_dataset))
print("Validation batches:", len(dev_loader))
```

```
Validation samples: 24844
Validation batches: 389
```

```
X_test, y_test = next(iter(train_loader))

X_test = X_test.to(device, non_blocking=True)
y_test = y_test.to(device, non_blocking=True)

model.eval()

with torch.no_grad():
    logits = model(X_test)

print("Input shape :", X_test.shape)
print("Output shape:", logits.shape)
print("Device      :", X_test.device)
print("Logits min  :", logits.min().item())
print("Logits max  :", logits.max().item())
```

```
Input shape : torch.Size([64, 1, 128, 251])
Output shape: torch.Size([64])
Device      : cuda:0
Logits min  : 0.7254846692085266
Logits max  : 2.1616785526275635

criterion = nn.BCEWithLogitsLoss(reduction="none")

sample_weights = torch.where(
    y_test == 0,
    torch.tensor(CLASS_WEIGHT_0, device=device),
    torch.tensor(CLASS_WEIGHT_1, device=device)
)

loss_per_sample = criterion(logits, y_test)

weighted_loss = (
    loss_per_sample * sample_weights
).mean()

print("Weighted loss:", weighted_loss.item())

---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
/tmp/ipykernel_4544/677021486.py in <cell line: 0>()
      3 sample_weights = torch.where(
      4     y_test == 0,
----> 5     torch.tensor(CLASS_WEIGHT_0, device=device),
      6     torch.tensor(CLASS_WEIGHT_1, device=device)
      7 )

NameError: name 'CLASS_WEIGHT_0' is not defined

# V3 class weights
CLASS_WEIGHT_0 = 14.414728682170542   # Bonafide
CLASS_WEIGHT_1 = 0.517966573816156    # Spoof

criterion = nn.BCEWithLogitsLoss(reduction="none")

sample_weights = torch.where(
    y_test == 0,
    torch.tensor(CLASS_WEIGHT_0, device=device),
    torch.tensor(CLASS_WEIGHT_1, device=device)
)

loss_per_sample = criterion(logits, y_test)

weighted_loss = (
    loss_per_sample * sample_weights
```

```
).mean()

print("Class weight 0 (Bonafide):", CLASS_WEIGHT_0)
print("Class weight 1 (Spoof):", CLASS_WEIGHT_1)
print("Weighted loss:", weighted_loss.item())
```

```
Class weight 0 (Bonafide): 14.414728682170542
Class weight 1 (Spoof): 0.517966573816156
Weighted loss: 2.0515189170837402
```

```
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import roc_auc_score, f1_score, accuracy_score

# ------------------------------------------------
# V3 Training Configuration
# ------------------------------------------------

MAX_EPOCHS = 50
EARLY_STOPPING_PATIENCE = 7

BEST_MODEL_PATH = "/content/voice_cnn_v3_best.pth"

CLASS_WEIGHT_0 = 14.414728682170542  # Bonafide
CLASS_WEIGHT_1 = 0.517966573816156   # Spoof

criterion = nn.BCEWithLogitsLoss(reduction="none")

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=2
)

history = {
    "train_loss": [],
    "val_loss": [],
    "val_auc": [],
    "val_f1": [],
    "val_accuracy": [],
    "learning_rate": []
}
```

```
best_val_auc = -float("inf")
epochs_without_improvement = 0

print("Starting V3 training...")
print("Device:", device)
print("Training samples:", len(train_dataset))
print("Validation samples:", len(dev_dataset))
```

```
Starting V3 training...
Device: cuda
Training samples: 74380
Validation samples: 24844
```

```
for epoch in range(1, MAX_EPOCHS + 1):

    # ================================================
    # TRAINING
    # ================================================

    model.train()

    running_loss = 0.0
    total_samples = 0

    for X, y in train_loader:

        X = X.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad()

        # Forward pass
        logits = model(X)

        # Per-sample BCE loss
        loss_per_sample = criterion(logits, y)

        # Apply class weights
        sample_weights = torch.where(
            y == 0,
            torch.tensor(
                CLASS_WEIGHT_0,
                device=device
            ),
            torch.tensor(
                CLASS_WEIGHT_1,
                device=device
            )
        )

        # Weighted loss
```

```
        loss = (
            loss_per_sample * sample_weights
        ).mean()

        # Backpropagation
        loss.backward()

        # Update parameters
        optimizer.step()

        batch_size = X.size(0)

        running_loss += loss.item() * batch_size
        total_samples += batch_size

    train_loss = running_loss / total_samples


    # ================================================
    # VALIDATION
    # ================================================

    model.eval()

    val_running_loss = 0.0
    val_samples = 0

    all_probs = []
    all_labels = []

    with torch.no_grad():

        for X, y in dev_loader:

            X = X.to(device, non_blocking=True)
            y = y.to(device, non_blocking=True)

            logits = model(X)

            # Validation loss
            loss_per_sample = criterion(logits, y)

            sample_weights = torch.where(
                y == 0,
                torch.tensor(
                    CLASS_WEIGHT_0,
                    device=device
                ),
                torch.tensor(
                    CLASS_WEIGHT_1,
                    device=device
```

```
                )
            )

            loss = (
                loss_per_sample * sample_weights
            ).mean()

            batch_size = X.size(0)

            val_running_loss += (
                loss.item() * batch_size
            )

            val_samples += batch_size

            # Convert logits → probabilities
            probs = torch.sigmoid(logits)

            all_probs.extend(
                probs.cpu().numpy()
            )

            all_labels.extend(
                y.cpu().numpy()
            )

    val_loss = val_running_loss / val_samples

    all_probs = np.asarray(all_probs)
    all_labels = np.asarray(all_labels)

    # ROC-AUC
    val_auc = roc_auc_score(
        all_labels,
        all_probs
    )

    # Threshold = 0.5 for monitoring
    val_preds = (
        all_probs >= 0.5
    ).astype(int)

    val_f1 = f1_score(
        all_labels,
        val_preds
    )

    val_accuracy = accuracy_score(
        all_labels,
        val_preds
```

```
    )


    # ================================================
    # LEARNING RATE SCHEDULER
    # ================================================

    scheduler.step(val_loss)

    current_lr = optimizer.param_groups[0]["lr"]


    # ================================================
    # SAVE HISTORY
    # ================================================

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_auc"].append(val_auc)
    history["val_f1"].append(val_f1)
    history["val_accuracy"].append(val_accuracy)
    history["learning_rate"].append(current_lr)


    # ================================================
    # BEST MODEL
    # ================================================

    if val_auc > best_val_auc:

        best_val_auc = val_auc

        epochs_without_improvement = 0

        torch.save(
            model.state_dict(),
            BEST_MODEL_PATH
        )

        best_marker = " ★ BEST"

    else:

        epochs_without_improvement += 1

        best_marker = ""


    # ================================================
    # PRINT RESULTS
    # ================================================
```

```
    print(
        f"Epoch {epoch:02d}/{MAX_EPOCHS} | "
        f"Train Loss: {train_loss:.4f} | "
        f"Val Loss: {val_loss:.4f} | "
        f"Val AUC: {val_auc:.6f} | "
        f"Val F1: {val_f1:.6f} | "
        f"Val Acc: {val_accuracy:.6f} | "
        f"LR: {current_lr:.6f}"
        f"{best_marker}"
    )


    # ================================================
    # EARLY STOPPING
    # ================================================

    if epochs_without_improvement >= EARLY_STOPPING_PATIENCE:

        print(
            f"\nEarly stopping triggered after "
            f"{epoch} epochs."
        )

        break


print("\nV3 training complete.")
print("Best validation ROC-AUC:", best_val_auc)
print("Best model:", BEST_MODEL_PATH)
```

```
Epoch 01/50 | Train Loss: 0.3584 | Val Loss: 1.0583 | Val AUC:
0.752871 | Val F1: 0.001882 | Val Acc: 0.103405 | LR: 0.001000 ★ BEST
Epoch 02/50 | Train Loss: 0.2700 | Val Loss: 8.0929 | Val AUC:
0.870540 | Val F1: 0.946249 | Val Acc: 0.898044 | LR: 0.001000 ★ BEST
Epoch 03/50 | Train Loss: 0.1689 | Val Loss: 0.3161 | Val AUC:
0.983388 | Val F1: 0.964521 | Val Acc: 0.938094 | LR: 0.001000 ★ BEST
Epoch 04/50 | Train Loss: 0.0875 | Val Loss: 0.1972 | Val AUC:
0.997386 | Val F1: 0.914561 | Val Acc: 0.858718 | LR: 0.001000 ★ BEST
Epoch 05/50 | Train Loss: 0.0609 | Val Loss: 0.0952 | Val AUC:
0.998276 | Val F1: 0.988902 | Val Acc: 0.980277 | LR: 0.001000 ★ BEST
Epoch 06/50 | Train Loss: 0.0422 | Val Loss: 0.0428 | Val AUC:
0.999564 | Val F1: 0.988661 | Val Acc: 0.979874 | LR: 0.001000 ★ BEST
Epoch 07/50 | Train Loss: 0.0508 | Val Loss: 0.0514 | Val AUC:
0.999567 | Val F1: 0.994843 | Val Acc: 0.990782 | LR: 0.001000 ★ BEST
Epoch 08/50 | Train Loss: 0.0276 | Val Loss: 0.0524 | Val AUC:
0.999509 | Val F1: 0.993298 | Val Acc: 0.988045 | LR: 0.001000
Epoch 09/50 | Train Loss: 0.0379 | Val Loss: 0.0792 | Val AUC:
0.999805 | Val F1: 0.998071 | Val Acc: 0.996538 | LR: 0.000500 ★ BEST
Epoch 10/50 | Train Loss: 0.0125 | Val Loss: 0.0259 | Val AUC:
```

```
0.999907 | Val F1: 0.998900 | Val Acc: 0.998028 | LR: 0.000500 ★ BEST
Epoch 11/50 | Train Loss: 0.0137 | Val Loss: 0.0596 | Val AUC:
0.999884 | Val F1: 0.998228 | Val Acc: 0.996820 | LR: 0.000500
Epoch 12/50 | Train Loss: 0.0194 | Val Loss: 0.0563 | Val AUC:
0.999915 | Val F1: 0.998587 | Val Acc: 0.997464 | LR: 0.000500 ★ BEST
Epoch 13/50 | Train Loss: 0.0101 | Val Loss: 0.0179 | Val AUC:
0.999939 | Val F1: 0.998653 | Val Acc: 0.997585 | LR: 0.000500 ★ BEST
Epoch 14/50 | Train Loss: 0.0124 | Val Loss: 0.0177 | Val AUC:
0.999951 | Val F1: 0.997844 | Val Acc: 0.996136 | LR: 0.000500 ★ BEST
Epoch 15/50 | Train Loss: 0.0090 | Val Loss: 0.0880 | Val AUC:
0.999953 | Val F1: 0.998701 | Val Acc: 0.997665 | LR: 0.000500 ★ BEST
Epoch 16/50 | Train Loss: 0.0147 | Val Loss: 0.0117 | Val AUC:
0.999976 | Val F1: 0.998158 | Val Acc: 0.996699 | LR: 0.000500 ★ BEST

---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_4544/2142648821.py in <cell line: 0>()
     49         batch_size = X.size(0)
     50
---> 51         running_loss += loss.item() * batch_size
     52         total_samples += batch_size
     53

KeyboardInterrupt:
```

```python
import torch
import os
import shutil
import numpy as np

# Paths
CHECKPOINT_PATH = "/content/voice_cloning_project_v3_resume.pth"
BEST_MODEL_PATH = "/content/voice_cnn_v3_best.pth"
HISTORY_PATH = "/content/v3_training_history.npz"

# Current epoch completed
CURRENT_EPOCH = 16

checkpoint = {
    # Model
    "model_state_dict": model.state_dict(),

    # Optimizer + scheduler
    "optimizer_state_dict": optimizer.state_dict(),
    "scheduler_state_dict": scheduler.state_dict(),

    # Training progress
    "epoch": CURRENT_EPOCH,
```

```
    "best_val_auc": best_val_auc,
    "epochs_without_improvement": epochs_without_improvement,

    # Training configuration
    "max_epochs": MAX_EPOCHS,
    "early_stopping_patience": EARLY_STOPPING_PATIENCE,
    "batch_size": BATCH_SIZE,
    "learning_rate": 0.001,
    "weight_decay": 1e-4,

    # Class weights
    "class_weight_0": CLASS_WEIGHT_0,
    "class_weight_1": CLASS_WEIGHT_1,

    # Complete history
    "history": history
}

torch.save(checkpoint, CHECKPOINT_PATH)

print("□ Resume checkpoint saved:")
print(CHECKPOINT_PATH)
```

```
□ Resume checkpoint saved:
/content/voice_cloning_project_v3_resume.pth
```

```
import os
import shutil
import numpy as np
import torch

# Google Drive destination
DRIVE_RESULTS = "/content/drive/MyDrive/voice_cloning_project/results"
os.makedirs(DRIVE_RESULTS, exist_ok=True)

# ==============================
# 1. Save resume checkpoint
# ==============================

resume_path = os.path.join(DRIVE_RESULTS, "v3_resume_epoch16.pth")

checkpoint = {
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "scheduler_state_dict": scheduler.state_dict(),

    "epoch": 16,
    "best_val_auc": best_val_auc,
    "epochs_without_improvement": epochs_without_improvement,
```

```
    "max_epochs": MAX_EPOCHS,
    "early_stopping_patience": EARLY_STOPPING_PATIENCE,

    "batch_size": BATCH_SIZE,
    "learning_rate": 0.001,
    "weight_decay": 1e-4,

    "class_weight_0": CLASS_WEIGHT_0,
    "class_weight_1": CLASS_WEIGHT_1,

    "history": history
}

torch.save(checkpoint, resume_path)


# ==============================
# 2. Save best model
# ==============================

best_drive_path = os.path.join(
    DRIVE_RESULTS,
    "voice_cnn_v3_best.pth"
)

shutil.copy2(
    BEST_MODEL_PATH,
    best_drive_path
)


# ==============================
# 3. Save training history
# ==============================

history_path = os.path.join(
    DRIVE_RESULTS,
    "v3_training_history.npz"
)

np.savez(
    history_path,
    train_loss=np.array(history["train_loss"]),
    val_loss=np.array(history["val_loss"]),
    val_auc=np.array(history["val_auc"]),
    val_f1=np.array(history["val_f1"]),
    val_accuracy=np.array(history["val_accuracy"]),
    learning_rate=np.array(history["learning_rate"])
)
```

```python
# ==============================
# 4. Verify
# ==============================

print("\n V3 FILES SAVED TO GOOGLE DRIVE\n")

for filename in [
    "v3_resume_epoch16.pth",
    "voice_cnn_v3_best.pth",
    "v3_training_history.npz"
]:
    path = os.path.join(DRIVE_RESULTS, filename)

    if os.path.exists(path):
        size_mb = os.path.getsize(path) / (1024**2)
        print(f" {filename}  ({size_mb:.2f} MB)")
    else:
        print(f" MISSING: {filename}")
```

```
 V3 FILES SAVED TO GOOGLE DRIVE

 v3_resume_epoch16.pth  (1.28 MB)
 voice_cnn_v3_best.pth  (0.43 MB)
 v3_training_history.npz  (0.00 MB)
```

```python
import os
import shutil

SOURCE = "/content/wavefake_v3"
DEST = "/content/drive/MyDrive/voice_cloning_project/features/wavefake_v3"

os.makedirs(DEST, exist_ok=True)

# Copy all 49 X chunks + 49 y chunks
files = sorted(os.listdir(SOURCE))

print(f"Files to copy: {len(files)}")

for i, filename in enumerate(files, 1):
    src = os.path.join(SOURCE, filename)
    dst = os.path.join(DEST, filename)

    shutil.copy2(src, dst)

    print(f"[{i}/{len(files)}] {filename}")

print("\n All WaveFake chunks copied to Google Drive.")
```

```
Files to copy: 98
[1/98] X_chunk_000.npy
[2/98] X_chunk_001.npy
[3/98] X_chunk_002.npy
[4/98] X_chunk_003.npy
[5/98] X_chunk_004.npy
[6/98] X_chunk_005.npy
[7/98] X_chunk_006.npy
[8/98] X_chunk_007.npy
[9/98] X_chunk_008.npy
[10/98] X_chunk_009.npy
[11/98] X_chunk_010.npy
[12/98] X_chunk_011.npy
[13/98] X_chunk_012.npy
[14/98] X_chunk_013.npy
[15/98] X_chunk_014.npy
[16/98] X_chunk_015.npy
[17/98] X_chunk_016.npy
[18/98] X_chunk_017.npy
[19/98] X_chunk_018.npy
[20/98] X_chunk_019.npy
[21/98] X_chunk_020.npy
[22/98] X_chunk_021.npy
[23/98] X_chunk_022.npy
[24/98] X_chunk_023.npy
[25/98] X_chunk_024.npy
[26/98] X_chunk_025.npy
```

```
from google.colab import drive
drive.mount('/content/drive')

import os
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import roc_auc_score, f1_score, accuracy_score

print("CUDA:", torch.cuda.is_available())
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Mounted at /content/drive
CUDA: True
Device: cuda
```

```
PROJECT_DIR = "/content/drive/MyDrive/voice_cloning_project"

FEATURE_DIR = os.path.join(PROJECT_DIR, "features")
WF_DIR = os.path.join(FEATURE_DIR, "wavefake_v3")
```

```
RESULTS_DIR = os.path.join(PROJECT_DIR, "results")

RESUME_PATH = os.path.join(
    RESULTS_DIR, "v3_resume_epoch16.pth"
)

BEST_PATH = os.path.join(
    RESULTS_DIR, "voice_cnn_v3_best.pth"
)

print("Resume checkpoint:", os.path.exists(RESUME_PATH))
print("Best model:", os.path.exists(BEST_PATH))
print("WaveFake directory:", os.path.exists(WF_DIR))
```

```
Resume checkpoint: True
Best model: True
WaveFake directory: True
```

```
import os
import numpy as np

WF_DIR =
"/content/drive/MyDrive/voice_cloning_project/features/wavefake_v3"

x_files = sorted([
    f for f in os.listdir(WF_DIR)
    if f.startswith("X_chunk_") and f.endswith(".npy")
])

y_files = sorted([
    f for f in os.listdir(WF_DIR)
    if f.startswith("y_chunk_") and f.endswith(".npy")
])

print("X chunks found:", len(x_files))
print("Y chunks found:", len(y_files))

total_x = 0
total_y = 0
errors = []

for i, (xf, yf) in enumerate(zip(x_files, y_files)):
    try:
        X = np.load(os.path.join(WF_DIR, xf), mmap_mode="r")
        y = np.load(os.path.join(WF_DIR, yf), mmap_mode="r")

        if X.shape != (1000, 128, 251):
            errors.append(f"{xf}: wrong X shape {X.shape}")

        if y.shape != (1000,):
            errors.append(f"{yf}: wrong y shape {y.shape}")
```

```
        total_x += len(X)
        total_y += len(y)

    except Exception as e:
        errors.append(f"Chunk {i}: {e}")

print("\n===== VERIFICATION =====")
print("Total X samples:", total_x)
print("Total y samples:", total_y)

print("\nExpected:")
print("X samples: 49,000")
print("Y samples: 49,000")

print("\nErrors:", len(errors))

if errors:
    for e in errors[:20]:
        print("", e)
else:
    print(" All chunks are valid")

if total_x == 49000 and total_y == 49000 and len(x_files) == 49 and
len(y_files) == 49 and not errors:
    print("\n WaveFake V3 DATA VERIFICATION PASSED")
else:
    print("\n⚠ Verification FAILED — do not continue yet.")
```

```
X chunks found: 49
Y chunks found: 49

===== VERIFICATION =====
Total X samples: 49000
Total y samples: 49000

Expected:
X samples: 49,000
Y samples: 49,000

Errors: 0
 All chunks are valid

 WaveFake V3 DATA VERIFICATION PASSED
```

```
import os
import numpy as np

FEATURE_DIR = "/content/drive/MyDrive/voice_cloning_project/features"

checks = {
```

```
    "X_train": ("X_train.npy", (25380, 128, 251)),
    "y_train": ("y_train.npy", (25380,)),
    "X_dev":   ("X_dev.npy",   (24844, 128, 251)),
    "y_dev":   ("y_dev.npy",   (24844,))
}

errors = []

print("===== ASVSPOOF DATA VERIFICATION =====")

for name, (filename, expected_shape) in checks.items():
    path = os.path.join(FEATURE_DIR, filename)

    if not os.path.exists(path):
        errors.append(f"{filename} missing")
        print(f" {name}: MISSING")
        continue

    arr = np.load(path, mmap_mode="r")

    print(f" {name}: shape={arr.shape}")

    if arr.shape != expected_shape:
        errors.append(
            f"{filename}: expected {expected_shape}, got {arr.shape}"
        )

print("\n===== RESULT =====")

if not errors:
    print(" ASVSPOOF DATA VERIFICATION PASSED")
else:
    print("⚠ VERIFICATION FAILED")
    for error in errors:
        print("", error)
```

```
===== ASVSPOOF DATA VERIFICATION =====
 X_train: shape=(25380, 128, 251)
 y_train: shape=(25380,)
 X_dev: shape=(24844, 128, 251)
 y_dev: shape=(24844,)

===== RESULT =====
 ASVSPOOF DATA VERIFICATION PASSED
```

```
checkpoint = torch.load(
    RESUME_PATH,
    map_location="cpu",
    weights_only=False
)
```

```python
required_keys = [
    "model_state_dict",
    "optimizer_state_dict",
    "scheduler_state_dict",
    "epoch",
    "best_val_auc",
    "epochs_without_improvement",
    "history"
]

print("===== CHECKPOINT VERIFICATION =====")

for key in required_keys:
    if key in checkpoint:
        print(f"□ {key}")
    else:
        print(f"□ {key}")

print("\nSaved epoch:", checkpoint["epoch"])
print("Best validation AUC:", checkpoint["best_val_auc"])
print("Epochs without improvement:",
      checkpoint["epochs_without_improvement"])

history = checkpoint["history"]

print("\nHistory lengths:")
for key, value in history.items():
    print(f"  {key}: {len(value)}")

print("\n===== RESULT =====")

if (
    all(key in checkpoint for key in required_keys)
    and checkpoint["epoch"] == 16
    and len(history["train_loss"]) == 16
    and len(history["val_auc"]) == 16
):
    print("□ CHECKPOINT VERIFICATION PASSED")
    print("□ Safe to resume from Epoch 17")
else:
    print("⚠ CHECKPOINT VERIFICATION FAILED")
```

```
===== CHECKPOINT VERIFICATION =====
□ model_state_dict
□ optimizer_state_dict
□ scheduler_state_dict
□ epoch
□ best_val_auc
□ epochs_without_improvement
□ history
```

```
Saved epoch: 16
Best validation AUC: 0.9999759726280179
Epochs without improvement: 0

History lengths:
  train_loss: 16
  val_loss: 16
  val_auc: 16
  val_f1: 16
  val_accuracy: 16
  learning_rate: 16

===== RESULT =====
 CHECKPOINT VERIFICATION PASSED
 Safe to resume from Epoch 17
```

```python
import torch
import torch.nn as nn

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class VoiceCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.pool = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, 1)
```

```
        )

    def forward(self, x):
        x = self.features(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x.squeeze(1)

model = VoiceCNN().to(device)

print("Model recreated successfully")
print("Device:", device)
```

```
Model recreated successfully
Device: cuda
```

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=2
)

CLASS_WEIGHT_0 = 14.414728682170542
CLASS_WEIGHT_1 = 0.517966573816156

criterion = nn.BCEWithLogitsLoss(reduction="none")

print("Optimizer:", optimizer.__class__.__name__)
print("Initial LR:", optimizer.param_groups[0]["lr"])
print("Weight decay:", optimizer.param_groups[0]["weight_decay"])
```

```
Optimizer: Adam
Initial LR: 0.001
Weight decay: 0.0001
```

```
RESUME_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v3_resume_epoch1
6.pth"
BEST_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/voice_cnn_v3_bes
t.pth"

checkpoint = torch.load(
```

```
    RESUME_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
scheduler.load_state_dict(checkpoint["scheduler_state_dict"])

start_epoch = checkpoint["epoch"] + 1
best_val_auc = checkpoint["best_val_auc"]
epochs_without_improvement = checkpoint["epochs_without_improvement"]
history = checkpoint["history"]

print("\n===== V3 RESUME STATUS =====")
print("Saved epoch:", checkpoint["epoch"])
print("Starting epoch:", start_epoch)
print("Best Val AUC:", best_val_auc)
print("Epochs without improvement:", epochs_without_improvement)
print("Current LR:", optimizer.param_groups[0]["lr"])
print("Device:", device)
```

```
===== V3 RESUME STATUS =====
Saved epoch: 16
Starting epoch: 17
Best Val AUC: 0.9999759726280179
Epochs without improvement: 0
Current LR: 0.0005
Device: cuda
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

FEATURE_DIR = "/content/drive/MyDrive/voice_cloning_project/features"
WF_DIR = os.path.join(FEATURE_DIR, "wavefake_v3")


class V3TrainDataset(Dataset):
    def __init__(self, asv_x_path, asv_y_path, wavefake_dir):

        self.asv_x = np.load(asv_x_path, mmap_mode="r")
        self.asv_y = np.load(asv_y_path, mmap_mode="r")

        self.x_files = sorted([
            f for f in os.listdir(wavefake_dir)
            if f.startswith("X_chunk_") and f.endswith(".npy")
        ])
```

```
        self.y_files = sorted([
            f for f in os.listdir(wavefake_dir)
            if f.startswith("y_chunk_") and f.endswith(".npy")
        ])

        self.wavefake_dir = wavefake_dir

        self.chunk_sizes = []

        for f in self.x_files:
            arr = np.load(
                os.path.join(wavefake_dir, f),
                mmap_mode="r"
            )
            self.chunk_sizes.append(arr.shape[0])

        self.chunk_offsets = np.cumsum(
            [0] + self.chunk_sizes
        )

        self.asv_len = len(self.asv_y)
        self.wavefake_len = sum(self.chunk_sizes)
        self.total_len = self.asv_len + self.wavefake_len

    def __len__(self):
        return self.total_len

    def __getitem__(self, idx):

        if idx < self.asv_len:

            x = self.asv_x[idx]
            y = self.asv_y[idx]

        else:

            wf_idx = idx - self.asv_len

            chunk_idx = (
                np.searchsorted(
                    self.chunk_offsets,
                    wf_idx,
                    side="right"
                ) - 1
            )

            local_idx = (
                wf_idx -
                self.chunk_offsets[chunk_idx]
            )
```

```
            x_file = self.x_files[chunk_idx]
            y_file = self.y_files[chunk_idx]

            X_chunk = np.load(
                os.path.join(
                    self.wavefake_dir,
                    x_file
                ),
                mmap_mode="r"
            )

            y_chunk = np.load(
                os.path.join(
                    self.wavefake_dir,
                    y_file
                ),
                mmap_mode="r"
            )

            x = X_chunk[local_idx]
            y = y_chunk[local_idx]

        x = torch.from_numpy(
            np.asarray(x, dtype=np.float32)
        ).unsqueeze(0)

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        return x, y


class ASVspoofDevDataset(Dataset):
    def __init__(self, x_path, y_path):

        self.X = np.load(
            x_path,
            mmap_mode="r"
        )

        self.y = np.load(
            y_path,
            mmap_mode="r"
        )

    def __len__(self):
        return len(self.y)
```

```
    def __getitem__(self, idx):

        x = self.X[idx]
        y = self.y[idx]

        x = torch.from_numpy(
            np.asarray(x, dtype=np.float32)
        ).unsqueeze(0)

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        return x, y

train_dataset = V3TrainDataset(
    os.path.join(FEATURE_DIR, "X_train.npy"),
    os.path.join(FEATURE_DIR, "y_train.npy"),
    WF_DIR
)

dev_dataset = ASVspoofDevDataset(
    os.path.join(FEATURE_DIR, "X_dev.npy"),
    os.path.join(FEATURE_DIR, "y_dev.npy")
)

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("===== DATALOADER VERIFICATION =====")
print("Training samples:", len(train_dataset))
print("Validation samples:", len(dev_dataset))
print("Training batches:", len(train_loader))
print("Validation batches:", len(dev_loader))
```

```
===== DATALOADER VERIFICATION =====
Training samples: 74380
Validation samples: 24844
Training batches: 1163
Validation batches: 389

X_test, y_test = next(iter(train_loader))

print("Batch X shape:", X_test.shape)
print("Batch y shape:", y_test.shape)
print("X dtype:", X_test.dtype)
print("y dtype:", y_test.dtype)
print("Bonafide:", (y_test == 0).sum().item())
print("Spoof:", (y_test == 1).sum().item())

assert X_test.shape == (64, 1, 128, 251)
assert y_test.shape == (64,)

print("□ Training DataLoader test passed")

Batch X shape: torch.Size([64, 1, 128, 251])
Batch y shape: torch.Size([64])
X dtype: torch.float32
y dtype: torch.float32
Bonafide: 2
Spoof: 62
□ Training DataLoader test passed

class V3TrainDatasetFast(Dataset):
    def __init__(self, asv_x_path, asv_y_path, wavefake_dir):

        self.asv_x = np.load(asv_x_path, mmap_mode="r")
        self.asv_y = np.load(asv_y_path, mmap_mode="r")

        self.wavefake_dir = wavefake_dir

        self.x_files = sorted([
            f for f in os.listdir(wavefake_dir)
            if f.startswith("X_chunk_") and f.endswith(".npy")
        ])

        self.y_files = sorted([
            f for f in os.listdir(wavefake_dir)
            if f.startswith("y_chunk_") and f.endswith(".npy")
        ])

        self.x_chunks = [
            np.load(
                os.path.join(wavefake_dir, f),
                mmap_mode="r"
            )
```

```
            for f in self.x_files
        ]

        self.y_chunks = [
            np.load(
                os.path.join(wavefake_dir, f),
                mmap_mode="r"
            )
            for f in self.y_files
        ]

        self.chunk_sizes = [
            len(x) for x in self.x_chunks
        ]

        self.chunk_offsets = np.cumsum(
            [0] + self.chunk_sizes
        )

        self.asv_len = len(self.asv_y)
        self.wavefake_len = sum(self.chunk_sizes)
        self.total_len = self.asv_len + self.wavefake_len

    def __len__(self):
        return self.total_len

    def __getitem__(self, idx):

        # ASVspoof
        if idx < self.asv_len:

            x = self.asv_x[idx]
            y = self.asv_y[idx]

        # WaveFake
        else:

            wf_idx = idx - self.asv_len

            chunk_idx = (
                np.searchsorted(
                    self.chunk_offsets,
                    wf_idx,
                    side="right"
                ) - 1
            )

            local_idx = (
                wf_idx -
                self.chunk_offsets[chunk_idx]
```

```
            )

            x = self.x_chunks[chunk_idx][local_idx]
            y = self.y_chunks[chunk_idx][local_idx]

        x = torch.from_numpy(
            np.asarray(x, dtype=np.float32)
        ).unsqueeze(0)

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        return x, y

train_dataset = V3TrainDatasetFast(
    os.path.join(FEATURE_DIR, "X_train.npy"),
    os.path.join(FEATURE_DIR, "y_train.npy"),
    WF_DIR
)

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

print("Training samples:", len(train_dataset))
print("Training batches:", len(train_loader))
```

```
Training samples: 74380
Training batches: 1163
```

```
import time

start = time.time()

for i, (X, y) in enumerate(train_loader):

    if i == 20:
        break

elapsed = time.time() - start

print(f"20 batches loaded in {elapsed:.2f} seconds")
print(f"Average per batch: {elapsed/20:.3f} seconds")
```

```
20 batches loaded in 116.99 seconds
Average per batch: 5.850 seconds
```

```
import os
import shutil
import time

DRIVE_WF_DIR =
"/content/drive/MyDrive/voice_cloning_project/features/wavefake_v3"
LOCAL_WF_DIR = "/content/wavefake_v3"

os.makedirs(LOCAL_WF_DIR, exist_ok=True)

x_files = sorted([
    f for f in os.listdir(DRIVE_WF_DIR)
    if f.startswith("X_chunk_") and f.endswith(".npy")
])

y_files = sorted([
    f for f in os.listdir(DRIVE_WF_DIR)
    if f.startswith("y_chunk_") and f.endswith(".npy")
])

print("X chunks:", len(x_files))
print("Y chunks:", len(y_files))

start = time.time()

for f in x_files + y_files:

    src = os.path.join(DRIVE_WF_DIR, f)
    dst = os.path.join(LOCAL_WF_DIR, f)

    if not os.path.exists(dst):
        shutil.copy2(src, dst)

elapsed = time.time() - start

print(f"\nCopy completed in {elapsed/60:.2f} minutes")
print("Local WaveFake directory:", LOCAL_WF_DIR)
```

```
X chunks: 49
Y chunks: 49

Copy completed in 0.85 minutes
Local WaveFake directory: /content/wavefake_v3
```

```
x_local = sorted([
    f for f in os.listdir(LOCAL_WF_DIR)
    if f.startswith("X_chunk_") and f.endswith(".npy")
])
```

```
y_local = sorted([
    f for f in os.listdir(LOCAL_WF_DIR)
    if f.startswith("y_chunk_") and f.endswith(".npy")
])

print("Local X chunks:", len(x_local))
print("Local Y chunks:", len(y_local))

total_x = sum(
    np.load(
        os.path.join(LOCAL_WF_DIR, f),
        mmap_mode="r"
    ).shape[0]
    for f in x_local
)

total_y = sum(
    np.load(
        os.path.join(LOCAL_WF_DIR, f),
        mmap_mode="r"
    ).shape[0]
    for f in y_local
)

print("Local X samples:", total_x)
print("Local Y samples:", total_y)

assert len(x_local) == 49
assert len(y_local) == 49
assert total_x == 49000
assert total_y == 49000

print("□ Local WaveFake cache verified")
```

```
Local X chunks: 49
Local Y chunks: 49
Local X samples: 49000
Local Y samples: 49000
□ Local WaveFake cache verified
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

class V3TrainDatasetFast(Dataset):
    def __init__(self, asv_x_path, asv_y_path, wavefake_dir):
        # ASVspoof memmaps
        self.asv_x = np.load(asv_x_path, mmap_mode="r")
```

```
        self.asv_y = np.load(asv_y_path, mmap_mode="r")

        # WaveFake chunks
        self.x_files = sorted([
            os.path.join(wavefake_dir, f)
            for f in os.listdir(wavefake_dir)
            if f.startswith("X_chunk_") and f.endswith(".npy")
        ])

        self.y_files = sorted([
            os.path.join(wavefake_dir, f)
            for f in os.listdir(wavefake_dir)
            if f.startswith("y_chunk_") and f.endswith(".npy")
        ])

        # Open each chunk once
        self.x_chunks = [
            np.load(f, mmap_mode="r") for f in self.x_files
        ]

        self.y_chunks = [
            np.load(f, mmap_mode="r") for f in self.y_files
        ]

        self.wavefake_sizes = [x.shape[0] for x in self.x_chunks]
        self.wavefake_offsets = np.cumsum(
            [0] + self.wavefake_sizes
        )

        self.asv_len = len(self.asv_y)
        self.wavefake_len = self.wavefake_offsets[-1]

        self.total_len = self.asv_len + self.wavefake_len

        print("ASVspoof samples :", self.asv_len)
        print("WaveFake samples :", self.wavefake_len)
        print("Total samples    :", self.total_len)

    def __len__(self):
        return self.total_len

    def __getitem__(self, idx):

        # ASVspoof
        if idx < self.asv_len:
            x = self.asv_x[idx]
            y = self.asv_y[idx]

        # WaveFake
        else:
```

```
            wf_idx = idx - self.asv_len

            chunk_idx = np.searchsorted(
                self.wavefake_offsets,
                wf_idx,
                side="right"
            ) - 1

            local_idx = wf_idx - self.wavefake_offsets[chunk_idx]

            x = self.x_chunks[chunk_idx][local_idx]
            y = self.y_chunks[chunk_idx][local_idx]

        x = torch.from_numpy(np.asarray(x)).float()
        y = torch.tensor(y, dtype=torch.float32)

        # CNN input: [1, 128, 251]
        x = x.unsqueeze(0)

        return x, y

TRAIN_X =
"/content/drive/MyDrive/voice_cloning_project/features/X_train.npy"
TRAIN_Y =
"/content/drive/MyDrive/voice_cloning_project/features/y_train.npy"

WAVEFAKE_LOCAL = "/content/wavefake_v3"

train_dataset = V3TrainDatasetFast(
    TRAIN_X,
    TRAIN_Y,
    WAVEFAKE_LOCAL
)

print("Dataset length:", len(train_dataset))
```

```
ASVspoof samples : 25380
WaveFake samples : 49000
Total samples    : 74380
Dataset length: 74380
```

```
import time
from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)
```

```
start = time.time()

for i, (X, y) in enumerate(train_loader):

    if i == 20:
        break

elapsed = time.time() - start

print(f"20 batches loaded in {elapsed:.2f} seconds")
print(f"Average per batch: {elapsed/20:.3f} seconds")
```

```
/tmp/ipykernel_2093/3582990473.py:73: UserWarning: The given NumPy
array is not writable, and PyTorch does not support non-writable
tensors. This means writing to this tensor will result in undefined
behavior. You may want to copy the array to protect its data or make
it writable before converting it to a tensor. This type of warning
will be suppressed for the rest of this program. (Triggered internally
at /pytorch/torch/csrc/utils/tensor_numpy.cpp:213.)
  x = torch.from_numpy(np.asarray(x)).float()
```

```
20 batches loaded in 6.56 seconds
Average per batch: 0.328 seconds
```

```
import os
import time
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

# ============================================================
# PATHS
# ============================================================

RESUME_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v3_resume_epoch1
6.pth"
BEST_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/voice_cnn_v3_bes
t.pth"
HISTORY_PATH =
"/content/drive/MyDrive/voice_cloning_project/results/v3_training_hist
ory.npz"

# ============================================================
# DEVICE
# ============================================================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print("Device:", device)

# ================================================================
# MODEL
# ================================================================

model = VoiceCNN().to(device)

# ================================================================
# OPTIMIZER
# ================================================================

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4
)

# ================================================================
# SCHEDULER
# ================================================================

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=2
)

# ================================================================
# LOSS
# ================================================================

CLASS_WEIGHT_0 = 14.414728682170542
CLASS_WEIGHT_1 = 0.517966573816156

criterion = nn.BCEWithLogitsLoss(reduction="none")

# ================================================================
# LOAD EPOCH-16 CHECKPOINT
# ================================================================

checkpoint = torch.load(
    RESUME_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

```
scheduler.load_state_dict(checkpoint["scheduler_state_dict"])

start_epoch = checkpoint["epoch"] + 1
best_auc = checkpoint["best_auc"]
epochs_without_improvement = checkpoint["epochs_without_improvement"]

history = checkpoint["history"]

print("\nCheckpoint restored")
print("-------------------------")
print("Saved epoch       :", checkpoint["epoch"])
print("Starting epoch    :", start_epoch)
print("Best Val AUC      :", best_auc)
print("No improvement    :", epochs_without_improvement)
print("Current LR        :", optimizer.param_groups[0]["lr"])
```

```
Device: cuda
```

```
---------------------------------------------------------------------------
-----
KeyError                                  Traceback (most recent call
last)
/tmp/ipykernel_2093/1293206903.py in <cell line: 0>()
     72
     73 start_epoch = checkpoint["epoch"] + 1
---> 74 best_auc = checkpoint["best_auc"]
     75 epochs_without_improvement =
checkpoint["epochs_without_improvement"]
     76

KeyError: 'best_auc'
```

```
print("Checkpoint type:", type(checkpoint))

print("\nCheckpoint keys:")
for key in checkpoint.keys():
    print(" -", key)

print("\nCheckpoint contents:")
for key, value in checkpoint.items():
    if isinstance(value, dict):
        print(f"{key}: dict with {len(value)} keys")
    elif isinstance(value, (list, tuple)):
        print(f"{key}: {type(value).__name__}, length={len(value)}")
    else:
        print(f"{key}: {type(value).__name__} = {value}")
```

```
Checkpoint type: <class 'dict'>

Checkpoint keys:
 - model_state_dict
```

```
 - optimizer_state_dict
 - scheduler_state_dict
 - epoch
 - best_val_auc
 - epochs_without_improvement
 - max_epochs
 - early_stopping_patience
 - batch_size
 - learning_rate
 - weight_decay
 - class_weight_0
 - class_weight_1
 - history

Checkpoint contents:
model_state_dict: dict with 25 keys
optimizer_state_dict: dict with 2 keys
scheduler_state_dict: dict with 15 keys
epoch: int = 16
best_val_auc: float64 = 0.9999759726280179
epochs_without_improvement: int = 0
max_epochs: int = 50
early_stopping_patience: int = 7
batch_size: int = 64
learning_rate: float = 0.001
weight_decay: float = 0.0001
class_weight_0: float = 14.414728682170542
class_weight_1: float = 0.517966573816156
history: dict with 6 keys
```

```
# ============================================================
# RESTORE CHECKPOINT — EPOCH 16
# ============================================================

model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
scheduler.load_state_dict(checkpoint["scheduler_state_dict"])

start_epoch = checkpoint["epoch"] + 1
best_auc = float(checkpoint["best_val_auc"])
epochs_without_improvement = checkpoint["epochs_without_improvement"]

history = checkpoint["history"]

print("\nCheckpoint restored successfully")
print("--------------------------------")
print("Saved epoch       :", checkpoint["epoch"])
print("Starting epoch    :", start_epoch)
print("Best Val AUC      :", best_auc)
print("No improvement    :", epochs_without_improvement)
```

```
print("Current LR        :", optimizer.param_groups[0]["lr"])
print("Batch size        :", checkpoint["batch_size"])
print("Max epochs        :", checkpoint["max_epochs"])
print("Early stopping    :", checkpoint["early_stopping_patience"])
```

```
Checkpoint restored successfully
------------------------------
Saved epoch       : 16
Starting epoch    : 17
Best Val AUC      : 0.9999759726280179
No improvement    : 0
Current LR        : 0.0005
Batch size        : 64
Max epochs        : 50
Early stopping    : 7
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

dev_loader = DataLoader(
    dev_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Train samples :", len(train_dataset))
print("Train batches :", len(train_loader))
print("Dev samples   :", len(dev_dataset))
print("Dev batches   :", len(dev_loader))
```

```
Train samples : 74380
Train batches : 1163
Dev samples   : 24844
Dev batches   : 389
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader

class V3TrainDatasetFast(Dataset):

    def __init__(self, asv_x_path, asv_y_path, wavefake_dir):
```

```
        # ASVspoof memmaps
        self.asv_x = np.load(asv_x_path, mmap_mode="r")
        self.asv_y = np.load(asv_y_path, mmap_mode="r")

        # WaveFake chunks
        self.x_files = sorted([
            os.path.join(wavefake_dir, f)
            for f in os.listdir(wavefake_dir)
            if f.startswith("X_chunk_") and f.endswith(".npy")
        ])

        self.y_files = sorted([
            os.path.join(wavefake_dir, f)
            for f in os.listdir(wavefake_dir)
            if f.startswith("y_chunk_") and f.endswith(".npy")
        ])

        # Open chunks once
        self.x_chunks = [
            np.load(f, mmap_mode="r")
            for f in self.x_files
        ]

        self.y_chunks = [
            np.load(f, mmap_mode="r")
            for f in self.y_files
        ]

        self.wavefake_sizes = [
            x.shape[0]
            for x in self.x_chunks
        ]

        self.wavefake_offsets = np.cumsum(
            [0] + self.wavefake_sizes
        )

        self.asv_len = len(self.asv_y)
        self.wavefake_len = self.wavefake_offsets[-1]

        self.total_len = self.asv_len + self.wavefake_len

        print("ASVspoof samples :", self.asv_len)
        print("WaveFake samples :", self.wavefake_len)
        print("Total samples    :", self.total_len)

    def __len__(self):
        return self.total_len
```

```
    def __getitem__(self, idx):

        # ----------------------------
        # ASVspoof
        # ----------------------------
        if idx < self.asv_len:

            x = self.asv_x[idx]
            y = self.asv_y[idx]

        # ----------------------------
        # WaveFake
        # ----------------------------
        else:

            wf_idx = idx - self.asv_len

            chunk_idx = np.searchsorted(
                self.wavefake_offsets,
                wf_idx,
                side="right"
            ) - 1

            local_idx = (
                wf_idx -
                self.wavefake_offsets[chunk_idx]
            )

            x = self.x_chunks[chunk_idx][local_idx]
            y = self.y_chunks[chunk_idx][local_idx]

        # IMPORTANT:
        # Make writable NumPy copy to avoid
        # PyTorch memmap warning
        x = torch.from_numpy(
            np.array(x, copy=True)
        ).float()

        y = torch.tensor(
            y,
            dtype=torch.float32
        )

        # [128,251] -> [1,128,251]
        x = x.unsqueeze(0)

        return x, y

TRAIN_X =
"/content/drive/MyDrive/voice_cloning_project/features/X_train.npy"
TRAIN_Y =
```

```
"/content/drive/MyDrive/voice_cloning_project/features/y_train.npy"

WAVEFAKE_LOCAL = "/content/wavefake_v3"

train_dataset = V3TrainDatasetFast(
    TRAIN_X,
    TRAIN_Y,
    WAVEFAKE_LOCAL
)
```

```
ASVspoof samples : 25380
WaveFake samples : 49000
Total samples    : 74380
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

print("Train samples :", len(train_dataset))
print("Train batches :", len(train_loader))
```

```
Train samples : 74380
Train batches : 1163
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

print("Train samples :", len(train_dataset))
print("Train batches :", len(train_loader))
```

```
Train samples : 74380
Train batches : 1163
```

```
X, y = next(iter(train_loader))

print("X shape :", X.shape)
print("X dtype :", X.dtype)
print("y shape :", y.shape)
print("y dtype :", y.dtype)
```

```
X shape : torch.Size([64, 1, 128, 251])
X dtype : torch.float32
```

```
y shape : torch.Size([64])
y dtype : torch.float32
```

```
print("History keys:")
for k, v in history.items():
    print(
        k,
        "->",
        type(v).__name__,
        "length =", len(v) if hasattr(v, "__len__") else "N/A"
    )
```

```
History keys:
train_loss -> list length = 16
val_loss -> list length = 16
val_auc -> list length = 16
val_f1 -> list length = 16
val_accuracy -> list length = 16
learning_rate -> list length = 16
```

```
# Reconstruct epoch numbers for the existing 16 completed epochs

history["epoch"] = list(range(1, len(history["train_loss"]) + 1))

print("History restored:")
for k, v in history.items():
    print(k, "->", len(v))
```

```
History restored:
train_loss -> 16
val_loss -> 16
val_auc -> 16
val_f1 -> 16
val_accuracy -> 16
learning_rate -> 16
epoch -> 16
```

```
history["epoch"].append(epoch)
history["train_loss"].append(train_loss)
history["val_loss"].append(val_loss)
history["val_auc"].append(val_auc)
history["val_f1"].append(val_f1)
history["val_accuracy"].append(val_acc)
history["learning_rate"].append(current_lr)

import time
import numpy as np
import torch
from sklearn.metrics import roc_auc_score, f1_score, accuracy_score

MAX_EPOCHS = 50
```

```
EARLY_STOPPING_PATIENCE = 7

print("Starting V3 training...")
print("Resuming from epoch:", start_epoch)
print("Best validation AUC:", best_auc)
print("Current learning rate:", optimizer.param_groups[0]["lr"])

for epoch in range(start_epoch, MAX_EPOCHS + 1):

    epoch_start = time.time()

    # ========================================================
    # TRAIN
    # ========================================================

    model.train()

    train_loss_sum = 0.0
    train_count = 0

    for X, y in train_loader:

        X = X.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)

        logits = model(X)

        losses = criterion(logits, y)

        sample_weights = torch.where(
            y == 0,
            torch.tensor(CLASS_WEIGHT_0, device=device),
            torch.tensor(CLASS_WEIGHT_1, device=device)
        )

        loss = (losses * sample_weights).mean()

        loss.backward()
        optimizer.step()

        train_loss_sum += loss.item() * X.size(0)
        train_count += X.size(0)

    train_loss = train_loss_sum / train_count

    # ========================================================
    # VALIDATION
    # ========================================================
```

```
    model.eval()

    val_loss_sum = 0.0
    val_count = 0

    all_probs = []
    all_labels = []

    with torch.no_grad():

        for X, y in dev_loader:

            X = X.to(device, non_blocking=True)
            y = y.to(device, non_blocking=True)

            logits = model(X)

            losses = criterion(logits, y)

            sample_weights = torch.where(
                y == 0,
                torch.tensor(CLASS_WEIGHT_0, device=device),
                torch.tensor(CLASS_WEIGHT_1, device=device)
            )

            loss = (losses * sample_weights).mean()

            val_loss_sum += loss.item() * X.size(0)
            val_count += X.size(0)

            probs = torch.sigmoid(logits)

            all_probs.append(probs.cpu().numpy())
            all_labels.append(y.cpu().numpy())

    val_loss = val_loss_sum / val_count

    all_probs = np.concatenate(all_probs)
    all_labels = np.concatenate(all_labels)

    # ========================================================
    # METRICS
    # ========================================================

    val_auc = roc_auc_score(all_labels, all_probs)

    val_pred = (all_probs >= 0.5).astype(int)

    val_f1 = f1_score(all_labels, val_pred)
    val_acc = accuracy_score(all_labels, val_pred)
```

```
    # ============================================================
    # SCHEDULER
    # ============================================================

    scheduler.step(val_loss)

    current_lr = optimizer.param_groups[0]["lr"]

    # ============================================================
    # HISTORY
    # ============================================================

    history["epoch"].append(epoch)
    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["val_auc"].append(val_auc)
    history["val_f1"].append(val_f1)
    history["val_accuracy"].append(val_acc)
    history["learning_rate"].append(current_lr)

    # ============================================================
    # BEST MODEL
    # ============================================================

    improved = val_auc > best_auc

    if improved:

        best_auc = val_auc
        epochs_without_improvement = 0

        marker = " ★ BEST"

        torch.save({
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "scheduler_state_dict": scheduler.state_dict(),
            "epoch": epoch,
            "best_val_auc": best_auc,
            "epochs_without_improvement":
                epochs_without_improvement,
            "max_epochs": MAX_EPOCHS,
            "early_stopping_patience":
                EARLY_STOPPING_PATIENCE,
            "batch_size": 64,
            "learning_rate": 0.001,
            "weight_decay": 1e-4,
            "class_weight_0": CLASS_WEIGHT_0,
            "class_weight_1": CLASS_WEIGHT_1,
            "history": history
```

```
        }, BEST_PATH)

    else:

        epochs_without_improvement += 1
        marker = ""

    # ============================================================
    # SAVE RESUME CHECKPOINT
    # ============================================================

    torch.save({
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "scheduler_state_dict": scheduler.state_dict(),
        "epoch": epoch,
        "best_val_auc": best_auc,
        "epochs_without_improvement":
            epochs_without_improvement,
        "max_epochs": MAX_EPOCHS,
        "early_stopping_patience":
            EARLY_STOPPING_PATIENCE,
        "batch_size": 64,
        "learning_rate": 0.001,
        "weight_decay": 1e-4,
        "class_weight_0": CLASS_WEIGHT_0,
        "class_weight_1": CLASS_WEIGHT_1,
        "history": history
    }, RESUME_PATH)

    # ============================================================
    # SAVE HISTORY
    # ============================================================

    np.savez(
        HISTORY_PATH,
        epoch=np.array(history["epoch"]),
        train_loss=np.array(history["train_loss"]),
        val_loss=np.array(history["val_loss"]),
        val_auc=np.array(history["val_auc"]),
        val_f1=np.array(history["val_f1"]),
        val_accuracy=np.array(history["val_accuracy"]),
        learning_rate=np.array(history["learning_rate"])
    )

    elapsed = time.time() - epoch_start

    print(
        f"Epoch {epoch:02d}/{MAX_EPOCHS} | "
        f"Train Loss: {train_loss:.4f} | "
```

```
        f"Val Loss: {val_loss:.4f} | "
        f"Val AUC: {val_auc:.6f} | "
        f"Val F1: {val_f1:.6f} | "
        f"Val Acc: {val_acc:.6f} | "
        f"LR: {current_lr:.6f} | "
        f"Time: {elapsed/60:.2f} min"
        f"{marker}"
    )

    # ============================================================
    # EARLY STOPPING
    # ============================================================

    if epochs_without_improvement >= EARLY_STOPPING_PATIENCE:

        print("\nEarly stopping triggered.")
        print("Best validation AUC:", best_auc)

        break

print("\nV3 training finished.")
print("Best validation AUC:", best_auc)
```

```
Starting V3 training...
Resuming from epoch: 17
Best validation AUC: 0.9999759726280179
Current learning rate: 0.0005
Epoch 17/50 | Train Loss: 0.0098 | Val Loss: 0.0161 | Val AUC:
0.999960 | Val F1: 0.998518 | Val Acc: 0.997343 | LR: 0.000500 | Time:
2.60 min
Epoch 18/50 | Train Loss: 0.0072 | Val Loss: 0.0136 | Val AUC:
0.999972 | Val F1: 0.998631 | Val Acc: 0.997545 | LR: 0.000250 | Time:
2.26 min
Epoch 19/50 | Train Loss: 0.0047 | Val Loss: 0.0108 | Val AUC:
0.999987 | Val F1: 0.999192 | Val Acc: 0.998551 | LR: 0.000250 | Time:
2.00 min ★ BEST
Epoch 20/50 | Train Loss: 0.0043 | Val Loss: 0.0236 | Val AUC:
0.999988 | Val F1: 0.999439 | Val Acc: 0.998994 | LR: 0.000250 | Time:
1.99 min ★ BEST
Epoch 21/50 | Train Loss: 0.0054 | Val Loss: 0.0161 | Val AUC:
0.999993 | Val F1: 0.999529 | Val Acc: 0.999155 | LR: 0.000250 | Time:
2.01 min ★ BEST
Epoch 22/50 | Train Loss: 0.0048 | Val Loss: 0.0165 | Val AUC:
0.999991 | Val F1: 0.992954 | Val Acc: 0.987442 | LR: 0.000125 | Time:
2.01 min
Epoch 23/50 | Train Loss: 0.0061 | Val Loss: 0.0089 | Val AUC:
0.999988 | Val F1: 0.997527 | Val Acc: 0.995572 | LR: 0.000125 | Time:
2.01 min
Epoch 24/50 | Train Loss: 0.0028 | Val Loss: 0.0138 | Val AUC:
0.999987 | Val F1: 0.999125 | Val Acc: 0.998430 | LR: 0.000125 | Time:
```

```
2.00 min
Epoch 25/50 | Train Loss: 0.0030 | Val Loss: 0.0177 | Val AUC:
0.999993 | Val F1: 0.999596 | Val Acc: 0.999275 | LR: 0.000125 | Time:
2.00 min
Epoch 26/50 | Train Loss: 0.0040 | Val Loss: 0.0173 | Val AUC:
0.999994 | Val F1: 0.999574 | Val Acc: 0.999235 | LR: 0.000063 | Time:
2.01 min ★ BEST
Epoch 27/50 | Train Loss: 0.0029 | Val Loss: 0.0117 | Val AUC:
0.999995 | Val F1: 0.999574 | Val Acc: 0.999235 | LR: 0.000063 | Time:
2.00 min ★ BEST
Epoch 28/50 | Train Loss: 0.0022 | Val Loss: 0.0106 | Val AUC:
0.999988 | Val F1: 0.996196 | Val Acc: 0.993198 | LR: 0.000063 | Time:
2.01 min
Epoch 29/50 | Train Loss: 0.0021 | Val Loss: 0.0071 | Val AUC:
0.999997 | Val F1: 0.999596 | Val Acc: 0.999275 | LR: 0.000063 | Time:
1.99 min ★ BEST
Epoch 30/50 | Train Loss: 0.0020 | Val Loss: 0.0090 | Val AUC:
0.999995 | Val F1: 0.999417 | Val Acc: 0.998953 | LR: 0.000063 | Time:
2.00 min
Epoch 31/50 | Train Loss: 0.0022 | Val Loss: 0.0053 | Val AUC:
0.999998 | Val F1: 0.999619 | Val Acc: 0.999316 | LR: 0.000063 | Time:
2.01 min ★ BEST
Epoch 32/50 | Train Loss: 0.0018 | Val Loss: 0.0091 | Val AUC:
0.999996 | Val F1: 0.999551 | Val Acc: 0.999195 | LR: 0.000063 | Time:
1.99 min
Epoch 33/50 | Train Loss: 0.0018 | Val Loss: 0.0086 | Val AUC:
0.999997 | Val F1: 0.999641 | Val Acc: 0.999356 | LR: 0.000063 | Time:
2.01 min
Epoch 34/50 | Train Loss: 0.0016 | Val Loss: 0.0076 | Val AUC:
0.999996 | Val F1: 0.999596 | Val Acc: 0.999275 | LR: 0.000031 | Time:
2.00 min
Epoch 35/50 | Train Loss: 0.0013 | Val Loss: 0.0058 | Val AUC:
0.999998 | Val F1: 0.999776 | Val Acc: 0.999597 | LR: 0.000031 | Time:
2.01 min ★ BEST
Epoch 36/50 | Train Loss: 0.0015 | Val Loss: 0.0112 | Val AUC:
0.999998 | Val F1: 0.999731 | Val Acc: 0.999517 | LR: 0.000031 | Time:
1.99 min
Epoch 37/50 | Train Loss: 0.0014 | Val Loss: 0.0095 | Val AUC:
0.999996 | Val F1: 0.999619 | Val Acc: 0.999316 | LR: 0.000016 | Time:
2.01 min
Epoch 38/50 | Train Loss: 0.0013 | Val Loss: 0.0090 | Val AUC:
0.999997 | Val F1: 0.999641 | Val Acc: 0.999356 | LR: 0.000016 | Time:
2.00 min
Epoch 39/50 | Train Loss: 0.0017 | Val Loss: 0.0080 | Val AUC:
0.999998 | Val F1: 0.999686 | Val Acc: 0.999436 | LR: 0.000016 | Time:
2.01 min
Epoch 40/50 | Train Loss: 0.0011 | Val Loss: 0.0102 | Val AUC:
0.999996 | Val F1: 0.999641 | Val Acc: 0.999356 | LR: 0.000008 | Time:
2.00 min
```

```
Epoch 41/50 | Train Loss: 0.0013 | Val Loss: 0.0097 | Val AUC:
0.999997 | Val F1: 0.999619 | Val Acc: 0.999316 | LR: 0.000008 | Time:
2.00 min
Epoch 42/50 | Train Loss: 0.0011 | Val Loss: 0.0099 | Val AUC:
0.999998 | Val F1: 0.999686 | Val Acc: 0.999436 | LR: 0.000008 | Time:
2.00 min

Early stopping triggered.
Best validation AUC: 0.9999983805727308

V3 training finished.
Best validation AUC: 0.9999983805727308
```

```python
import os
import shutil
from pathlib import Path

# ==============================
# PROJECT DIRECTORIES
# ==============================
DRIVE_ROOT = Path("/content/drive/MyDrive/voice_cloning_project")
RESULTS_DIR = DRIVE_ROOT / "results"
FEATURES_DIR = DRIVE_ROOT / "features"
ARCHIVE_DIR = DRIVE_ROOT / "checkpoints_archive"

ARCHIVE_DIR.mkdir(parents=True, exist_ok=True)

print("Checking important files...\n")

# ==============================
# IMPORTANT FILES
# ==============================
important_files = [
    RESULTS_DIR / "voice_cnn_v3_best.pth",
    RESULTS_DIR / "v3_resume_epoch16.pth",
    RESULTS_DIR / "v3_training_history.npz",
    RESULTS_DIR / "v2_2021_detailed_eval.npz",

    FEATURES_DIR / "X_train.npy",
    FEATURES_DIR / "y_train.npy",
    FEATURES_DIR / "X_dev.npy",
    FEATURES_DIR / "y_dev.npy",
]

for f in important_files:
    if f.exists():
        size_mb = f.stat().st_size / (1024**2)
        print(f"□ {f.name:<30} {size_mb:,.2f} MB")
    else:
        print(f"□ MISSING: {f}")
```

```
# ==============================
# COPY CRITICAL V3 CHECKPOINTS
# ==============================
files_to_archive = [
    RESULTS_DIR / "voice_cnn_v3_best.pth",
    RESULTS_DIR / "v3_resume_epoch16.pth",
    RESULTS_DIR / "v3_training_history.npz",
]

for src in files_to_archive:
    if src.exists():
        dst = ARCHIVE_DIR / src.name
        shutil.copy2(src, dst)
        print(f"□ Archived: {src.name}")

print("\n□ Critical V3 checkpoints archived.")
```

```
Checking important files...

□ voice_cnn_v3_best.pth           1.29 MB
□ v3_resume_epoch16.pth           1.29 MB
□ v3_training_history.npz         0.00 MB
□ v2_2021_detailed_eval.npz       161.14 MB
□ X_train.npy                     1,555.27 MB
□ y_train.npy                     0.19 MB
□ X_dev.npy                       1,522.42 MB
□ y_dev.npy                       0.19 MB
□ Archived: voice_cnn_v3_best.pth
□ Archived: v3_resume_epoch16.pth
□ Archived: v3_training_history.npz

□ Critical V3 checkpoints archived.
```

```
import torch
import torch.nn as nn
from pathlib import Path

# ==============================
# Paths
# ==============================
BEST_V3_PATH = Path(

"/content/drive/MyDrive/voice_cloning_project/results/voice_cnn_v3_bes
t.pth"
)

# ==============================
# Device
# ==============================
```

```python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# ==============================
# Recreate model architecture
# ==============================
model = VoiceCNN().to(device)

# ==============================
# Load BEST V3 checkpoint
# ==============================
checkpoint = torch.load(
    BEST_V3_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

print("\n BEST V3 MODEL LOADED")
print("Best validation AUC:", checkpoint["best_val_auc"])
print("Saved training epoch:", checkpoint["epoch"])
print("Model device:", device)
```

```
Device: cuda

 BEST V3 MODEL LOADED
Best validation AUC: 0.9999983805727308
Saved training epoch: 35
Model device: cuda
```

```python
# ==========================================
# STEP 2 — VERIFY 2021 DF EVALUATION SETUP
# ==========================================

import os
from pathlib import Path
import numpy as np

# Your project root
PROJECT_ROOT = Path(
    "/content/drive/MyDrive/voice_cloning_project"
)

RESULTS_DIR = PROJECT_ROOT / "results"

# Existing V2 detailed evaluation
V2_EVAL_PATH = RESULTS_DIR / "v2_2021_detailed_eval.npz"

print("Project root:", PROJECT_ROOT)
```

```python
print("V2 evaluation file:", V2_EVAL_PATH)
print()

# Check V2 saved evaluation
if V2_EVAL_PATH.exists():

    data = np.load(V2_EVAL_PATH, allow_pickle=True)

    print("Existing V2 evaluation:")
    print("--------------------------------")

    for key in data.files:
        arr = data[key]
        print(f"{key:<15} shape={arr.shape} dtype={arr.dtype}")

    print()
    print("Total predictions:", len(data["y_true"]))

else:
    print("□ V2 evaluation file not found.")
```

```
Project root: /content/drive/MyDrive/voice_cloning_project
V2 evaluation file:
/content/drive/MyDrive/voice_cloning_project/results/v2_2021_detailed_
eval.npz

Existing V2 evaluation:
--------------------------------
y_true          shape=(480000,) dtype=int64
y_prob          shape=(480000,) dtype=float64
attack_ids      shape=(480000,) dtype=<U12
vocoders        shape=(480000,) dtype=<U32
codecs          shape=(480000,) dtype=<U8
speakers        shape=(480000,) dtype=<U7
subsets         shape=(480000,) dtype=<U8
paths           shape=(480000,) dtype=<U17
failed_indices  shape=(17,) dtype=int64

Total predictions: 480000
```

```python
# ============================================
# STEP 3 — V3 INFERENCE ON ASVSPOOF 2021 DF
# ============================================

import numpy as np
import torch
from pathlib import Path
from tqdm.auto import tqdm

# Best V3 checkpoint
BEST_V3_PATH = Path(
```

```
"/content/drive/MyDrive/voice_cloning_project/results/voice_cnn_v3_best.pth"
)

# Load best V3
checkpoint = torch.load(
    BEST_V3_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

print("✅ Best V3 model loaded")
print("Best Val AUC:", checkpoint["best_val_auc"])
print()

# ----------------------------------------
# IMPORTANT:
# Use your EXISTING 2021 DF evaluation loader
# ----------------------------------------

# Replace `df_eval_loader` with the loader
# from your previous V2 evaluation code.

all_probs = []

with torch.no_grad():

    for batch_x in tqdm(
        df_eval_loader,
        desc="V3 inference"
    ):

        batch_x = batch_x.to(device, non_blocking=True)

        logits = model(batch_x)

        probs = torch.sigmoid(logits)

        all_probs.append(
            probs.cpu().numpy()
        )

# Combine predictions
v3_y_prob = np.concatenate(all_probs)

print("\n==================================")
print("V3 INFERENCE COMPLETE")
```

```
print("================================")

print("Predictions:", len(v3_y_prob))
print("Min prob:", v3_y_prob.min())
print("Max prob:", v3_y_prob.max())
print("Mean prob:", v3_y_prob.mean())
```

```
 Best V3 model loaded
Best Val AUC: 0.9999983805727308
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2093/537603798.py in <cell line: 0>()
     40
     41     for batch_x in tqdm(
---> 42         df_eval_loader,
     43         desc="V3 inference"
     44     ):

NameError: name 'df_eval_loader' is not defined
```

```
# Find variables related to ASVspoof / DF / evaluation
vars_found = [
    name for name in globals()
    if any(k in name.lower() for k in [
        "df", "asv", "eval", "dataset", "loader"
    ])
]

print("Potential existing variables:")
for name in sorted(vars_found):
    try:
        obj = globals()[name]
        print(f"{name:<35} {type(obj)}")
    except:
        pass
```

```
Potential existing variables:
ASVspoofDevDataset                  <class 'type'>
DataLoader                          <class 'type'>
Dataset                             <class 'type'>
V2_EVAL_PATH                        <class 'pathlib._local.PosixPath'>
V3TrainDataset                      <class 'type'>
V3TrainDatasetFast                  <class 'type'>
__loader__                          <class 'NoneType'>
dev_dataset                         <class
'__main__.ASVspoofDevDataset'>
dev_loader                          <class
```

```
'torch.utils.data.dataloader.DataLoader'>
train_dataset                           <class
'__main__.V3TrainDatasetFast'>
train_loader                            <class
'torch.utils.data.dataloader.DataLoader'>
```

```
# Inspect existing Dataset classes and their source code

import inspect

classes_to_check = [
    "ASVspoofDevDataset",
    "V3TrainDataset",
    "V3TrainDatasetFast",
]

for class_name in classes_to_check:
    cls = globals().get(class_name)

    if cls is not None:
        print("\n" + "=" * 70)
        print(class_name)
        print("=" * 70)

        try:
            print(inspect.getsource(cls))
        except Exception as e:
            print("Could not retrieve source:", e)
```

```
======================================================================
ASVspoofDevDataset
======================================================================
Could not retrieve source: source code not available

======================================================================
V3TrainDataset
======================================================================
Could not retrieve source: source code not available

======================================================================
V3TrainDatasetFast
======================================================================
Could not retrieve source: source code not available
```

```
# ============================================================
# STEP 3 — EXACT ASVSPOOF 2021 DF PIPELINE USED FOR V2
# ============================================================

import numpy as np
import torch
```

```
import torchaudio
from datasets import load_dataset
from pathlib import Path

# ------------------------------------------------------------
# Paths
# ------------------------------------------------------------
PROJECT_ROOT = Path(
    "/content/drive/MyDrive/voice_cloning_project"
)

RESULTS_DIR = PROJECT_ROOT / "results"

V3_BEST_PATH = RESULTS_DIR / "voice_cnn_v3_best.pth"

# ------------------------------------------------------------
# Device
# ------------------------------------------------------------
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

# ------------------------------------------------------------
# Load BEST V3 model
# ------------------------------------------------------------
checkpoint = torch.load(
    V3_BEST_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

model = model.to(device)
model.eval()

print("□ Best V3 model loaded")
print("Best validation AUC:", checkpoint["best_val_auc"])

# ------------------------------------------------------------
# EXACT preprocessing from V2
# ------------------------------------------------------------
TARGET_SR = 16000
TARGET_LENGTH = 4 * TARGET_SR

N_MELS = 128
```

```
N_FFT = 1024
HOP_LENGTH = 256

mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=TARGET_SR,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    n_mels=N_MELS
)

def preprocess_audio(audio, sr):

    # Convert to float32
    audio = np.asarray(
        audio,
        dtype=np.float32
    )

    # Resample if necessary
    if sr != TARGET_SR:

        waveform = torch.tensor(
            audio
        ).unsqueeze(0)

        waveform = torchaudio.functional.resample(
            waveform,
            sr,
            TARGET_SR
        )

        audio = waveform.squeeze(0).numpy()

    # Convert to tensor
    waveform = torch.tensor(
        audio,
        dtype=torch.float32
    )

    # Pad or crop to exactly 4 seconds
    if len(waveform) < TARGET_LENGTH:

        waveform = torch.nn.functional.pad(
            waveform,
            (
                0,
                TARGET_LENGTH - len(waveform)
            )
        )
```

```
    else:

        waveform = waveform[
            :TARGET_LENGTH
        ]

    # Log-Mel
    mel = mel_transform(
        waveform.unsqueeze(0)
    )

    log_mel = torch.log(
        mel + 1e-9
    )

    return log_mel


print("✅ Preprocessing function ready")

# ------------------------------------------------------------
# Load official 2021 DF test split
# ------------------------------------------------------------
ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print("✅ ASVspoof 2021 DF streaming dataset loaded")

# Inspect first sample
sample = next(iter(ds))

print("\nFirst sample:")
print("Path:", sample["path"])
print("Label:", sample["label"])
print("Notes:", sample["notes"])
print(
    "Sample rate:",
    sample["audio"]["sampling_rate"]
)
print(
    "Audio samples:",
    len(sample["audio"]["array"])
)
```

```
Device: cuda
✅ Best V3 model loaded
```

```
Best validation AUC: 0.9999983805727308
 Preprocessing function ready

{"model_id":"b6a5712323104cec8d12a6c5ebe33c32","version_major":2,"vers
ion_minor":0}

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id":"25994db1b6ad4c0db14611a2527cc600","version_major":2,"vers
ion_minor":0}

{"model_id":"4e6c710748ad47f0840cc0220b989da2","version_major":2,"vers
ion_minor":0}

 ASVspoof 2021 DF streaming dataset loaded

First sample:
Path: DF_E_2000011.flac
Label: 1
Notes: {"utterance_id": "DF_E_2000011", "speaker_id": "LA_0023",
"subset": "progress", "codec": "nocodec", "source": "asvspoof",
"attack_id": "A14", "vocoder": "traditional_vocoder"}
Sample rate: 16000
Audio samples: 22801

# ============================================================
# STEP 3B — PREPROCESSING SANITY TEST
# ============================================================

feature = preprocess_audio(
    sample["audio"]["array"],
    sample["audio"]["sampling_rate"]
)

print("Feature shape :", feature.shape)
print("Feature dtype :", feature.dtype)
print("Feature min   :", feature.min().item())
print("Feature max   :", feature.max().item())
print("Feature mean  :", feature.mean().item())

assert feature.shape == (1, 128, 251), \
    f"Unexpected feature shape: {feature.shape}"

print("\n PREPROCESSING SANITY TEST PASSED")
```

```
Feature shape : torch.Size([1, 128, 251])
Feature dtype : torch.float32
Feature min   : -20.7232666015625
Feature max   : 8.868060111999512
Feature mean  : -14.463459014892578

 PREPROCESSING SANITY TEST PASSED
```

```python
# ============================================================
# STEP 4 — RESUMABLE V3 INFERENCE ON ASVSPOOF 2021 DF
# ============================================================

import os
import json
import numpy as np
import torch
import torchaudio
from pathlib import Path
from tqdm.auto import tqdm

# ------------------------------------------------------------
# Paths
# ------------------------------------------------------------
PROJECT_ROOT = Path(
    "/content/drive/MyDrive/voice_cloning_project"
)

RESULTS_DIR = PROJECT_ROOT / "results"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

V3_EVAL_DIR = RESULTS_DIR / "v3_2021_eval"
V3_EVAL_DIR.mkdir(parents=True, exist_ok=True)

CHECKPOINT_FILE = V3_EVAL_DIR / "progress.npz"

print("Evaluation directory:")
print(V3_EVAL_DIR)

# ------------------------------------------------------------
# Load BEST V3 model
# ------------------------------------------------------------
checkpoint = torch.load(
    V3_BEST_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(
    checkpoint["model_state_dict"]
)
```

```
model.to(device)
model.eval()

print("\n Best V3 model loaded")
print("Best validation AUC:", checkpoint["best_val_auc"])

# ------------------------------------------------------------
# Fresh streaming dataset
# ------------------------------------------------------------
ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(" 2021 DF test stream ready")

# ------------------------------------------------------------
# Storage
# ------------------------------------------------------------
y_true = []
y_prob = []
attack_ids = []
vocoders = []
codecs = []
speakers = []
subsets = []
paths = []

# ------------------------------------------------------------
# Inference
# ------------------------------------------------------------
SAVE_EVERY = 5000

for idx, item in enumerate(
    tqdm(ds, desc="V3 2021 DF inference")
):

    try:

        # Audio
        audio = item["audio"]["array"]
        sr = item["audio"]["sampling_rate"]

        # Preprocess
        feature = preprocess_audio(
            audio,
            sr
        )
```

```
        # [1,128,251] -> [1,1,128,251]
        x = feature.unsqueeze(0).to(
            device,
            non_blocking=True
        )

        # Inference
        with torch.no_grad():

            logit = model(x)

            prob = torch.sigmoid(
                logit
            ).item()

        # Metadata
        notes = json.loads(
            item["notes"]
        )

        y_true.append(
            int(item["label"])
        )

        y_prob.append(
            float(prob)
        )

        attack_ids.append(
            notes.get("attack_id", "unknown")
        )

        vocoders.append(
            notes.get("vocoder", "unknown")
        )

        codecs.append(
            notes.get("codec", "unknown")
        )

        speakers.append(
            notes.get("speaker_id", "unknown")
        )

        subsets.append(
            notes.get("subset", "unknown")
        )

        paths.append(
```

```
            item["path"]
        )

        # ----------------------------------------------------
        # Periodic checkpoint
        # ----------------------------------------------------
        if (idx + 1) % SAVE_EVERY == 0:

            np.savez_compressed(
                CHECKPOINT_FILE,
                y_true=np.array(
                    y_true,
                    dtype=np.int64
                ),
                y_prob=np.array(
                    y_prob,
                    dtype=np.float64
                ),
                attack_ids=np.array(
                    attack_ids
                ),
                vocoders=np.array(
                    vocoders
                ),
                codecs=np.array(
                    codecs
                ),
                speakers=np.array(
                    speakers
                ),
                subsets=np.array(
                    subsets
                ),
                paths=np.array(
                    paths
                ),
                processed=np.array(
                    [idx + 1],
                    dtype=np.int64
                )
            )

            print(
                f"\n Checkpoint saved: "
                f"{idx + 1:,} samples"
            )

    except Exception as e:

        print(
```

```
            f"\n⚠ Failed sample {idx}: {e}"
        )

# ------------------------------------------------------------
# Final save
# ------------------------------------------------------------
v3_eval_file = (
    RESULTS_DIR /
    "v3_2021_detailed_eval.npz"
)

np.savez_compressed(
    v3_eval_file,

    y_true=np.array(
        y_true,
        dtype=np.int64
    ),

    y_prob=np.array(
        y_prob,
        dtype=np.float64
    ),

    attack_ids=np.array(
        attack_ids
    ),

    vocoders=np.array(
        vocoders
    ),

    codecs=np.array(
        codecs
    ),

    speakers=np.array(
        speakers
    ),

    subsets=np.array(
        subsets
    ),

    paths=np.array(
        paths
    )
)

print("\n" + "=" * 60)
```

```python
print("□ V3 2021 DF INFERENCE COMPLETE")
print("=" * 60)

print(
    "Predictions:",
    len(y_prob)
)

print(
    "Probability min:",
    np.min(y_prob)
)

print(
    "Probability max:",
    np.max(y_prob)
)

print(
    "Probability mean:",
    np.mean(y_prob)
)

print(
    "Saved:",
    v3_eval_file
)
```

```
Evaluation directory:
/content/drive/MyDrive/voice_cloning_project/results/v3_2021_eval

□ Best V3 model loaded
Best validation AUC: 0.9999983805727308
```


```
{"model_id":"6987b73da2ad4dc88bc746fe333c0289","version_major":2,"vers
ion_minor":0}

{"model_id":"157f5533f22f427da76338d7135be533","version_major":2,"vers
ion_minor":0}
```


```
□ 2021 DF test stream ready
```


```
{"model_id":"24065348d8ad495f8a8a925c7a1d6605","version_major":2,"vers
ion_minor":0}
```


```
□ Checkpoint saved: 5,000 samples

□ Checkpoint saved: 10,000 samples

□ Checkpoint saved: 15,000 samples
```

```
 Checkpoint saved: 20,000 samples

⚠ Failed sample 20504: decodeAVFrame, /__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder: Invalid data found when processing input

 Checkpoint saved: 25,000 samples

⚠ Failed sample 26942: decodeAVFrame, /__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder: Invalid data found when processing input

⚠ Failed sample 29320: decodeAVFrame, /__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder: Invalid data found when processing input

 Checkpoint saved: 30,000 samples

 Checkpoint saved: 35,000 samples

 Checkpoint saved: 40,000 samples

 Checkpoint saved: 45,000 samples

 Checkpoint saved: 50,000 samples

 Checkpoint saved: 55,000 samples

 Checkpoint saved: 60,000 samples

⚠ Failed sample 61912: decodeAVFrame, /__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder: Invalid data found when processing input

 Checkpoint saved: 65,000 samples

 Checkpoint saved: 70,000 samples

 Checkpoint saved: 75,000 samples

 Checkpoint saved: 80,000 samples

⚠ Failed sample 84702: decodeAVFrame, /__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder: Invalid data found when processing input
```

```
 Checkpoint saved: 85,000 samples

 Checkpoint saved: 90,000 samples

 Checkpoint saved: 95,000 samples

 Checkpoint saved: 100,000 samples

 Checkpoint saved: 105,000 samples

 Checkpoint saved: 110,000 samples

 Checkpoint saved: 115,000 samples

 Checkpoint saved: 120,000 samples

 Checkpoint saved: 125,000 samples

⚠ Failed sample 128812: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 130,000 samples

 Checkpoint saved: 135,000 samples

 Checkpoint saved: 140,000 samples

 Checkpoint saved: 145,000 samples

 Checkpoint saved: 150,000 samples

⚠ Failed sample 153414: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 155,000 samples

 Checkpoint saved: 160,000 samples

 Checkpoint saved: 165,000 samples

⚠ Failed sample 167807: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 170,000 samples
```

```
 Checkpoint saved: 175,000 samples

⚠ Failed sample 176936: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 180,000 samples

 Checkpoint saved: 185,000 samples

 Checkpoint saved: 190,000 samples

 Checkpoint saved: 195,000 samples

 Checkpoint saved: 200,000 samples

 Checkpoint saved: 205,000 samples

 Checkpoint saved: 210,000 samples

 Checkpoint saved: 215,000 samples

 Checkpoint saved: 220,000 samples

⚠ Failed sample 220394: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 225,000 samples

 Checkpoint saved: 230,000 samples

 Checkpoint saved: 235,000 samples

 Checkpoint saved: 240,000 samples

 Checkpoint saved: 245,000 samples

⚠ Failed sample 249230: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 250,000 samples

 Checkpoint saved: 255,000 samples

 Checkpoint saved: 260,000 samples
```

```
 Checkpoint saved: 265,000 samples

⚠ Failed sample 267139: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 270,000 samples

 Checkpoint saved: 275,000 samples

⚠ Failed sample 279312: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 280,000 samples

 Checkpoint saved: 285,000 samples

⚠ Failed sample 289756: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

 Checkpoint saved: 290,000 samples

 Checkpoint saved: 295,000 samples

 Checkpoint saved: 300,000 samples

 Checkpoint saved: 305,000 samples

 Checkpoint saved: 310,000 samples

 Checkpoint saved: 315,000 samples

 Checkpoint saved: 320,000 samples

 Checkpoint saved: 325,000 samples

 Checkpoint saved: 330,000 samples

 Checkpoint saved: 335,000 samples

 Checkpoint saved: 340,000 samples

 Checkpoint saved: 345,000 samples

 Checkpoint saved: 350,000 samples
```

```
□ Checkpoint saved: 355,000 samples

⚠ Failed sample 356544: decodeAVFrame,
/__w/torchcodec/torchcodec/meta-pytorch/torchcodec/src/torchcodec/
_core/SingleStreamDecoder.cpp:1397, Could not push packet to decoder:
Invalid data found when processing input

□ Checkpoint saved: 360,000 samples

□ Checkpoint saved: 365,000 samples

□ Checkpoint saved: 370,000 samples

□ Checkpoint saved: 375,000 samples

□ Checkpoint saved: 380,000 samples

□ Checkpoint saved: 385,000 samples

□ Checkpoint saved: 390,000 samples

□ Checkpoint saved: 395,000 samples

□ Checkpoint saved: 400,000 samples
```

```python
from pathlib import Path
import numpy as np

progress_file = Path(
    "/content/drive/MyDrive/voice_cloning_project/"
    "results/v3_2021_eval/progress.npz"
)

print("Exists:", progress_file.exists())

if progress_file.exists():
    p = np.load(progress_file, allow_pickle=False)

    print("\nKeys:")
    print(p.files)

    print("\nProcessed recorded:")
    print(int(p["processed"][0]))

    print("\nArray lengths:")
    for key in p.files:
        if key != "processed":
            print(f"{key:12s}: {len(p[key]):,}")
```

```
Exists: False
```

```python
from pathlib import Path

root = Path(
    "/content/drive/MyDrive/voice_cloning_project/results"
)

print("Searching for V3 evaluation files...\n")

for p in root.rglob("*"):
    if p.is_file():
        name = p.name.lower()

        if (
            "v3" in name
            or "progress" in name
            or "2021" in name
            or "eval" in name
        ):
            print(p)
```

```
Searching for V3 evaluation files...
```

```python
from google.colab import drive
drive.mount('/content/drive')

# ============================================================
# STEP 4 — RESUMABLE V3 INFERENCE ON ASVSPOOF 2021 DF
# ============================================================

import os
import json
import numpy as np
import torch
import torchaudio
from pathlib import Path
from tqdm.auto import tqdm

# ------------------------------------------------------------
# Paths
# ------------------------------------------------------------
PROJECT_ROOT = Path(
    "/content/drive/MyDrive/voice_cloning_project"
)

RESULTS_DIR = PROJECT_ROOT / "results"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

V3_EVAL_DIR = RESULTS_DIR / "v3_2021_eval"
V3_EVAL_DIR.mkdir(parents=True, exist_ok=True)
```

```
CHECKPOINT_FILE = V3_EVAL_DIR / "progress.npz"

print("Evaluation directory:")
print(V3_EVAL_DIR)

# ------------------------------------------------------------
# Load BEST V3 model
# ------------------------------------------------------------
checkpoint = torch.load(
    V3_BEST_PATH,
    map_location=device,
    weights_only=False
)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

model.to(device)
model.eval()

print("\n Best V3 model loaded")
print("Best validation AUC:", checkpoint["best_val_auc"])

# ------------------------------------------------------------
# Fresh streaming dataset
# ------------------------------------------------------------
ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

print(" 2021 DF test stream ready")

# ------------------------------------------------------------
# Storage
# ------------------------------------------------------------
y_true = []
y_prob = []
attack_ids = []
vocoders = []
codecs = []
speakers = []
subsets = []
paths = []

# ------------------------------------------------------------
# Inference
# ------------------------------------------------------------
```

```
SAVE_EVERY = 5000

for idx, item in enumerate(
    tqdm(ds, desc="V3 2021 DF inference")
):

    try:

        # Audio
        audio = item["audio"]["array"]
        sr = item["audio"]["sampling_rate"]

        # Preprocess
        feature = preprocess_audio(
            audio,
            sr
        )

        # [1,128,251] -> [1,1,128,251]
        x = feature.unsqueeze(0).to(
            device,
            non_blocking=True
        )

        # Inference
        with torch.no_grad():

            logit = model(x)

            prob = torch.sigmoid(
                logit
            ).item()

        # Metadata
        notes = json.loads(
            item["notes"]
        )

        y_true.append(
            int(item["label"])
        )

        y_prob.append(
            float(prob)
        )

        attack_ids.append(
            notes.get("attack_id", "unknown")
        )
```

```
        vocoders.append(
            notes.get("vocoder", "unknown")
        )

        codecs.append(
            notes.get("codec", "unknown")
        )

        speakers.append(
            notes.get("speaker_id", "unknown")
        )

        subsets.append(
            notes.get("subset", "unknown")
        )

        paths.append(
            item["path"]
        )

        # ----------------------------------------------------
        # Periodic checkpoint
        # ----------------------------------------------------
        if (idx + 1) % SAVE_EVERY == 0:

            np.savez_compressed(
                CHECKPOINT_FILE,
                y_true=np.array(
                    y_true,
                    dtype=np.int64
                ),
                y_prob=np.array(
                    y_prob,
                    dtype=np.float64
                ),
                attack_ids=np.array(
                    attack_ids
                ),
                vocoders=np.array(
                    vocoders
                ),
                codecs=np.array(
                    codecs
                ),
                speakers=np.array(
                    speakers
                ),
                subsets=np.array(
                    subsets
                ),
```

```
                paths=np.array(
                    paths
                ),
                processed=np.array(
                    [idx + 1],
                    dtype=np.int64
                )
            )

            print(
                f"\n Checkpoint saved: "
                f"{idx + 1:,} samples"
            )

    except Exception as e:

        print(
            f"\n⚠ Failed sample {idx}: {e}"
        )

# ------------------------------------------------------------
# Final save
# ------------------------------------------------------------
v3_eval_file = (
    RESULTS_DIR /
    "v3_2021_detailed_eval.npz"
)

np.savez_compressed(
    v3_eval_file,

    y_true=np.array(
        y_true,
        dtype=np.int64
    ),

    y_prob=np.array(
        y_prob,
        dtype=np.float64
    ),

    attack_ids=np.array(
        attack_ids
    ),

    vocoders=np.array(
        vocoders
    ),

    codecs=np.array(
```

```
        codecs
    ),

    speakers=np.array(
        speakers
    ),

    subsets=np.array(
        subsets
    ),

    paths=np.array(
        paths
    )
)

print("\n" + "=" * 60)
print("□ V3 2021 DF INFERENCE COMPLETE")
print("=" * 60)

print(
    "Predictions:",
    len(y_prob)
)

print(
    "Probability min:",
    np.min(y_prob)
)

print(
    "Probability max:",
    np.max(y_prob)
)

print(
    "Probability mean:",
    np.mean(y_prob)
)

print(
    "Saved:",
    v3_eval_file
)
```

```
Evaluation directory:
/content/drive/MyDrive/voice_cloning_project/results/v3_2021_eval

------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
```

```
last)
/tmp/ipykernel_1269/844748491.py in <cell line: 0>()
     33 # ------------------------------------------------------------
     34 checkpoint = torch.load(
---> 35     V3_BEST_PATH,
     36     map_location=device,
     37     weights_only=False

NameError: name 'V3_BEST_PATH' is not defined
```

```python
from pathlib import Path
import os

eval_dir = Path(

"/content/drive/MyDrive/voice_cloning_project/results/v3_2021_eval"
)

print("=" * 70)
print("V3 2021 DF CHECKPOINT CHECK")
print("=" * 70)

print("\nDirectory:")
print(eval_dir)

print("\nDirectory exists:", eval_dir.exists())

if not eval_dir.exists():
    print("❌ Directory does not exist.")
else:
    files = sorted([
        f for f in eval_dir.rglob("*")
        if f.is_file()
    ])

    print("Files found:", len(files))

    if not files:
        print("❌ NO FILES FOUND")
    else:
        print("\nFiles:")

        for f in files:
            size_mb = f.stat().st_size / (1024 ** 2)
            print(
                f"  {f.name:<40} "
                f"{size_mb:.2f} MB"
            )

print("\n" + "=" * 70)
```

```
print("CHECK COMPLETE")
print("=" * 70)
```

```
======================================================================
V3 2021 DF CHECKPOINT CHECK
======================================================================

Directory:
/content/drive/MyDrive/voice_cloning_project/results/v3_2021_eval

Directory exists: True
Files found: 0
 NO FILES FOUND

======================================================================
CHECK COMPLETE
======================================================================
```